

STREAMLINED, SECURED, READY FOR SERVICE



**תיכון מקיף לאומנויות ולמדעים על שם יגאל אלון רמת  
השרון**

**מערכת לניהול נתוני מלש"בים וקשר מלש"בים עם הצבא -  
IlayDF**

**מגיש: עילי נגלר**

**ת.ז: 330635707**

**מורים מנחים: פלדברג עידן, רז עודי, סגל ליאת, רפסון רום,  
שוצמן יעקב.**

**שם החלופה: הגנת סייבר**

**תאריך הגשה: 25.5.2025**

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>מבוא.....</b>                                           | <b>4</b>  |
| <b>ייזום הפרויקט.....</b>                                  | <b>4</b>  |
| בחירת הפרויקט.....                                         | 4         |
| תיאור המערכת.....                                          | 5         |
| הגדרת הלקוח.....                                           | 5         |
| מטרת הפרויקט.....                                          | 6         |
| אתגרי הפרויקט.....                                         | 6         |
| סקירה של פתרונות קיימים.....                               | 7         |
| מגבלות המערכת.....                                         | 7         |
| סקירת טכנולוגיית הפרויקט.....                              | 7         |
| <b>פירוט תיאור המערכת.....</b>                             | <b>8</b>  |
| תיאור מפורט של המערכת.....                                 | 8         |
| יכולות המערכת.....                                         | 8         |
| תכנון וניהול לוח זמנים.....                                | 9         |
| פירוט הבדיקות (קופסה שחורה) ותרחשי קצה.....                | 9         |
| <b>תיאור תחום הידע.....</b>                                | <b>13</b> |
| יכולות הלקוח.....                                          | 14        |
| יכולות השרת.....                                           | 17        |
| <b>מבנה וארכיטקטורה של הפרויקט.....</b>                    | <b>21</b> |
| <b>תיאור הארכיטקטורה.....</b>                              | <b>21</b> |
| תיאור מילולי של החומרה.....                                | 21        |
| שרטוט גרפי של המודל.....                                   | 23        |
| <b>טכנולוגיות הפרויקט.....</b>                             | <b>23</b> |
| שפת תכנות.....                                             | 23        |
| מערכת הפעלה.....                                           | 23        |
| תקשורת.....                                                | 24        |
| JSON – מבנה הנתונים.....                                   | 25        |
| שליחת דואר אלקטרוני.....                                   | 25        |
| Threads) תהליכים.....                                      | 25        |
| GUI) ממשק המשתמש.....                                      | 26        |
| SQLite – מוד הנתונים.....                                  | 26        |
| <b>זרימת המידע במערכת.....</b>                             | <b>27</b> |
| טבלה המציגה בקצרה את זרימת המידע במערכת עבור כל יכולת..... | 27        |
| גרף המציג באופן כללי את זרימת המידע במערכת.....            | 34        |
| <b>אלגוריתמים מרכזיים בפרויקט.....</b>                     | <b>35</b> |
| הצפנה היברידית בפרויקט.....                                | 35        |
| מערכת בקשות דינמית במערכת.....                             | 38        |
| ולידציה: אימות תקינות נתוני משתמש – צד לקוח ושרת.....      | 39        |
| <b>סביבת פיתוח הפרויקט.....</b>                            | <b>41</b> |
| <b>תיאור פרוטוקול התקשורת.....</b>                         | <b>41</b> |
| תיאור מילולי של הפרוטוקול.....                             | 42        |
| פירוט הודעות במערכת.....                                   | 42        |
| <b>מסכי הפרויקט.....</b>                                   | <b>44</b> |
| <b>תיאור גרפי של מסכי הפרויקט.....</b>                     | <b>62</b> |

|                                                                                   |            |
|-----------------------------------------------------------------------------------|------------|
| <b>תיאור מבני הנתונים.....</b>                                                    | <b>63</b>  |
| usersDB.....                                                                      | 63         |
| user_data.....                                                                    | 63         |
| auth_codes.....                                                                   | 64         |
| meyunim.....                                                                      | 64         |
| meyunim_assignments.....                                                          | 65         |
| meyunim_requests.....                                                             | 65         |
| messages.....                                                                     | 65         |
| <b>סקירת חולשות ואיומים.....</b>                                                  | <b>66</b>  |
| הגנה על שכבת האפליקציה.....                                                       | 66         |
| הגנה על שכבת התעבורה.....                                                         | 68         |
| הצפנה.....                                                                        | 69         |
| <b>מימוש הפרויקט.....</b>                                                         | <b>72</b>  |
| סקירת מודלים ומחלקות במערכת.....                                                  | 72         |
| מודלים ומחלקות מיובאים.....                                                       | 72         |
| מודלים ומחלקות בפיתוח אישי שלי.....                                               | 74         |
| תרשים של מחלקות בפיתוח אישי שלי.....                                              | 85         |
| <b>בעיות אלגוריתמיות מרכזיות בפרויקט, קטעי קוד מיוחדים ופתרונות מעניינים.....</b> | <b>85</b>  |
| לתקשורת מאובטחת (RSA + AES) מנגנון הצפנה משולב.....                               | 85         |
| ניהול סשנים מאובטח וטיפול בחיבורים מרובים.....                                    | 89         |
| לוגיקת הטיפול המורכבת בבקשות מיונים ועדכון.....                                   | 91         |
| מנגנון ההתראות למשתמש.....                                                        | 94         |
| טיפול מקיף בשגיאות ותקשורת אמינה בצד הלקוח.....                                   | 96         |
| <b>מסמך בדיקות.....</b>                                                           | <b>99</b>  |
| בדיקות מתוך מסמך הבדיקות המקורי.....                                              | 99         |
| בדיקות נוספות (שלא הופיעו בתכנון).....                                            | 100        |
| <b>מדריך למשתמש.....</b>                                                          | <b>102</b> |
| עץ קבצים.....                                                                     | 102        |
| התקנת המערכת.....                                                                 | 103        |
| דרישות בסיס.....                                                                  | 103        |
| אופן הפעלת המערכת.....                                                            | 104        |
| <b>סיכום אישי ורפלקציה.....</b>                                                   | <b>104</b> |
| רפלקציה.....                                                                      | 104        |
| תודות.....                                                                        | 105        |
| <b>ביבליוגרפיה.....</b>                                                           | <b>106</b> |
| <b>נספחים.....</b>                                                                | <b>106</b> |
| main.py:.....                                                                     | 106        |
| admin_dashboard.py:.....                                                          | 138        |
| network_client.py:.....                                                           | 168        |
| network_server.py:.....                                                           | 175        |
| database.py:.....                                                                 | 199        |
| send_email.py:.....                                                               | 220        |
| ui_elements.py:.....                                                              | 222        |
| validators.py:.....                                                               | 224        |
| add_initial_user.py:.....                                                         | 226        |

## מבוא

### ייזום הפרויקט

#### בחירת הפרויקט

המערכת IlayDF נבנתה מתוך צורך ממשי לשפר את תהליך הגיוס, המיון והשיבוץ של מיועדים לשירות ביטחון (מלש"בים) לצה"ל.

מקור השם, נובע משילוב בין שמי הפרטי, Ilay, לבין שם הצבא הנהדר שלנו, IDF.

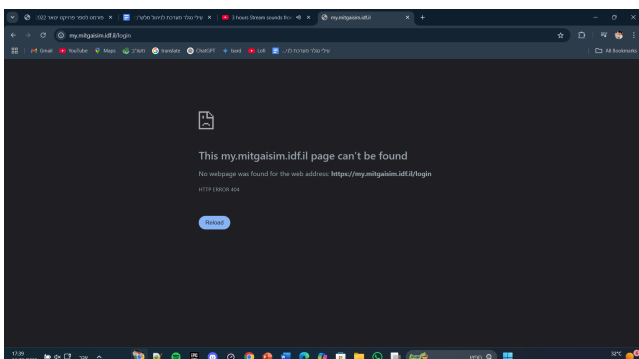
כמועמד בעצמי, הרגשתי כי יש צורך במתן חווית גיוס אחרת במובן השירותים הדיגיטליים. חווית גיוס שתהיה נוחה יותר, יעילה יותר, וכמובן גם יציבה ומאובטחת. ההיכרות שלי עם המערכת נחשבת חלקה למדי ביחס לחברי, ועדיין חוויתי לא מעט קשיים בתקשורת מול הצבא. אף אני הייתי צריך לדבר במשך למעלה מחודשיים טלפונית עם אחד ממוקדי השירות, לשלוח פניות גם לאתר וגם למייל, רק על מנת לקבל מיון שהייתי זכאי לו מלכתחילה עקב עניין טכני. בהליך המיון הקיים היום יש בעיות כמו:

- **אתר ואפליקציה שנמצאים בתחזוקה לקויה:** האתר והאפליקציה הקיימים עובדים בצורה רצופת בעיות אשר יוצרת שיבושים בתקשורת בין הצבא למלש"ב.
- **ריבוי מערכות מסורבל:** כמה מוקדים טלפונים שונים (לחלק מהחילות יש מוקד טלפוני ייעודי ולחלק לא), דפים ברשתות החברתיות השונות, צאטים בפייסבוק, אפליקציה, כתובות מייל מרובות, ימי צהל ברשות ועוד, הופכים את התקשורת עם הצבא לקשה ומעייפת לצד המועמד עם הרבה מקרי "נפילה בין הכיסאות" של מלש"בים. כל סוג ואופי תקשורת או בקשה מתבצעים בצורה שונה (לדוגמה: על מנת לקבל מיון של חיל האוויר יש לשלוח פניית מייל למייל של חיל האוויר, על מנת לדחות מיון מסויים יש לדבר עם המוקד הטלפוני של החיל אליו המיון במידה ויש כזה ובמידה ואין יש לדבר עם המוקד הטלפוני המרכזי, ועל מנת לצפות בנתונים אישיים יש להיכנס לאתר), מלש"בים אשר לא משקיעים שעות בשבוע על מנת לתקשר עם המוקדים השונים עלולים להיפגע מכך משמעותית בתצורה של חוסר קבלת הליך מיון מתאים. בנוסף לכך, ריבוי זה מקשה מאוד על הצבא ומאט את עבודתו, שכל כך צריך את ההתנהלות היעילה הזו.

**מתוך אלו, נולד הרצון ליצור מערכת דיגיטלית מודרנית, מאוחדת לכל, שתיתן מענה אמיתי – גם למלש"בים וגם לגורמי הצבא.** המלש"בים ירוויחו מערכת נוחה ויכולו להשקיע פחות מאמצים בהליכי המיון ויותר מאמצים בהכנה מתאימה, והצבא יוכל לפנות כוח אדם שהוא כך כך צריך במיוחד בתקופה זו ולעבוד ביעילות ומהירות.

#### דוגמאות חיות לבעיות בשעת עריכת הספר:

כעת אני עורך את הספר, ובמקרה ניסיתי להתחבר אל האתר לאחר שקיבלתי הודעה כי עודכן לי זימון חדש. האתר אינו עובד, ואפילו אינו מציג מסך מתאים.



בעיה נוספת שנתקלתי בעת עריכה שנייה, שבוע לאחר הבעיה הקודמת:

נסיתי להתחבר לאתר, שנראה שעובד כרגיל, אך פעם אחר פעם האתר הציג שגיאה לאחר הזנת קוד האימות שקיבלתי אל הטלפון. לאחר שהתייאשתי, נכנסתי אל חשבון האינסטגרם שלי וראיתי הודעה של מיטב בחשבון האינסטגרם הרשמי שלהם (בחלון הנקרא "סטורי") הודעה על התקלה. שוב- בעיות באתר ללא הודעה מתאימה באתר, ריבוי מקורות מסורבל וריבוי תקלות. כל אלו רק חיזקו לי את ההבנה כמה דרושה מערכת כמו IlayDF.



### תיאור המערכת

מערכת IlayDF הינה פלטפורמה כפולה, דיגיטלית מאובטחת לניהול תהליך הגיוס, המיון והשיבוץ של מלש"בים, תוך יצירת תקשורת ישירה, נוחה ויעילה בינם לבין גורמי הצבא. המערכת מאפשרת למלש"ב להתחבר בצורה מאובטחת, לצפות בנתוני האישיים וביבוציו למיונים עתידיים, להגיש בקשות לשינוי מועד או ביטול מיון, ולשלוח הודעות ישירות לאדמין. במקביל, היא מעניקה לצבא כלים מתקדמים לניהול משתמשים, יצירת מיונים, טיפול בבקשות, תגובה להודעות, והצגת נתונים סטטיסטיים דרך גרפים ברורים. כל המידע במערכת מועבר ומאוחסן בהצפנה מלאה, תוך הקפדה על חוויית משתמש ברורה, ממשק גרפי נעים ושמירה מוחלטת על פרטיות ואבטחת המידע של כל הצדדים.

### הגדרת הלקוח

המערכת משרתת שני סוגי לקוחות:

- המלש"בים – אשר מקבלים ממשק אישי דרכו הם יכולים להתחבר בצורה מאובטחת ששומרת על נתוניהם, לצפות בנתוניהם האישיים, לראות מיונים עתידיים, להגיש בקשות (כגון דחייה או ביטול), ולשלוח הודעות לצבא(למשל לבקש מיונים, לערער על תוצאות שיבוץ).
- הצבא – אשר מנהל את כלל המערכת. הוא אחראי על הוספת משתמשים, שיבוצם למיונים, טיפול בבקשות, תגובה להודעות, הצגת נתונים סטטיסטיים, ניהול מידע אישי וצבאי של המשתמשים, ועוד.

## מטרת הפרויקט

המטרה המרכזית היא להפוך את תהליך הגיוס לצה"ל לנגיש, מהיר, ומאובטח, בצורה שתורמת גם למלש"ב וגם לצה"ל.

מצד אחד, היא מאפשרת לכל מלש"ב לדעת בדיוק היכן הוא עומד, אילו מיונים קיבל, ולפנות לצבא בקלות, ומצד שני, היא מספקת לצבא מערכת ניהול יעילה, אינטואיטיבית, גמישה ומדויקת, כזו שתומכת בעבודה בקנה מידה רחב ומצמצמת עבודה ידנית מיותרת. בנוסף, יש לצבא אופציה להעניק גישה לקהל רחב של מנהלים, בזכות ביטול הצורך של מחשב המנהל להיות בחיבור ישיר למסד הנתונים והשרת. על יכולת זו ארחיב בהמשך, אבל יתרונה ברור – כל אחד מהמנהלים יכול להשתמש במערכת במחשבו הפרטי.

בכך שאין צורך שמנהל יהיה מחובר ישירות למצוינים לעיל, יש שני יתרונות מרכזיים:

- **הגברת הגמישות של כל מנהל** – יכול לעבוד ממחשבו הפרטי מכל מקום ובכל זמן. גמישות זו תאפשר עבודה יעילה ומהירה יותר.
- **הגברת האבטחה** – גם במידה של פריצה למחשב של אחד המנהלים, מסד הנתונים והשרת לא שמורים שם, כך שפעולות התוקף מוגבלות ואין לו גישה לנתונים יקרי ערך.

## אתגרי הפרויקט

### הגנה על פרטיות המשתמשים ונתונים

מערכת IlayDF עוסקת בנתונים אישיים רגישים במיוחד: גיל, כתובת, תעודת זהות, פרופיל צבאי, דפ"ר, ציון עברית ומידע רפואי-תפקודי. מדובר במידע שלא רק מעורר רגישות משפטית, אלא גם מוסרית. כבר מתחילת הדרך היה לי ברור שכל גישה לנתונים אלה חייבת להיעשות באחריות גבוהה, ולכן הוטמעה הצפנה חזקה, מנגנוני אימות כפולים, וחוסר שימוש במידע מעבר למה שנדרש תפעולית. יתר על כן, המידע רגיש גם מעבר לפן האישי. חשיפה של מידע מהסוג הזה יכולה להביא לפגיעה חמורה בצה"ל ובביטחון המדינה.

### שקיפות מול המשתמשים

אחת הסוגיות האתיות שעלו במהלך הפיתוח היא גבול השקיפות מול המשתמש: מה לחשוף למשתמש? באיזה שלב? למשל האם עליו לראות את סטטוס הבקשות שלו? או מה לומר כאשר בקשה נדחת? בחרתי לבנות מערכת שבה כל פעולה משתקפת מיידית אצל המשתמש – בצורה שגם מכבדת אותו ואת זכותו לדעת מה הם השיקולים בנוגע לעתידו בצבא, אך שומרת לצבא את הזכות ללקבוע את כמות המידע שהיא חושפת למשתמש מתוך הבנה כי שיקולים מסוימים חסויים, וחשיפה שלהם יכולה לגרום לנזק. בין אם לצבא או למשתמש.

המערכת מסבירה בשפה פשוטה, על מנת שכולם יוכלו להשתמש בה בלי קשר ליכולתם הטכנולוגית. המטרה הייתה לבנות מערכת שלא רק יעילה – אלא גם מכבדת ותומכת.

### הפרדה ברורה בין סמכות למידע

במערכת קיימים שני סוגי משתמשים – מלש"ב ואדמ"ן – ואף אחד מהם לא אמור לגשת אל מידע שאינו בסמכותו. האדמ"ן, למרות תפקידו, אינו חשוף הסיסמאות של המשתמשים, אינו רואה הודעות פרטיות שנחקקו, ואינו יכול להיכנס "בשם" המשתמש. המשתמש, חשוף אך ורק לנתונים האישיים, אך יכול להיות בטוח כי אף אחד לא יכול לגשת לנתוניו האישיים.

ההפרדה הזו, שמבוססת גם בקוד וגם ברמת ההרשאות, מבטאת תפיסה ברורה של אחריות מערכתית.

### **אבטחת מידע לא כמשהו טכני, אלא עקרוני**

הבחירה להשתמש בהצפנה מתקדמת לא נעשתה רק משיקולים טכנולוגיים. היה לי חשוב להראות למשתמש שפרטיותו לא רק נדרשת על פי חוק – אלא גם נלקחת ברצינות. זו הייתה בחירה ערכית לא פחות ממקצועית.

### **איזון בין אוטומציה למעורבות אנושית**

המערכת מאפשרת אישור בקשות בלחיצת כפתור – אך בחרתי שלא לאשר בקשות בצורה אוטומטית. הסיבה היא שלא כל בקשה דומה לקודמתה. השארתי את האדמין כחלק פעיל בתהליך – מתוך מחשבה שאמפתיה אנושית והבנה של נסיבות הן מרכיב שאי אפשר להחליף בקוד.

### **סקירה של פתרונות קיימים**

בצה"ל קיימים כיום מספר כלים דיגיטליים, אך יש בהן שתי בעיות מרכזיות שהן מערכות רצופות בעיות ולא נוחות לשימוש, וריבוי סביבות קשר של הצבא למלש"ב(הרחבה נמצאת בסיבות לבחירת הפרויקט)

IlayDF שונה בכך שהיא מערכת דו־כיוונית, פתוחה ומאובטחת – כזו שמבוססת על הצפנה מלאה של כל מידע, ממשק גרפי נוח, והפרדה ברורה בין הרשאות המשתמשים.

### **מגבלות המערכת**

המערכת דורשת התקנת סביבת Python עם ספריות חיצוניות. היא פועלת במחשב בלבד ואינה זמינה כרגע למובייל או כדפדפן. כמו כן, מדובר בפרויקט עצמאי שאינו מחובר בפועל למסדי הנתונים של צה"ל.

### **סקירת טכנולוגיית הפרויקט**

פרויקט IlayDF עוסק במספר תחומים משמעותיים, אשר דורשים ידע ויישום טכנולוגיות ממספר תחומים שונים:

- **רשתות תקשורת:** המערכת מבוססת על תקשורת TCP socket בין לקוח לשרת. כל הודעה נשלחת מלקוח לשרת ונענית חזרה, כולל הצפנה ופענוח. נעשה שימוש במבנה JSON להעברת נתונים, עם הצפנה דו־שלבית (Hybrid Encryption) לכל הודעה.
- **אבטחת מידע:** יישום מלא של הצפנה היברידית – שילוב בין RSA (להצפנת מפתחות סימטריים) ו-AES (להצפנת תוכן). סיסמאות מגובבות עם SHA256. המערכת מגבילה גישה לפי תפקיד, דורשת אימות דו־שלבי, ומתנתקת לאחר חוסר פעילות.
- **מערכות הפעלה:** המערכת פועלת בסביבת מערכת הפעלה שולחנית (Windows / Linux / Mac), תוך גישה ישירה לקבצים, קריאת משתנים מקובץ .env, שימוש בחלונות מערכת ואייקונים, וממשק משתמש גרפי בסביבת Tkinter.
- **GUI – ממשק משתמש:** פיתוח ממשק משתמש מלא עם CustomTkinter כולל תפריטים, טפסים, popup, תיבות גלילה, תצוגות גרף, אייקונים וסגנון אחיד לכל רכיב במערכת.
- **מודי נתונים:** ניהול מידע באמצעות SQLite – כולל יצירת טבלאות, קשרים בין טבלאות (foreign keys), שאילתות מותאמות פר משתמש, עדכון סטטוס, שמירת חותמות זמן ואחזור מידע סטטיסטי.

## המערכת אינה עוסקת בתחומים הבאים:

- אינה פועלת ברשתות אינטרנט ציבוריות (אין שימוש בפרוטוקולי HTTP / HTTPS).
- אינה כוללת ממשק אינטרנטי (אין HTML / CSS / JavaScript).
- אינה מבצעת אימות ביומטרי (טביעת אצבע, זיהוי פנים).
- אינה שומרת קבצים בענן או משתפת מידע עם שרתים חיצוניים.
- אינה מתחברת למערכות הצבא האמיתיות או ל-API ממשלתי.

## סייגים טכנולוגיים שקיימים בהפעלת המערכת כוללים:

- **צורך במחשב מקומי:** המערכת אינה פועלת בדפדפן או בענן, אלא רק כתוכנה שמותקנת על מחשב עם Python.
- **חיבור לרשת מקומית:** כדי שהלקוח יוכל לתקשר עם השרת, שניהם חייבים להיות באותה רשת.
- **תלות בגורמי חוץ לשליחת מיילים:** המערכת מסתמכת על שירות SMTP של Gmail.
- **אי-תמיכה מובנית במובייל או טאבלט:** הממשק מבוסס Tkinter, ולכן מותאם להפעלה על מחשב אישי בלבד.

## פירוט תיאור המערכת

### תיאור מפורט של המערכת

IlayDF כפי שנכתב קודם, היא מערכת לניהול תהליך הגיוס, המיון והשיבוץ של מועמדים לשירות ביטחון תוך ניהול נתונים אישיים, זרימת מידע דו-כיוונית בין משתמשים מנהליים לבין מועמדים, והצגת מידע בזמן אמת באופן מאובטח ומבוקר.

המערכת נועדה לארגן, לאחסן, לנתח ולתאם בין רכיבי המידע הרבים שמעורבים בתהליך הגיוס לצה"ל. היא ממפה קשרים בין נתונים אישיים לבין פעולות ניהוליות (כגון שיבוץ למיונים), ומבצעת אינטגרציה של מגוון מרכיבים – תקשורת, אבטחת מידע, ממשק משתמש, ועיבוד נתונים – לכדי מערכת אחת קוהרנטית.

ברמה הלוגית, המערכת מאפשרת שליטה דו-צדדית על בסיס מידע מוגדר מראש, תיעוד פעולות, ניהול זרימת תהליכים (כמו שיבוץ ← בקשה ← אישור ← תגובה), והפקת דוחות ותצוגות ויזואליות. כל הרכיבים הללו מתואמים ומוגנים תחת מערכת הרשאות קפדנית, מבנה נתונים מסודר, ופרוטוקול תקשורת מוצפן.

בפועל, המערכת מתפקדת כגוף מתווך בין מאגר הנתונים הפנימי לבין גורמי קצה, תוך ניהול מלא של כל שלב בתהליך – החל מהזדהות ואימות כפול, דרך שליפת נתונים, ועד אישור או דחיית פעולות משתמש. כל זה מתבצע ללא גישה לוקאלית לשרת, תוך שמירה על פרטיות מוחלטת, תיעוד מלא של כל פעולה, והתאמה לשימוש יומיומי.

IlayDF מיישמת באופן פרקטי עקרונות של מערכות מידע ארגוניות: גישה לפי תפקיד, הפרדה בין ממשק למנוע לוגי, אינטגרציה של הצפנה, תיעוד כל שינוי, ניתוח נתונים, ותגובה בזמן אמת למידע דינמי.

### יכולות המערכת

**יכולות משתמש רגיל (מלש"ב):**

- קבלת שם משתמש וסיסמה ראשונית למייל(מחליף את קבלת הצו הראשון האמיתי ופרטי ההתחברות לאתר לכתובת הפיזית של המלש"ב כנהוג היום)
- אימות התחברות ראשונית לפי מספר תעודת זהות.
- התחברות מאובטחת עם סיסמה ואימות דו־שלבי.
- איפוס סיסמה.
- צפייה בנתונים האישיים(לדוגמה ציון ראיון או פרופיל רפואי).
- צפייה במיונים עתידיים אליהם שובץ.
- הגשת בקשה לדחיית מיון.
- הגשת בקשה לויתור על מיון.
- שליחת הודעות ישירות לצבא בכל נושא.
- צפייה בתגובות מהצבא להודעות.
- קבלת התראות על שינויי סטטוס\קבלת התרעות.
- קבלת תזכורות למיונים קרובים.
- התנתקות מסודרת וניתוק אוטומטי לאחר חוסר פעילות.

### **יכולות משתמש אדמין (צבא):**

- הפעלת של שרת התקשורת בלחיצה.
- כניסה לממשק ניהול גרפי ייעודי.
- התחברות מאובטחת עם סיסמה ואימות דו־שלבי.
- איפוס סיסמה.
- הוספת משתמשים חדשים למערכת.
- שליחת פרטי התחברות ראשונית למייל של משתמש חדש(מחליף את שליחת הצו הראשון האמיתי ופרטי ההתחברות לאתר לכתובת הפיזית של המלש"ב כנהוג היום).
- עריכת והזנה של נתונים אישיים וצבאיים של משתמשים.
- יצירה וניהול של מיונים חדשים.
- יצירה של מיונים.
- שיבוץ משתמשים למיונים.
- צפייה בבקשות דחייה / ויתור בנוגע למיונים לפי סטטוס וטיפול בהן.
- קביעת תאריכים חדשים למיונים.
- צפייה בכל המיונים הפעילים.
- צפייה ותגובה להודעות ממשתמשים.
- הצגת גרפים סטטיסטיים מגוונים מתוך בסיס הנתונים.
- גיבוי וניהול עקיף של מסד הנתונים.

### **תכנון וניהול לוח זמנים**

התחלת העבודה- סוף חודש אוקטובר 24 עד תחילת ינואר 25. בוצעה הכנה ראשונית של המערכת, ללא פיצרים מתקדמים שעובדים, רק הקמה של מסד נתונים ראשוני וממשק לקוח ואדמין ראשוניים.

עיבוי המערכת והכנסת פיצרים מתקדמים- ינואר 25 עד מרץ 25. בוצעה הטמעה של פיצרים מתקדמים יותר, ברמה לוקאלית ללא תקשורת. במקביל בנייה של מערכת תקשורת עובדת ללא חיבור למערכת.

שילוב המערכות- מרץ 25 עד תחילת מאי 25. בוצעה הטמעה של התקשורת במערכת, הטמעת תהליכים, ובנייה של מערכת ההצפנה וייצוב של המערכת.

שילוב הצפנה, הגנת סייבר מתקדמת וסגירת קצוות- מאי 25 עד יוני 25. בוצע שילוב מלאה של מערכת ההצפנה, התווספו הגנות כנגד מתקפות סייבר שונות והמערכת נבדקה באופן מקיף.

## **פירוט הבדיקות (קופסה שחורה) ותרחישי קצה**

המערכת עברה תכנון וביצוע של מערך בדיקות בשיטת "קופסה שחורה" – כלומר, בדיקות המתמקדות בהתנהגות החיצונית של המערכת בהתאם לקלט שמוזן לה, ללא גישה לקוד הפנימי. מטרת הבדיקות היא לוודא שכל תהליך מגיב באופן תקין לקלטים תקינים, ומטפל באופן מאובטח ויציב בקלטים חריגים או לא צפויים. בנוסף, נבחנו תרחישי קצה המדמים מצבים גבוליים, נדירים או שגויים – כדי לוודא שהמערכת שומרת על אמינותה בכל מצב. הבדיקות בוצעו על ידי הפעלת המערכת בפועל, הזנת קלטים דרך ממשק המשתמש או שליחת בקשות יזומות, תוך תצפית על תגובת המערכת, ללא שימוש בבדיקות קוד פנימיות.

### **רשימת הבדיקות ומטרתן עם אופן ביצוע:**

1. **התחברות עם סיסמה שגויה**  
מטרת הבדיקה: האם המערכת מזהה סיסמה לא נכונה ומונעת גישה למערכת.  
שיטת הבדיקה: מזינים שם משתמש תקני עם סיסמה שגויה.
2. **התחברות עם שם משתמש לא קיים**  
מטרת הבדיקה: לבדוק האם המערכת מונעת התחברות של משתמש לא רשום.  
שיטת הבדיקה: מזינים שם שאינו קיים במסד.
3. **התחברות עם קוד אימות שגוי**  
מטרת הבדיקה: לבדוק האם המערכת בודקת את הקוד בצורה תקינה ואינה מאשרת קוד שגוי.  
שיטת הבדיקה: מזינים סיסמה נכונה ואז קוד אקראי.
4. **התחברות עם קוד שפג תוקף**  
מטרת הבדיקה: לבדוק האם מגבלת הזמן על קוד האימות עובדת.  
שיטת הבדיקה: מחכים 10 דקות לפני הזנת הקוד ואז מזינים את הקוד שקיבלנו.
5. **התחברות תקינה מלאה**  
מטרת הבדיקה: לבדוק האם המערכת פועלת תקין בתנאים רגילים של התחברות.  
שיטת הבדיקה: מזינים פרטים נכונים ומוודאים מעבר למסך ראשי.

6. **התחברות עם קוד שכבר שומש**  
מטרת הבדיקה: לבדוק האם הקוד נצרך פעם אחת בלבד.  
שיטת הבדיקה: מזינים את אותו הקוד פעמיים.
7. **שליחת הודעה לאדמין**  
מטרת הבדיקה: לבדוק האם ההודעה נשלחת ונשמרת במסד.  
שיטת הבדיקה: שולחים הודעה מתפריט המשתמש.
8. **צפייה בתגובה מהאדמין**  
מטרת הבדיקה: לבדוק האם התגובה נשלחת חזרה ונשמרת במסד.  
שיטת הבדיקה: מתחברים כאדמין ובודקים האם אנו רואים את ההודעה שנשלחה.
9. **שליחת הודעה ריקה**  
מטרת הבדיקה: לבדוק האם המערכת בולמת שליחה של תוכן ריק.  
שיטת הבדיקה: מנסים לשלוח הודעה ללא טקסט.
10. **שליחת בקשת דחייה על מיון**  
מטרת הבדיקה: לבדוק האם הבקשה נרשמת ונשלחת לאדמין.  
שיטת הבדיקה: בוחרים מיון עתידי ולוחצים "בקש דחייה".
11. **שליחת בקשת ויתור על מיון**  
מטרת הבדיקה: לראות האם הבקשה נרשמת והסטטוס משתנה.  
שיטת הבדיקה: נשלחת בקשה דרך כפתור "וותר".
12. **אישור בקשה מצד האדמין**  
מטרת הבדיקה: לבדוק האם המערכת מעדכנת את סטטוס הבקשה.  
שיטת הבדיקה: לוחצים כאדמין על "אשר" ומוודאים שינוי(גם מצד המנהל ולאחר מכן מצד המשתמש)
13. **דחיית בקשה מצד האדמין**  
מטרת הבדיקה: לבדוק האם סטטוס הבקשה משתנה ל-Rejected  
שיטת הבדיקה: לוחצים כאדמין על "דחה" ומוודאים שינוי(גם מצד המנהל ולאחר מכן מצד המשתמש).

**14. שינוי תאריך במיון**

מטרת הבדיקה: לבדוק האם המערכת מעדכנת את התאריך במאגר.  
שיטת הבדיקה: משנים תאריך כאדמין ומוודאים שינוי(גם מצד המנהל ולאחר מכן מצד המשתמש).

**15. יצירת מיון חדש**

מטרת הבדיקה: לבדוק האם המיון נרשם ומופיע במערכת  
שיטת הבדיקה: יוצרים מיון חדש כאדמין ומוודאים שניתן לראותו תקינה(גם מצד המנהל ולאחר מכן מצד המשתמש).

**16. יצירת מיון ללא תאריך**

מטרת הבדיקה: לבדוק האם המערכת בולמת מיון לא תקני.  
שיטת הבדיקה: יוצרים מיון חדש כאדמין, ללא תאריך.

**17. שיבוץ משתמש למיון**

מטרת הבדיקה: לבדוק האם המשתמש מקבל את השיבוץ במסך.  
שיטת הבדיקה: יוצרים מיון חדש כאדמין ומוודאים שניתן לראותו תקינה מצד המשתמש.

**18. מחיקת מיון ועדכון בצד המשתמש**

מטרת הבדיקה: לבדוק האם המיון מוסר מהמסכים הרלוונטיים.  
שיטת הבדיקה: מוחקים מיון כאדמין ומוודאים שהוא הוסר(גם מצד המנהל ולאחר מכן מצד המשתמש).

**19. הוספת משתמש חדש עם פרטים תקינים**

מטרת הבדיקה: לבדוק האם ההוספה למערכת מתבצעת כהלכה.  
שיטת הבדיקה: מוסיפים כאדמין משתמש חדש, ולאחר מכן מנסים להתחבר כמשתמש ההוא.

**20. ניסיון להוסיף שם משתמש קיים**

מטרת הבדיקה: לבדוק האם המערכת מונעת כפילויות של שמות משתמשים.  
שיטת הבדיקה: מוסיפים כאדמין שם משתמש שכבר קיים.

**21. שליחת פרטי התחברות ראשוניים למייל**

מטרת הבדיקה: לבדוק האם מייל ההצטרפות למערכת יוצא ומתקבל ביעד כהלכה.  
שיטת הבדיקה: מוסיפים כאדמין משתמש חדש ובודקים האם קיבלנו למייל זה את פרטי ההתחברות.

**22. קבלת קוד אימות למייל**

מטרת הבדיקה: לבדוק האם השרת שולח קוד אימות למייל כהלכה.  
שיטת הבדיקה: מתחברים עם משתמש קיים ובודקים האם קיבלנו קוד למייל.

**23. ניתוק אוטומטי לאחר חוסר פעילות**

מטרת הבדיקה: בדיקת תקינות סגירת session לפי טיימר.  
שיטת הבדיקה: נמצאים בחוסר פעילות למשך 10 דקות.

**24. Logout יזום של משתמש**

מטרת הבדיקה: לבדוק האם המשתמש חוזר למסך הראשי כרצוי.  
שיטת הבדיקה: לחיצה על כפתור Logout.

**25. הצגת נתונים אישיים לאחר התחברות**

מטרת הבדיקה: בדיקת שליפה תקינה של נתונים מהמסד.  
שיטת הבדיקה: כניסה למסך "הנתונים האישיים שלי" כמשתמש.

**26. עדכון נתוני משתמש ע"י אדמין**

מטרת הבדיקה: לבדוק האם לאחר שינוי פרטים מצד האדמין הם נשמרים כהלכה.  
שיטת הבדיקה: עורכים כאדמין פרטי משתמש חדש ובודקים אם השתנו תקינה(גם דרך מסך צפיית הנתונים של האדמין ולאחר מכן דרך כניסה לאותו המשתמש וצפייה בנתונים האישיים).

**27. טעינת גרפים סטטיסטיים תקינה.**

מטרת הבדיקה: לבדוק שהגרפים עולים והנתונים אשר מוצגים בהם נכונים  
שיטת הבדיקה: ניגשים ללשונית הגרפים בלוח האדמין, ובודקים האם הגרפים עולים והאם הם משקפים את הנתונים הקיימים במסד.

**28. אובדן תקשורת כתוצאה משרת נכבה במהלך הפעילות**

מטרת הבדיקה: לבדוק האם המערכת מציגה שגיאה כרצוי.  
שיטת הבדיקה: הפעלת שרת באופן תקין, העלאת המערכת, כיבוי השרת וצפייה האם הודעה מתאימה מוצגת.

**29. שרת כבוי בעת ההרצה**

מטרת הבדיקה: לבדוק האם המערכת מציגה שגיאה כרצוי.  
שיטת הבדיקה: מנסים להעלות את המערכת כאשר השרת כבוי.

**30. קלט שגוי בשדות מספריים**

מטרת הבדיקה: לבדוק האם המערכת מונעת הזנת טקסט לא חוקי.  
שיטת הבדיקה: מנסים להזין אותיות בשדה גיל/טלפון וצופים אם מופיעה הודעה מתאימה.

**תיאור תחום הידע**

מערכת llayDF מיועדת לשני סוגי משתמשים – משתמשים רגילים (מלש"בים) ומנהלי מערכת (אדמינים) – כל אחד מהם מקבל חוויית שימוש שונה, המותאמת לצרכיו ולתפקידו.

בשלב הקודם הוצגה רשימת היכולות של המערכת ברמת כותרת. כעת, נעבור לשלב המפורט יותר – ניתוח של כל יכולת מהותית, כפי שהיא מיושמת בצד השרת ובצד הלקוח. עבור כל יכולת יוצג שמה, מטרתה בפועל, הפעולות המרכיבות אותה והאובייקטים הדרושים למימושה. הניתוח הזה מבהיר את המבנה הפנימי של המערכת ואת ריבוי המרכיבים שדרושים כדי לגרום לכל פעולה להתרחש – גם אם עבור המשתמש הסופי היא נראית פשוטה.

הצגת היכולות נעשית בהפרדה בין צד השרת לצד הלקוח, כאשר כל צד כולל תיאור מדויק של היכולות הרלוונטיות לו, כפי שבאו לידי ביטוי במימוש המעשי של המערכת.

**יכולות הלקוח**

להלן פירוט מלא של כלל היכולות שממשק לקוח במערכת תומך בהן. כל יכולת מתארת פעולה שהלקוח מבצע – כגון שליחת בקשה לשרת, קבלת נתונים, או הצגת ממשק גרפי למשתמש. כל פריט ברשימה כולל תיאור מהות היכולת, היכולות הטכניות הנדרשות לביצועה, והרכיבים המעורבים בתהליך – כגון כפתורים, טפסים, רכיבי GUI, ופונקציות תקשורת. רשימה זו משקפת את ממשק המשתמש ואת כל נקודות הממשק של הלקוח מול השרת, ומהווה בסיס לפיתוח, בדיקה ושיפור חוויית המשתמש.

**קבלת שם משתמש וסיסמה ראשונית למייל(משתמש רגיל + אדמין)**

**מהות היכולת:** שליחת פרטי התחברות ראשוניים (שם משתמש וסיסמה זמנית) ישירות למייל של המלש"ב, במקום שליחה פיזית בדואר.

**אוסף יכולות נדרשות:** יצירת משתמש חדש על ידי האדמין, הפקת סיסמה זמנית, שליחת הפרטים למייל בצורה אוטומטית, שמירת הסיסמה במסד לאחר גיבוב.

**אובייקטים נחוצים:** טופס יצירת משתמש, SMTP, database, send\_email.py.

**אימות התחברות ראשונית לפי מספר תעודת זהות(משתמש רגיל + אדמין)**

**מהות היכולת:** בדיקת תקינות המשתמש על סמך ת"ז לפני הזנת סיסמה אוסף יכולות נדרשות: שדה לקליטת ת"ז במסך ההתחברות, אימות ת"ז מול רשומה במסד, הצגת שדות המשך רק לאחר אימות.

**נחוצים:** main.py, usersDB, בקשת login\_step\_1, תיבת קלט

**איפוס סיסמה**

**מהות היכולת:** שליחת סיסמה חדשה למשתמש במקרה ששכח את הסיסמה, באמצעות מייל מאומת בלבד.

**אוסף יכולות נדרשות:** ממשק להזנת תעודת זהות, אימות קיומו של המשתמש במסד, הפקת סיסמה

חדשה, גיבוב הסיסמה, עדכון הסיסמה במסד, שליחת הסיסמה למייל של המשתמש, הצגת חיווי הצלחה או שגיאה.  
אובייקטים נחוצים: טופס איפוס סיסמה, `usersDB`, `send_email.py`, פונקציית `reset_password`, הצפנה, תקשורת עם השרת.

## התחברות למערכת (משתמש רגיל + אדמין)

**מהות היכולת:** התחברות עם שם משתמש וסיסמה, ולאחר מכן אימות עם קוד חד-פעמי.  
**אוסף יכולות נדרשות:** ממשק התחברות גרפי, קליטת קלט מהמשתמש, גיבוב סיסמה, שליחת בקשת `login` לשרת, קבלת קוד במייל, הזנת קוד ושליחת `verify_code`, פענוח תגובה והצגת מסך המשך.  
**אובייקטים נחוצים:** `main.py`, `network_client`, `GUI` התחברות, טיימר, קובץ תקשורת

## תפריט ראשי למשתמש(משתמש רגיל)

**מהות היכולת:** מסך המשמש "תפריט" לבחירה בין האפשרויות השונות ומציג התראות חדשות(כדוגמת מיון קרוב או תשובה שהתקבלה)  
**אוסף יכולות נדרשות:** הצגת שם המשתמש, הצגת התראות חמישה כפתורים: "הנתונים האישיים שלי", "המיונים שלי", "היסטוריית בקשות", "שליחת הודעות למנהל", "Logout".  
**אובייקטים נחוצים:** `GUI`, `CustomTkinter`, פונקציות ניווט, כפתורים.

## הצגת נתונים אישיים(משתמש רגיל)

**מהות היכולת:** הצגת פרטים אישיים כמו סימול עברית, פרופיל, דפ"ר ועוד  
**אוסף יכולות נדרשות:** שליחת בקשה `get_user_data`, קבלת הנתונים מהשרת, עיבוד `JSON` להצגת שדות, טעינת התצוגה לתוך טופס קריא.  
**אובייקטים נחוצים:** `network_client`, `user_data_frame`, תוויות `GUI`.

## הצגת מיונים(משתמש רגיל)

**מהות היכולת:** תצוגה של כל המיונים הפעילים למשתמש  
**אוסף יכולות נדרשות:** שליחת בקשה `get_user_meyunim`, קבלת רשימת מיונים, הצגת כל מיון ככרטיס עם כפתורים לפעולה.  
**אובייקטים נחוצים:** `GUI` כרטיסי מיון, `Frame` גלילה, פונקציות בקשות.

## בקשת דחייה למיון(משתמש רגיל)

**מהות היכולת:** המשתמש מבקש לדחות מועד מיון מסוים  
**אוסף יכולות נדרשות:** כפתור "בקש דחייה" לצד מיון, פתיחת חלון `popup` להזנת הסבר, שליחת בקשת `submit_request` עם `delay`.  
**אובייקטים נחוצים:** `MyButton`, `BasePopup`, בקשה לשרת, `Validators`.

## בקשת ויתור על מיון(משתמש רגיל)

**מהות היכולת:** המשתמש מבקש להסיר את עצמו מהמיון.  
**אוסף יכולות נדרשות:** כפתור "וותר" ליד כל מיון, חלון הסבר, שליחת בקשה עם סוג cancel.  
**אובייקטים נחוצים:** popup, network\_client, כפתור GUI, שדות קלט.

### שליחת הודעה ישירה לצבא(משתמש רגיל)

**מהות היכולת:** המשתמש שולח שאלה/פנייה דרך הממשק.  
**אוסף יכולות נדרשות:** טופס הודעה עם נושא ותוכן, שליחת הודעה לשרת, הצגת חיווי הצלחה.  
**אובייקטים נחוצים:** שדות טקסט, כפתור שליחה, network\_client

### צפייה בתגובות מהאדמין(משתמש רגיל)

**מהות היכולת:** תצוגת ההודעות שנשלחו עם תגובות.  
**אוסף יכולות נדרשות:** שליחת בקשה לקבלת כל ההודעות, עיבוד הרשומות לתצוגה, הפרדת הודעות עם תגובה מאלה שעדיין ממתינות.  
**אובייקטים נחוצים:** GUI תיבת הודעות, JSON, scrollable frame.

### ניתוק מהמערכת(משתמש רגיל + אדמין)

**מהות היכולת:** יציאה יזומה או אוטומטית לאחר 10 דקות מסשן הפעילות.  
**אוסף יכולות נדרשות:** כפתור Logout, איפוס משתנים פנימיים, ניתוב למסך ההתחברות, ניתוק אוטומטי לאחר 10 דקות (טיימר).  
**אובייקטים נחוצים:** פונקציית reset, הגדרת timeout, תפריט ראשי.

### הוספת משתמש חדש(אדמין בלבד)

**מהות היכולת:** יצירת משתמש חדש במסד ושליחת סיסמה זמנית למייל  
**אוסף יכולות נדרשות:** מילוי טופס יצירה, בדיקת תקינות שדות (שם משתמש, ת"ז, מייל), שליחת הבקשה לשרת, יצירת המשתמש במסד, שליחת סיסמה זמנית למייל, הצגת הודעת הצלחה/שגיאה.  
**אובייקטים נחוצים:** טופס יצירת משתמש, network\_client, send\_email.py, validators.py.

### צפייה ברשימת המשתמשים(אדמין בלבד)

**מהות היכולת:** תצוגת טבלת כל המשתמשים במערכת.  
**אוסף יכולות נדרשות:** שליחת בקשה לשרת לשליפת כל המשתמשים, קבלת נתונים ועיבוד לתצוגה, הצגת טבלה עם שדות: שם, מייל, טלפון, גיל.  
**אובייקטים נחוצים:** scrollable table, usersDB, GUI, רשימה, תגיות טבלה.

### עריכת נתוני משתמש והזנת נתונים חדשים(אדמין בלבד)

**מהות היכולת:** עדכון או הוספה של גיל, כתובת, טלפון, נתוני פרופיל, דפ"ר, עברית ויום המאה.  
**אוסף יכולות נדרשות:** בחירת משתמש מהרשימה, טעינת נתוניו לתוך טופס, עריכת שדות, שליחת

נתונים מעודכנים לשרת, שמירת העדכונים במסד.  
אובייקטים נחוצים: `user_data`, טופס GUI לעריכה, כפתור שמירה, פונקציית `update`.

### יצירת מיון חדש(אדמין בלבד)

מהות היכולת: הוספת מיון חדש למסד עם כותרת, תיאור ותאריך.  
אוסף יכולות נדרשות: מילוי טופס מיון, בחירת תאריך וכתובת תיאור, שליחת נתונים לשרת, יצירת מיון בטבלת `meyunim`.  
אובייקטים נחוצים: טופס מיון, `DatePicker`, כפתור יצירה, טבלת `meyunim`

### שיבוץ משתמשים למיון(אדמין בלבד)

מהות היכולת: שיוך משתמשים ספציפיים למיון שנוצר  
אוסף יכולות נדרשות: טעינת רשימת משתמשים, סימון המשתמשים הרלוונטיים, שליחת רשימת `user_ids` לשרת, יצירת רשומות בטבלת `assignments`.  
אובייקטים נחוצים: `multi-select GUI`, `assignments`, כפתור שיבוץ, `network_client`.

### צפייה בכל המיונים הפעילים(אדמין בלבד)

מהות היכולת: תצוגה של כלל המיונים הפעילים/עבריים במערכת  
אוסף יכולות נדרשות: בקשה מהשרת לקבלת רשימת מיונים, הצגה של כותרת, תאריך, כמות משתתפים, לחצן למחיקה או צפייה בשיבוצים.  
אובייקטים נחוצים: GUI כרטיסי מיון, טבלת `meyunim`, כפתור `delete/view`.

### צפייה בבקשות דחייה / ויתור שהגישו משתמשים(אדמין בלבד)

מהות היכולת: טיפול בבקשות פתוחות ממשתמשים וצפייה בבקשות קודמות.  
אוסף יכולות נדרשות: קבלת רשימת בקשות עם סטטוס "Pending", הצגת נתוני המשתמש, סוג הבקשה והסבר, אפשרות לאשר, לדחות, או לקבוע תאריך חדש.  
אובייקטים נחוצים: `meyunim_requests`, כפתורים `Approve/Reject`, `DateEntry`.

### תגובה להודעות ממשתמשים(אדמין בלבד)

מהות היכולת: שליחת תגובה אישית להודעות שהתקבלו  
אוסף יכולות נדרשות: תצוגת כל ההודעות עם אפשרות סינון, שדה כתיבה לתגובה, שליחת תגובה לשרת, עדכון שדה `response` במסד.  
אובייקטים נחוצים: `messages`, `TextBox`, כפתור שליחה.

### צפייה בסטטיסטיקות וגרפים(אדמין בלבד)

**מהות היכולת:** תצוגת נתונים גרפיים על פעילות המערכת.  
**אוסף יכולות נדרשות:** שליחה של בקשות קבלת נתונים לשרת, עיבוד הנתונים ל-`lists`, בניית גרפים עם `matplotlib`, הטמעתם בחלון ממשק האדמין.  
**אובייקטים נחוצים:** `usersDB`, `requests`, `matplotlib`, `GUI frame` לגרפים.

### מחיקת מיון(אדמין בלבד)

**מהות היכולת:** הסרת מיון מסד הנתונים וכל השיבוצים הקשורים אליו  
**אוסף יכולות נדרשות:** בחירת מיון מרשימה, שליחת בקשת `delete` לשרת, מחיקת המיון וכל ההקשרים ממסד הנתונים, רענון התצוגה.  
**אובייקטים נחוצים:** `meyunim`, `assignments`, `Delete`, כפתור.

## יכולות השרת

להלן פירוט מלא של כלל היכולות שהשרת במערכת תומך בהן. כל יכולת מייצגת סוג בקשה שהלקוח יכול לשלוח לשרת, וכוללת תיאור מהות הפעולה, היכולות הנדרשות למימושה, והרכיבים המעורבים (פונקציות, טבלאות מסד נתונים, ספריות וכו'). רשימה זו משקפת את כלל ממשק ה-API שהשרת חושף בפני הלקוח, ומהווה בסיס לתכנון ולבקרה של צד הלקוח במערכת.

### get\_public\_key

**מהות היכולת:** שליחת המפתח הציבורי של השרת ללקוח.  
**אוסף יכולות נדרשות:** קריאת קובץ `PEM`, המרה ל-`PEM string`, שליחה.  
**אובייקטים נחוצים:** `PUBLIC_KEY`

### login

**מהות היכולת:** בדיקת סיסמה ופרטי התחברות.  
**אוסף יכולות נדרשות:** בדיקת שם משתמש וסיסמה, שליפת מזהה וסוג המשתמש.  
**אובייקטים נחוצים:** `check_user_credentials`, `usersDB`.

### complete\_first\_login

**מהות היכולת:** עדכון ת"ז, טלפון, כתובת, גיל וסיסמה למשתמש חדש.  
**אוסף יכולות נדרשות:** אימות ת"ז מול מסד, עדכון רשומות בטבלת `usersDB`.  
**אובייקטים נחוצים:** `hash_id_number`, `hash_password`

### send\_code\_to\_user

**מהות היכולת:** שליחת קוד אימות זמני למייל.  
**אוסף יכולות נדרשות:** יצירת קוד, שמירה במסד, שליחת מייל.  
**אובייקטים נחוצים:** `generate_and_store_code`, `send_verification_email`

### verify\_code

**מהות היכולת:** אימות הקוד שהוזן על ידי המשתמש.  
**אוסף יכולות נדרשות:** שליפה ממסד ובדיקה לפי זמן.  
**אובייקטים נחוצים:** `verify_code`

## **request\_reset\_code**

מהות היכולת: שליחת קוד לאיפוס סיסמה למייל של המשתמש.  
אוסף יכולות נדרשות: שליפת מייל לפי שם משתמש, שליחת קוד.  
אובייקטים נחוצים: generate\_and\_store\_code, send\_verification\_email

## **reset\_password**

מהות היכולת: עדכון סיסמה חדשה לאחר אימות.  
אוסף יכולות נדרשות: בדיקת התאמת שם משתמש + מייל, שמירת סיסמה חדשה.  
אובייקטים נחוצים: reset\_user\_password

## **get\_user\_data**

מהות היכולת: שליפת נתוני מערכת של משתמש (דפ"ר, עברית וכו').  
אוסף יכולות נדרשות: שאילתת SELECT לפי user\_id.  
אובייקטים נחוצים: get\_user\_data, user\_data

## **get\_meyunim**

מהות היכולת: שליפת המיונים של המשתמש כולל סטטוס ובקשות אחרונות.  
אוסף יכולות נדרשות: שאילתות JOIN בין meyunim, meyunim\_assignments, meyunim\_requests.  
אובייקטים נחוצים: get\_user\_meyunim

## **submit\_request**

מהות היכולת: שמירת בקשת דחייה / ויתור במסד.  
אוסף יכולות נדרשות: הכנסת רשומה חדשה לטבלת meyunim\_requests.  
אובייקטים נחוצים: submit\_request

## **send\_message**

מהות היכולת: שליחת הודעה כללית לאדמין.  
אוסף יכולות נדרשות: הכנסת רשומה לטבלת messages.  
אובייקטים נחוצים: send\_message\_to\_admin

## **get\_messages**

מהות היכולת: שליפת ההודעות של המשתמש עצמו.  
אוסף יכולות נדרשות: שאילתת SELECT לפי sender\_id.  
אובייקטים נחוצים: get\_user\_messages

## **get\_my\_requests**

מהות היכולת: שליפת היסטוריית בקשות של המשתמש.  
אוסף יכולות נדרשות: JOINים על meyunim\_requests, meyunim, meyunim\_assignments.  
אובייקטים נחוצים: get\_requests\_by\_user

## **add\_user**

**מהות היכולת:** יצירת משתמש חדש ושליחת מייל עם סיסמה.  
**אוסף יכולות נדרשות:** בדיקה כפילות, הכנסת רשומה ל-usersDB, שליחת מייל.  
**אובייקטים נחוצים:** hash\_password, send\_new\_user\_email

### get\_all\_users

**מהות היכולת:** שליפת כל המשתמשים במערכת.  
**אוסף יכולות נדרשות:** שאילתת SELECT מ-usersDB.  
**אובייקטים נחוצים:** get\_all\_users

### load\_user\_data

**מהות היכולת:** שליפת נתוני משתמש מלאים לפי שם משתמש.  
**אוסף יכולות נדרשות:** SELECT כפול מ-usersDB ו-user\_data.  
**אובייקטים נחוצים:** connect\_db

### update\_user\_data

**מהות היכולת:** עדכון פרטי משתמש אישיים ונתוני מערכת.  
**אוסף יכולות נדרשות:** UPDATE בטבלאות usersDB ו-user\_data.  
**אובייקטים נחוצים:** connect\_db

### create\_meyun

**מהות היכולת:** יצירת מיון חדש עם כותרת, תיאור, תאריך ושיבוץ משתמשים.  
**אוסף יכולות נדרשות:** הכנסת מיון חדש, הכנסת שיבוצים.  
**אובייקטים נחוצים:** connect\_db, meyunim, meyunim\_assignments

### get\_all\_meyunim

**מהות היכולת:** שליפת כל המיונים + רשימת משובצים לכל אחד.  
**אוסף יכולות נדרשות:** SELECT + JOIN על כל הטבלאות הרלוונטיות.  
**אובייקטים נחוצים:** meyunim, meyunim\_assignments, usersDB

### delete\_meyun

**מהות היכולת:** מחיקת מיון קיים מכל הטבלאות.  
**אוסף יכולות נדרשות:** מחיקת שיבוצים, בקשות, מיון עצמו.  
**אובייקטים נחוצים:** delete\_meyun

### get\_all\_requests

**מהות היכולת:** שליפת כל הבקשות מכל המשתמשים.  
**אוסף יכולות נדרשות:** SELECT עם JOINים וטיפול בערכים חסרים.  
**אובייקטים נחוצים:** get\_all\_requests

### update\_request\_status

**מהות היכולת:** שינוי סטטוס של בקשה מסוימת.  
**אוסף יכולות נדרשות:** UPDATE לפי ID בטבלת meyunim\_requests.  
**אובייקטים נחוצים:** update\_request\_status

### **approve\_cancel\_request**

מהות היכולת: אישור בקשת ביטול כולל הסרת שיבוץ.  
אוסף יכולות נדרשות: SELECT לשיבוץ, מחיקה, עדכון סטטוס.  
אובייקטים נחוצים: connect\_db, meyunim\_assignments, meyunim\_requests

### **update\_date\_for\_request**

מהות היכולת: עדכון תאריך מיון עקב בקשת דחייה.  
אוסף יכולות נדרשות: SELECT ל-meyun\_id, UPDATE ל-meyunim.  
אובייקטים נחוצים: update\_date\_for\_request

### **get\_all\_messages**

מהות היכולת: שליפת כל ההודעות שהמשתמשים שלחו.  
אוסף יכולות נדרשות: JOIN בין messages ל-usersDB.  
אובייקטים נחוצים: get\_all\_messages

### **reply\_to\_message**

מהות היכולת: שמירת תגובת האדמין להודעה.  
אוסף יכולות נדרשות: UPDATE שדה response בטבלת messages.  
אובייקטים נחוצים: connect\_db

### **get\_age\_distribution**

מהות היכולת: שליפת גילאים לכל המשתמשים לצורך גרף.  
אוסף יכולות נדרשות: SELECT ו-filter של null/ריקים.  
אובייקטים נחוצים: usersDB

### **get\_request\_type\_stats**

מהות היכולת: ספירת בקשות לפי סוג (דחייה/ויתור).  
אוסף יכולות נדרשות: GROUP BY על טבלת meyunim\_requests.  
אובייקטים נחוצים: meyunim\_requests

### **get\_request\_status\_stats**

מהות היכולת: ספירת בקשות לפי סטטוס (Approved וכו').  
אוסף יכולות נדרשות: GROUP BY על שדה status.  
אובייקטים נחוצים: meyunim\_requests

### **get\_meyunim\_per\_month**

מהות היכולת: ספירת מיונים לפי חודש (גרף).  
אוסף יכולות נדרשות: SELECT עם strftime.  
אובייקטים נחוצים: meyunim

### **get\_messages\_per\_week**

**מהות היכולת:** ספירת הודעות לפי שבוע (גרף).  
**אוסף יכולות נדרשות:** SELECT עם strftime לפי timestamp.  
**אובייקטים נחוצים:** messages

### get\_user\_notifications

**מהות היכולת:** שליפת התראות למשתמש (מיון קרוב, תגובות, בקשות).  
**אוסף יכולות נדרשות:** SELECT מכמה טבלאות, UPDATE לדגלי notified\_user.  
**אובייקטים נחוצים:** meyunim, meyunim\_assignments, meyunim\_requests, messages

## מבנה וארכיטקטורה של הפרויקט

### תיאור הארכיטקטורה

#### תיאור מילולי של החומרה

שרת מרכזי (Server Machine):

מכונה זו אחראית על קבלת ועיבוד בקשות מכלל הלקוחות, ניהול הלוגיקה העסקית, אימות משתמשים, אבטחת התקשורת, והאינטראקציה הישירה עם מסד הנתונים (users.db המאוחסן מקומית על מכונה זו). בנוסף, השרת אחראי ליצירת קשר עם שרתי דואר חיצוניים לצורך שליחת הודעות דוא"ל למשתמשים, כגון קודי אימות ופרטי התחברות. מכונת השרת דורשת חיבור רשת יציב וזמינות גבוהה, וכן משאבי עיבוד וזיכרון מספקים להתמודדות עם חיבורים מרובים בו-זמנית.

מריץ את network server.py.

מכיל את כל מסד הנתונים (users.db) בקבצי SQLite ומבצע את האינטראקציה הישירה איתו.

מבצע את כלל הלוגיקה שקשורה למסד הנתונים: החל מהתחברות ועד לניהול משתמשים וניתוח נתונים וסטטיסטיקות דרך נתוני המסד.

אחראי על כל הליך שליחת המיילים (דרך send\_email.py) באמצעות חיבור SMTP ל-Gmail.

#### לקוחות (Client Machines):

מריצים את main.py.

כוללים ממשק גרפי (GUI) הבנוי באמצעות Tkinter + CustomTkinter.

מקיימים תקשורת עם השרת דרך מחלקת network\_client.py.

מצפינים את הנתונים שנשלחים באמצעות RSA (ומפענחים את התשובות עם AES).

יכולים לפעול בו זמנית במספר מחשבים.

#### קבצים נוספים:

validators.py – ולידציה של קלטים מהמשתמש.

ui\_elements.py – כפתורים וחלונות מותאמים.

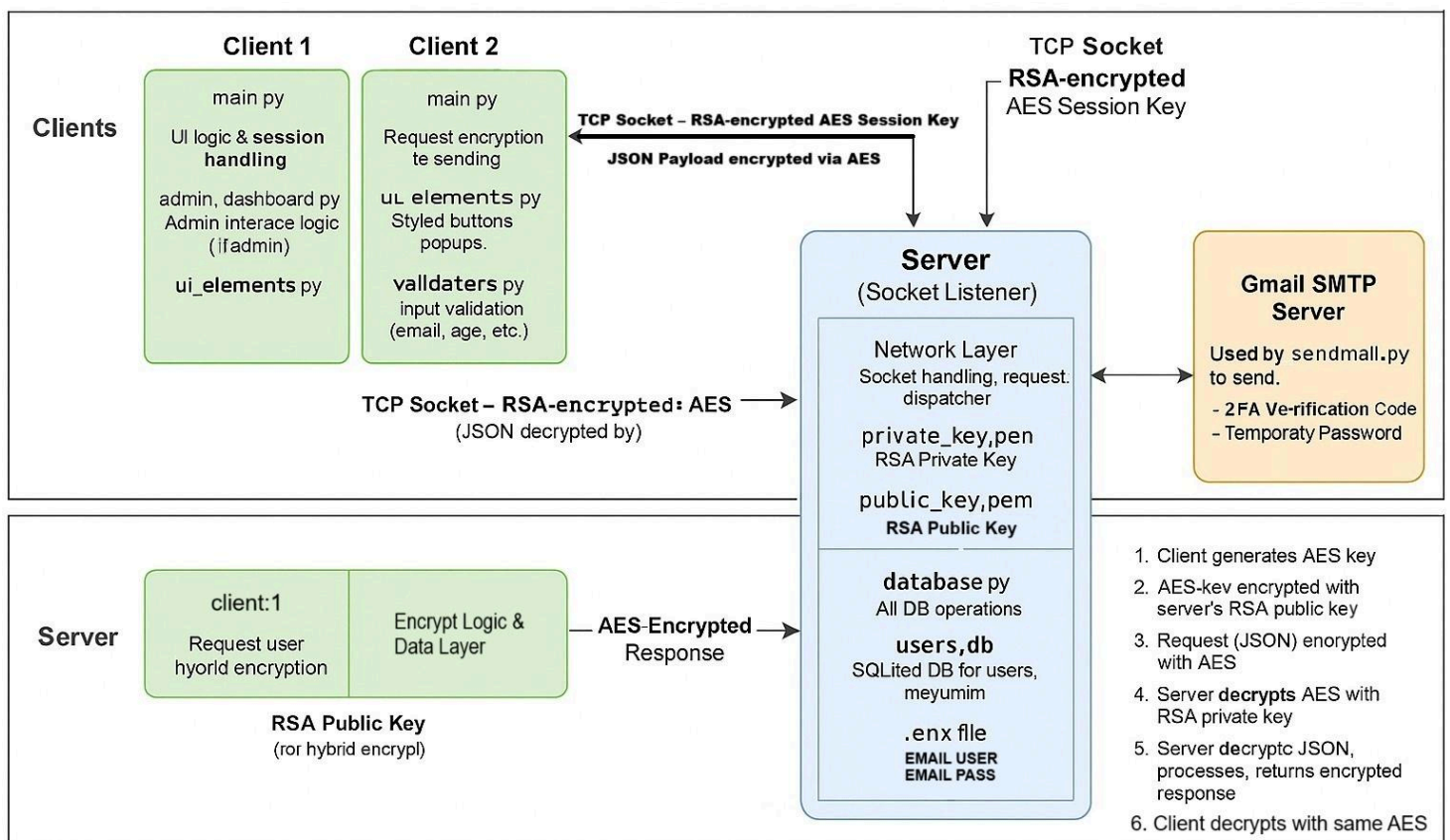
admin\_dashboard.py – רכיבי GUI ייחודיים לאדמין (ניהול משתמשים, גרפים, הודעות).

send\_email.py – שליחת מיילים לאימות וסיסמה.

database.py – כל שאילתות ה-SQL שמבוצעות בצד השרת בלבד.

## שרטוט גרפי של המודל

### IlayDF System Architecture Secure Client/'Server Model



## טכנולוגיות הפרויקט

### שפת תכנות

המערכת נכתבה בשפת Python – שפה דינמית, מונחית עצמים, המתאימה במיוחד לפיתוח מהיר של אפליקציות שולחניות, מערכות תקשורת וממשקים גרפיים. היא מציעה תחביר קריא וברור, אוסף עצום של ספריות מובנות, ותמיכה קהילתית רחבה. בזכות כך ניתן היה לפתח את כל המערכת – גם צד הלקוח, גם צד השרת וגם תקשורת השרת – בשפה אחת אחידה. Python היא שפה בין-פלטפורמית, מה שמבטיח שהמערכת תוכל לרוץ גם על מערכות הפעלה שונות.

## מערכת הפעלה

המערכת פותחה על מערכת Windows 11, אך היא תואמת גם ל-Linux ו-macOS, שכן כל הקוד מבוסס Python ואינו תלוי בפלטפורמה. אין שימוש בספריות בינאריות או דרייברים מערכתיים – הכל מופעל מתוך קוד פתוח. ספריות כמו socket, tkinter ו-sqlite3 כלולות כבר בהתקנה הרגילה של Python, ולכן קל מאוד להעביר ולהתקין את המערכת גם במחשבים אחרים.

## תקשורת

מערכת llayDF מבוססת על תקשורת **TCP socket** דו כיוונית, בין תוכנת הלקוח לתוכנת השרת. כל פעולה שהמשתמש מבצע – החל מהתחברות, דרך שליחת בקשות, ועד צפייה בנתונים – מתבצעת דרך שליחת הודעות מוצפנות לשרת, והחזרת תגובה מוצפנת חזרה.

שכבת התקשורת אחראית על:

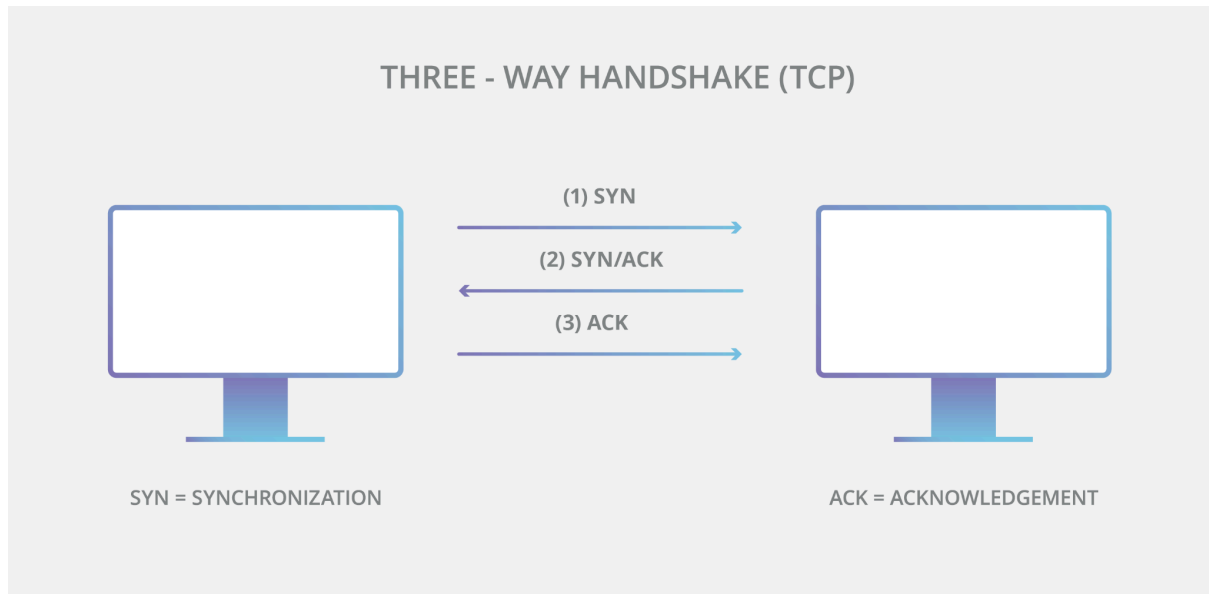
- יצירת החיבור בין הלקוח לשרת
- הגנה על המידע המועבר.
- ניהול חיבורים מרובים בו-זמנית.
- פירוק והרכבת נתונים מתאימים לעיבוד.

## מה זה TCP?

TCP (ראשי תיבות של **Transmission Control Protocol**) הוא פרוטוקול רשת שמספק תקשורת **אמינה, מסודרת, ומבוססת חיבורים** בין שני צדדים (במקרה זה: הלקוח והשרת). הוא אחראי לדברים הבאים:

- יצירת "חיבור" (Handshake) בין הצדדים.
- חלוקה של מידע לחבילות (Packets) ושליחה לפי הסדר.
- וידוא שכל חבילה התקבלה, ואם לא – שליחה מחדש.
- הרכבת כל החבילות בצד המקבל – לפי הסדר המדויק.

היתרון ב-TCP לעומת פרוטוקולים כמו UDP הוא **האחריות שלו על שלמות הנתונים והסדר**, דבר חיוני למערכת כמו פרויקט זה שמעבירה מידע רגיש.



### מה זה Socket?

Socket הוא ממשק תכנותי (API) שמאפשר לתוכנה לבצע תקשורת דרך הרשת. כל צד בתקשורת (שרת או לקוח) יוצר socket ומגדיר בו:

- כתובת IP
- מספר פורט (כמו ערוץ תקשורת)
- סוג הפרוטוקול (TCP)

ב-llayDF:

- השרת יושב ב-Socket במצב listen, ממתיין לחיבורים נכנסים.
- כל לקוח מתחבר לכתובת ה-IP והפורט של השרת, ויוצר תעלת תקשורת פרטית.

### מבנה הנתונים – JSON

הנתונים נשלחים בפורמט JavaScript Object Notation - **JSON** - פורמט טקסט קל, קריא, ונפוץ מאוד להעברת מידע בין שרת ללקוח. כל הודעה כוללת מילון עם שדות כמו:

```
{  
  "type": "login",  
  "username": "user123",  
  "password": "abc123"  
}
```

היתרונות של JSON:

- קריא על ידי בני אדם.
- נתמך כמעט בכל שפת תכנות.

## שליחת דואר אלקטרוני

במערכת llayDF קיימת יכולת לשליחת דוא"ל אוטומטי למשתמשים, שנועדה לתמוך באימות דו-שלבי (2FA) ובאיפוס סיסמה. השליחה מתבצעת דרך שרת ה-SMTP של Gmail באמצעות חיבור מאובטח בעזרת ספריית smtpplib של פייתון. גוף ההודעה מורכב באמצעות מודול email.mime, שמאפשר לבנות מיילים בפורמט תקני עם נושא, גוף טקסטואלי ומבנה ברור. כל מייל שנשלח מכיל פרטים מותאמים אישית: שם המשתמש, קוד אימות זמני או סיסמה אקראית, והוראות פעולה – והכל מנוסח בצורה מאובטחת וידידותית.

לצורך אבטחת הסיסמה של חשבון המייל השולח (במקרה זה- המייל היעודי שנוצר עבור הפרויקט), נעשה שימוש בקובץ env (משתני סביבה), אשר נטען על-ידי הספרייה python-dotenv. כך, פרטי ההתחברות לחשבון Gmail אינם מופיעים בקוד עצמו, מה שמגן עליהם מפני חשיפה. קובץ send\_email.py מרכז את כל הלוגיקה לשליחת הודעות, ומטפל בו כל מקרה כשל – החל מבעיות ברשת, דרך סיסמה לא תקינה, ועד למניעת שליחה במקרה של שגיאה פנימית. כל שליחת דוא"ל משולבת בתהליך הלוגי של השרת. יש שימוש בשליחת הדואר האלקטרוני דרך המערכת בשלושה אופנים שונים: שליחת פרטים ראשוניים, שליחת קוד אימות להתחברות, שליחת קוד אימות למען איפוס סיסמה.

## תהליכונים (Threads)

תהליכון (באנגלית: **Thread**) הוא יחידת ביצוע עצמאית בתוך תהליך (Process). כל תהליך בפעולה – לדוגמה, הרצת תוכנה – יכול להכיל מספר תהליכונים, שכל אחד מהם פועל במקביל (או לכאורה במקביל) עם שאר התהליכונים של אותו תהליך. כולם משתפים את אותו זיכרון ואת אותם משאבים, אך כל תהליכון מבצע משימה משלו – מה שמאפשר ריבוי משימות בתוך אותה תוכנה. לדוגמה, תהליכון אחד יכול להאזין לחיבור רשת, תהליכון שני יכול לטפל בבקשת משתמש, ותהליכון שלישי לשלוח מייל – והכול בו זמנית, מבלי להפריע זה לזה.

תהליכונים שימושיים במיוחד כאשר יש פעולות שדורשות זמן, כמו תקשורת רשת, גישה לקבצים, או המתנה לתשובת שרת. אם היינו מבצעים הכול בתהליכון אחד בלבד – כל פעולה "חוסמת" הייתה עוצרת את כל התוכנית עד להשלמתה. בעזרת threading אפשר להמשיך להפעיל את הממשק, לעבד מידע נוסף, או לקבל חיבורים חדשים תוך כדי. במערכת llayDF זה בא לידי ביטוי בכך שכל לקוח שמתחבר לשרת מקבל תהליכון נפרד שמטפל בו, ובצד הלקוח – כל פעולה של תקשורת (כמו login או שליחת בקשה) נשלחת בתהליכון עצמאי כדי שה-GUI לא יתקע.

בשרת המערכת, תהליכונים (Threads) משמשים כדי לאפשר **ניהול מקבילי של מספר לקוחות בו זמנית**. כאשר לקוח מתחבר לשרת דרך socket, השרת יוצר תהליכון נפרד שמטפל אך ורק בו – כך ששאר הלקוחות אינם נתקעים או מושפעים מתגובה איטית של לקוח אחר. כל חיבור נכנס נרשם בלוג השרת ומוקצה לו thread ייעודי שמאזין לבקשות, מפענח את ההודעות, ומבצע את הפעולה הרלוונטית (לדוגמה: התחברות, שליפת מיונים, שליחת הודעה). גישה זו מבטיחה שהשרת מסוגל לשרת מספר לקוחות במקביל בצורה רציפה, אמינה ונטולת עיכובים.

## ממשק המשתמש (GUI):

ממשק המשתמש במערכת llayDF נבנה בעזרת שימוש בספריות Tkinter ו-CustomTkinter, שמספקות כלים נוחים לבניית יישומי GUI גרפיים בסביבת פייתון. בעוד ש-Tkinter היא הספרייה המובנית בפייתון ומאפשרת יצירת חלונות, כפתורים, שדות קלט, תיבות הודעה ותפריטים בסיסיים,

CustomTkinter מוסיפה שכבת עיצוב מודרנית המשלבת צבעים אחידים, פינות מעוגלות, תמיכה במצבים כהים ובהירים, אנימציות בסיסיות, מראה נקי יותר ועוד. תכנון הממשק התבצע תוך דגש על חוויית משתמש ברורה וידידותית, הן למלש"ב והן למנהל מטעם הצבא, מתוך תפיסה אישית שלי כי מערכת המספקת חווית משתמש שכזו תהפוך את כלל הליך השימוש במערכת ליעיל יותר, ותעניק תחושה מקצועית, אמינה ומאובטחת.

עבור המשתמש הרגיל, ה-GUI כולל מסכים רבים. כל פעולה נפתחת בחלון ברור, עם הודעות הצלחה או שגיאה, ולידציה מידית על הקלט (למשל עבור ת"ז, גיל, מייל וכו'). אצל האדמין, הממשק כולל תפריט צד עם ניווט בין תתי-מסכים: ניהול משתמשים, ניתוח נתונים, ניהול מיונים, מערכת הודעות, ודשבורד גרפי שמציג גרפים דינמיים בעזרת matplotlib. לכל אובייקט ויזואלי נבנה רכיב מותאם (למשל כפתור) תוך שמירה על עיצוב אחיד. בנוסף, נבנו רכיבי `popup` (חלונות קופצים) בהתאמה אישית – לדוגמה להזנת תאריך חדש או תגובה להודעה – שמעניקים שימוש קל וברור יותר.

## מסד הנתונים – SQLite:

מערכת IlayDF עושה שימוש במסד נתונים מסוג **SQLite**, שהוא מסד נתונים רלציוני (מבוסס טבלאות וקשרים ביניהן) אך שונה ממסדי נתונים גדולים כמו MySQL או PostgreSQL בכך שהוא **"קל משקל", עצמאי, וללא צורך בשרת**. כלומר – כל המסד נשמר בתוך **קובץ מקומי בודד** (`users.db`), ואין צורך בהתקנה של שירות מסדי נתונים חיצוני או בתקשורת נפרדת. זהו פתרון אידיאלי לפרויקטים מקומיים, אפליקציות דסקטופ, מערכות ניסוי ופרויקטים חינוכיים, בדיוק כמו בפרויקט IlayDF.

SQLite מבוסס על שפת SQL מלאה – עם תמיכה ב-`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `JOIN`, `GROUP BY`, וכו'. הוא תומך באינדקסים, מפתחות זרים (`foreign keys`), שלמות טבלאות, ומאפשר גישה מקבילה של קריאה ממספר תהליכים. המנוע עצמו מובנה בפיתון באמצעות הספרייה `sqlite3`, כך שאין צורך להוריד תוספים – פשוט לייבא את המודול ולהתחיל להשתמש בו. SQLite גם מהיר מאוד לעבודה מקומית, בטוח מבחינת גישה (הוא מקבל רק פקודות מהשרת שלנו), ומתאים לסביבות שבהן אין הרבה גישות בו-זמנית לכתיבה.

במערכת IlayDF יש שימוש רחב ומובנה ב-SQLite כבסיס לכל המידע: החל מהתחברות המשתמש, דרך נתוני מערכת אישיים, ועד למיונים, הודעות ובקשות.

המסד כולל שבע טבלאות עיקריות:

- `usersDB` – טבלת המשתמשים הכוללת שם משתמש, סיסמה מגובבת, ת"ז, אימייל, סוג הרשאה (`is_admin`) וסטטוס התחברות ראשונית.
- `user_data` – נתוני מערכת אישיים כמו דפ"ר, פרופיל, עברית, ראיון ו-100 יום.
- `auth_codes` – קודי אימות זמניים לשימוש ב-2FA.
- `meyunim` – טבלת המיונים שיצר האדמין.
- `meyunim_assignments` – שיוך משתמשים למיונים.
- `meyunim_requests` – בקשות המשתמשים לדחייה או ביטול מיון.
- `messages` – הודעות שהמשתמשים שלחו לאדמין, כולל תגובות.

כל פעולה שהשרת מבצע (התחברות, עדכון, שליפה, מחיקה) מתבצעת דרך שאילתות על הטבלאות הללו. השרת עוטף כל גישה במסגרות בטוחות (try/except), סוגר את החיבור למסד אחרי כל פעולה, ומונע גישה במקביל שיכולה להפריע ליציבות.

## זרימת המידע במערכת

טבלה המציגה בקצרה את זרימת המידע במערכת עבור כל יכולת

| היכולת                                    | זרימת מידע במערכת                                                                                               |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| קבלת שם משתמש וסיסמה ראשונית למייל        | האדמין יוצר משתמש -> נשלחת בקשה לשרת -> השרת שומר את המשתמש במסד וגובה את הסיסמה -> נשלח מייל עם הסיסמה למשתמש. |
| אימות התחברות ראשונית לפי מספר תעודת זהות | המשתמש מזין ת"ז -> הלקוח שולח לשרת לאימות -> אם תקין נחשפים שדות המשך.                                          |
| איפוס סיסמה                               | המשתמש מזין שם משתמש -> נשלחת בקשה לשרת -> נשלח קוד למייל -> לאחר אימות נשלחת סיסמה חדשה לשרת ונשמרת במסד.      |
| התחברות למערכת                            | המשתמש מזין שם משתמש -> נשלחת בקשה לשרת -> נשלח קוד למייל -> לאחר אימות נשלחת סיסמה חדשה לשרת ונשמרת במסד.      |
| תפריט ראשי למשתמש                         | לאחר התחברות, מוצגים כפתורים ונתוני התראה -> כל כפתור מוביל לבקשה מתאימה לשרת.                                  |
| הצגת נתונים אישיים                        | נשלחת בקשה get_user_data -> השרת מחזיר JSON עם נתוני מערכת -> הנתונים מוצגים למשתמש.                            |
| הצגת מיונים                               | נשלחת בקשה get_user_meyunim -> מתקבלים כל המיונים עם סטטוס ובקשות קודמות -> מוצגים כרטיסים עם פעולות.           |
| בקשת דחייה למיון                          | המשתמש לוחץ "בקש דחייה" -> נפתח popup להזנת סיבה -> נשלחת בקשת submit_request עם סוג "delay".                   |

|                           |                                                                                                |
|---------------------------|------------------------------------------------------------------------------------------------|
| בקשת ויתור על מיון        | כמו הקודם, אך סוג הבקשה הוא "cancel".                                                          |
| שליחת הודעה ישירה לצבא    | המשתמש מזין נושא ותוכן <- נשלחת בקשה לשרת <- נשמר במסד ההודעות.                                |
| צפייה בתגובות מהאדמין     | נשלחת בקשה get_messages <- מתקבלות ההודעות מהשרת ומוצגות לפי תגובה/ללא תגובה.                  |
| ניתוק מהמערכת             | לחיצה על Logout או תום 10 דקות <- איפוס משתנים וסגירת session <- מעבר למסך התחברות.            |
| הוספת משתמש חדש           | האדמין ממלא טופס <- נשלחת בקשה לשרת <- השרת מוסיף למסד ושולח מייל עם סיסמה זמנית.              |
| צפייה ברשימת המשתמשים     | נשלחת בקשה get_all_users <- מתקבלות רשומות <- מוצגות בטבלה.                                    |
| עריכת נתוני משתמש         | האדמין בוחר משתמש <- נשלחת בקשה load_user_data <- מתבצעת עריכה <- נשלחת בקשה update_user_data. |
| יצירת מיון חדש            | האדמין ממלא טופס מיון <- בוחר תאריך <- נשלחת בקשה create_meyun <- נשמר במסד.                   |
| שיבוץ משתמשים למיון       | האדמין בוחר משתמשים למיון חדש <- נשלחת רשימה לשרת יחד עם מזהה מיון.                            |
| צפייה בכל המיונים הפעילים | נשלחת בקשה get_all_meyunim <- מוצגים כל המיונים עם פרטי שיבוץ.                                 |
| מחיקת מיון                | לחיצה על מחיקה <- נשלחת בקשה delete_meyun <- השרת מוחק את כל השיבוצים והבקשות הקשורים.         |

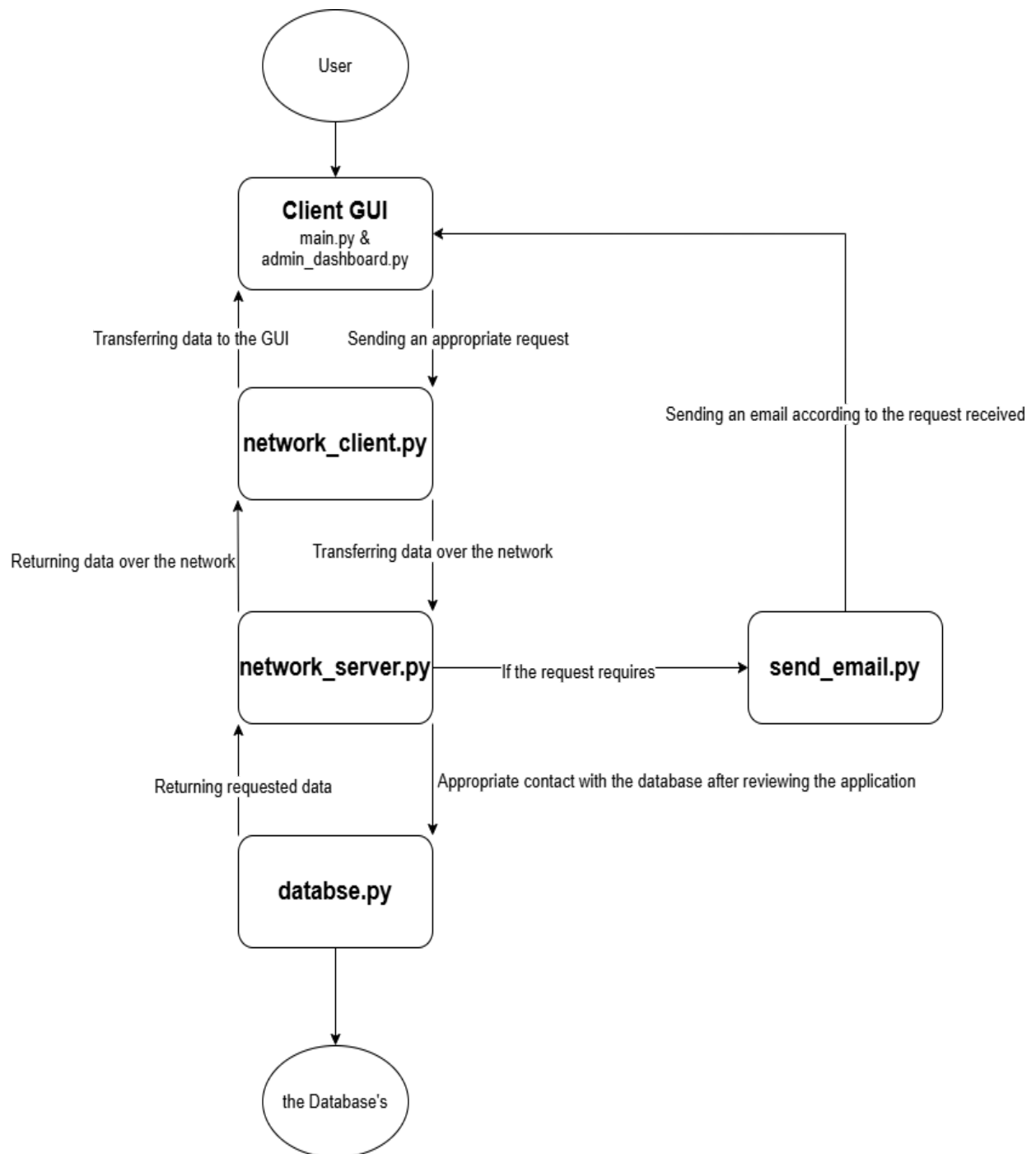
|                            |                                                                                                                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| צפייה בבקשות דחייה / ויתור | נשלחת בקשה get_all_requests -> מוצגות בקשות בפילטרים לפי סוג וסטטוס.                                                                                                                              |
| תגובה להודעות ממשתמשים     | נשלחת בקשה get_all_messages -> מוצגות הודעות עם אפשרות תגובה -> התגובה נשמרת במסד.                                                                                                                |
| צפייה בסטטיסטיקות וגרפים   | נשלחות בקשות get_age_distribution וכו' -> מתקבלות תוצאות -> מוצגות בגרפים עם matplotlib.                                                                                                          |
| get_public_key             | הלקוח שולח בקשה לא מוצפנת מסוג "get_public_key" -> השרת טוען את המפתח הציבורי מקובץ PEM -> השרת מחזיר את המפתח כ-string -> הלקוח שומר בזיכרון כ-self.public_key.                                  |
| login                      | המשתמש מזין שם משתמש וסיסמה -> נשלחת בקשה מוצפנת עם "type=login" -> השרת בודק מול מסד הנתונים -> אם הצליח, השרת מחזיר תשובת "ok" עם user_id, email ו-is_admin -> הלקוח שומר נתונים להמשך התחברות. |
| complete_first_login       | המשתמש מזין ת"ז, טלפון, כתובת, גיל וסיסמה חדשה -> נשלחת בקשה מוצפנת לשרת -> השרת מאמת ת"ז מול מסד -> אם תקין, מעדכן את שאר השדות בטבלת usersDB -> השרת מחזיר הצלחה -> הלקוח חוזר למסך התחברות.    |
| send_code_to_user          | לאחר התחברות בסיסית, הלקוח שולח בקשה עם user_id -> השרת מייצר קוד זמני, שומר אותו במסד ושולח למייל של המשתמש -> השרת מחזיר סטטוס הצלחה/שגיאה -> הלקוח מציג למשתמש תיבת קוד.                       |
| verify_code                | המשתמש מזין קוד שקיבל -> הלקוח שולח את הקוד עם user_id -> השרת מאמת מול auth_codes בטווח של 10 דקות -> מחזיר אם הקוד תקף או לא -> הלקוח ממשיך לתפריט או מציג שגיאה.                               |
| request_reset_code         | המשתמש מבקש לאפס סיסמה -> הלקוח שולח שם משתמש -> השרת מאתר את המייל במסד, מייצר קוד חדש ושולח למייל -> מחזיר ללקוח את user_id והמייל לצורך המשך תהליך.                                            |

|                 |                                                                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reset_password  | לאחר אימות, הלקוח שולח שם משתמש, מייל וסיסמה חדשה<br>-> השרת מאמת את הנתונים -> אם תקין, מגבה את הסיסמה ושומר במסד -> מחזיר סטטוס -> הלקוח מעדכן את המשתמש ופותח מסך התחברות. |
| get_user_data   | לאחר התחברות, הלקוח שולח בקשה עם user_id -> השרת שולף נתונים מטבלת user_data (דפ"ר, פרופיל, עברית וכו') -> מחזיר כ-JSON -> הלקוח מציג בתצוגה גרפית.                           |
| get_meyunim     | הלקוח שולח בקשה עם user_id -> השרת מצרף את כל השיבוצים והבקשות האחרונות -> מחזיר רשימה של כרטיסי מיון עם סטטוס ועדכון אחרון -> הלקוח מציג כל מיון ככרטיס פעולה.               |
| submit_request  | המשתמש פותח popup וממלא הודעה -> נשלחת בקשה לשרת עם סוג ("cancel"/"delay") והסבר -> השרת מאחסן בטבלת meyunim_requests ומחזיר סטטוס -> הלקוח מציג חיווי ו/או רענון.            |
| send_message    | המשתמש שולח נושא ותוכן -> נשלחת בקשה לשרת עם הנתונים -> השרת מוסיף רשומה חדשה בטבלת messages -> מחזיר סטטוס -> הלקוח מעדכן הודעת הצלחה.                                       |
| get_messages    | הלקוח שולח בקשה עם user_id -> השרת שולף את כל ההודעות מהמשתמש כולל תגובות -> מחזיר JSON -> הלקוח מציג ברשימה ממוסגרת לפי תאריך ומצב תגובה.                                    |
| get_my_requests | הלקוח שולח בקשה עם user_id -> השרת מבצע JOIN על השיבוצים, המיונים והבקשות -> מחזיר את היסטוריית הבקשות -> הלקוח מציג את כולן לפי סדר כרונולוגי.                               |
| add_user        | האדמין ממלא טופס -> נשלחת בקשה לשרת עם הנתונים -> השרת בודק כפילות, מגבה סיסמה ושומר במסד -> שולח מייל עם הסיסמה למשתמש -> מחזיר הודעת הצלחה -> הלקוח מרענן את הרשימה.        |
| get_all_users   | הלקוח שולח בקשה -> השרת שולף את כל המשתמשים מהמסד -> מחזיר רשימת tuples עם פרטים -> הלקוח מציג                                                                                |

|                           |                                                                                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| טבלה או קומבובוקס לבחירה. |                                                                                                                                                            |
| load_user_data            | הלקוח שולח שם משתמש -> השרת שולף גם מטבלת usersDB וגם מ-user_data -> מחזיר את הכל במבנה JSON -> מוצג בטופס עריכה בצד האדמין.                               |
| update_user_data          | האדמין עורך את הנתונים ושולח בקשה -> השרת מעדכן גם את usersDB וגם את user_data לפי הצורך (UPDATE או INSERT) -> מחזיר סטטוס -> הלקוח מציג חיווי הצלחה.      |
| create_meyun              | האדמין ממלא טופס מיון ובוחר משתמשים -> נשלחת בקשה עם כל הנתונים -> השרת יוצר מיון חדש ומוסיף שיבוצים לפי user_ids -> מחזיר סטטוס -> הלקוח מרענן את התצוגה. |
| get_all_meyunim           | נשלחת בקשה -> השרת שולף את כל המיונים כולל המשתמשים שובצו -> מחזיר רשימה מסודרת -> הלקוח מציג כל מיון עם לחצן מחיקה.                                       |
| delete_meyun              | האדמין לוחץ Delete -> נשלחת בקשה עם מזהה מיון -> השרת מוחק את המיון, כל השיבוצים והבקשות הקשורות -> מחזיר סטטוס -> הלקוח מסיר מהתצוגה.                     |
| get_all_requests          | הלקוח שולח בקשה -> השרת שולף את כל הבקשות כולל מידע על המשתמשים והמיונים -> מחזיר JSON -> מוצג בממשק אדמין עם פילטרים.                                     |
| update_request_status     | האדמין לוחץ Reject -> נשלחת בקשה עם request_id וסטטוס חדש -> השרת מעדכן בטבלה -> מחזיר סטטוס -> הלקוח מסיר את הבקשה מהתצוגה.                               |
| approve_cancel_request    | האדמין לוחץ Approve -> נשלחת בקשה עם request_id -> השרת מאתר את השיבוצ, מוחק אותו ומעדכן סטטוס -> מחזיר סטטוס -> הלקוח מציג עדכון.                         |
| update_date_for_request   | האדמין בוחר תאריך חדש ל-delay -> נשלחת בקשה עם request_id ותאריך חדש -> השרת מאתר את המיון ומעדכן                                                          |

|                                                                                                                             |                          |
|-----------------------------------------------------------------------------------------------------------------------------|--------------------------|
| תאריך + סטטוס -> מחזיר סטטוס -> הלקוח מרענן.                                                                                |                          |
| נשלחת בקשה לשרת -> השרת מבצע JOIN לטבלת messages ו-usersDB -> מחזיר כל ההודעות -> מוצגות לאדמין עם כפתור תגובה.             | get_all_messages         |
| האדמין כותב תגובה ולוחץ Send -> נשלחת בקשה עם תוכן התגובה והמזהה -> השרת מעדכן את השדה בטבלה -> מחזיר סטטוס -> הלקוח מרענן. | reply_to_message         |
| נשלחת בקשה -> השרת שולף גילאים מטבלת usersDB ומסנן ריקים -> מחזיר רשימה -> הלקוח יוצר היסטוגרמה ב-matplotlib.               | get_age_distribution     |
| בקשה נשלחת -> השרת מבצע GROUP BY לפי סוג -> מחזיר רשימה [(delay, x), (cancel, y)] -> נבנה גרף עמודות.                       | get_request_type_stats   |
| בדומה לקודם, GROUP BY לפי Approved/Pending/Rejected -> מחזיר תוצאה לגרף.                                                    | get_request_status_stats |
| בקשה נשלחת -> השרת מפעיל strftime על תאריכי המיונים ומחשב ספירה חודשית -> מחזיר תוצאה -> מופיע כגרף קו.                     | get_meyunim_per_month    |
| נשלחת בקשה -> השרת מבצע GROUP BY לפי שבוע מה-timestamp של ההודעות -> מחזיר תוצאה -> מוצג כגרף.                              | get_messages_per_week    |

גרף המציג באופן כללי את זרימת המידע במערכת



## אלגוריתמים מרכזיים בפרויקט

### הצפנה היברידית בפרויקט

#### ניסוח הבעיה

כאשר משתמש שולח נתונים רגישים דרך האינטרנט או אפילו ברשת מקומית (TCP Socket), קיימת סכנה ממשית שמישהו "יזין" לנתונים. במערכת IlayDF מדובר בפרטים רגישים מאוד: סיסמאות, מספרי זהות, בקשות אישיות למיזונים, ואפילו אימות דו-שלבי.

אך ישנה בעיה! מצד אחד, צריך לוודא שהמידע **לא נחשף** במהלך הדרך. מצד שני, אנחנו עובדים עם תקשורת **Raw Socket** ולא פרוטוקול מוצפן כמו HTTPS. כלומר, אין לנו מנגנון אבטחה אוטומטי. לכן, נצטרך לפתח מנגנון הצפנה בעבור המערכת.

אם נצפין את הנתונים – נדרשת הצפנה שתהיה גם חזקה, וגם מהירה.

אבל לכל שיטה יתרון וחסרון:

**RSA (הצפנה אסימטרית)** מאוד בטוחה, אך איטית, ולא מתאימה להודעות גדולות.  
**AES (הצפנה סימטרית)** מהירה מאוד, אבל לא ניתן להשתמש בה בלי לשתף מראש מפתח בין הצדדים – וזה בדיוק מה שאנחנו מנסים להימנע ממנו.

#### אלגוריתמים קיימים לפתרון

RSA נועד להצפין מפתחות, לא מידע בנפח גבוה.  
AES מצוין להצפנה של זרמים גדולים – אם יש לך מפתח מראש.  
TLS (כמו HTTPS) נותן פתרון מלא, אבל דורש שכבת תקשורת שלמה, תעודות, ואינו משתלב בקלות בסביבת סוקט פשוטה.  
לכן הפתרון המקובל בעולם, שמצאתי הכי מתאים לפרויקט שלי הוא **הצפנה היברידית**.

מהו הפתרון ההיברידי?

1. בצד הלקוח נוצר מפתח AES חדש לגמרי.
2. הלקוח מצפין את המפתח הזה באמצעות המפתח הציבורי של השרת (RSA).
3. ההודעה עצמה – כלומר הבקשה – מוצפנת באמצעות המפתח AES.
4. המפתח המוצפן ב-RSA מצורף לתחילת הבקשה.
5. השרת, שמחזיק את המפתח הפרטי (RSA private key), מפענח את מפתח ה-AES.
6. באמצעות אותו מפתח AES הוא מפענח את ההודעה עצמה.

זזהו – מעתה ואילך התקשורת עם השרת (לאותו session) יכולה להשתמש ב-AES בלבד.

### מדוע זה הפתרון שבחרתי?

כל פתרון אחר פשוט נפל באחד התנאים:

- RSA בלבד – היה מצריך לחתוך את הבקשה לגושים קטנים מאוד (למשל פחות מ-245 בתים), והיה איטי מאוד. בתחילת העבודה עבדתי ככה, והכל עבד כראוי, אך כאשר נפח התקשורת או מספר המשתמשים עלה נוצרו עבירות רבות.
- AES בלבד – בלתי אפשרי, כי איך אחליף את המפתח אם אין חיבור בטוח?
- TLS/SSL – לא מתאים. המערכת רצה כ-TCP socket פשוט. חיבור SSL היה דורש תשתית אחרת לגמרי, עם תעודות ואימותים. בנוסף, שליטתי בשיטה זו מוגבלת.

הצפנה היברידית מספקת גם אבטחה ברמה גבוהה, גם ביצועים טובים, וגם **עצמאות מלאה. בנוסף, לא הכרתי את השיטה הזו עד לתחילת העבודה על הפרויקט, וחשבתי שזה יכול להיות מרכיב למידה עצמי מעניין מאוד, שמציג שיטת עבודה המקובלת בעולם האמיתי.**

קשיים שנתקלתי בהם במהלך המימוש

#### 1. אורך מפתח RSA מוגבל

- לא ניתן להצפין יותר מ-245 בתים (ב-RSA 2048 ביט).
- פתרתי ע"י הצפנת המפתח בלבד, לא כל ההודעה.

#### 2. התמודדות עם bytes ו-str

- JSON לא תומך ב-bytes, אז נדרש קידוד ל-base64.
- המפתח המוצפן, ה-iv, וה-ciphertext עוברים קידוד לפני שליחה.

#### 3. ניהול IV לכל בקשה

- השתמשתי ב-IV אקראי לכל בקשה – מה שמונע התקפות של שחזור הודעות.
- ה-IV נשלח יחד עם ההודעה.

#### 4. פענוח שגוי ו-Padding Errors

- אם צד אחד טועה ב-IV או מפתח – הפענוח נכשל.
- נדרשה טיפול שגיאות מדויקת + try/except בכל שלב קריפטוגרפי.

#### 5. שימור המפתח AES בצד השרת

- השרת לא יכול פשוט "לזכור" את המפתח AES – כי אין sessions גלויים.
- פתרתי ע"י צירוף aes\_key\_encrypted עם כל בקשה, במקום לשמור מצב.
- לא הכי יעיל – אבל הרבה יותר בטוח ופשוט ליישום.

## סיכום מימוש מקורי בפרויקט

במקום לייצר קשר מוצפן רציף (persistent session), כל בקשה ב-IlayDF היא **חבילה מוצפנת עצמאית (stateless)**.

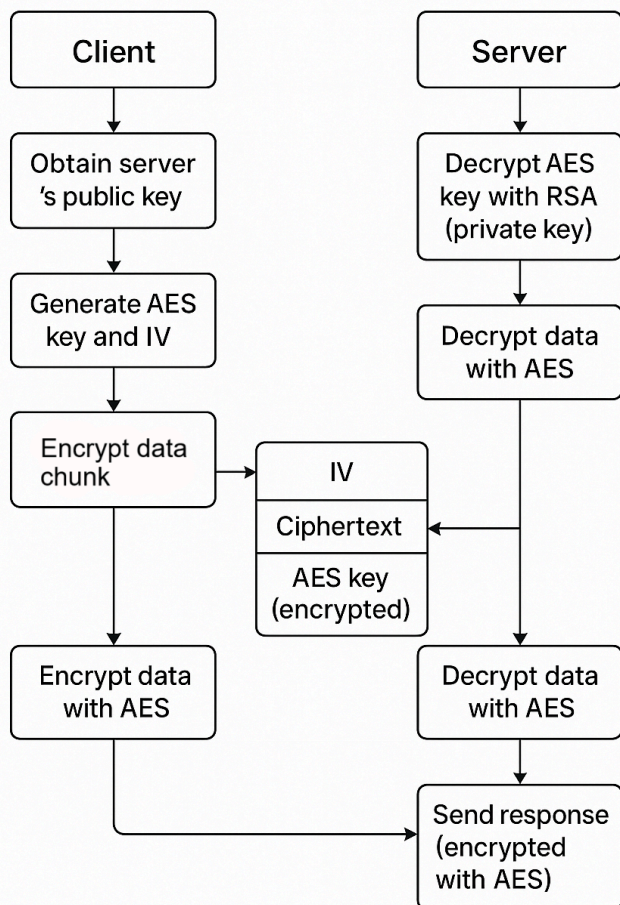
כלומר, גם אם מישהו מאזין לבקשה אחת, הוא לא יכול להבין בקשה אחרת – כי ה-IV משתנה, המפתח משתנה, והכל מוצפן מחדש.

הוספת היכולת להצפין גם **את תשובת השרת** הופכת את המערכת ל-end-to-end encrypted.

לסיכום, האלגוריתם ההיברידי בפרויקט מייצג פתרון בטוח, אלגנטי ויעיל – שמתאים בדיוק למערכת בגודל הזה.

תרשים קצר לנוחות:

## Hybrid Encryption



## Hybrid Encryption

## מערכת בקשות דינמית במערכת

### ניסוח הבעיה

במערכת גיוס כמו llayDF, ישנם מיונים שנקבעים מראש לכל משתמש. אך המציאות משתנה: ייתכן שהמשתמש אינו יכול להגיע, או שאינו מעוניין להשתתף במיון מסוים.

המערכת נדרשת לאפשר למשתמש:

- לבקש דחיית מיון (delay) עם תיאור / הסבר אישי.
- לבקש לבטל מיון (cancel) – כלומר לוותר עליו.
- לקבל אישור או דחייה מהאדמין.
- לקבל הודעה על השינוי.

ובמקביל:

- האדמין צריך לנהל בקשות באופן ממוקד ויעיל, עם מידע ברור על כל בקשה.
- אסור לאפשר למשתמש להגיש אינספור בקשות זהות על מנת "לזרז" את ההליך.
- על כל מיון של משתמש יכולה להיות רק בקשה אחת פעילה.

הבעיה האלגוריתמית היא:

כיצד לנהל תהליך מבוקר, מבוסס סטטוס, בין משתמש לאדמין, בצורה אסינכרונית, על מיון נתון.

### תיאור אלגוריתמים קיימים לפתרון בעיה כזו

מערכות בקשות כאלה קיימות בעולם במגוון תחומים (מערכות Helpdesk, אישור תורים, קריאות שירות). לרוב מבוססות על המושגים הבאים לפי המחקר שביצעת:

- **State Machine (מכונת מצבים)** – כל בקשה יכולה להיות בסטטוס אחד מתוך קבוצה סופית (Pending / Approved / Rejected).
- **Workflow Engine** – מנגנון שמגדיר מי רשאי לבצע פעולה בכל שלב.
- **Database Versioning / History Log** – כדי לעקוב אחר שינויים.

### סקירת הפתרון הנבחר עבורי במערכת

בפרויקט נבחר פתרון פשוט, יעיל ונגיש ליישום, המבוסס על טבלת meyunim\_requests במסד.

כל בקשה כוללת:

- מזהה שיבוץ (assignment\_id)
- סוג (delay / cancel)
- תוכן / הסבר
- סטטוס: Pending / Approved / Rejected
- תאריך יצירה
- נתוני קונטקסט (שם משתמש, שם מיון, תאריך מיון)

## מדוע בחרתי בפתרון זה

לאחר מחקר אינטרנטי גיליתי כי זו תהיה הדרך הפשוטה ביותר לבצע את ההליך. ביצוע ההליך עם שיטות אחרות הצריך ידע שלא היה לי, ומפאת זמן לא יכולתי ללמוד.

## קשיים ודרכי התמודדות

כפילויות בבקשות:

- נפתר על-ידי בדיקה בטבלת `meyunim_requests` – לפני הוספה חדשה, נבדק האם קיימת בקשה פתוחה לאותו `assignment_id`.

רענון תצוגת בקשות לאחר אישור/דחייה:

- על כל פעולה אדמין נדרשת קריאה מחדש מהשרת כדי להימנע מצפייה בנתונים מיושנים.

שמירה על עקביות בין הטבלאות:

- אישור ביטול מוחק שיבוץ, לכן חובה לשמור זאת אטומית עם עדכון הבקשה – אחרת תיווצר אי הלימה (המשתמש לא יראה שהבקשה אושרה כי הוא כבר לא משובץ למיון, וזהו מצב שלא רציתי שיקרה).

ממשק גרפי עם הרבה פרטים באדמין יוצר עומס בעין:

- שילוב של תוויות, כפתורים, תאריכים, צבעים, והצגת פרטים רלוונטיים בלבד. נפתר בעזרת פילטרים חכמים (לפי סוג וסטטוס), תוויות צבע, והצגת "אין בקשות" במידת הצורך.

## לסיכום

מערכת הבקשות הדינמית בפרויקט מהווה מודל מצוין לניהול זרימת עבודה אמין, פשוט, ואינטואיטיבי. היא מאפשרת אינטראקציה מלאה בין משתמש לאדמין – תוך שמירה על בקרה, שקיפות, והיסטוריה.

הפתרון מבוסס על אלגוריתם סטנדרטי של `stateful requests` – אך הותאם למגבלות מערכת קטנה, תוך שמירה על פשטות, אבטחה, והתאמה מלאה ל-`SQLite`.

## ולידציה: אימות תקינות נתוני משתמש – צד לקוח ושרת

ניסוח הבעיה

במערכת שבה משתמשים מזינים מידע אישי (שם משתמש, כתובת, ת"ז, סיסמה, מספר טלפון, ציוני סימול עברית, פרופיל, דפ"ר), נדרשת מערכת אמינה שתוודא:

שהנתונים תקינים מבחינה תחבירית, שהנתונים מתאימים לתחום הערכים האפשרי, שהשדות הנדרשים אינם חסרים, שהמידע לא מסוכן (`SQL injection`, תווים אסורים וכו').

אם מידע שגוי יוזן – זה עלול לגרום ל: שגיאות במסד (לדוגמה, שדה גיל = "abc"), חוויית משתמש מתסכלת (הודעת שגיאה רק אחרי שליחה), קריסה/שיבוש תהליכים (מיון עם תאריך לא תקין), חורי אבטחה.

הבעיה האלגוריתמית הינה כיצד לבצע אימות מידע מוקדם ככל האפשר (בד"כ בצד לקוח), תוך וידוא שאף מידע מזויף או שגוי לא חודר לשרת או למסד.

### תיאור אלגוריתמים קיימים לפתרון בעיה זו

אימות נתונים (Validation) הוא תחום רחב מאוד שכולל:

- Regular Expressions Regex – לבדוק מבנה מייל, טלפון, ת"ז.
- Type Checking – המרה למספר שלם, תאריך וכו'.
- Range Checking – טווחים: גיל בין 1 ל-119, פרופיל 0 עד 97.
- Set Membership – האם ערך שייך לקבוצת ערכים מוגדרת (פרופילים מותרים, דרגות עברית).
- Mandatory Fields – שדות שלא יכולים להיות ריקים.
- Conditional Logic – למשל: אם המשתמש לא מסמן checkbox -> אסור להזין הערה.

### סקירת הפתרון הנבחר עבורי במערכת

הוספתי:

- קובץ validators.py - שמסייע בוולידציה על פונקציות שונות
- הפרדה בין סוגי שדות: שדות חובה (username, email, id) – נבדקים תחילה, שדות אופציונליים (interview notes, phone) – נבדקים רק אם מולאו.
- ולידציה ב-GUI עצמו: ב-main.py וב-admin\_dashboard.py, לפני שליחת בקשה – מתבצע אימות.
- הודעת שגיאה מיידית עם messagebox.showerror(...) אם משהו שגוי.
- מחסומים כפולים: גם לקוח גם שרת.

### יתרונות הפתרון:

פשוט, קל להבנה, לא דורש ספריות חיצוניות, מאוד מודולרי – ניתן להוסיף כל בדיקה לפונקציה ייעודית.

ולידציה בצד השרת בלבד – הייתה יוצרת חוויית משתמש גרועה, כי כל שגיאה נראית רק אחרי השליחה.

ולידציה אוטומטית מבוססת GUI – לא אמינה מספיק, לא גמישה למבני נתונים משתנים.

### קשיים ודרכי התמודדות

- מילוי שדות בטפסים מ-Tkinter לעיתים קולט רווחים מיותרים <- כל קלט עובר .strip() לפני בדיקה.

- ולידציה של ערכים ריקים מול ערכים אופציונליים  
-> הפונקציות בודקות if value: לפני כל ניסיונות המרה או cast.
- קלט שגוי בפורמט מספרי  
-> שימוש ב-int(value) עם try: except ValueError: כדי למנוע קריסה.
- בדיקות כפולות (גם לקוח וגם שרת)  
-> זה נדרש לצורכי אבטחה, אך מחייב תחזוקה כפולה – אז שמרנו על שמות תואמים ואחידים לפונקציות.
- ולידציה של ערכים שלא ברור מהם החוקים שלהם (למשל interview note)  
-> קבענו שהם אופציונליים לגמרי, אך הגבלנו את האורך לתצוגה בלבד.

## נקודת ייחוד בפרויקט

כל ולידציה היא פונקציה עצמאית – ניתן לבדוק בנפרד, לכתוב עליה בדיקות יחידה, ולהרחיב בקלות.

שמירה על פשטות: המשתמש מקבל מיידית פידבק ברור מה GUI – עם פירוט מלא של מה לא תקין.

המערכת יודעת להתמודד גם עם חוסר קלט, גם עם קלט לא תקין, וגם עם קלט לא צפוי – הכל מטופל.

## לסיכום

מנגנון הולידציה בפרויקט מהווה תשתית בסיסית אך איתנה, ששומרת על איכות, אבטחה ויציבות המידע.  
הוא משלב גמישות לקליטים אמיתיים של משתמשים – עם בקורות חכמות שמונעות תקלות ובעיות עתידיות.

הפתרון ממומש בקפדנות ומתוחזק היטב – ומהווה חלק בלתי נפרד מהאמינות הכוללת של המערכת.

## סביבת פיתוח הפרויקט

במהלך הפיתוח של מערכת llayDF השתמשתי בסביבת פיתוח פשוטה אך מאוד ממוקדת ונוחה. את הקוד כתבתי בשפת Python 3.12, בגלל הגמישות שלה והאפשרות להשתמש בה גם לבניית GUI, גם לתקשורת בין לקוח לשרת, וגם להצפנה.

את הפיתוח עצמו עשיתי בסביבת העבודה PyCharm, שם עבדתי בתוך סביבה וירטואלית מבודדת שיצרתי עם venv. זה איפשר לי להתקין רק את הספריות הרלוונטיות לפרויקט בלי להשפיע על התקנות אחרות במחשב. כל הספריות הותקנו עם pip.

מבחינת מבנה הקוד, חילקתי אותו למודולים לפי תחומים. יש קובץ ראשי שמנהל את כל ממשק המשתמש, קובץ admin\_dashboard.py שאחראי על צד האדמין, קבצים נפרדים לתקשורת, קובץ נפרד למסד הנתונים, ועוד. זה עזר לי לשמור על סדר, ובעיקר להפוך כל חלק לנפרד ולבדיק. ככה כאשר הייתה לי טעות, ידעתי לבדוק אותה בצורה ממוקדת יותר.

כדי לבדוק שהמערכת באמת פועלת כמו שצריך, ביצעתי הרבה בדיקות ידניות. בדקתי כל תהליך – החל מהתחברות, קבלת קוד אימות למייל, איפוס סיסמה, ועד לשליחת הודעות, עריכת נתוני משתמשים, ושיבוץ למיונים. בדקתי גם את התגובות של המערכת למקרי קצה מגוונים, על מנת להפוך אותה לאמינה ויציבה ככל האפשר.

## תיאור פרוטוקול התקשורת

### תיאור מילולי של הפרוטוקול

זוהי הרחבה למצויין בפירוט טכנולוגיות הפרויקט: תקשורת.

מערכת התקשורת בפרויקט llayDF מבוססת על פרוטוקול מותאם אישית הפועל מעל חיבור TCP. הלקוח שולח בקשות לשרת בפורמט JSON מוצפן, והשרת משיב עם תגובות גם כן במבנה JSON. כל הבקשות יוזמות מהצד של הלקוח, בעוד השרת פועל כמאזין פסיבי שמגיב לכל בקשה לפי סוגה. הבקשות כוללות פעולה מסוג "type", ולעיתים נתונים נלווים כמו מזהה משתמש, פרטי התחברות או שדות מערכתיים נוספים.

על מנת להבטיח אבטחה, כל תקשורת בין הלקוח לשרת מוצפנת באמצעות הצפנה היברידית: הלקוח יוצר מפתח AES חדש, מצפין אותו עם המפתח הציבורי של השרת (RSA), ואז משתמש ב-AES להצפנת גוף ההודעה. השרת מפענח את המפתח באמצעות המפתח הפרטי שלו, ולאחר מכן מפענח את גוף ההודעה. כל הודעה נשלחת כחבילה אחת המכילה את המפתח המוצפן, וקטור האתחול (IV) והטקסט המוצפן. כך מתקבלת תקשורת מוצפנת, יעילה ומבוססת בקשות-תגובות.

סדר הפעולות לפני כל שליחה:

1. הלקוח בונה אובייקט Python מסוג dict עם הנתונים.
2. האובייקט מומר למחרוזת JSON.
3. המחרוזת מוצפנת עם AES.
4. מפתח ה-AES עצמו מוצפן באמצעות RSA.
5. שלושת החלקים נארזים למחרוזת JSON נוספת ונשלחים כ-bytes.

השרת מקבל את ההודעה, מפענח באותה דרך בדיוק, ומחזיר תגובה בפורמט זהה.

אין שימוש בהפרדה של כמות בתים או Header קבוע. כל הודעה מועברת כ-JSON מוצפן אחד שלם, והצד המקבל קורא את כל המידע בבת אחת.

### פירוט הודעות במערכת

להלן כלל סוגי ההודעות, עם שם, כיוון, מבנה שדות ותגובת השרת:

| התגובה(כיוון: מהשרת ללקוח)                              | הבקשה לשרת (כיוון: מהלקוח לשרת) | סוג ההודעה     |
|---------------------------------------------------------|---------------------------------|----------------|
| status": "ok", "key": "<PEM public" }<br>{ "<key string | { "type": "get_public_key" }    | get_public_key |

|                      |                                                                                                                                                |                                                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| login                | type: "login", "username": "...",<br>{ "..." : "password"                                                                                      | status: "ok"   "error", "message": "<br>...", "is_admin": 1 0, "user_id": 1,<br>"email": "...", "first_login": true false<br>{ |
| complete_first_login | type: "complete_first_login",<br>"user_id": 1, "id_number": "...", "phone":<br>"...", "address": "...", "age": 18,<br>{ "..." : "new_password" | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| send_code_to_user    | type: "send_code_to_user",<br>{ "user_id": 1                                                                                                   | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| verify_code          | type: "verify_code", "user_id": 1,<br>{ "code": "123456"                                                                                       | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| request_reset_code   | type: "request_reset_code",<br>{ "..." : "username"                                                                                            | status: "ok"   "error", "user_id": 1,<br>{ "..." : "email": "...", "message"                                                   |
| reset_password       | type: "reset_password", "username": "<br>..." : "...", "email": "...", "new_password"<br>{                                                     | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| get_user_data        | { type: "get_user_data", "user_id": 1 }                                                                                                        | { [...] : "status": "ok"   "error", "data" }                                                                                   |
| get_meyunim          | { type: "get_meyunim", "user_id": 1 }                                                                                                          | { [...] : "status": "ok", "data" }                                                                                             |
| submit_request       | type: "submit_request",<br>"assignment_id": 1, "request_type":<br>{ "..." : "delay", "message"                                                 | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| send_message         | type: "send_message", "user_id": 1,<br>{ "..." : "subject": "...", "content"                                                                   | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| get_messages         | { type: "get_messages", "user_id": 1 }                                                                                                         | { [...] : "status": "ok", "data" }                                                                                             |
| get_my_requests      | type: "get_my_requests", "user_id": "<br>1                                                                                                     | { [...] : "status": "ok", "data" }                                                                                             |
| add_user             | type: "add_user", "username": "...",<br>"email": "...", "password": "...",<br>"raw_password": "...", "id_number": "...",<br>{ "is_admin": 0    | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| get_all_users        | { "type": "get_all_users" }                                                                                                                    | { [...] : "status": "ok", "data" }                                                                                             |
| load_user_data       | :"type": "load_user_data", "username" }<br>{ "..."                                                                                             | { [...] : "status": "ok"   "error", "data" }                                                                                   |
| update_user_data     | type: "update_user_data",<br>"username": "...", "account_data": {...},<br>{ [...] : "system_data"                                              | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| create_meyun         | type: "create_meyun", "title": "...",<br>"description": "...", "date":<br>{ ["YYYY-MM-DD", "user_ids": [1,2                                    | :"status": "ok"   "error", "message" }<br>{ "..."                                                                              |
| get_all_meyunim      | { "type": "get_all_meyunim" }                                                                                                                  | { [...] : "status": "ok", "data" }                                                                                             |

|                          |                                                                                         |                                                   |
|--------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------|
| delete_meyun             | type": "delete_meyun", "meyun_id": 1" }                                                 | :"status": "ok"   "error", "message" }<br>{ "..." |
| get_all_requests         | { "type": "get_all_requests" }                                                          | { [...] : "status": "ok", "data" }                |
| update_request_status    | type": "update_request_status", " }<br>"request_id": 1, "new_status":<br>{ ""Approved   | :"status": "ok"   "error", "message" }<br>{ "..." |
| approve_cancel_request   | type": "approve_cancel_request", " }<br>{ "request_id": 1                               | :"status": "ok"   "error", "message" }<br>{ "..." |
| update_date_for_request  | type": "update_date_for_request", " }<br>"request_id": 1, "new_date":<br>{ ""YYYY-MM-DD | :"status": "ok"   "error", "message" }<br>{ "..." |
| get_all_messages         | { "type": "get_all_messages" }                                                          | { [...] : "status": "ok", "data" }                |
| reply_to_message         | type": "reply_to_message", " }<br>{ "...": ""message_id": 1, "response                  | :"status": "ok"   "error", "message" }<br>{ "..." |
| get_age_distribution     | { "type": "get_age_distribution" }                                                      | { [...] : "status": "ok", "ages" }                |
| get_request_type_stats   | { "type": "get_request_type_stats" }                                                    | { [...] : "status": "ok", "data" }                |
| get_request_status_stats | { "type": "get_request_status_stats" }                                                  | { [...] : "status": "ok", "data" }                |
| get_meyunim_per_month    | { "type": "get_meyunim_per_month" }                                                     | { [...] : "status": "ok", "data" }                |
| get_messages_per_week    | { "type": "get_messages_per_week" }                                                     | { [...] : "status": "ok", "data" }                |
| get_user_notifications   | type": "get_user_notifications", " }<br>{ "user_id": 1                                  | { [...] : "status": "ok", "notifications" }       |

## מסכי הפרויקט

### מסך התחברות ראשי

דרך מסך זה ניתן להתחבר למערכת או לעבור לאיפוס סיסמא.

**מסך מילוי פרטים לאחר התחברות ראשונית**

במידה והפרטים שהוזנו במסך ההתחברות הם של משתמש חדש, המשתמש יעבור למסך זה, בו יאמת את תעודת הזהות שלו וימלא פרטים אישיים. במידה ותעודת הזהות לא תתאים לתעודת הזהות השמורה במערכת, לא ניתן יהיה להתחבר למערכת. בנוסף ישנו כפתור חזרה למסך הראשי.

Login System

Complete Your Profile & Set Password

ID Number (9 digits)

Phone (9-10 digits)

Address

New Password

Submit Information

Back to Login

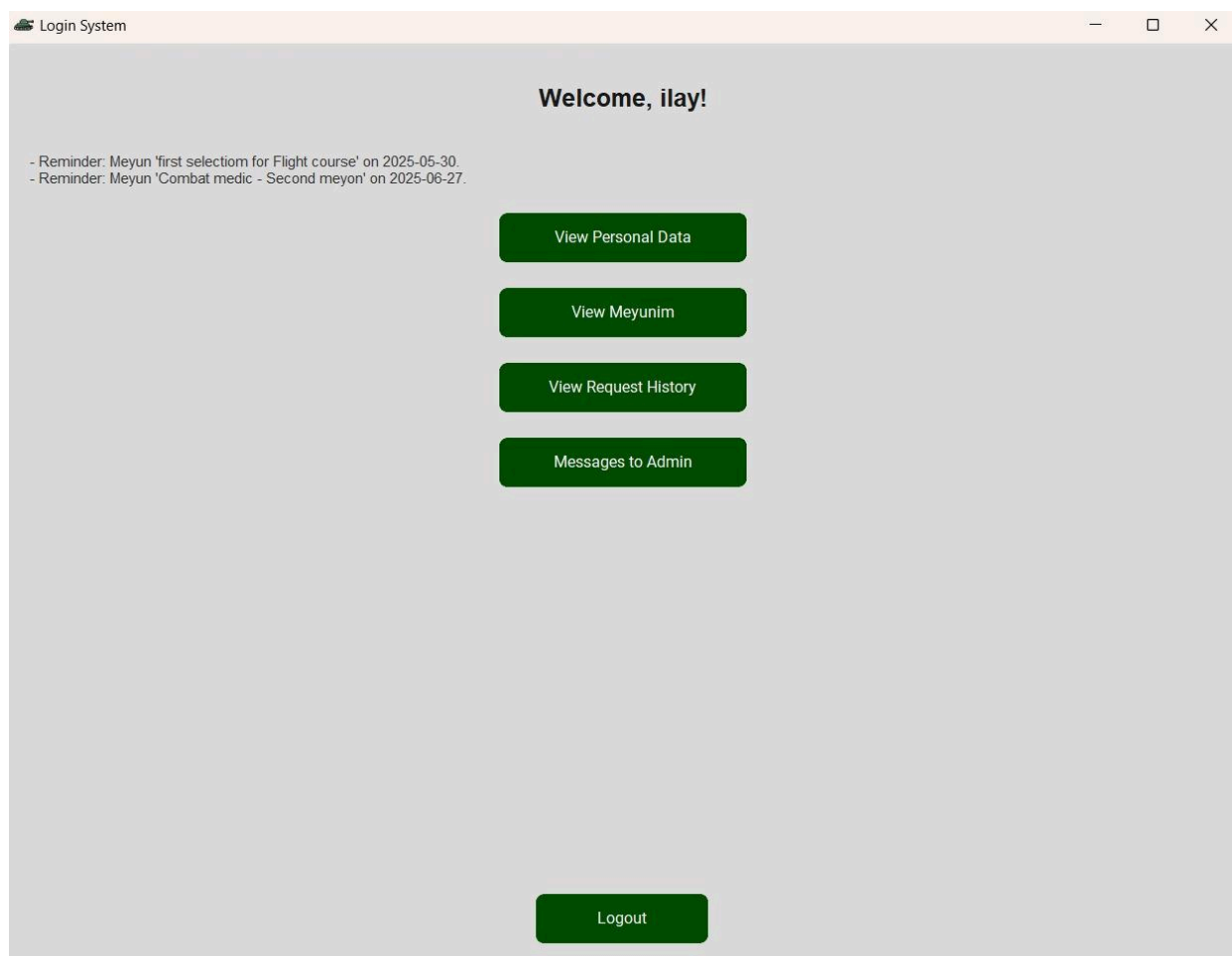
## מסך אימות דו שלבי

לאחר הזנה תקינה של שם משתמש וסיסמא, המשתמש יעבור למסך זה בו ידרש להזין את הקוד בן 6 הספרות שקיבל לכתובת האימייל שלו הכתובה במערכת.

The screenshot shows a web browser window titled "Login System". The main heading is "Enter Verification Code for Login". Below the heading is a large, empty text input field. Underneath the input field is a green button labeled "Verify". Below the "Verify" button is a message: "A 6-digit code has been sent to your email." At the bottom of the form is another green button labeled "Back to Login".

## מסך תפריט ראשי בעבור מלש"ב

במסך זה יראה המלש"ב התראות חדשות, ותפריט המציג בפניו 4 אופציות: צפייה בנתונים האישיים, צפייה במיונים, צפייה בהיסטוריית הבקשות ודף קשר עם הצבא.



## מסך נתונים אישיים בעבור מלש"ב

במסך זה יכול לצפות המלש"ב בנתוניו האישיים

Login System

DAPAR

90

PROFILE

45

HEBREW

8.0

INTERVIEW

Highly suitable for combat Service

100 DAY

Completed

Back to Main Menu

## מסך צפייה במיונים בעבור מלש"ב

במסך זה יכול לראות המשתמש את כל מיוניו, לבקש לדחות או לבטל מיון ולראות את סטטוס בקשותיו בנוגע למיונים.

Login System

Your Meyunim

**first selection for Flight course (2025-05-30)**  
Description: Please bring a medical form, food and water.  
Status: Scheduled (Pending Delay)

**Combat medic - Second meyon (2025-06-27)**  
Description: 9:00 - Medical corps base  
Status: Scheduled

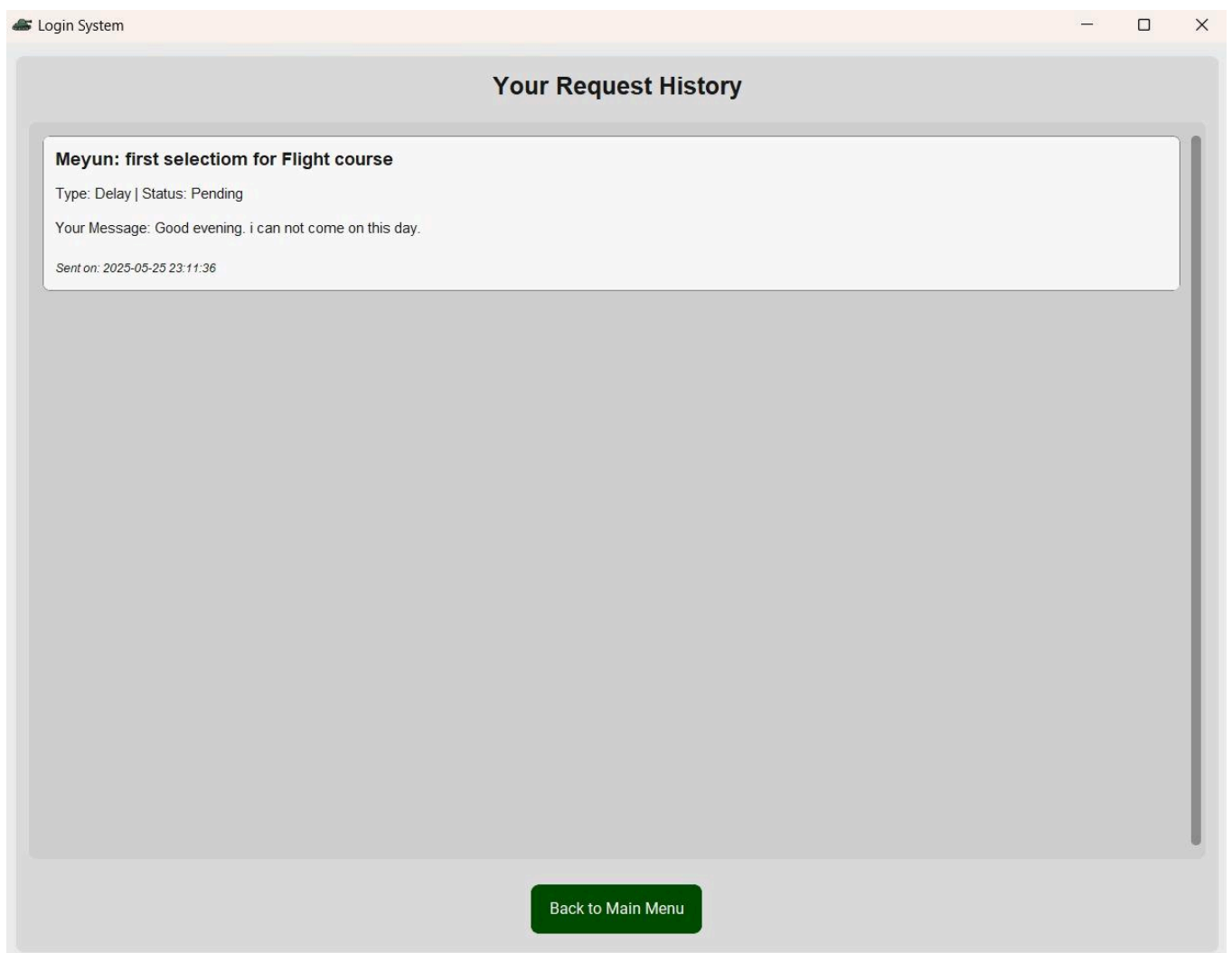
Request Delay

Request Cancellation

Back to Main Menu

## מסך היסטוריית בקשות בעבור מלש"ב

במסך זה יכול מלש"ב לצפות בכל היסטוריית הבקשות שלו.



## מסך תקשורת עם הצבא בעבור מלש"ב

במסך זה יכול המלש"ב לתקשר עם הצבא. הוא יכול לשלוח הודעות אל הצבא, לצפות בהודעות שנשלחו ובתגובות אליהן.

Login System

Send a Message to Admin

Subject

Send

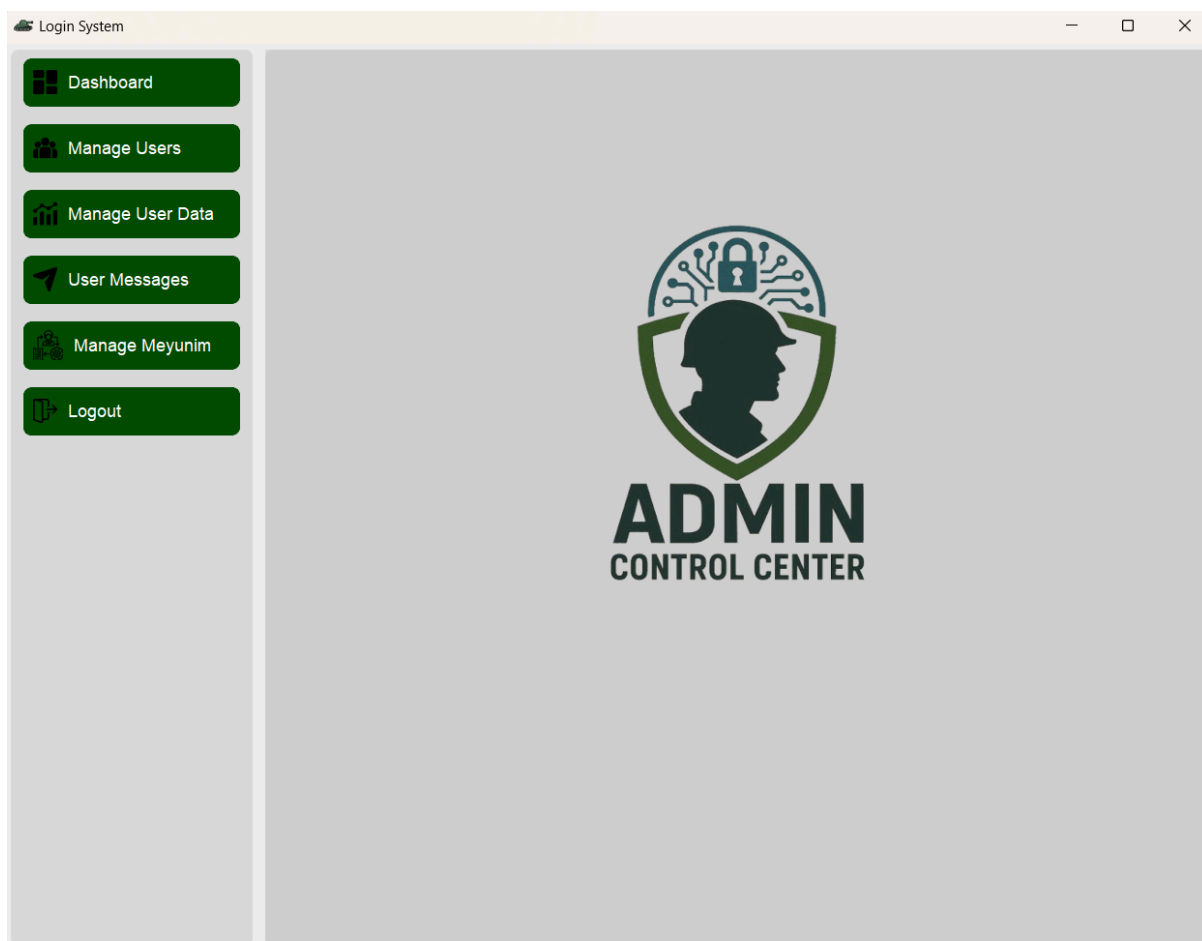
Your Message History

**Subject: vacation**  
Your Message: i am on a vacation next week, just for gou to know.  
Sent: 2025-05-25T23:18:59.494360  
**Admin's Reply: have fun.**

Back to Main Menu

## מסך תפריט ראשי בעבור אדמין

לאחר כניסה של אדמין, מוצג לו מסך תפריט ראשי בו כפתורי מעבר לחלונות הבאים.



## מסך ניהול משתמשים בעבור אדמין

במסך זה יכול המנהל מטעם הצבא להוסיף משתמשים חדשים למערכת ולשלוח להם את פרטי ההתחברות למערכת אל כתובת המייל(מעין צו ראשון דיגיטלי). בנוסף לכך, הוא יכול להוסיף משתמשי אדמין אחרים. יתר על כן, רואה המנהל את כלל המשתמשים הרשומים למערכת עם פרטיהם, מלבד משתמשי אדמין כמובן.

Login System

Dashboard

Manage Users

Manage User Data

User Messages

Manage Meyunim

Logout

Manage Users

Username

Email

ID Number (9 digits)

☐ Is Admin

Add User

| ID | Username | Role | Phone      | Address                      | Age |
|----|----------|------|------------|------------------------------|-----|
| 2  | ilay     | User | 0544678934 | Ben Gorion 1, Zichron Yaakov | 19  |
| 3  | yonatan  | User | 0534538876 | Palmach 18, Haifa            | 19  |
| 4  | avner    | User | 0526678923 | Olive rode 12, Beit Shemesh  | 20  |
| 5  | yair     | User | 0545698234 | Hashlom 22, Tel Aviv         | 18  |

## מסך ניהול נתוני משתמשים בעבור אדמין

במסך זה יכול האדמין להזין ולעדכן פרטים של משתמשים. החל מעדכון כתובת לאחר מעבר דירה, ועד להזנת נתונים רפואיים לאחר הצו הראשון.

Login System

Dashboard

Manage Users

Manage User Data

User Messages

Manage Meyunim

Logout

Manage User Data

Search Username:

ilay

Email: nagler.ilay@gmail.com

Age: 19

Address: Ben Gorion 1, Zichron Yaakov

Phone: 0544678934

DAPAR: 90

Hebrew: 8.0

Profile: 45

Interview: Highly suitable for combat Service

✓ 100 DAY Completed

Save Changes

Reset Changes

## מסך תקשורת עם מלש"בים בעבור אדמין

במסך זה יכול המנהל לצפות בהודעות מלשבים ולענות להן. הוא רואה בצורה מסודרת ולפי צבעים את הבקשות להן הגיב ואת הבקשות שלא ניתן להן מענה עדיין.

Login System

Dashboard

Manage Users

Manage User Data

User Messages

Manage Meyunim

Logout

Messages from Users

Inbox

From: avner

Received: 2025-05-25T23:18:59.494360

Subject: vacation

Message: i am on a vacation next week, just for gou to know.

Your Reply: have fun.

From: yonatan

Received: 2025-05-25T23:16:39.038396

Subject: early recruitment

Message: i would like to join the army as fast that i could.

Reply

From: ilay

Received: 2025-05-25T23:13:03.371514

Subject: air force

Message: hi! i would like to join the air force.

Reply

**מסך ניהול מיונים בעבור האדמין הכולל כל הקשור לניהול המיונים של הצבא. המסך כולל 3 תתי מסכים**

**מסך יצירת מיונים**

מסך יצירת המיונים מאפשר למנהל ליצור מיונים חדשים, לקבוע את שמם, לתאר אותם, להזין תאריך ולשבץ אליהם מלש"בים

Login System

Dashboard

Manage Users

Manage User Data

User Messages

Manage Meyunim

Logout

Create Meyun View Requests All Meyunim

### Create New Meyun

first selection for Flight course

Please bring a medical form, food and water.

Date: 2025-05-30

Assign to Users

- ☒ ilay
- ☐ yonatan
- ☒ avner
- ☒ yair

Create Meyun and Assign

## מסך ניהול בקשות דחייה וביטול ממלשבים

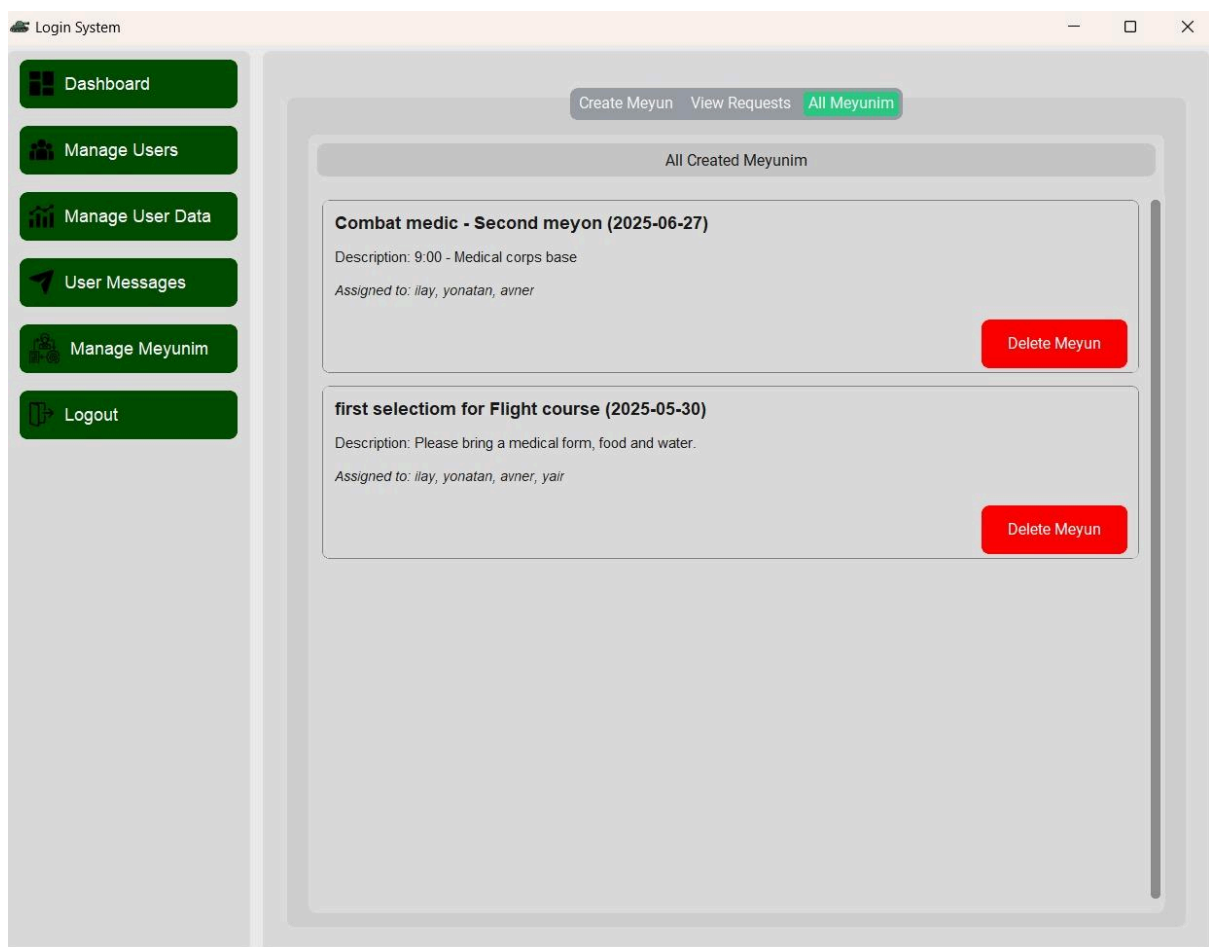
במסך זה, יכול האדמין לצפות בכלל הבקשות אשר מסודרות לפי סטטוס ומוצגות בצבעים שונים. הוא יכול לאשר ולדחות בקשות. תחת כל בקשה מופיע הסטטוס שלה.

The screenshot shows a web application window titled "Login System". On the left is a sidebar with navigation buttons: Dashboard, Manage Users, Manage User Data, User Messages, Manage Meyunim, and Logout. The main content area is titled "User Requests" and contains a table of requests. At the top of the main area are buttons for "Create Meyun", "View Requests" (highlighted in green), and "All Meyunim". Below these are filters for "Type" (set to "All") and "Status" (set to "All").

| User Requests                                                                                                                                                                                 |                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>User: ilay   Meyun: first selection for Flight course (2025-05-30) [DELAYED TO: 2025-05-30]</b><br>Request: Delay   Status: Approved<br>Message: Good evening. i can not come on this day. |                |
| <b>User: yonatan   Meyun: Combat medic - Second meyon (2025-06-27)</b><br>Request: Delay   Status: Rejected<br>Message: i dont want to be a medic!                                            |                |
| <b>User: avner   Meyun: Combat medic - Second meyon (2025-06-27)</b><br>Request: Cancel   Status: Pending<br>Message: i want to be a pilot.                                                   | Approve Reject |
| <b>User: avner   Meyun: first selection for Flight course (2025-05-30)</b><br>Request: Delay   Status: Pending<br>Message: i am on a vacation.                                                | Approve Reject |

## מסך כל המיונים

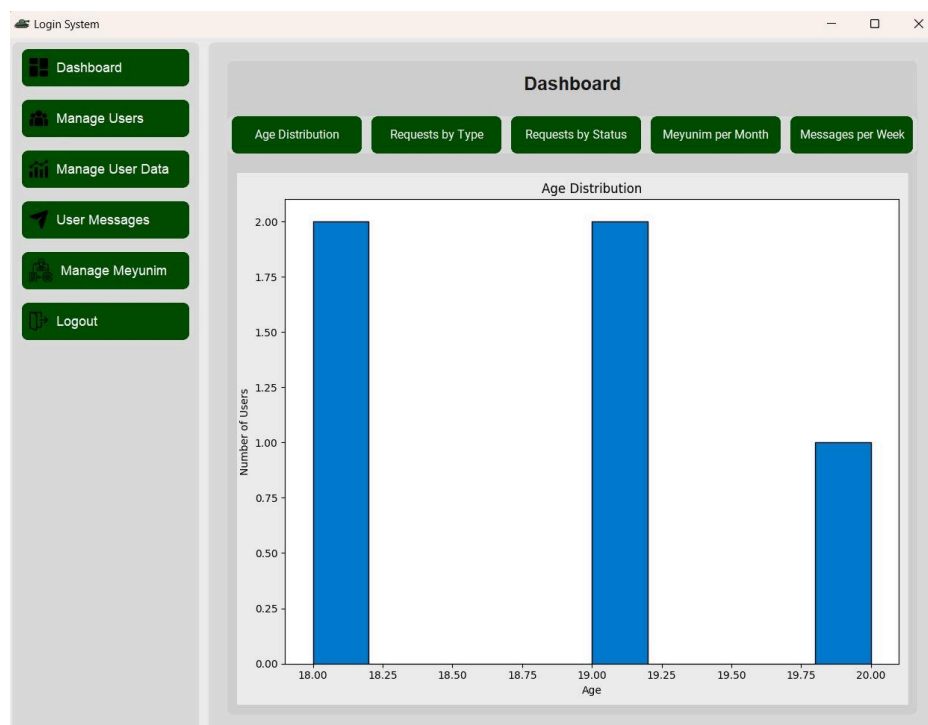
המסך מציג את כל המיונים הקיימים ואופציה לביטולם.



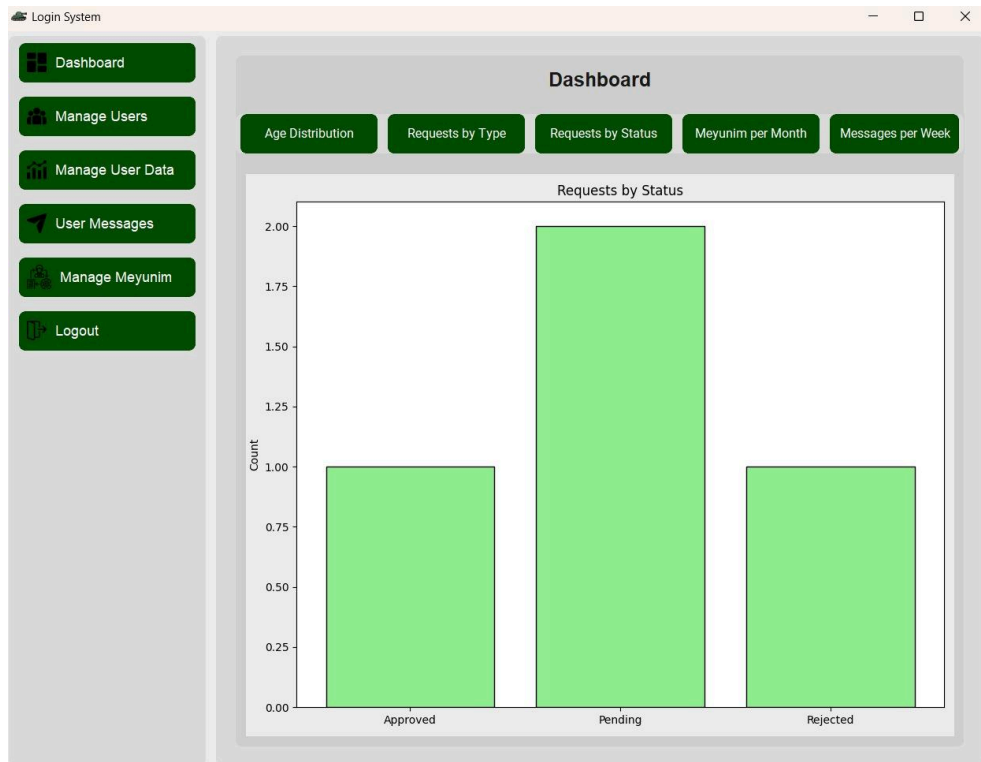
מסך דשבורד בעבור האדמין הכולל הצגה גרפית של נתונים. המסך כולל 5 תתי מסכים.

כלל המסכים הבאים מציגים את הנתונים בצורה גרפית למען ניהול יעיל ונוח.

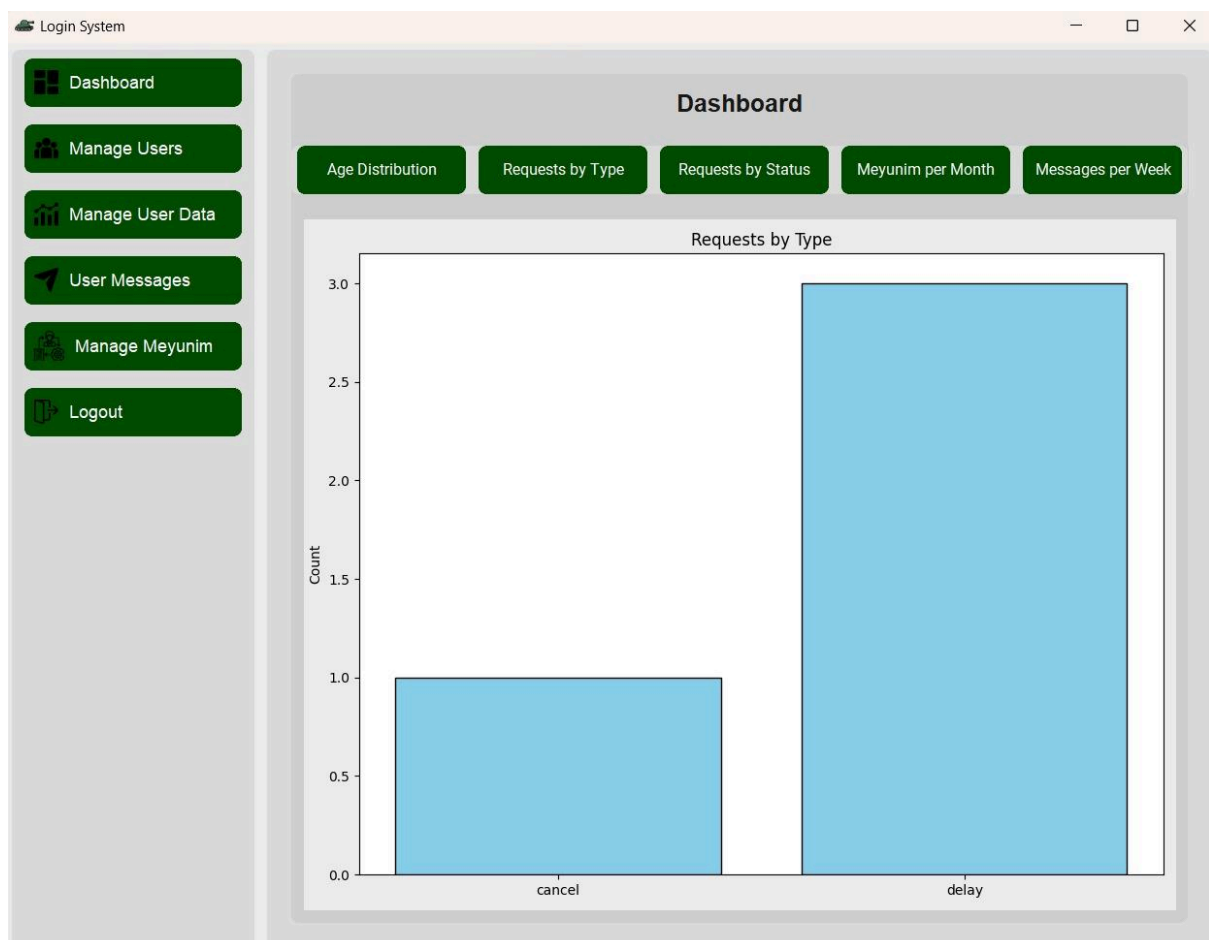
גרף גרף עמודות של משתמשים לפי גילאים



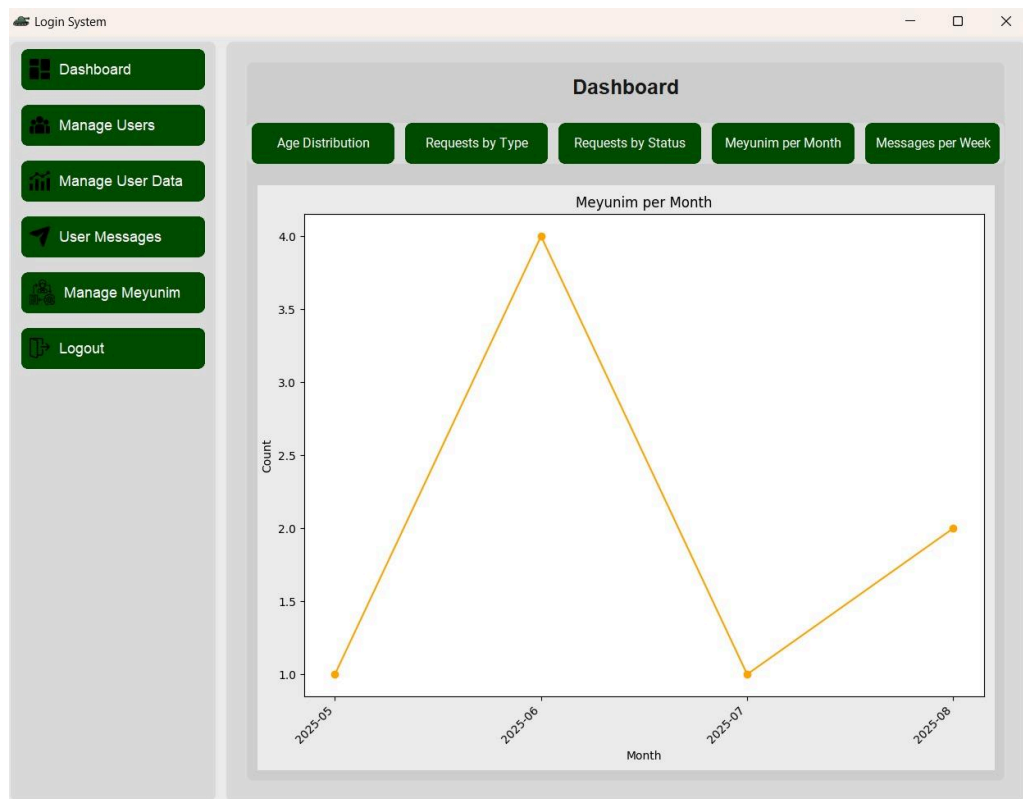
גרף עמודות של בקשות לפי סטטוס



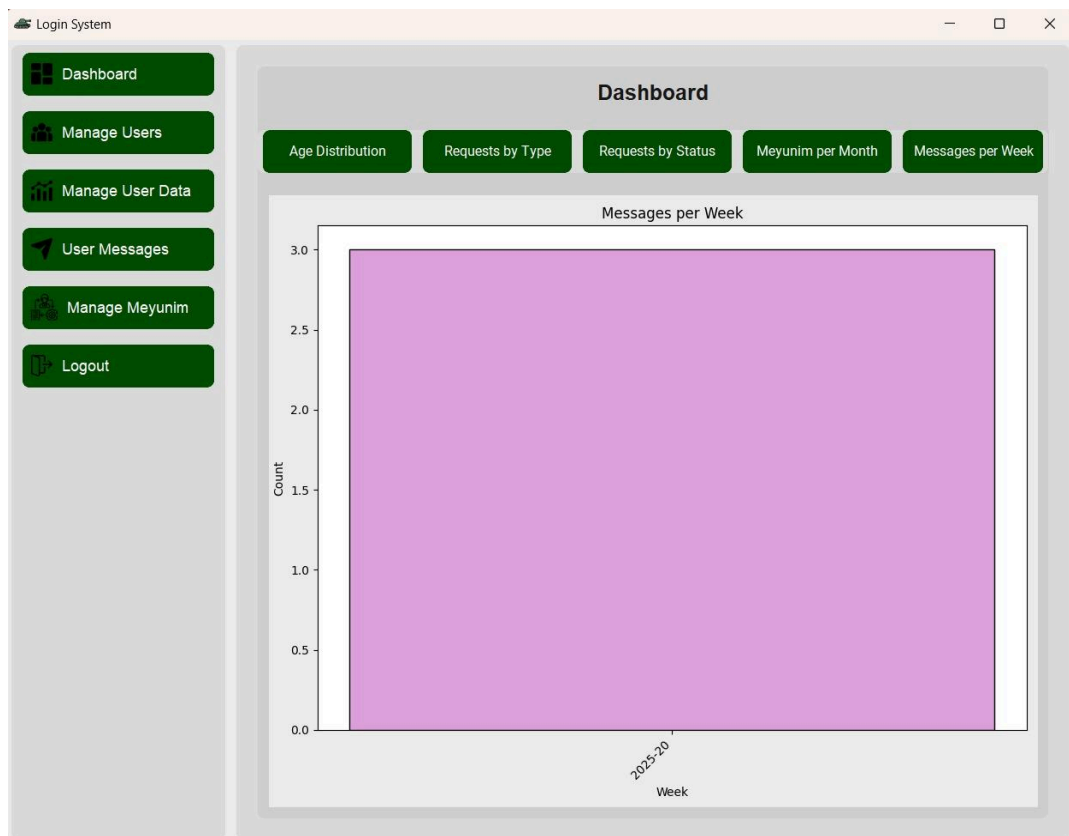
גרף עמודות של בקשות לפי סוג



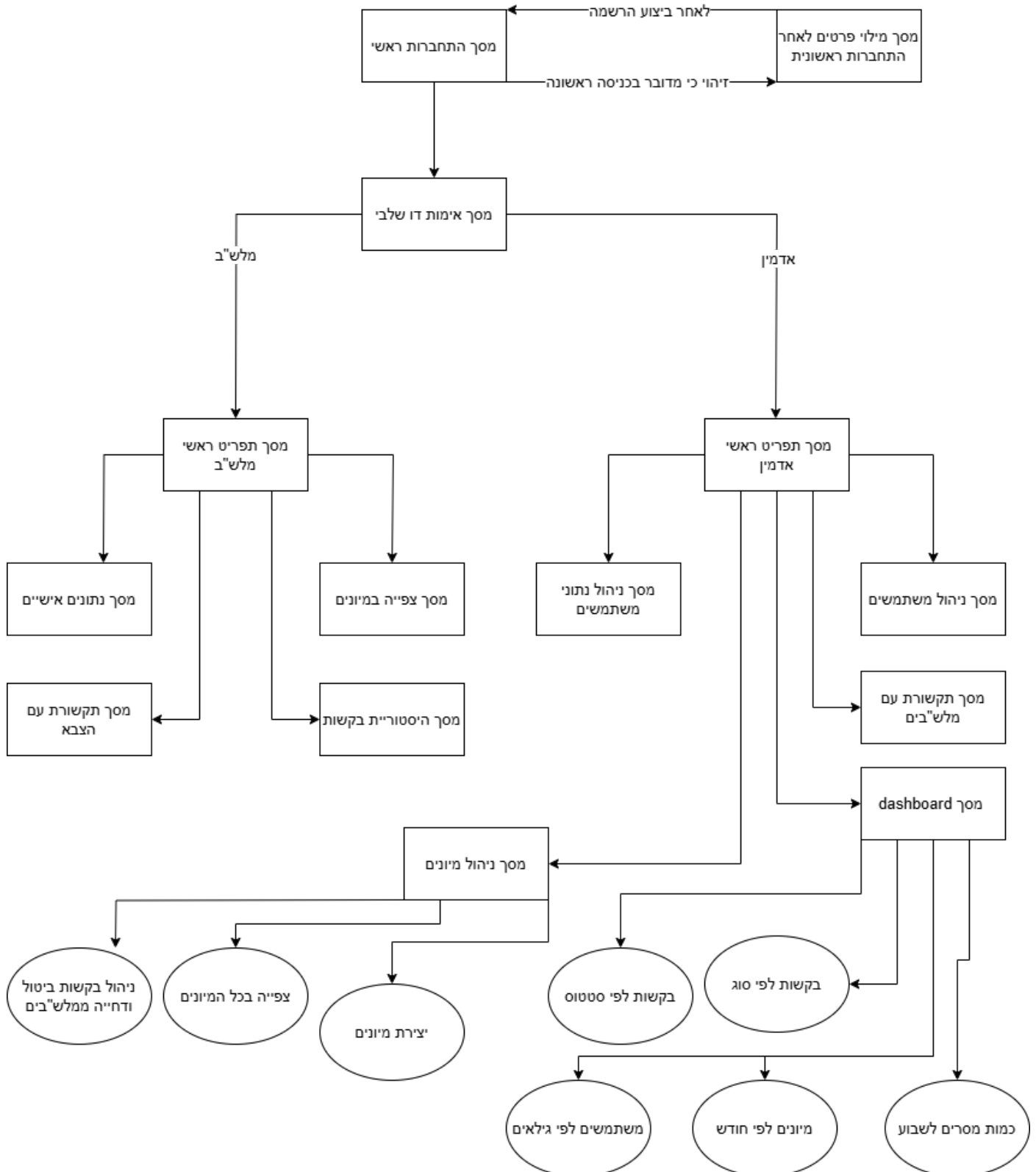
## גרף קווי מגמה של כמות מיונים לחודש



## גרף עמודות של כמות מסרים לשבוע



## תיאור גרפי של מסכי הפרויקט



## תיאור מבני הנתונים

המערכת עושה שימוש במסד נתונים מסוג SQLite בשם users.db, שמכיל את כל המידע הקבוע של המשתמשים, המיונים, הבקשות וההודעות. נוסף על כך, נעשה שימוש בקובץ env לאחסון פרטי התחברות רגישים (כגון כתובת וסיסמה למייל), וכן בקובצי מפתחות RSA (public\_key.pem, private\_key.pem) לצורך הצפנה.

כל שאר הנתונים – בקשות, שדות קלט זמניים, מפתחות AES – נשמרים רק בזיכרון (RAM) במהלך הריצה.

להלן פירוט מלא של כלל הטבלאות במסד:

### usersDB

מכיל את כל המשתמשים הרשומים במערכת

| שם שדה    | טיפוס   | דוגמא                 | הערה                     |
|-----------|---------|-----------------------|--------------------------|
| id        | INTEGER | 1                     | מפתח ראשי (primary key)  |
| username  | TEXT    | "ilay22"              | שם ייחודי                |
| email     | TEXT    | "ilaynag@gmail.com"   |                          |
| password  | TEXT    | "hashed password"     | סיסמא מגובבת לפי SHA-256 |
| id_number | TEXT    | "hashed id"           | ת"ז מגובבת לפי SHA-256   |
| is_admin  | TEXT    | or 1 0                | 1 = אדמין, 0 = מלש"ב     |
| phone     | INTEGER | "0542235897"          | אופציונלי                |
| address   | TEXT    | "herzel 24. Tel Aviv" | אופציונלי                |
| age       | INTEGER | 18                    | אופציונלי                |

### user\_data

נתונים מערכתיים של כל משתמש

| שם שדה  | טיפוס   | דוגמא | הערה                      |
|---------|---------|-------|---------------------------|
| id      | INTEGER |       | מפתח ראשי                 |
| user_id | INTEGER | 1     | מפתח זר ל-usersDB         |
| dapar   | INTEGER | 60    | כפולות של 10 בין 10 ל-90. |
| profile | INTEGER | 97    | ערכים אפשריים: 0, 21,     |

|                            |                            |         |             |
|----------------------------|----------------------------|---------|-------------|
| 72, 64, 45, 25, 24, 97, 82 |                            |         |             |
| ציוני עברית מ 5 ל 8        | 7                          | INTEGER | interview   |
| שדה טקסט חופשי             | "התאמה גבוהה לשירות בקרבי" | TEXT    | interview   |
| האם בוצע יום המאה          | or 1 0                     | INTEGER | hundred_day |

## auth\_codes

קודי אימות זמניים לאימות דו שלבי

| שם שדה    | טיפוס   | דוגמא                      | הערה                 |
|-----------|---------|----------------------------|----------------------|
| id        | INTEGER |                            | מפתח ראשי            |
| user_id   | INTEGER | "1"                        | מפתח זר ל-usersDB    |
| code      | TEXT    | "338459"                   | קוד אימות חד פעמי    |
| timestamp | TEXT    | "2025-05-01"<br>"10:22:15" | לצורך בדיקת תוקף קוד |

## meyunim

טבלת המיונים שנוצרו על ידי הצבא

| שם שדה      | טיפוס   | דוגמא                    | הערה                 |
|-------------|---------|--------------------------|----------------------|
| id          | INTEGER | 1                        | מפתח ראשי            |
| title       | TEXT    | "מבחן מיון לחיל הים"     |                      |
| description | TEXT    | "יש להגיע עם בגדי ספורט" |                      |
| date        | TEXT    | "2025-06-15"             | בפורמט<br>YYYY-MM-DD |

## meyunim\_assignments

שיבוץ של משתמשים למיונים

| שם שדה   | טיפוס   | דוגמא | הערה              |
|----------|---------|-------|-------------------|
| id       | INTEGER | 1     | מפתח ראשי         |
| user_id  | INTEGER | 1     | מפתח זר ל-usersDB |
| meyun_id | INTEGER | 2     | מפתח זר ל-meyunim |

## meyunim\_requests

בקשות דחייה או ויתור על מיון

| שם שדה        | טיפוס   | דוגמא           | הערה                          |
|---------------|---------|-----------------|-------------------------------|
| id            | INTEGER |                 | מפתח ראשי                     |
| assignment_id | INTEGER | 3               | מפתח זר ל-meyunim_assignments |
| type          | TEXT    | "delay"         | "cancel" או "delay"           |
| message       | TEXT    | "אני בחוץ לארץ" | הסבר של המשתמש                |
| status        | TEXT    | "pending"       | "Approved", "Rejected"        |
| timestamp     | TEXT    |                 | תאריך בקשה                    |
| notified_user | INTEGER | 0               | האם המשתמש עודכן (0/1)        |

## messages

הודעות מהמשתמשים לאדמין ולהפך

| שם שדה | טיפוס | דוגמא | הערה |
|--------|-------|-------|------|
|--------|-------|-------|------|

|                            |                     |         |                    |
|----------------------------|---------------------|---------|--------------------|
| מפתח ראשי                  |                     | INTEGER | id                 |
| מפתח זר ל-usersDB          | "1"                 | INTEGER | sender_id          |
|                            | "בעיה במיון"        | TEXT    | subject            |
| תוכן ההודעה                | "אני לא יכול להגיע" | TEXT    | content            |
| תגובת האדמין (אם קיימת)    | "נדחה בהצלחה"       | TEXT    | response           |
| זמן שליחת ההודעה           |                     | TEXT    | timestamp          |
| זמן תגובת האדמין (אם קיים) |                     | TEXT    | admin_responded_at |
| האם המשתמש ראה את התגובה   | 0                   | INTEGER | notified_user      |

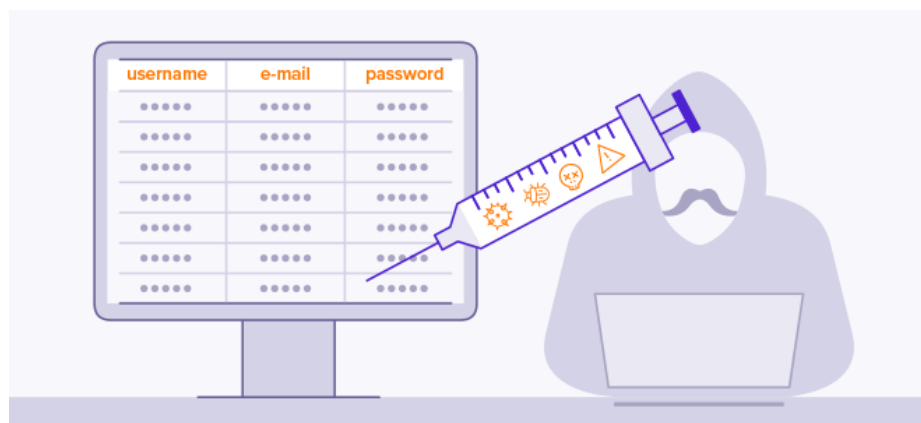
כל הנתונים שמורים בפורמט טקסטואלי פשוט או מספרי, והמערכת משתמשת בשאילתות רגילות מסוג SELECT, INSERT, UPDATE, ו-DELETE על כל אחת מהטבלאות לפי הצורך. השימוש במסד נתונים אחד מרכזי מונע כפילויות, שומר על קשרים בין הנתונים, ומאפשר שליפה יעילה של כל פרט הדרוש להצגת מידע, קבלת החלטות מצד האדמין או שמירת היסטוריה של פעולות משתמשים.

## סקירת חולשות ואיומים

### הגנה על שכבת האפליקציה

במערכת כמו זו, אחת הסכנות המרכזיות היא SQL Injection – מצב שבו תוקף מזין לתוך שדות קלט ערכים שמתחזים לקוד SQL, במטרה לשנות שאילתות הרצה. כדי להתמודד עם האיום הזה, ביצעתי הגנה כפולה:

1. בכל מקום שבו נשלחים נתונים למסד נעשה שימוש בפרמטרים עם placeholders מסודרים, באמצעות סימן שאלה (prepared statements). למשל: במקום להכניס ערכים ישירות לשאילתת SELECT או INSERT, המשתמש מוכנס לשאילתה בצורה מבוקרת כפרמטר. כך נטרלנו את האפשרות להזרקת קוד למסד.
2. בעזרת מערכת וולידציה קלט לא תקין נעצר כבר בשלב ממשק המשתמש וב-99.9% מהמקרים לא יצליח להגיע אל השאילתה (אני עוד לא מצאתי פירצה כזו)



בנוסף, המערכת לא מתקשרת עם אתרי אינטרנט חיצוניים, ולכן אינה חשופה ישירות לאיומים טיפוסיים של מערכות Web כמו Cross-Site Scripting (XSS) או קישורים מזויפים.

### מניעת התקפות שירות

המערכת מיישמת מנגנון בסיסי למניעת התקפות מניעת שירות (DoS/DDoS) פשוטות, המבוסס על הגבלת קצב בקשות (Rate Limiting) מכתובות IP בודדות. כך זה עובד:

#### 1. מעקב אחר חיבורים ("connection\_log"):

השרת משתמש במילון מסוג defaultdict list בשם connection\_log. המפתח במילון הוא כתובת ה-IP של הלקוח, והערך הוא רשימה של חותמות זמן (timestamps) של החיבורים האחרונים מאותה כתובת.

#### 2. חלון זמן וסף ("RATE\_WINDOW", "RATE\_LIMIT"):

- מוגדר "חלון זמן" ("RATE\_WINDOW", לדוגמה 10 שניות).
- מוגדר "סף בקשות" ("RATE\_LIMIT", לדוגמה 30 בקשות).

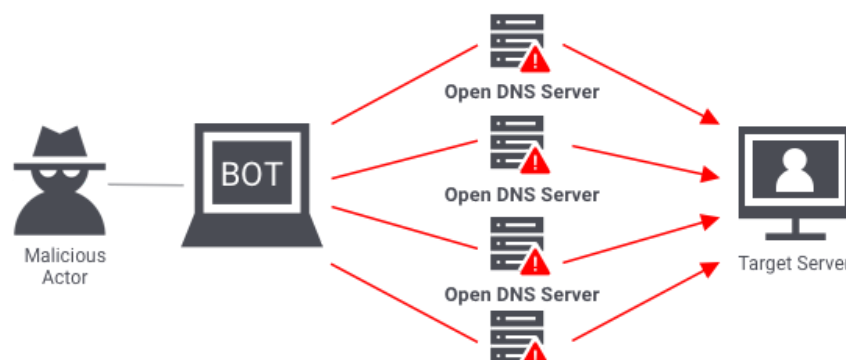
#### 3. בדיקה בכל חיבור חדש:

- כאשר מתקבל חיבור חדש מכתובת IP מסוימת: השרת מסנן מרשימת חותמות הזמן של אותו IP את כל החיבורים שקרו לפני יותר מ-"RATE\_WINDOW" שניות (כלומר, שומר רק את החיבורים שקרו בתוך חלון הזמן האחרון).
- חותמת הזמן של החיבור הנוכחי מתווספת לרשימה.
- השרת בודק את אורך הרשימה המעודכנת (מספר החיבורים בחלון הזמן האחרון).

#### 4. חסימה ("blocked\_ips", "BLOCK\_TIME"):

אם מספר החיבורים בחלון הזמן האחרון ("len(connection\_log[ip])") עולה על "RATE\_LIMIT":

- כתובת ה-IP נחשבת כחשודה ומנסה להעמיס על השרת.
- כתובת ה-IP מתווספת למילון "blocked\_ips", כאשר הערך הוא חותמת הזמן שבה החסימה תסתיים ("BLOCK\_TIME"). "now + BLOCK\_TIME" הוא משך זמן החסימה (לדוגמה, 60 שניות).
- הרשימה של ה-IP מ-"connection\_log" נמחקת.
- החיבור הנוכחי מאותו IP נדחה והשרת לא ממשיך לטפל בו.
- בכל חיבור חדש, השרת בודק תחילה אם ה-IP של הלקוח נמצא ב-"blocked\_ips" ואם זמן החסימה שלו טרם פג. אם כן, החיבור נדחה מיד.
- אם IP היה חסום וזמן החסימה שלו פג, הוא מוסר מ-"blocked\_ips".



## שליחת מיילים מאובטחת

שליחת המיילים נעשית דרך Gmail תוך שימוש ב-SMTP מאובטח (smtp.gmail.com עם TLS). כדי למנוע חשיפת סיסמאות: פרטי חשבון המייל (EMAIL\_USER + EMAIL\_PASS) מאוחסנים בקובץ .env, שאינו חלק מהקוד, הקובץ עצמו נקרא דרך dotenv ונטען בזיכרון בלבד והמיילים עצמם אינם כוללים מידע אישי מעבר לנדרש – והם נכתבים בפורמט ברור אך מאובטח.

## אימות דו שלבי

האימות הדו-שלבי (2FA), המיושם במערכת באמצעות שליחת קוד אימות ייחודי ומוגבל בזמן לדוא"ל המשתמש בעת התחברות או איפוס סיסמה, מהווה נדבך קריטי באסטרטגיית האבטחה הכוללת של הפרויקט. תרומתו המרכזית היא בהגנה מפני גישה בלתי מורשית לחשבונות משתמשים, גם במקרה שפרטי ההתחברות העיקריים (שם משתמש וסיסמה) נחשפו או נגנבו. על ידי דרישת גורם אימות נוסף (משהו שהמשתמש יודע - הסיסמה, ומשהו שלמשתמש יש גישה אליו - חשבון הדוא"ל לקבלת הקוד), המערכת מקטינה משמעותית את הסיכון להשתלטות על חשבונות. הגנה זו חיונית במיוחד במערכת המנהלת מידע רגיש על משתמשים ותהליכי מיון, שכן פריצה לחשבון עלולה לאפשר לתוקף לצפות במידע אישי, לשנות נתונים, להגיש בקשות בשם המשתמש, או במקרה של חשבון אדמין – לגרום נזק נרחב יותר למערכת כולה. בכך, ה-2FA לא רק מגן על פרטיות המשתמשים ושלמות הנתונים, אלא גם מחזק את אמינות המערכת ומונע ניצול לרעה של



חשבונות למטרות זדוניות.

## הגנה על שכבת התעבורה

שכבת התעבורה במודל ה-TCP/IP, ובפרט פרוטוקול TCP - Transmission Control Protocol, אחראית על יצירת חיבור אמין וסדור בין שני יישומים הרצים על מחשבים שונים ברשת. הפרויקט הנוכחי משתמש ב-TCP כבסיס לתקשורת בין הלקוח (network\_client.py) לשרת (network\_server.py).

### 1. פרוטוקול TCP - מאפיינים רלוונטיים לאבטחה:

מבוסס חיבור (Connection-Oriented): לפני העברת נתונים, TCP מבצע "לחיצת יד משולשת" (Three-Way Handshake) כדי לכוון חיבור לוגי בין שני הצדדים. החיבור נשמר עד לסיומו המפורש.

אמינות (Reliability) - הפרוטוקול מבטיח שכל הנתונים שנשלחו יגיעו ליעדם ובסדר הנכון. הוא משתמש במספרי סדר (Sequence Numbers) ובאישורי קבלה (Acknowledgements - ACKs) כדי לעקוב אחר חבילות ולשלוח מחדש חבילות שאבדו או הגיעו פגומות.

בקרת זרימה וגודש (Flow and Congestion Control) - הפרוטוקול מנהל את קצב שליחת הנתונים כדי למנוע הצפה של המקבל או של הרשת.

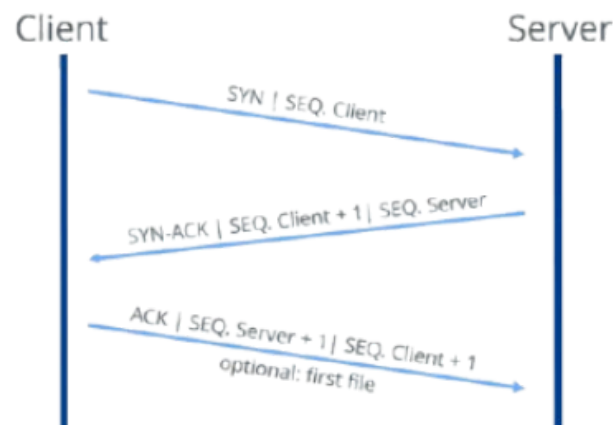
2. לחיצת היד המשולשת (Three-Way Handshake):

תהליך זה חיוני לכינון חיבור TCP:

- שלב 1 (SYN): הלקוח שמעוניין ליצור חיבור שולח לשרת חבילת TCP עם דגל SYN (Synchronize) מורם ומספר סדר התחלתי (Client\_ISN - Initial Sequence Number) אקראי.
- שלב 2 (SYN-ACK): אם השרת מוכן לקבל את החיבור, הוא מגיב בחבילת TCP עם דגלי SYN ו-Ack מורמים. החבילה כוללת מספר סדר התחלתי משלו (Server\_ISN), אישור קבלה (ACK) למספר הסדר של הלקוח ( $Client\_ISN + 1$ ).
- שלב 3 (ACK): הלקוח מקבל את תגובת השרת ושולח חבילת TCP עם דגל ACK מורם, המאשרת את קבלת מספר הסדר של השרת ( $Server\_ISN + 1$ ).

לאחר שלב זה, החיבור כונן ושני הצדדים יכולים להתחיל להעביר נתונים.

## TCP connection establishment ( Three way handshake)



## הצפנה

### מנגנוני הצפנה בפרויקט והגנה על המערכת

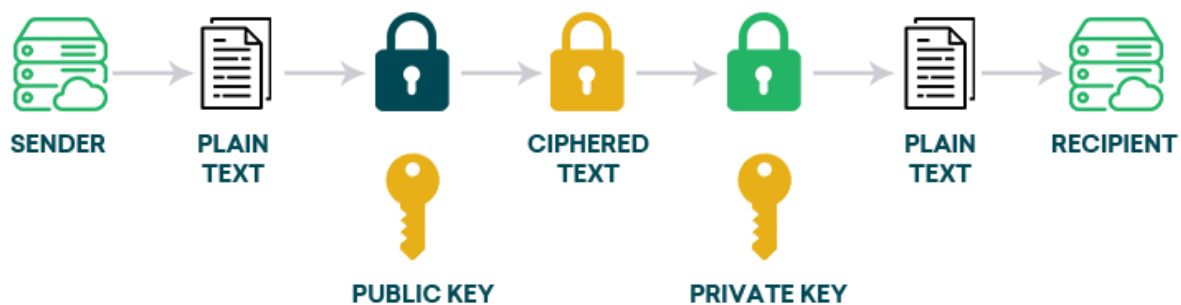
הפרויקט משלב טכניקות הצפנה חיוניות להגנה על סודיות המידע, שלמותו, ואימות. ההצפנה מיושמת הן בתקשורת בין הלקוח לשרת והן באחסון נתונים רגישים במסד הנתונים.

### 1. הצפנת תקשורת בין לקוח לשרת

מטרתה להבטיח שנתונים המועברים ברשת יהיו בלתי קריאים לצד שלישי. זה מושג באמצעות הצפנה היברידית של כלל הנתונים באמצעות שילוב שיטות ההצפנה AES ו-RSA.

כיצד עובדות השיטות?**RSA**

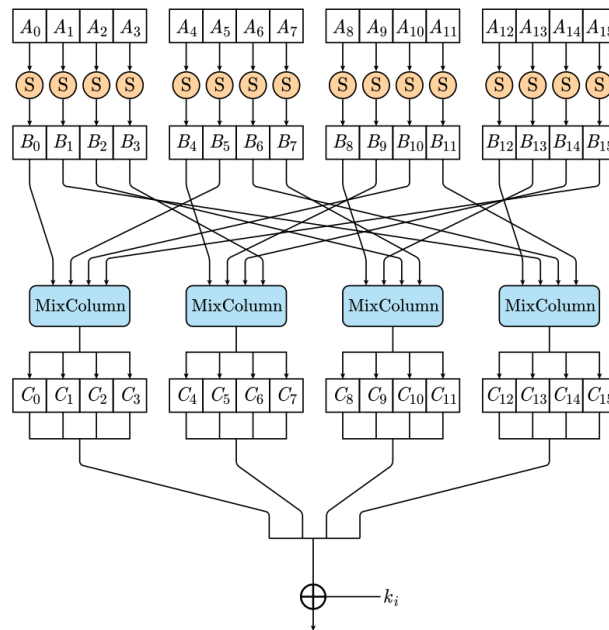
RSA היא שיטת הצפנה שמבוססת על עקרונות מתמטיים פשוטים יחסית אבל עם מספרים מאוד גדולים. זה מתחיל בבחירה של שני מספרים ראשוניים גדולים נסמן אותם כ- $p$  ו- $q$  ומהם מחשבים את המכפלה שלהם מספר שנקרא  $n$  שהוא חלק מהמפתח הציבורי. אחר כך מחשבים גם את פונקציית אוילר של  $n$  מספר שמשמש לבניית המפתח הפרטי אבל נשמר בסוד. בחרים מספר נוסף  $e$  שהוא המפתח הציבורי והוא נבחר כך שאין לו מחלקים משותפים עם פונקציית אוילר של  $n$ . עכשיו יוצרים את המפתח הפרטי  $d$  כך שכאשר מרימים את ההודעה בחזקת  $e$  ואז  $d$  מקבלים את ההודעה המקורית. ההצפנה נעשית על ידי העלאת ההודעה בחזקת  $e$  מודולו  $n$  והפענוח על ידי העלאה בחזקת  $d$  מודולו  $n$ . כל מי שמחזיק את המפתח הפרטי יכול לפענח אבל כדי לחשב אותו מתוך המפתח הציבורי צריך לפרק את  $n$  לגורמים שלו  $p$  ו- $q$  וזה בדיוק מה שהופך את השיטה לבטוחה כי



קשה מאוד לעשות את זה כש- $n$  מספיק גדול.

**AES**

AES היא שיטת הצפנה סימטרית כלומר אותו מפתח משמש גם להצפנה וגם לפענוח. השיטה מבוססת על פעולות מתמטיות בבסיס בינארי על סיביות ופועלת על בלוקים בגודל קבוע של 128 ביט שזה שווה ל 16 בתים. כדי להצפין מחלקים את המידע לבלוקים כאלה וכל בלוק עובר סדרה של שלבים שמערבבים מחליפים ומזיזים את הסיביות לפי סדר מסוים שתלוי במפתח. השלבים כוללים פעולות כמו החלפת בתים בטבלת ערכים קבועה הזזת שורות ערבוב עמודות והוספת מפתח וכל זה חוזר בכמה סבבים 10 12 או 14 תלוי באורך המפתח שיכול להיות 128 או 192 או 256 ביט. הפענוח פשוט מבצע את אותם שלבים בסדר הפוך. AES נחשבת מאוד בטוחה בין היתר כי היא מהירה מאוד גם בחומרה וגם בתוכנה וקשה מאוד לפענח אותה בלי לדעת את המפתח גם בעזרת מחשבים חזקים מאוד. כל עוד המפתח שמור כמו שצריך קשה מאוד לפרוץ את ההצפנה.



### שלבי ההצפנה בפרויקט

#### א. החלפת מפתחות מאובטחת באמצעות RSA (הצפנה אסימטרית):

- השרת מחזיק זוג מפתחות RSA קבוע. הלקוח מייצר זוג RSA ארעי בתחילת סשן
- בתהליך אתחול התקשורת, הלקוח שולח את מפתחו הציבורי לשרת ומקבל את המפתח הציבורי של השרת.
- לאחר מכן, הלקוח מייצר מפתח AES סימטרי חד-פעמי (256 ביט).
- הלקוח מצפין את מפתח ה-AES באמצעות המפתח הציבורי של השרת (עם ריפוד OAEP) ושולח אותו לשרת יחד עם IV ראשוני והבקשה הראשונה מוצפנת AES.
- השרת מפענח את מפתח ה-AES באמצעות מפתחו הפרטי. כעת, לשניהם יש מפתח AES סודי משותף.
- הגנה בשלב זה מבטיחה את סודיות מפתח ה-AES ומספקת אימות חלקי של השרת.

#### ב. הצפנת נתונים שוטפת באמצעות AES (הצפנה סימטרית):

- לאחר החלפת המפתחות, כל התקשורת מוצפנת עם אלגוריתם AES-256 במצב הפעלה CBC.

- לכל הודעה מוצפנת נוצר וקטור אתחול (IV) חדש ואקראי (16 בתים), הנשלח עם הנתונים המוצפנים. זה מבטיח שטקסט מוצפן יהיה שונה גם עבור הודעות זהות.
- לפני הצפנת AES, הנתונים מרופדים באמצעות PKCS#7 להתאמה לגודל הבלוק של AES. תהליך הסרת הריפוד גם בודק את תקינותו.
- הגנה בשלב זה מבטיחה שכל תוכן התקשורת יהיה סודי. שיבוש נתונים במהלך ההעברה יתגלה לרוב.

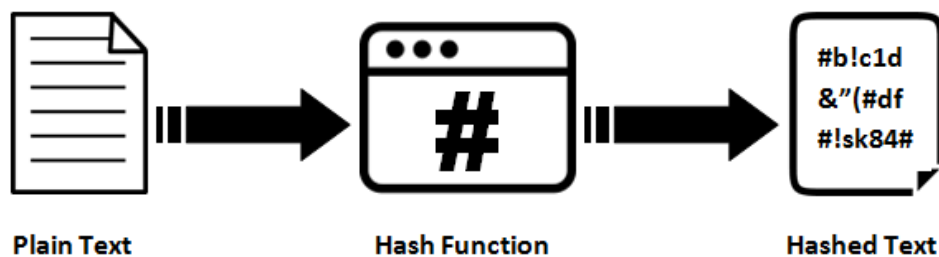
## 2. גיבוב (Hashing) לאחסון סיסמאות ומספרי זהות

אלגוריתם: SHA-256.

אחסון סיסמאות: סיסמאות אינן נשמרות במקור. ערך הגיבוב SHA-256 של הסיסמה נשמר במסד הנתונים. בעת התחברות, הסיסמה המוזנת מגובבת ומושווית לערך השמור.

אחסון מספרי זהות: מספרי זהות מגובבים גם הם עם SHA-256 לפני שמירתם.

הגנה: מונע חשיפת נתונים מקוריים במקרה של פריצה למסד הנתונים. פונקציות גיבוב הן חד-כיווניות.



## תרומת מנגנוני ההצפנה לאבטחת הפרויקט

הגנה מציתות (Eavesdropping): הצפנת AES מונעת קריאת מידע רגיש ברשת.

הגנה מהתקפות (Man-in-the-Middle (MitM): השילוב של RSA ו-AES מקשה על יירוט, פענוח או שינוי התקשורת מבלי שהדבר יתגלה.

הגנה על נתונים במנוחה (Data at Rest): גיבוב סיסמאות ומספרי זהות מגן עליהם אם מסד הנתונים נפרץ.

הבטחת שלמות נתונים: מצב CBC של AES ובדיקת ריפוד PKCS#7 מסייעים בזיהוי שיבוש נתונים.

בניית אמון משתמשים: שימוש בהצפנה חזקה מגביר את אמון המשתמשים באבטחת המידע שלהם.

## מימוש הפרויקט

## סקירת מודלים ומחלקות במערכת

## מודלים ומחלקות מיובאים

| שם המודל                           | תיאור כללי של המודל                                                      | ייעוד בפרויקט של המודל                                                                        |
|------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <u>os</u>                          | מספק פונקציונליות תלויה במערכת ההפעלה, כמו עבודה עם קבצים ונתיבים.       | משמש לבניית נתיבים לקבצי מפתחות, בדיקת קיומם, יצירת נתונים אקראיים (IV), וגישה למשתני סביבה.  |
| <u>socket</u>                      | מאפשר תכנות רשת ברמה נמוכה ליצירת חיבורי TCP/IP.                         | יוצר ומנהל את חיבורי התקשורת (TCP/IP sockets) בין השרת ללקוחות.                               |
| <u>json</u>                        | מאפשר המרה בין אובייקטים של Python למחרוזות בפורמט JSON ולהיפך.          | משמש לסריאליזציה ודה-סריאליזציה של הודעות ובקשות המועברות בין הלקוח לשרת.                     |
| <u>concurrent.futures</u>          | מספק ממשק לביצוע אסינכרוני של משימות באמצעות מאגרי תהליכים או תהליכים.   | ThreadPooExecutor מנהל מאגר תהליכים בשרת לטיפול בחיבורי לקוחות מרובים במקביל.                 |
| <u>time</u>                        | מספק פונקציות הקשורות למדידה וטיפול בזמן.                                | משמש בשרת למנגנון הגבלת קצב הבקשות (Rate Limiting) ולניהול חסימת כתובות IP.                   |
| <u>collections</u>                 | מספק מבני נתונים ייחודיים כמו defaultdict.                               | defaultdict משמש בשרת לתיעוד היסטוריית חיבורים מכתובות IP (עבור Rate Limiting).               |
| <u>חלקים שונים של cryptography</u> | ספרייה מקיפה לפעולות קריפטוגרפיות כמו הצפנה, גיבוב, וטעינת מפתחות.       | משמשת לטעינת מפתחות RSA, הצפנת RSA (להחלפת מפתח AES), הצפנת AES (לתקשורת).                    |
| <u>base64</u>                      | מאפשר קידוד ופענוח נתונים בינאריים ל או מ פורמט Base64 להעברה טקסטואלית. | מקודד ומפענח רכיבי תקשורת מוצפנים (מפתח AES, IV, נתונים) ל/מ Base64 לצורך שילובם ב-JSON.      |
| <u>hashlib</u>                     | מספק אלגוריתמי גיבוב קריפטוגרפיים מאובטחים.                              | משמש לגיבוב (SHA256) של סיסמאות ומספרי זהות לפני שמירתם במסד הנתונים.                         |
| <u>customtkinter</u>               | ספרייה לבניית ממשקי משתמש גרפיים מודרניים המבוססת על Tkinter.            | בונה את כל רכיבי הממשק הגרפי (GUI) של האפליקציה, הן בצד הלקוח הראשי והן בלוח הבקרה של האדמין. |
| <u>tkinter.messagebox</u>          | מודול של Tkinter להצגת דיאלוגים סטנדרטיים למשתמש.                        | מציג הודעות מידע, שגיאה, אזהרה ושאלות אישור למשתמש בממשק הגרפי.                               |

|                                  |                                                                   |                                                                                                   |
|----------------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>חלקים שונים של matplotlib</b> | ספרייה ליצירת גרפים ודיאגרמות דו-ממדיות ותלת-ממדיות.              | משמשת ליצירה והצגה של גרפים סטטיסטיים בלוח הבקרה של האדמין.                                       |
| <b>PIL</b>                       | ספרייה לעיבוד תמונות, המאפשרת פתיחה, שינוי, ושמירה של קבצי תמונה. | טוענת וממירה קבצי תמונה (לוגואים, אייקונים) לשימוש בממשק הגרפי של CustomTkinter.                  |
| <b>tkcalendar</b>                | מספק ווידג'ט לבחירת תאריך עם ממשק לוח שנה עבור Tkinter.           | מאפשר למנהל לבחור תאריכים בעת יצירת מיון חדש או עדכון תאריך לבקשת דחייה.                          |
| <b>datetime</b>                  | מודול ליבה של Python לעבודה עם תאריכים, שעות, והפרשי זמן.         | משמש לניהול חותמות זמן של קודי אימות, בקשות, הודעות, ותאריכים לוגיים במערכת.                      |
| <b>random</b>                    | מספק פונקציות ליצירת מספרים ורצפים אקראיים.                       | משמש ליצירת סיסמאות אקראיות חזקות בעת הוספת משתמש חדש (בצד האדמין).                               |
| <b>string</b>                    | מכיל קבועים שימושיים של מחרוזות (אותיות, ספרות וכו').             | מספק את מאגר התווים ליצירת סיסמאות אקראיות.                                                       |
| <b>sqlite3</b>                   | מספק ממשק למסד הנתונים SQLite המבוסס קבצים.                       | מנהל את כל האינטראקציות עם מסד הנתונים (יצירה, קריאה, עדכון, מחיקה).                              |
| <b>smtplib</b>                   | מממש את פרוטוקול SMTP בצד הלקוח לשליחת דואר אלקטרוני.             | שולח הודעות דוא"ל (קודי אימות להתחברות ואיפוס סיסמא, פרטי משתמש חדש) דרך שרת SMTP חיצוני (Gmail). |
| <b>חלקים שונים של email.mime</b> | מספק כלים ליצירת הודעות דוא"ל בפורמט MIME.                        | בונה את מבנה הודעות הדוא"ל, כולל כותרות ותוכן טקסטואלי.                                           |
| <b>dotenv</b>                    | טוען משתני סביבה מקובץ .env.                                      | טוען פרטי התחברות רגישים (לדוא"ל) מקובץ .env כדי לא לשמור אותם בקוד.                              |
| <b>re</b>                        | מספק תמיכה בביטויים רגולריים לחיפוש והתאמת תבניות טקסט.           | משמש לבדיקת תקינות בסיסית של פורמט כתובות דוא"ל.                                                  |
| <b>threading</b>                 | מאפשר יצירה וניהול של תהליכונים (threads) לביצוע מקבילי.          | משמש להרצת פעולות רשת (בקשות לשרת) בתהליכון נפרד כדי למנוע "קפיאה" של ממשק המשתמש בלקוח.          |

## מודלים ומחלקות בפיתוח אישי שלי

### מודולים

המערכת בנויה ממספר מודולים ברורים, שכל אחד מהם אחראי על רכיב ייחודי בתפקוד הכולל:

| שם המודל                 | תיאור של המודל                                                                                                                                           |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>network_client.py</b> | מודול network_client.py אחראי על ניהול התקשורת של צד הלקוח מול השרת באמצעות פרוטוקול TCP. המודול בנוי כך שכל תקשורת בין ממשק המשתמש לבין השרת תעבור דרכו |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| בלבד, תוך ניהול יציב של חיבורים ושמירה על אבטחת המידע. הוא כולל גם טיפול בשגיאות רשת בצורה מבוקרת, כדי למנוע קריסה של ממשק המשתמש במקרה של ניתוק מהשרת.                                                                                                                                                                                                                                                                                                                                                                               |                                           |
| מודול network_server.py אחראי על צד השרת של המערכת, והוא זה שמאזין לבקשות מהלקוחות ומטפל בהן. הוא פועל כשרת TCP ומבצע את כל תהליך האימות, הפענוח והתגובה לבקשות שמגיעות מהלקוח – בצורה מוצפנת ובטוחה. בנוסף, המודול כולל מנגנוני הגנה כמו הגבלת קצב, חסימת כתובות IP חשודות, וטיימאאוטים לחיבורים לא פעילים. המודול הזה הוא רכיב הליבה בתיווך בין ממשק המשתמש למסד הנתונים.                                                                                                                                                           | <u><a href="#">network_server.py</a></u>  |
| מודול database.py מנהל את כל הפעולות מול מסד הנתונים של המערכת, שהוא SQLite. הוא אחראי על יצירת הטבלאות השונות, וכן על ביצוע כל השאילתות שנדרשות להפעלת הלוגיקה של המערכת. המודול כולל גם פונקציות עזר לניהול המערכת. מתבצעת דרך פונקציה מתוך המודול הזה. השימוש ב-SQLite יחד עם התחברות מבוקרת, מאפשר למערכת לפעול בצורה מהירה, בטוחה ויציבה.                                                                                                                                                                                        | <u><a href="#">database.py</a></u>        |
| מודול admin_dashboard.py אחראי על ממשק הניהול הגרפי של המערכת עבור המשתמשים מסוג אדמין. המודול מממש את כל הפעולות בצד האדמין תוך שימוש בתקשורת עם השרת בלבד, ללא גישה ישירה למסד הנתונים. הוא כולל שימוש נרחב ברכיבי GUI של CustomTkinter לניהול מתקדם. ממשק זה מאפשר שליטה מלאה על המערכת, בצורה אינטואיטיבית ומאובטחת.                                                                                                                                                                                                              | <u><a href="#">admin_dashboard.py</a></u> |
| מודול main.py מהווה את נקודת הכניסה הראשית לממשק המשתמש של המערכת. הוא מנהל את כל הזרימה הגרפית מצד הלקוח. כל פעולה שנעשית מצד המשתמש נשלחת לשרת דרך מודול network_client.py, ואין לממשק גישה ישירה למסד הנתונים. המודול כולל גם התנהגות שונה למשתמשים רגילים ואדמינים, בהתאם להרשאות שנקבעו להם בעת ההתחברות.                                                                                                                                                                                                                        | <u><a href="#">main.py</a></u>            |
| מודול validators.py כולל אוסף של פונקציות עזר לבדיקת תקינות קלטים שמוזנים על ידי המשתמשים במערכת. מטרת המודול היא לוודא שהנתונים שהמשתמש מזין עומדים בדרישות שהוגדרו מראש, ולמנוע הכנסת מידע שגוי או לא תקין למערכת. הפונקציות במודול זה משולבות בכל חלקי הממשק – הן בצד ההתחברות והרישום הראשוני, והן בטפסים למילוי או עריכה של נתונים. השימוש ב-validators מאפשר חוויית משתמש נקייה ויציבה תוך שמירה על אחידות בבדיקות.                                                                                                             | <u><a href="#">validators.py</a></u>      |
| מודול send_email.py אחראי על שליחת הודעות הדוא"ל מהמערכת, והוא משמש בעיקר לשני מצבים מרכזיים: שליחת קוד אימות חד-פעמי (2FA) למשתמש בעת התחברות או איפוס סיסמה, ושליחת פרטי התחברות ראשוניים למשתמש חדש שנוסף על ידי אדמין. המודול משתמש ב-SMTP דרך Gmail, כאשר פרטי ההתחברות לחשבון הדוא"ל מאוחסנים בקובץ env. ומטוענים בצורה מאובטחת. כל שליחה מבוצעת בצורה סינכרונית ומותאמת אישית, כולל טקסט ברור בעברית, בהתאם להקשר שבו מתבצעת השליחה. המודול מבודד משאר רכיבי המערכת, ומאפשר להרחיב או להחליף את שירות הדוא"ל בעתיד בצורה נוחה. | <u><a href="#">send_email.py</a></u>      |
| מודול ui_elements.py מכיל רכיבי GUI מותאמים אישית שממשים את כל ממשק המשתמש בפרויקט. השימוש ברכיבים מתוך מודול זה מבטיח שכל כפתור או חלון במערכת ייראה ויתנהג באותו האופן, ובכך שומר על אחידות וסטנדרט עיצובי.                                                                                                                                                                                                                                                                                                                         | <u><a href="#">ui_elements.py</a></u>     |

## מחלקות

### מחלקת NetworkClient:

מחלקה זו, אחראית על ניהול תקשורת עם השרת – כולל התחברות, שליחת בקשות מוצפנות, קבלת תגובות ופיענוח. היא פועלת בשיטת הצפנה היברידית עבור הצפנה (RSA להצפנת מפתח AES, ו-AES להצפנת התוכן).

תכונות המחלקה:

| שם התכונה           | תיאור התכונה                                                                     |
|---------------------|----------------------------------------------------------------------------------|
| self.socket         | אובייקט TCP socket שמחובר לשרת – משמש לשליחה וקבלה של נתונים.                    |
| self.lock           | מנעול threading.Lock שמבטיח גישה בלעדית לסוקט – כדי למנוע התנגשויות בין תהליכים. |
| self.lock           | מפתח RSA פרטי של הלקוח – משמש לפענוח, נוצר בעת התחברות.                          |
| self.public_key_pem | המפתח הפומבי (public) של הלקוח, בפורמט PEM – נשלח לשרת.                          |
| self.public_key     | המפתח הפומבי של השרת, שנשלף מהשרת ומאוחסן בצד הלקוח – משמש להצפנת AES.           |

פעולות המחלקה:

| שם הפעולה                       | טענת כניסה                                                 | טענת יציאה                                                                                                                                                                                    |
|---------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| __init__(self)                  | לא מקבלת פרמטרים חיצוניים.                                 | יוצרת socket ומתחברת לשרת, מייצרת מפתחות RSA, שולחת את המפתח הפומבי לשרת, ומקבלת את המפתח הפומבי של השרת. אם משהו נכשל, היא מעלה ServerConnectionError.                                       |
| fetch_public_key_internal(self) | לא מקבלת פרמטרים.                                          | שולחת בקשת get_public_key לשרת, שומרת את המפתח הפומבי שהתקבל באובייקט self.public_key. אם התהליך נכשל, היא מעלה ServerConnectionError.                                                        |
| send_raw_request(self, data)    | data – מילון JSON שמייצג בקשה פשוטה (ללא הצפנה).           | שולח את הנתונים לשרת ומחזיר את תגובת השרת כמילון. אם החיבור נכשל או התגובה פגומה, היא מעלה ServerConnectionError.                                                                             |
| send_request(self, data_dict)   | data_dict – מילון JSON עם פרטי הבקשה (type, user_id) וכו'. | מצפין את הנתונים עם AES, מצפין את מפתח ה-AES עם RSA, שולח לשרת, מפענח את התגובה (אם יש), מחזיר אותה כ-dict. במקרה של תקלה, מחזיר מילון עם status: error והודעת שגיאה מתאימה, ולא מעלה חריגות. |
| close(self)                     | לא מקבלת פרמטרים.                                          | סוגרת את החיבור (socket) אם נפתח. לא מחזירה ערך.                                                                                                                                              |

**מחלקת ServerConnectionError:**

זוהי מחלקת חריגה מותאמת אישית (custom exception) שמוגדרת כדי לאותת על כשל בתקשורת עם השרת – למשל כשל בהתחברות, פענוח, או קבלת מפתח.

תכונות המחלקה:

אין תכונות מותאמות אישית. המחלקה יורשת מ-Exception ומתנהגת כמו חריגה רגילה.

| שם הפעולה                                 | טענת כניסה                                                              | טענת יציאה                                                                      |
|-------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>__init__(self, args, kwargs)</code> | מקבלת את אותם פרמטרים כמו Exception רגיל (טקסט שגיאה או אובייקט חריגה). | בונה אובייקט שמכיל את הודעת השגיאה, ומשמש לזריקת חריגות ברורות בעת כשל בתקשורת. |

**מחלקת AdminDashboard:**

מחלקה זו, אחראית על הממשק הגרפי של אדמין במערכת. דרכה ניתן לראות ולבצע את כל פעולות הניהול במערכת. כל הפעולות מתבצעות דרך תקשורת עם השרת.

תכונות המחלקה הראשיות:

| שם התכונה                            | תיאור התכונה                                                                               |
|--------------------------------------|--------------------------------------------------------------------------------------------|
| <code>self.root</code>               | חלון האב של ממשק האדמין (Tk root)                                                          |
| <code>self.user</code>               | מידע על המשתמש המחובר (מילון עם שדות כמו username, is_admin, user_id).                     |
| <code>self.app</code>                | מופע של מחלקת App – דרכו נשלחות בקשות לשרת ( <code>self.app.safe_network_request</code> ). |
| <code>self.icons</code>              | מילון של אובייקטי תמונה (אייקונים לגרפים, משתמשים, לוגו וכו').                             |
| <code>self.on_logout_callback</code> | פונקציה שתיקרא בעת לחיצה על Logout.                                                        |
| <code>self.sidebar</code>            | מסגרת הצד (תפריט ניווט).                                                                   |
| <code>self.content_frame</code>      | אזור התוכן המרכזי שבו מוצגים כל המסכים.                                                    |
| תכונות דינמיות נוספות                | כמו שדות קלט, תיבות טקסט, גרפים – לפי מסך שנבחר                                            |

פעולות המחלקה:

| שם הפעולה                                                        | טענת כניסה                                                                                                               | טענת יציאה                                                                          |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>__init__(self, root, user, app_instance, on_logout)</code> | TK. root – חלון user – מילון עם פרטי המשתמש.<br>app_instance – מופע App.<br>on_logout – פונקציה שתיקרא כשיוצאים מהמערכת. | מאתחלת את ממשק האדמין, טוענת אייקונים, בונה את התפריט הצדדי ואת אזור התוכן הראשוני. |
| <code>load_icons(self)</code>                                    | לא מקבלת פרמטרים.                                                                                                        | טוען את קובצי האייקון ממערכת הקבצים ומחזיר מילון של תמונות <code>CTkImage</code> .  |
| <code>clear_screen(self)</code>                                  | לא מקבלת פרמטרים.                                                                                                        | מוחק את כל ה-widgets מה-root. משמש בעת מעבר מסך.                                    |

|                                                                                 |                                  |                                          |
|---------------------------------------------------------------------------------|----------------------------------|------------------------------------------|
| יצר את תפריט הניווט בצד – כולל כפתורים למעבר בין מסכים.                         | לא מקבלת פרמטרים.                | create_sidebar(self)                     |
| יצר את אזור התוכן המרכזי שבו יוצגו הגרפים, המשתמשים, המיונים וכו'.              | לא מקבלת פרמטרים.                | create_content_frame(self)               |
| מנקה את התוכן מתוך content_frame.                                               | לא מקבלת פרמטרים.                | clear_content(self)                      |
| מציג את מסך הגרפים הסטטיסטיים.                                                  | לא מקבלת פרמטרים.                | show_dashboard(self)                     |
| מציג את מסך ניהול המשתמשים.                                                     | לא מקבלת פרמטרים.                | show_manage_users(self)                  |
| שולף את רשימת המשתמשים מהשרת ומכניס אותם לתוך תיבת הטקסט בטבלת המשתמשים.        | לא מקבלת פרמטרים.                | populate_users_table(self)               |
| שולח בקשה לשרת להוספת משתמש חדש לפי הטופס שמולא. מוודא תקינות מייל ותעודת זהות. | לא מקבלת פרמטרים.                | add_user(self)                           |
| מציג את מסך ניהול נתוני המשתמשים.                                               | לא מקבלת פרמטרים.                | show_manage_user_data(self)              |
| טוען את כל המשתמשים הלא-אדמינים לתפריט הנפתח (ComboBox), לפי חיפוש אם קיים.     | לא מקבלת פרמטרים.                | load_users_to_selector(self)             |
| מפעיל מחדש את load_users_to_selector כדי לרענן את רשימת המשתמשים.               | שינויים בשדה החיפוש.             | update_user_selector(self, args)         |
| מנקה את כל שדות הקלט של עריכת נתוני המשתמש שנבחר.                               | לא מקבלת פרמטרים.                | clear_user_data_fields(self)             |
| שולח בקשה לשרת לטעינת הנתונים של המשתמש הנבחר וממלא את השדות בטופס.             | שם משתמש נבחר מהקומבובוקס.       | load_user_data(self, event=None)         |
| מאפס את ערכי השדות לערכים המקוריים שהגיעו מהשרת.                                | לא מקבלת פרמטרים.                | reset_user_data_fields(self)             |
| בודק תקינות כל השדות, שולח עדכון לשרת עבור המשתמש הנבחר, ומרענן את המידע.       | לא מקבלת פרמטרים.                | save_user_data(self)                     |
| מציג את מסך ניהול המיונים.                                                      | לא מקבלת פרמטרים.                | show_manage_meyunim(self)                |
| שולח בקשה לשרת ליצור מיון חדש עם תיאור, תאריך ורשימת משתמשים.                   | לא מקבלת פרמטרים.                | create_meyun_and_assign(self)            |
| בונה טופס עם שדות למילוי ו-Checkbox של משתמשים לשיבוץ.                          | לשונית גרפית ליצירת מיון.        | build_create_meyun_tab(self, parent_tab) |
| בונה מסך לצפייה בבקשות ותגובה עליהן.                                            | לשונית גרפית לבקשות.             | build_requests_tab(self, parent_tab)     |
| טוען מהשרת את כל הבקשות לפי סינון, מציג אותן ומאפשר אישור/דחייה.                | לא מקבלת פרמטרים.                | refresh_requests_ui(self)                |
| טוען את המיונים מהשרת ומציגם ברשימה – כולל תיאור, משתמשים, ומחיקת מיון.         | לשונית גרפית לכל המיונים שנוצרו. | build_all_meyunim_tab(self, parent_tab)  |

|                                                 |                                         |                                                                        |
|-------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------|
| confirm_delete_meyun(self, meyun_id)            | מזהה מיון.                              | שולח לשרת בקשה למחיקת המיון וכל שיבוציו, עם הודעת אישור מקדימה.        |
| handle_rejection(self, request_id)              | מזהה בקשה.                              | שולח בקשה לעדכון סטטוס לדחייה, ומרענן את רשימת הבקשות.                 |
| handle_approval(self, request_id, request_type) | מזהה בקשה וסוג הבקשה (cancel או delay). | עבור cancel – מוחק את השיבוץ; עבור delay – פותח חלון לבחירת תאריך חדש. |
| show_user_messages(self)                        | לא מקבלת פרמטרים.                       | מציג את כל ההודעות שהתקבלו ממשתמשים כולל אפשרות מענה.                  |

**מחלקת NewDatePopup:**

מחלקה זו, מייצגת חלון קופץ (popup) שמאפשר לאדמין לבחור תאריך חדש לבקשה לדחיית מיון. היא יורשת מ-BasePopup, ומציגה רכיב לוח שנה (DateEntry) וכפתור לשליחת העדכון לשרת.

תכונות המחלקה:

| שם התכונה                   | תיאור התכונה                                        |
|-----------------------------|-----------------------------------------------------|
| self.request_id             | מזהה הבקשה לדחייה שנבחרה.                           |
| self.on_success_callback    | פונקציה שתרוץ לאחר עדכון מוצלח – לרוב ריענון תצוגה. |
| self.safe_request           | הפניה לפונקציה מתוך App לשליחת בקשות מאובטחות לשרת. |
| self.critical_error_handler | פונקציה שתופעל אם מתגלה שגיאת שרת קריטית.           |
| self.date_picker            | רכיב לבחירת תאריך מסוג DateEntry.                   |

פעולות המחלקה:

| שם הפעולה                                                                                                           | טענת כניסה                                                                                                                                                                                                                                                                | טענת יציאה                                                                  |
|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| init__(self, master, __request_id, on_success_callback, safe_request_method, critical_error_handler, (args, kwargs) | master – חלון האב של הפופאפ<br>request_id – מזהה הבקשה לדחייה<br>on_success_callback – פונקציה להפעלה במקרה של הצלחה<br>safe_request_method – פונקציה למשלוח בקשות<br>critical_error_handler – פונקציה לניהול שגיאות קריטיות<br>args, kwargs – פרמטרים נוספים ל-BasePopup | בונה את חלון הפופאפ, מגדיר רכיב בחירת תאריך, ומקשר את הכפתור לפעולת השליחה. |

|                                                                                                                                                                                             |                   |                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------|
| שולפת את התאריך שנבחר, שולחת בקשת update_date_for_request לשרת עם מזהה הבקשה והתאריך החדש. אם הבקשה מצליחה – מוצגת הודעת הצלחה ונקראת הפונקציה on_success_callback. אחרת מוצגת הודעת שגיאה. | לא מקבלת פרמטרים. | save_date(self) |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------|

**מחלקת App:**

מחלקה זו מנהלת את כל ממשק המשתמש של הצד הלקוח. היא אחראית על התחברות, אימות דו-שלבי, הצגת מסכים למשתמש רגיל או אדמין, שליחת בקשות לשרת, קבלת תגובות, טיפול בשגיאות רשת, וניהול זמן הפעולה של כל ששן. כל הפעולה במערכת – החל מהתחברות ועד הודעה למנהל – עוברת דרכה.

תכונות המחלקה:

| שם התכונה                     | תיאור התכונה                                            |
|-------------------------------|---------------------------------------------------------|
| self.master                   | חלון ה-Tk הראשי של האפליקציה.                           |
| self.network                  | מופע של NetworkClient, דרכו מתבצעת כל תקשורת עם השרת.   |
| self.server_available         | מציין אם הייתה התחברות מוצלחת לשרת בעת האתחול.          |
| self.server_error_active      | דגל למניעת ריבוי טיפול בשגיאה רשתית (ברגע שהיא מתרחשת). |
| self.session_timer            | מזהה של טיימר לאיפוס אוטומטי לאחר 10 דקות.              |
| self.session_duration         | משך הסשן בשניות (600 = עשר דקות).                       |
| self.current_frame            | הפניה למסך הנוכחי שמוצג (Frame פעיל).                   |
| self.pending_user             | מידע זמני על המשתמש שמנסה להתחבר, לפני קוד אימות.       |
| self.pending_code_context     | מציין אם הקוד שנשלח הוא להתחברות או לשחזור סיסמה.       |
| self.current_user_id_session  | מזהה המשתמש המחובר כרגע.                                |
| self.current_username_session | שם המשתמש המחובר כרגע.                                  |
| תכונות דינמיות נוספות         | כמו שדות קלט, תיבות טקסט וכו'.                          |

פעולות המחלקה:

| שם הפעולה              | טענת כניסה                      | טענת יציאה                                                                         |
|------------------------|---------------------------------|------------------------------------------------------------------------------------|
| __init__(self, master) | master: מופע Tk של החלון הראשי. | מאתחלת את המערכת הגרפית, יוצרת חיבור לשרת דרך NetworkClient, טוענת את מסך ההתחברות |

|                                                                                                                                                                         |                                                                                                                    |                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| ומפעילה טיימר ששן של 10 דקות. אם השרת לא זמין – מוצג חלון שגיאה.                                                                                                        |                                                                                                                    |                                                                                                       |
| מפסיק את הסשן, מנתק את החיבור לשרת, מנקה את החלון הראשי, ומציג popup שמתריע על קריסת תקשורת.                                                                            | <code>error_source_message</code> : מחרוזת שמתארת מאיפה השגיאה נוצרה.                                              | <code>handle_server_critical_error(self, error_source_message="")</code>                              |
| מנסה לשלוח את הבקשה לשרת דרך <code>self.network</code> , ומחזירה את התשובה. אם יש שגיאת תקשורת – מופעל מנגנון כשל קריטי.                                                | <code>request_data</code> : מילון עם בקשה לשרת.<br><code>source_description</code> : תיאור טקסטואלי של מקור הבקשה. | <code>safe_network_request(self, request_data, source_description="")</code>                          |
| מפעיל טיימר של 10 דקות (600 שניות) שבעקבותיהן תתבצע ניתוק אוטומטי מהמערכת.                                                                                              | לא מקבלת פרמטרים.                                                                                                  | <code>start_session_timer(self)</code>                                                                |
| מאפס את הטיימר של הסשן – מאריך את הזמן לעוד 10 דקות.                                                                                                                    | לא מקבלת פרמטרים.                                                                                                  | <code>reset_session_timer(self)</code>                                                                |
| בודק אם יש משתמש מחובב, ואם כן – מציג הודעה על סיום סשן ומחזיר למסך התחברות.                                                                                            | לא מקבלת פרמטרים.                                                                                                  | <code>handle_session_timeout(self)</code>                                                             |
| מנתק את המשתמש הנוכחי, מאפס משתני סשן, ומציג מחדש את מסך ההתחברות.                                                                                                      | לא מקבלת פרמטרים.                                                                                                  | <code>logout_and_show_login(self)</code>                                                              |
| סוגר את הסשן, מבטל את הטיימר, וסוגר את התוכנה (לרוב בלחיצה על X).                                                                                                       | לא מקבלת פרמטרים.                                                                                                  | <code>on_closing(self)</code>                                                                         |
| מוחק את כל ה- <code>widgets</code> מהמסך הראשי ומאפס משתנים גרפיים של המסך.                                                                                             | לא מקבלת פרמטרים.                                                                                                  | <code>clear_screen(self)</code>                                                                       |
| בונה את ממשק ההתחברות הגרפי – כולל שדות משתמש וסיסמה, כפתור התחברות, וכפתור שכחתי סיסמה.                                                                                | לא מקבלת פרמטרים.                                                                                                  | <code>create_login_screen(self)</code>                                                                |
| הופך את מצב התצוגה של שדה הסיסמה (גלוי ↔ מוסתר) ומעדכן את האייקון בהתאם.                                                                                                | לא מקבלת פרמטרים.                                                                                                  | <code>toggle_password_visibility(self)</code>                                                         |
| בודק האם שדות המשתמש והסיסמה מולאו; אם כן – מפעיל תהליך התחברות (ב- <code>Thread</code> ).                                                                              | לא מקבלת פרמטרים.                                                                                                  | <code>validate_login(self)</code>                                                                     |
| שולח בקשת <code>login</code> לשרת. אם ההתחברות מוצלחת – שומר פרטי משתמש, ושולח בקשת <code>send_code_to_user</code> במידה והמשתמש חדש – שולח אותו למסך "התחברות ראשונה". | <code>username</code> : שם המשתמש.<br><code>password</code> : סיסמה.                                               | <code>process_login(self, username, password)</code>                                                  |
| מציג טופס השלמת פרטים ראשוניים: ת"ז, טלפון, כתובת, גיל וסיסמה חדשה.                                                                                                     | לא מקבלת פרמטרים.                                                                                                  | <code>create_first_login_screen(self)</code>                                                          |
| בודק את שדות הטופס לפי כללי ולידציה, ומפעיל את <code>process_submit_first_login</code> ב- <code>Thread</code> .                                                         | לא מקבלת פרמטרים.                                                                                                  | <code>submit_first_login_info(self)</code>                                                            |
| שולח לשרת את הבקשה <code>complete_first_login</code> . אם הצליח – מחזיר למסך ההתחברות.                                                                                  | נתוני המשתמש כפי שהוזנו בטופס ההשלמה.                                                                              | <code>_process_submit_first_login(self, user_id, id_number, phone, address, age, new_password)</code> |

|                                                                                           |                                                               |                                                           |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------|
| מציג טופס להזנת קוד אימות (2FA) שהתקבל במייל.                                             | לא מקבלת פרמטרים.                                             | create_code_verification_screen(self)                     |
| בודק אם הוזן קוד חוקי – אם כן, שולח בקשה לשרת ומפעיל תהליך אימות ב-Thread.                | לא מקבלת פרמטרים.                                             | verify_code_action(self)                                  |
| שולח לשרת את הבקשה verify_code. אם הצליח – מפעיל את handle_successful_code_verification_. | user_id: מזהה המשתמש.<br>code: קוד אימות שהוזן.               | _process_verify_code(self, user_id, code)                 |
| אם המשתמש אדמין – פותח את AdminDashboard. אם רגיל – עובר למסך בחירה בין נתונים למיונים.   | לא מקבלת פרמטרים.                                             | _handle_successful_code_verification(self)                |
| מציג תפריט ראשי למשתמש רגיל – עם אפשרויות לצפייה בנתונים אישיים, מיונים, בקשות והודעות.   | user_id: מזהה המשתמש.<br>username: שם המשתמש.                 | show_user_dashboard_options(self, user_id, username)      |
| יוזם Thread שנועד להביא את ההתראות האחרונות של המשתמש מהשרת.                              | user_id_to_fetch: מזהה המשתמש.                                | _fetch_user_notifications(self, user_id_to_fetch)         |
| שולח בקשת get_user_notifications לשרת. אם הצליח – מציג התראות במסך המשתמש.                | uid: מזהה המשתמש.                                             | _send_fetch_notifications_request(self, uid)              |
| מציג את רשימת המיונים של המשתמש, כולל תיאור, סטטוס, ובאפשרות שליחת בקשות דחייה או ביטול.  | user_id: מזהה המשתמש.<br>username: שם המשתמש.                 | create_user_dashboard(self, user_id, username)            |
| מציג את נתוני המשתמש בממשק כרטיסים (כמו דפ"ר, פרופיל, עברית וכו').                        | user_id: מזהה המשתמש.<br>username: שם המשתמש.                 | create_user_data_dashboard(self, user_id, username)       |
| מציג טופס שליחת הודעה למנהל + היסטוריית הודעות שנשלחו עם תגובות.                          | user_id: מזהה המשתמש.<br>username: שם המשתמש.                 | create_user_messages_screen(self, user_id, username)      |
| אוסף את שדות ההודעה מהטופס, שולח לשרת ב-Thread, ומעדכן תצוגת הפלט.                        | user_id_for_msg: מזהה המשתמש.                                 | submit_user_message_action(self, user_id_for_msg)         |
| שולח בקשת send_message לשרת. אם הצליח – מנקה את השדות ומרענן את ההיסטוריה.                | uid: מזהה המשתמש.<br>subj: נושא ההודעה.<br>cont: תוכן ההודעה. | _process_submit_user_message(self, uid, subj, cont)       |
| שולח בקשת get_messages לשרת ומציג את ההודעות במסך.                                        | user_id_to_load: מזהה המשתמש.                                 | _load_user_message_history_display(self, user_id_to_load) |
| מציג את כל בקשות העבר של המשתמש, כולל סוג, סטטוס, תוכן ותאריך.                            | user_id: מזהה המשתמש.<br>username: שם המשתמש.                 | show_request_history(self, user_id, username)             |

|                                                                                 |                                                                                |                                                              |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------|
| מציג טופס להזנת שם משתמש לשליחת קוד שחזור סיסמה.                                | לא מקבלת פרמטרים.                                                              | open_forgot_password_screen(self)                            |
| קורא ל- process_send_reset_code ב-Thread כדי לשלוח קוד שחזור לכתובת מייל.       | לא מקבלת פרמטרים.                                                              | send_reset_code_thread(self)                                 |
| שולח בקשת request_reset_code לשרת. אם התקבל אישור – עובר למסך אימות קוד.        | שם: username<br>המשתמש.                                                        | _process_send_reset_code(self, username)                     |
| מציג שדה להזנת סיסמה חדשה והפעלת כפתור איפוס.                                   | שם: username<br>המשתמש.<br>email: כתובת האימייל<br>המשויכת.                    | create_new_password_screen(self, username, email)            |
| שולח את הבקשה לאיפוס סיסמה ב-Thread – לאחר בדיקת תקינות בסיסית של הסיסמה החדשה. | שם: username<br>המשתמש.<br>email: כתובת האימייל.                               | reset_password_action_thread(self, username, email)          |
| שולח בקשת reset_password לשרת. אם הצליח – מחזיר למסך ההתחברות עם הודעת הצלחה.   | שם: username<br>המשתמש.<br>email: כתובת אימייל.<br>new_password: הסיסמה החדשה. | _process_reset_password(self, username, email, new_password) |

**מחלקת SubmitRequestPopup:**

מחלקה זו מייצגת חלון קופץ (popup) שמאפשר למשתמש רגיל לשלוח בקשה לדחיית מיון או ביטול מיון. היא מופיעה מתוך מסך המיונים, ומאפשרת להזין הודעה ולשלוח אותה לשרת.

תכונות המחלקה:

| שם התכונה                   | תיאור התכונה                                               |
|-----------------------------|------------------------------------------------------------|
| self.assignment_id          | מזהה השיבוץ שאליו מתייחסת הבקשה.                           |
| self.request_type           | סוג הבקשה – "delay" או "cancel".                           |
| self.safe_request           | פונקציית שליחה לשרת מתוך מחלקת App.                        |
| self.user_id_for_context    | מזהה המשתמש השולח את הבקשה – נדרש לעדכון המסך לאחר השליחה. |
| self.username_for_context   | שם המשתמש – משמש בתצוגה לאחר השליחה.                       |
| self.on_success_callback    | פונקציה שתופעל לאחר שליחה מוצלחת – לרוב ריענון המיונים.    |
| self.critical_error_handler | פונקציה שתופעל במקרה של שגיאת תקשורת חמורה.                |
| self.message_entry          | תיבת טקסט להזנת תוכן הבקשה מצד המשתמש.                     |

| שם הפעולה                                                                                                                                                                                                      | טענת כניסה                                                                                                                                                                                                                                                                                                                                                                                                                            | טענת יציאה                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| init__(self, master, __<br>assignment_id, request_type,<br>safe_request_method,<br>user_id_for_context,<br>username_for_context,<br>on_success_callback,<br>critical_error_handler_callback<br>(, args, kwargs | master: חלון האב של הפופאפ.<br>assignment_id: מזהה השיבוץ שאליו<br>מתייחסת הבקשה.<br>request_type: "delay" או "cancel".<br>safe_request_method: פונקציית שליחה<br>מאובטחת לשרת.<br>user_id_for_context: מזהה המשתמש.<br>username_for_context: שם המשתמש.<br>on_success_callback: פונקציה שתפעל<br>אם הבקשה נשלחה בהצלחה.<br>critical_error_handler_callback:<br>פונקציה שתפעל במקרה של כשל חמור.<br>args, kwargs: מועברים למחלקת-האם. | יוצר את חלון הפופאפ עם שדה<br>להזנת הודעה וכפתור שליחה,<br>ושומר את ההקשרים הדרושים.                                                                                                                            |
| submit(self)                                                                                                                                                                                                   | לא מקבלת פרמטרים.                                                                                                                                                                                                                                                                                                                                                                                                                     | קורא את תוכן תיבת הטקסט,<br>מוודא שאינה ריקה, ושולח בקשה<br>מסוג submit_request לשרת עם<br>פרטי הבקשה. אם נשלחה<br>בהצלחה – סוגר את הפופאפ<br>ומפעיל את<br>on_success_callback. אם<br>נכשלה – מציג הודעת שגיאה. |

**מחלקת MyButton:**

מחלקה זו מרחיבה את CTkButton מספריית customtkinter, ומגדירה כפתור מעוצב במראה אחיד למערכת כולה. כל הכפתורים במערכת – גם למשתמשים רגילים וגם לאדמינים – מבוססים על מחלקה זו.

המחלקה אינה מוסיפה תכונות חדשות מעבר למה שמוגדר ב-CTkButton, אבל היא כן מגדירה ערכים ברירת מחדל בעת יצירה:

| תכונה אוטומטית | ערך ברירת מחדל             |
|----------------|----------------------------|
| fg_color       | "#004d00" (ירוק כהה)       |
| hover_color    | "#006400" (ירוק בהיר יותר) |
| text_color     | "white"                    |
| height         | 40                         |

| שם הפעולה                                 | טענת כניסה                                                                       | טענת יציאה                                                                                                            |
|-------------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>__init__(self, args, kwargs)</code> | <code>args, kwargs</code> : כל הפרמטרים שניתן להעביר ל- <code>CTkButton</code> . | מגדיר את הכפתור עם ערכי עיצוב ברירת מחדל אם לא הוגדרו, ואז מפעיל את <code>__init__</code> של <code>CTkButton</code> . |

### מחלקת `BasePopup`:

מחלקה זו מרחיבה את `CTkToplevel` (חלון קופץ עצמאי) ומספקת בסיס מעוצב לחלונות `popup` במערכת. היא כוללת הגדרות מראה, מיקום במסך, טיפול באייקון, הפעלת מצב מודאלי, וניהול סגירה.

תכונות המחלקה:

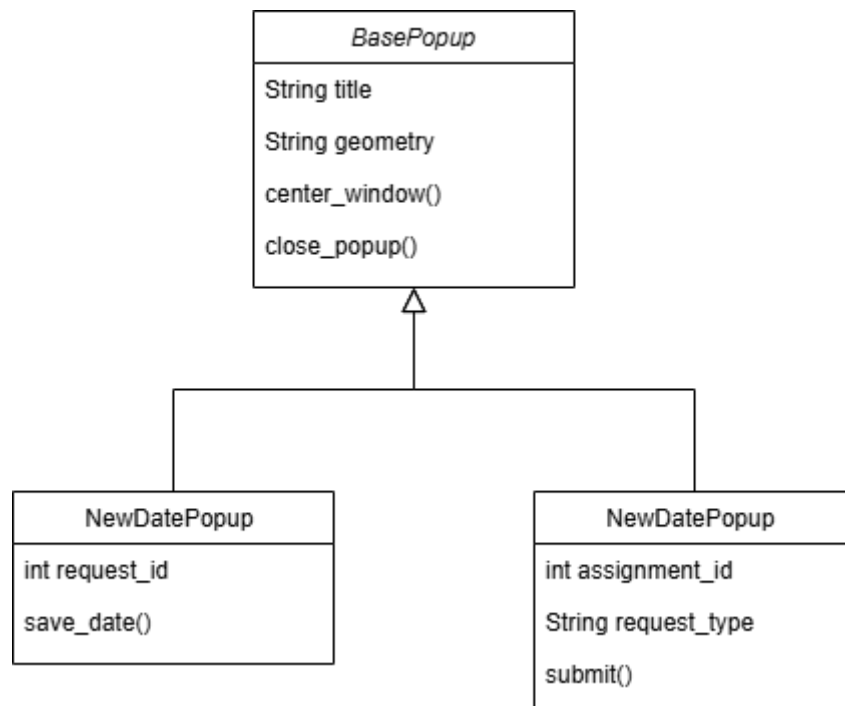
| שם התכונה                          | תיאור התכונה                                                                                                                              |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>self</code> (עצמה)           | כילה את חלון הפופאפ הפעיל ( <code>CTkToplevel</code> ). אין תכונות מותאמות נוספות באובייקט עצמו, אבל ההגדרות שלה משפיעות מיידית על הממשק. |
| רקע ( <code>fg_color</code> )      | מוגדר כברירת מחדל ל- <code>#E8F5E9</code> .                                                                                               |
| אייקון ( <code>iconbitmap</code> ) | מוגדר ל- <code>ico_logo.ico</code> אם הקובץ קיים.                                                                                         |
| תצוגה מודאלית                      | הפעלת <code>grab_set</code> ו- <code>focus_force</code> כדי שהחלון יהיה בחזית ויחסום פעולה מחוץ לו.                                       |
| מיקום                              | החלון ממרכז אוטומטית במסך באמצעות <code>center_window</code> .                                                                            |

פעולות המחלקה:

| שם הפעולה                                                                          | טענת כניסה                                                                                                                                                                                                                     | טענת יציאה                                                                                     |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>init__(self, master, title="Popup", geometry="500x350", args, kwargs)</code> | <code>master</code> : חלון האב של הפופאפ.<br><code>title</code> : כותרת החלון (ברירת מחדל: "Popup").<br><code>geometry</code> : גודל ברירת מחדל של החלון.<br><code>args, kwargs</code> : מועברים ל- <code>CTkToplevel</code> . | יוצר חלון קופץ עם הגדרות עיצוב, מיקום אוטומטי, אייקון, ומצב מודאלי.                            |
| <code>center_window(self)</code>                                                   | לא מקבלת פרמטרים.                                                                                                                                                                                                              | מחשב את מיקום החלון כך שיופיע במרכז המסך (מעט גבוה יותר), ומעדכן את הגיאומטריה של החלון בהתאם. |

|                                                |                   |                   |
|------------------------------------------------|-------------------|-------------------|
| מסיר את ה־grab מהחלון וסוגר אותו<br>(destroy). | לא מקבלת פרמטרים. | close_popup(self) |
|------------------------------------------------|-------------------|-------------------|

### תרשים של מחלקות בפיתוח אישי שלי



## בעיות אלגוריתמיות מרכזיות בפרויקט, קטעי קוד מיוחדים ופתרונות מעניינים

### מנגנון הצפנה משולב (RSA + AES) לתקשורת מאובטחת

הסבר:

המערכת מממשת פרוטוקול הצפנה ייחודי ברמת היישום כדי לאבטח את התקשורת בין הלקוח לשרת. התהליך מתחיל בהחלפת מפתחות ציבוריים RSA בין הצדדים. לאחר מכן, הלקוח מייצר מפתח AES סימטרי חד-פעמי (מפתח סשן), מצפין אותו באמצעות המפתח הציבורי של השרת, ושולח אותו לשרת. השרת מפענח את מפתח ה-AES באמצעות המפתח הפרטי שלו. מרגע זה, כל התקשורת השוטפת (בקשות ותגובות) מוצפנת באמצעות מפתח ה-AES המשותף, תוך שימוש

בוקטור אתחול (IV) ייחודי לכל הודעה ובריפוד PKCS#7. מנגנון זה מבטיח סודיות ושלמות בסיסית של הנתונים המועברים. להרחבה ניתן לעיין בהבסבר שנכתב על כך בנושא האלגוריתמים המרכזיים.

קטעי קוד מרכזיים:

`network\_client.py` שלחת בקשה מוצפנת (בתוך - `network\_client.py`):

```
```python
# ... (לאחר בדיקות ראשוניות, send_request בתוך)
aes_key = os.urandom(32)
request_iv = os.urandom(16)
json_data_bytes = json.dumps(data_dict).encode()

cipher_encrypt = Cipher(algorithms.AES(aes_key), modes.CBC(request_iv),
    backend=default_backend())

encryptor = cipher_encrypt.encryptor()

padded_data = pkcs7_pad(json_data_bytes)

encrypted_data = encryptor.update(padded_data) + encryptor.finalize()

encrypted_key = self.public_key.encrypt( # self.public_key הוא המפתח הציבורי
של השרת
    aes_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

payload = json.dumps({
    "key": base64.b64encode(encrypted_key).decode(),
```

```

    "iv": base64.b64encode(request_iv).decode(),
    "data": base64.b64encode(encrypted_data).decode()
}).encode()

# ... (וקבלת תגובה payload שליחת) ...

# aes_key פענוח תגובה מהשרת (מוצפנת עם אותו)
response_json = json.loads(response_bytes.decode())
if "iv" in response_json and "data" in response_json:
    response_iv = base64.b64decode(response_json["iv"])
    encrypted_response_data = base64.b64decode(response_json["data"])

    cipher_decrypt = Cipher(algorithms.AES(aes_key),
                             modes.CBC(response_iv), backend=default_backend())

    decryptor = cipher_decrypt.decryptor()

    padded_decrypted_data = decryptor.update(encrypted_response_data) +
    decryptor.finalize()

    decrypted_data = pkcs7_unpad(padded_decrypted_data)

    return json.loads(decrypted_data.decode())
...

`network_server.py` - (בתוך - `handle_client` קבלת ופענוח בקשה מוצפנת):
```python
# ... (while True של handle_client בתוך לולאת)

if is_encrypted_payload:
    # ... (קיים [ip] client_public_keys בדיקה אם)
    try:

```

```

payload = json.loads(request_json_str)
encrypted_aes_key_b64 = payload["key"]
request_iv_b64 = payload["iv"]
encrypted_data_b64 = payload["data"]

encrypted_aes_key = base64.b64decode(encrypted_aes_key_b64)
request_iv = base64.b64decode(request_iv_b64)
encrypted_data = base64.b64decode(encrypted_data_b64)

aes_key_session = PRIVATE_KEY.decrypt( # PRIVATE_KEY הוא המפתח הפרטי של השרת
    encrypted_aes_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

cipher_decrypt = Cipher(algorithms.AES(aes_key_session),
    modes.CBC(request_iv), backend=default_backend())

decryptor = cipher_decrypt.decryptor()

padded_decrypted_data = decryptor.update(encrypted_data) +
    decryptor.finalize()

request_data_bytes = pkcs7_unpad(padded_decrypted_data)

request_dict = json.loads(request_data_bytes.decode())

except (json.JSONDecodeError, KeyError, base64.binascii.Error,
    ValueError) as e_dec:

    # ... טיפול בשגיאות פענוח ...

```

...

וקריאה `encrypt_response_with_aes` הצפנת תגובה (פונקציה - `network_server.py` - `handle_client`):

```
python

def encrypt_response_with_aes(response_dict, aes_key):

    response_iv = os.urandom(16)

    cipher_encrypt = Cipher(algorithms.AES(aes_key),
                             modes.CBC(response_iv), backend=default_backend())

    encryptor = cipher_encrypt.encryptor()

    json_data_bytes = json.dumps(response_dict).encode("utf-8")

    padded_data = pkcs7_pad(json_data_bytes)

    encrypted_data = encryptor.update(padded_data) + encryptor.finalize()

    return {

        "iv": base64.b64encode(response_iv).decode("utf-8"),

        "data": base64.b64encode(encrypted_data).decode("utf-8")

    }
```

## ניהול ששנים מאובטח וטיפול בחיבורים מרובים

### הסבר:

השרת מתוכנן לטפל במספר לקוחות במקביל באמצעות `ThreadPoolExecutor`. עבור כל חיבור לקוח, השרת מנהל ששן הצפנת AES ייחודי. בנוסף, מיושם מנגנון הגבלת קצב בקשות (Rate Limiting) בסיסי: השרת עוקב אחר מספר החיבורים מכל כתובת IP בפרק זמן נתון, ואם מספר זה חורג מסף מוגדר, כתובת ה-IP נחסמת לפרק זמן קצוב כדי למנוע העמסת יתר פשוטה.

### קטעי קוד מרכזיים:

`network_server.py` - מנגנון Rate Limiting (בתחילת `handle_client`):

```
python
```

```
[ip = addr[0
```

```

()now = time.time

:[if ip in blocked_ips and now < blocked_ips[ip]
(".print(f"[BLOCKED] {ip} is currently blocked. Connection refused
()conn.close
return
:[elif ip in blocked_ips and now >= blocked_ips[ip]
[del blocked_ips[ip]
(".print(f"[UNBLOCKED] {ip} is no longer blocked

[connection_log[ip] = [t for t in connection_log[ip] if now - t < RATE_WINDOW
(connection_log[ip].append(now

:[if len(connection_log[ip]) > RATE_LIMIT
print(f"[DDOS PROTECTION] Blocking IP {ip} for {BLOCK_TIME} seconds
(".due to high request rate
blocked_ips[ip] = now + BLOCK_TIME
[if ip in connection_log: del connection_log[ip]
()conn.close
return
...

:(`start_server`-ב) `ThreadPoolExecutor`-שימוש ב- `network_server.py`
python```
(executor = ThreadPoolExecutor(max_workers=10
... #
:try
91

```

```

:while True

:try

(conn, addr = server.accept

(executor.submit(handle_client, conn, addr

... #

... #

:finally

... #

(executor.shutdown(wait=True

```

## לוגיקת הטיפול המורכבת בבקשות מיונים ועדכון

### הסבר:

המערכת מאפשרת למשתמשים להגיש בקשות לדחייה או ביטול של מיונים אליהם שובצו, ולמנהלים לאשר או לדחות בקשות אלו. אישור בקשת ביטול מסיר את שיבוץ המשתמש מהמיון. אישור בקשת דחייה מצריך מהמנהל לבחור תאריך חדש, והמערכת מעדכנת את תאריך המיון הראשי (ולא רק את השיבוץ הספציפי) ואת סטטוס הבקשה. הלוגיקה כוללת אינטראקציה עם מספר טבלאות במסד הנתונים ('meyunim', 'meyunim\_assignments', 'meyunim\_requests') כדי לשמור על עקביות הנתונים.

### קטעי קוד מרכזיים:

```

(`network_server.py` - טיפול בבקשת אישור ביטול (בתוך `handle_request`):

python```

:"elif rtype == "approve_cancel_request

("request_id_cancel = request.get("request_id

(conn_db = connect_db

```

```

()cursor = conn_db.cursor

:try

cursor.execute("SELECT assignment_id FROM meyunim_requests
((,WHERE id=?", (request_id_cancel

()assign_row = cursor.fetchone

:if not assign_row

return {"status": "error", "message": "Request not found or no
{"assignment linked

[assignment_id_to_delete = assign_row[0

:if assignment_id_to_delete

cursor.execute("DELETE FROM meyunim_assignments WHERE
((,id=?", (assignment_id_to_delete

cursor.execute("UPDATE meyunim_requests SET status="Approved"
((,WHERE id=?", (request_id_cancel

()conn_db.commit

return {"status": "ok", "message": "Cancel request approved, assignment
{"removed

(טיפול בשגיאות וסגירת חיבור) ... #
...

(`network_server.py` - טיפול בבקשת עדכון תאריך (בתוך `handle_request`
python``

:"elif rtype == "update_date_for_request

("request_id_delay = request.get("request_id

("new_date_delay = request.get("new_date

# (success = update_date_for_request(request_id_delay, new_date_delay
קריאה לפונקציה מ-database.py

```

```

if success: return {"status": "ok", "message": "Delay request approved and
{"date updated

else: return {"status": "error", "message": "Failed to update date for delay
{"request

...

```

update\_date\_for\_request` פונקציית `database.py`

python``

```

:(def update_date_for_request(request_id, new_date

```

```

()conn = connect_db

```

```

:if conn

```

```

()cursor = conn.cursor

```

```

:try

```

# נביא את השיבוץ מתוך הבקשה

```

cursor.execute("SELECT assignment_id FROM meyunim_requests
(,WHERE id=?", (request_id

```

```

()row = cursor.fetchone

```

```

if not row: return False

```

```

[assignment_id = row[0

```

# נביא את מזהה המיון מתוך השיבוץ

```

cursor.execute("SELECT meyun_id FROM meyunim_assignments
(,WHERE id=?", (assignment_id

```

```

()row = cursor.fetchone

```

```

if not row: return False

```

```

[meyun_id = row[0

```

# נעדכן את תאריך המיון

```
cursor.execute("UPDATE meyunim SET date=? WHERE id=?",
(((new_date, meyun_id
```

# נעדכן את סטטוס הבקשה ל-Approved

```
cursor.execute("UPDATE meyunim_requests SET status='Approved'
(,WHERE id=?", (request_id
```

```
()conn.commit
```

```
return True
```

# ... (טיפול בשגיאות וסגירת חיבור)

```
return False
```

```
...
```

`admin\_dashboard.py` - קריאה לטיפול באישור בקשה (בתוך  
:('AdminDashboard.handle\_approval'

```
python'''
```

```
... #
```

```
:"elif request_type == "delay
```

```
,NewDatePopup(self.root, request_id
```

```
,on_success_callback=self.refresh_requests_ui
```

```
,safe_request_method=self.app.safe_network_request
```

```
(critical_error_handler=self.app.handle_server_critical_error
```

```
...
```

והלוגיקה בתוך `NewDatePopup.save\_date` ששולחת בקשת  
`update\_date\_for\_request` לשרת.

**מנגנון ההתראות למשתמש**

**הסבר:**

השרת מספק נקודת קצה (`get\_user\_notifications`) שאוספת עבור משתמש ספציפי התראות רלוונטיות ממספר מקורות: תזכורות על מיונים קרובים, עדכונים על סטטוס בקשות שהוגשו (כגון "אושר" או "נדחה"), ותגובות ממנהל המערכת להודעות שהמשתמש שלח. לאחר שליפת ההתראות, הן מסומנות במסד הנתונים כהתראות שהמשתמש קיבל (`notified\_user = 1`), כדי למנוע הצגתן החוזרת כהתראות "חדשות".

### קטעי קוד מרכזיים:

```
`network_server.py` - טיפול בבקשת `get_user_notifications` (בתוך
`handle_request`):

python'''

:elif rtype == "get_user_notifications

("user_id = request.get("user_id

()conn_db = connect_db(); cursor = conn_db.cursor

[] = notifications

:try

# שליפת תזכורות למיונים קרובים

cursor.execute("SELECT m.title, m.date FROM meyunim_assignments a
JOIN meyunim m ON a.meyun_id = m.id WHERE a.user_id = ? AND
((date(m.date) >= date("now")) ORDER BY date(m.date) ASC", (user_id

for title, date_str in cursor.fetchall(): notifications.append(f"Reminder:
({title}) on {date_str}

# שליפת עדכונים על בקשות

"""cursor.execute

SELECT m.title, r.type, r.status, r.id as request_id

FROM meyunim_requests r

JOIN meyunim_assignments a ON r.assignment_id = a.id

JOIN meyunim m ON a.meyun_id = m.id

WHERE r.status != "Pending" AND a.user_id = ? AND (r.notified_user =
0 OR r.notified_user IS NULL
```

```

((user_id), ""

[] = req_ids_to_mark

:()for title, req_type, status, req_id in cursor.fetchall

notifications.append(f"Update: Your {req_type} request for \"{title}\" was
(\".{status.lower

(req_ids_to_mark.append(req_id

# סימון בקשות כנצפו
cursor.execute(f"UPDATE meyunim_requests SET notified_user = 1
(WHERE id IN ({','.join(['?']*len(req_ids_to_mark))})", req_ids_to_mark

# שליפת תגובות אדמין להודעות

cursor.execute("SELECT subject, id as message_id FROM messages
WHERE sender_id = ? AND response IS NOT NULL AND (notified_user = 0
((OR notified_user IS NULL) ORDER BY timestamp DESC", (user_id

[] = msg_ids_to_mark

:()for subject, msg_id in cursor.fetchall

("){notifications.append(f"Admin replied to your message: \"{subject

(msg_ids_to_mark.append(msg_id

# סימון הודעות כנצפו
cursor.execute(f"UPDATE messages SET notified_user = 1 WHERE id
(IN ({','.join(['?']*len(msg_ids_to_mark))})", msg_ids_to_mark

(conn_db.commit

{return {"status": "ok", "notifications": notifications

# ... (טיפול בשגיאות וסגירת חיבור)
...

(הערה: הקריאה ל-connect_db() והשמת הסמן נעשות בצורה מקוצרת כאן, בקוד המלא
הן בתוך ה-try/finally)

```

## טיפול מקיף בשגיאות ותקשורת אמינה בצד הלקוח

## הסבר:

אפליקציית הלקוח (במחלקה `App` בקובץ `main.py`) כוללת לוגיקה לטיפול בחוסן ובאמינות התקשורת מול השרת. המתודה `safe\_network\_request` עוטפת כל קריאה לשרת, בודקת את זמינות אובייקט הרשת, ומטפלת בתגובות שגיאה מהשרת. היא מבחינה בין שגיאות "רגילות" (למשל, קלט לא תקין) לבין שגיאות קריטיות המצביעות על אובדן קשר מלא עם השרת. במקרה של שגיאה קריטית, מופעלת המתודה `handle\_server\_critical\_error` שמנקה את המסך, סוגרת משאבים, ומציגה הודעת שגיאה ברורה למשתמש, תוך מניעת הצפה של הודעות נוספות.

## קטעי קוד מרכזיים:

```
`main.py` - מתודת `App.safe_network_request`:  
  
python```  
  
:("""def safe_network_request(self, request_data, source_description  
  
if self.server_error_active: # If error already handled, don't make new  
requests  
  
return None  
  
  
:if not self.network  
  
)self.master.after(0, lambda: self.handle_server_critical_error  
  
(({f"Network client not initialized ({source_description  
  
return None  
  
  
:try  
  
(response = self.network.send_request(request_data  
  
  
Check for specific error messages that indicate total connection loss #  
  
:"if isinstance(response, dict) and response.get("status") == "error  
  
(msg = response.get("message", "").lower  
  
] = critical_phrases
```

```

,"network communication failed"
,"server response timed out"
,"server closed connection"
"client not properly initialized"

[
:(if any(phrase in msg for phrase in critical_phrases
)self.master.after(0, lambda: self.handle_server_critical_error
(("{f"{response.get("message")}"}) ({source_description
return None

return response

except Exception as e: # Catch any other unexpected exception from
send_request itself

)self.master.after(0, lambda: self.handle_server_critical_error
(("{f"Unexpected error during network request: {e}"}) ({source_description
return None
...

```

:`App.handle\_server\_critical\_error` מתודת - `main.py`

python``

```

:("""=def handle_server_critical_error(self, error_source_message
:if self.server_error_active
return # Already handling or handled
self.server_error_active = True
("{print(f"CRITICAL SERVER ERROR: {error_source_message

```

```

:if self.session_timer
(self.master.after_cancel(self.session_timer

```

```
self.session_timer = None
```

```
# ניסיון לסגור חלונות קופצים מודאליים
():current_grab = self.master.grab_current
:if current_grab and current_grab != self.master
():current_grab.grab_release
():current_grab.destroy
:except Exception as e
("{print(f"Error trying to release grab / destroy popup: {e
```

```
self.clear_screen() # Clear current content of the main window
```

```
# סגירת חיבור רשת
:if self.network and self.network.socket
:try
():self.network.close
:except Exception as e
("{print(f"Error closing network connection during critical error: {e
self.network = None # Mark network as unusable
```

```
# הצגת הודעת שגיאה למשתמש
self._show_server_unavailable_popup(error_source_message)
```

```
...
```

```
`main.py` - מתודת `App._show_server_unavailable_popup`
```

אחראית על יצירת והצגת חלון ה-qtcpsocket המודיע למשתמש שהשרת אינו זמין.

**שאר הפתרונות לבעיות שהוצגו בחלק האתגרים הוצגו יחד עם הפתרון.**

**מסמך בדיקות**

בתיאור הבדיקות המתוכננות הוצגו כלל הבדיקות עם מטרתן ואופי בדיקתן, לעיון ניתן לחזור לדף תכנון הבדיקות.

**בדיקות מתוך מסמך הבדיקות המקורי**

| שם הבדיקה                   | תוצאה | בעיה שנמצאה                  | תיקון שבוצע                         |
|-----------------------------|-------|------------------------------|-------------------------------------|
| התחברות עם סיסמה שגויה      | נכשל  | המערכת אפשרה כניסה           | תוקן ע"י בדיקת סיסמה מגובבת מול מסד |
| התחברות עם שם משתמש לא קיים | עבר   |                              |                                     |
| התחברות עם קוד אימות שגוי   | נכשל  | לא נשלחה התראה למייל         | תוקן שליחת דוא"ל לשרת SMTP          |
| התחברות עם קוד שפג תוקף     | נכשל  | הקוד התקבל גם לאחר 10 דקות   | נוסף מנגנון מחיקה לקודים שפג תוקפם  |
| התחברות תקינה מלאה          | עבר   |                              |                                     |
| התחברות עם קוד שכבר שומש    | נכשל  | המערכת אפשרה שימוש חוזר בקוד | נוספה בדיקת שימוש חד-פעמי לקוד      |
| שליחת הודעה לאדמין          | עבר   |                              |                                     |
| צפייה בתגובה מהאדמין        | עבר   |                              |                                     |
| שליחת הודעה ריקה            | נכשל  | המערכת אפשרה שליחה           | נוספה בדיקה שהשדה לא ריק לפני שליחה |
| שליחת בקשת דחייה על מיון    | עבר   |                              |                                     |
| שליחת בקשת ויתור על מיון    | עבר   |                              |                                     |
| אישור בקשה מצד האדמין       | עבר   |                              |                                     |
| דחיית בקשה מצד האדמין       | עבר   |                              |                                     |

|                                 |      |                             |                                   |
|---------------------------------|------|-----------------------------|-----------------------------------|
| שינוי תאריך במיון               | עבר  |                             |                                   |
| יצירת מיון חדש                  | עבר  |                             |                                   |
| יצירת מיון ללא תאריך            | נכשל | המערכת קיבלה מיון ללא תאריך | נוספה בדיקה למניעת יצירת מיון ריק |
| שיבוץ משתמש למיון               | נכשל | לא נשלחה בקשה לשרת          | תוקן קריאה לפונקציה המתאימה       |
| מחיקת מיון ועדכון בצד המשתמש    | עבר  |                             |                                   |
| הוספת משתמש חדש עם פרטים תקינים | עבר  |                             |                                   |
| ניסיון להוסיף שם משתמש קיים     | נכשל | לא נשלחה בקשה לשרת          | תוקן קריאה לפונקציה המתאימה       |

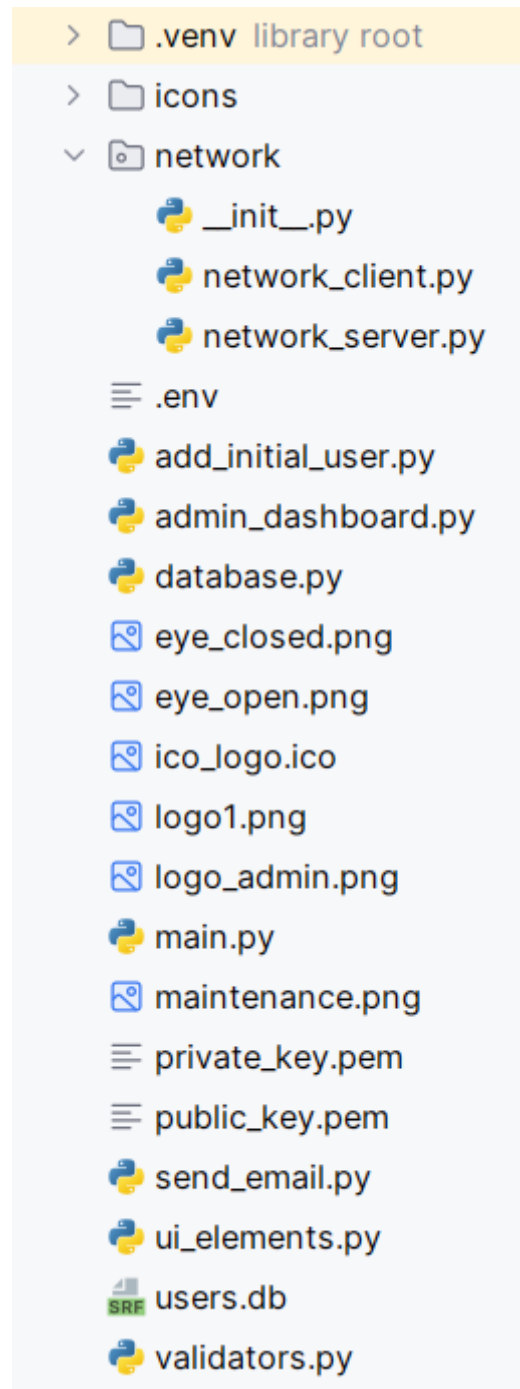
## בדיקות נוספות (שלא הופיעו בתכנון)

| שם הבדיקה                   | תוצאה | בעיה שנמצאה                       | תיקון שבוצע                           |
|-----------------------------|-------|-----------------------------------|---------------------------------------|
| שליחת קוד אימות ללא התחברות | נכשל  | השרת קיבל בקשה גם ללא התחברות     | נוספה בדיקה שהמשתמש מחובר לפני שליחה  |
| כניסה למיון שבוטל           | נכשל  | המשתמש הצליח לגשת למסך מיון שבוטל | נוספה בדיקת סטטוס שיבוץ לפני כניסה    |
| שינוי סיסמה ללא אימות קוד   | נכשל  | הייתה גישה לשדה סיסמה לפני הקוד   | שדה הוסתר עד אימות מלא                |
| חיפוש משתמש באותיות קטנות   | עבר   |                                   |                                       |
| חיפוש משתמש ריק             | נכשל  | השדה הריק לא הוביל לתוצאה מתאימה  | הוחזרו כל המשתמשים כשארן מחרוזת חיפוש |
| בקשת דחייה עם תאריך עבר     | נכשל  | המערכת קיבלה תאריך ישן            | נוספה ולידציה לתאריכים עתידיים בלבד   |
| תגובה ריקה מצד אדמין        | נכשל  | תגובה ריקה נשמרה במסד             | נוספה בדיקה בצד שרת לפני שמירה        |

|                                  |                                       |      |                       |
|----------------------------------|---------------------------------------|------|-----------------------|
|                                  |                                       | עבר  | צפייה בגרף בלי נתונים |
|                                  |                                       | עבר  | שמות משתמש בעברית     |
| נוספה בדיקת user_id בתשובה מהשרת | משתמש הצליח לראות נתונים של משתמש אחר | נכשל | גישה לנתוני משתמש אחר |

## מדריך למשתמש

### עץ קבצים



## התקנת המערכת

### דרישות בסיס

דרישות תוכנה, סביבת עבודה ורשת:

- Python 3.12 ומעלה, מותקן תקין.
- ניתן להריץ בסביבת פיתוח או הרצה ישירה ממערכת הקבצים.
- מערכת הפעלה תומכת: Windows, Linux, MacOS.
- חיבור לרשת המקומית בה פועל השרת (לדוגמה localhost או רשת LAN).
- התקנת הספריות המתאימות לפי הייבוא בראש כל קובץ. - אם יש זמן לתקן ולהוסיף איזה

### דרישות קוד:

הקבצים שצריכים להימצא עבור שימוש של משתמש רגיל(מלש"ב):

- `main.py` – קובץ הפעלת ממשק המשתמש.
- `network_client.py` – קובץ התקשורת מול השרת.
- `validators.py` – בדיקות תקינות לשדות (משותף גם ללקוח).
- `ui_elements.py` – כפתורים וחלונות מוקפצים.
- `eye_open, eye_closed, logo1, maintenance` מוג `png` – רצוי, במידה ולא קיים תוצג הודעת שגיאה. מומלץ למען נראה תקינה.
- `ico_logo.ico` - רצוי, במידה ולא קיים תוצג הודעת שגיאה. מומלץ למען נראה תקינה.

דרישות להפעלת המערכת כמנהל - האדמין יכול לפעול בשתי דרכים:

הקבצים שצריכים להימצא עבור שימוש של אדמין בגישת "עבודה מהבית: מחשב אישי":

גישה זו מאפשרת עבודה כמעט מלאה לחלוטין של אדמין ללא גישה ישירה לקודים של מסד הנתונים, השרת, שליחת האימיילים ועוד. היא תאפשר את כלל פעולות האדמין מלבד הפעלת השרת וצפייה בקוד עצמו.

כל אשר משתמש אדמין צריך בעבור גישה זו יתר על הקבצים והתמונות הבסיסיים, הוא הקובץ `admin_dashboard.py` אשר מכיל את הממשק הגרפי הייחודי לאדמין על שלל פעולותיו ותיקיית `icons` המכילה תמונות לשיפור הנראות(המערכת יכולה לפעול ללא התיקייה- משפיעה רק על נראות).

לצורך גישה "שרת":

גישה זו מאפשרת עבודה מלאה לחלוטין של אדמין. היא תאפשר שליטה מלאה על המערכת, כמו הרצת השרת וצפייה בקוד הגולמי, ואופציות מתקדמות כגון הוספת משתמש ראשוני על מנת להתחבר למערכת, שינוי מפתחות ההצפנה ועוד.

קבצי קוד דרושים:

עילי נגלר llayDF

**network\_server.py** – קובץ ההפעלה של השרת.

**database.py** – גישה לכל שאילתות בסיס הנתונים.

**send\_email.py** – שליחת מיילים.

קבצים אחרים:

env – קובץ חובה המכיל:

**EMAIL\_USER** (כתובת Gmail פעילה)

**EMAIL\_PASS** (סיסמת האפליקציה של האימייל)

קבצי מפתחות ההצפנה:

public key.pem

private\_key.pem

חשבון מייל:

חובה שהאדמין יפעיל חשבון Gmail עם הרשאה לשליחת הודעות דרך SMTP. כמובן שיש מייל מופעל ועובד ייעודי לפרויקט llayDF.

מסד הנתונים: קובץ מסד הנתונים מסוג **SQLite**.

הפעלה ראשונה של המערכת: קובץ **add\_initial\_user.py** לצורך יצירת משתמש אדמין ראשון במסד, חובה להפעלה חד-פעמית.

## אופן הפעלת המערכת

אופן הפעלת המערכת פשוט במיוחד וכל אדם עם יכולת טכנולוגית בסיסית ידע להשתמש בה בהצלחה. על מנת להריץ את המערכת מצד הצבא, כל שיש לעשות הוא להריץ את קובץ השרת. לאחר מכן, להריץ את הקובץ הראשי (main), וכעת ניתן להשתמש במערכת בהצלחה.

במידה ומדובר בהרצה ראשונית של המערכת יש להריץ קודם את **add\_initial\_user.py**.

בחלק "מסכי הפרויקט" מופיעים כלל מסכי המערכת, ולכן לא אצרף אותם מחדש כאן על מנת לא ליצור כפילות. המעבר בניהם, והשימוש בהם, פשוט במיוחד עם הוראות באנגלית ולכן גם כן לא אצרף אותם פה.

## סיכום אישי ורפלקציה

### רפלקציה

כשאני מנסה לסכם את העבודה שלי על הפרויקט הזה, llayDF, שהפך כבר להרבה יותר מעוד רעיון עם שם מדליק כמו שהיה בהתחלה, אני מגלה שזה הרבה יותר מפרויקט תכנות. זה סיפור שלם.

סיפור של למידה, ומסע של נחישות והתמודדות. בהתחלה חשבתי שאני הולך "רק" לכתוב מערכת, לבנות קוד, לחבר בין חלקים – אבל לא הבנתי כמה זה ידרוש ממני. בין אם מבחינה רגשית ומנטלית, ובין אם מבחינת ידע.

את החודש חודשיים הראשונים אפשר לסכם בהרבה בלבול. לא בגלל שלא ידעתי תכנות, אלא כי הבנתי שמה שלמדתי עד עכשיו הוא ממש קצה הקרחון. פתאום נדרשתי לא רק לדעת לבצע משימות קטנות כמו איך להצפין או לעבוד עם מחלקות, אלא איך לבנות מערכת שלמה: שמדברת עם עצמה, שמאובטחת מקצה לקצה, שנראית טוב, שמגיבה כמו שצריך. מערכת כזו זה משהו בהיקף שלא דמיינתי. ההצלחה לא הגיעה בבום. היו לא מעט רגעים של תסכול ואכזבה. לילות של בהייה בקוד וניסיון להבין מדוע פיצר מסוים לא עובד, כשכבר אין לך אף אחד לשאול אותו. אבל הרגעים האלו מתגמדים לעומת הרגעים הטובים, שבהם אתה סוף סוף מצליח. אין גאווה כמו לראות בעיה שעבדת קשה בשביל לפתור נפתרת.

אני חושב שאחת התובנות הגדולות שלי הייתה להבין שההבדל בין הצלחה לכישלון הוא לא תמיד בידע או בהשגה של המטרה הסופית, אלא בהחלטה להמשיך גם כשקשה. ברגעים האלו בניתי לא רק את המערכת, אלא גם את עצמי.

במהלך העשייה של הפרויקט למדתי יכולות שאני כבר יכול לראות כיצד יעזרו לי בהמשך, כמו איך לבצע מחקר, איך לעבוד עם הרבה מידע, איך לחלק משימות, איך לחלק את הזמן שלי, איך ליישם טיפים ממכרים ועוד עשרות יכולות, אך בעיקר בעיקר, לא לוותר. כל הכלים האלו, מגמדים את הידע התכנותי והטכנולוגי העצום שצברתי לאורך הדרך.

הפרויקט נתן לי תחושת מסוגלות. הוא גרם לי לראות שאני יכול. גם אם זה מורכב, גם אם זה קשוח – אני מסוגל להתמודד עם זה, להבין את זה, ולבנות מזה משהו אמיתי. למדתי להעריך את הדרך, לא רק את התוצאה. כל תקלה – כל באג, כל לילה לבן של עבודה, היו שיעורים על עבודה קשה ודבקות במטרה.

אני לא אדם שאוהב להיות חכם בדיעבד, אבל במבט לאחור הייתי מתכנן יותר מראש. הייתי לומד לעצור ולנשום, ולא להילחץ מכל תקלה. הייתי משתף אחרים יותר בתהליך, ומתייעץ יותר עם צוות המורים. אבל, כל חלק כזה בדרך בנה אותי. מבחינה תכנותית, לו הייתי מתחיל את הפרויקט מחדש, הייתי יוצר מבנה תיקיות ברור יותר מלכתחילה, ובנוסף, הייתי משלב מערכת בדיקות אוטומטית מוקדם יותר (unit tests), מה שהיה מונע לי לא מטע באגים בשלב מאוחר יותר של הפיתוח. אם היה לי עוד זמן, הייתי מוסיף בעיקר עוד פיצרים לניהול בעבור הצבא, מתוך תפיסה כי מערכת יעילה באמת עבור הצבא יכולה להיות דבר משמעותי להצלחת הצבא. השאלה המרכזית שאני שואל את עצמי כרגע מבחינה תכנותית, היא אילו פרצות יש למערכת. מאוד מעניין אותי כיצד אפשר לפרוץ את המערכת שאני עומד עליה.

## תודות

בראש ובראשונה, אני רוצה להודות תודה ענקית לצוות המורים הנהדר של המגמה, על תמיכה בלתי מובנת מאליה בהכנת הפרויקט. העבודה היסודית והבלתי מתפשרת שלכם הפכה את הפרויקט הזה לאפשרי. על העזרה בכל שעה, הייעוץ והתמיכה, בין אם במובן האישי או הטכנולוגי. לעידן פלדברג רכז המגמה, שהוביל אותנו כמגמה בהצלחה אל עבר היעד בלי להתייאש. לעודי רז, שהוא מורה המחשבים הראשון שלי בחיים, זה שהכיר לי לראשונה את התחום, תודה על התמיכה והייעוץ בכל שלב, ובעיקר על כך שאף פעם לא וויתרת עלי או נתת לי להרים ידיים ולעגל פינות. לליאת, שהפכה לסוג של אמא של כולם במגמה, על ההשקעה האדירה בנו והתמיכה גם שקשה. לרום רופסון, שהיה המורה הראשון שלי לשפת הפיתוח, על העזרה והייעוץ על אף שאינו מלמד אותי השנה. לקובי שוצמן, שמגיע בכל שישי בשביל ללמד אותנו ולהעשיר, על הזמן היקר שלך וההעשרה המדהימה שנבעה מידע וניסיון שאין לו תחליף.

תודה לחברי למגמה על כך שבזכותכם נהנתי לבוא אל השיעורים, ללמוד, וגם לעבוד קשה. על 3 שנים של הנאה ולמידה מטורפת, וחוויות שלא אשכח.

לחברי הקרובים יאיר, אבנר ואיתי, על התמיכה המוראלית בשעת הכנת ספר זה בפרט, ובתקופת פרויקט זה בכלל. על זה שתמיד האמנתם בי אפילו שלא באמת ידעתם במה מדובר, ודאגתם לשלומי.

וכמובן, למשפחתי היקרה, שתמיד עומדים מאחורי בכל שלב או החלטה בחיים, שמהווים עבורי עוגן וללא תמיכתם ודאגתם לא הייתי יכול לבצע פרויקט בהיקף שכזה. ואיך אפשר שלא, על זה שסבלתם עוד קצת מהחפירות שלי.

## ביבליוגרפיה

Apna College. (2022, June 10). *Multithreading vs multiprocessing in Python* [Video]. YouTube. <https://www.youtube.com/watch?v=STEOavXqXkQ>

CodingsFun. (2023, February 14). *How to use CustomTkinter in Python* [Video]. YouTube. [https://www.youtube.com/watch?v=Miydkti\\_QVE](https://www.youtube.com/watch?v=Miydkti_QVE)

Code With Harry. (2020, August 20). *Build CRUD app with SQLite in Python* [Video]. YouTube. <https://www.youtube.com/watch?v=byHcYRpMgl4>

GeeksforGeeks. (2024). *Build GUI using CustomTkinter module in Python*. <https://www.geeksforgeeks.org/build-a-basic-form-gui-using-customtkinter-module-in-python>

NeuralNine. (2020, July 13). *Build a TCP server and client in Python* [Video]. YouTube. <https://www.youtube.com/watch?v=3QiPPX-KeSc>

Python Software Foundation. (2025). *sqlite3 – DB-API 2.0 interface for SQLite databases*. <https://docs.python.org/3/library/sqlite3.html>

Tech With Tim. (2019, January 28). *Python socket programming tutorial* [Video]. YouTube. <https://www.youtube.com/watch?v=Lbfe3-v7yE0>

Tech With Tim. (2019, December 15). *Threading in Python tutorial* [Video]. YouTube. <https://www.youtube.com/watch?v=IEEhzQoKtQU>

## סופחים

### main.py:

```
import customtkinter as ctk

from tkinter import messagebox, Label

from PIL import Image, ImageTk

import threading
```

```

import time

from network.network_client import NetworkClient, ServerConnectionError

from admin_dashboard import AdminDashboard

from ui_elements import MyButton, BasePopup

from validators import is_strong_password

# from tkcalendar import DateEntry # Not directly used in user-facing part of this file now

import datetime

import os # Required for icon path check


class SubmitRequestPopup(BasePopup):

    def __init__(self, master, assignment_id, request_type, safe_request_method, user_id_for_context,
                 username_for_context, on_success_callback, critical_error_handler_callback, *args, **kwargs):

        super().__init__(master, title="Submit Request", geometry="400x280", *args, **kwargs)

        self.assignment_id = assignment_id

        self.request_type = request_type

        self.safe_request = safe_request_method # Use the safe_request_method from App

        self.user_id_for_context = user_id_for_context

        self.username_for_context = username_for_context

        self.on_success_callback = on_success_callback

        self.critical_error_handler = critical_error_handler_callback


        ctk.CTkLabel(self, text=f"Enter your message for {request_type} request:", font=("Arial", 14)).pack(
            pady=(10, 5))

        self.message_entry = ctk.CTkTextbox(self, height=120)

        self.message_entry.pack(pady=5, padx=10, fill="both", expand=True)

        self.message_entry.focus()


        MyButton(self, text="Submit", command=self.submit, height=35).pack(pady=(5, 10))


    def submit(self):

        message = self.message_entry.get("1.0", "end-1c").strip()

```

```

if not message:

    messagebox.showerror("Error", "Message cannot be empty.", parent=self)

    return


response = self.safe_request({ # Use the safe_request method

    "type": "submit_request",

    "assignment_id": self.assignment_id,

    "request_type": self.request_type,

    "message": message

}, source_description="SubmitRequestPopup.submit")


if response is None: # Critical server error was handled by App

    self.close_popup() # Close this popup as the main app will show server down

    return


if response.get("status") == "ok":

    messagebox.showinfo("Success", "Request submitted.", parent=self)

    if self.on_success_callback:

        self.on_success_callback(self.user_id_for_context, self.username_for_context)

    self.close_popup()

else:

    messagebox.showerror("Error", response.get("message", "Failed to submit request."), parent=self)

```

```

class App:

    def __init__(self, master):

        self.master = master

        self.network = None

        self.server_available = False

        self.server_error_active = False # Flag to prevent multiple error popups


    try:

```

```

self.network = NetworkClient()

self.server_available = True

except ServerConnectionError:

    self.server_available = False

    # self.network remains None

    return

# ---- This part only runs if server_available is True ----

self.master.title("Login System")

self.master.protocol("WM_DELETE_WINDOW", self.on_closing)

self.session_timer = None

self.session_duration = 600

try:

    self.master.iconbitmap("ico_logo.ico")

except Exception:

    print("Icon file 'ico_logo.ico' not found or platform issue.")

width = 1000

height = 750

x_position = (self.master.winfo_screenwidth() - width) // 2

y_position = (self.master.winfo_screenheight() - height) // 2 - 30

self.master.geometry(f"{width}x{height}+{x_position}+{y_position}")

ctk.set_appearance_mode("light")

ctk.set_default_color_theme("green")

self.show_password = False

self.current_frame = None

self.pending_user = None

self.pending_code_context = None

self.current_user_id_session = None

```

```

self.current_username_session = None

self.create_login_screen()

self.start_session_timer()

import os
from PIL import Image, ImageTk

import os
from PIL import Image, ImageTk
from tkinter import Label

def _show_server_unavailable_popup(self, error_source_message=""):
    self.master.withdraw()

    popup_title = "Server Unavailable"
    popup_message = (
        "The server is currently unavailable or connection was lost.\n"
        "Please try again later.\n"
    )
    if error_source_message:
        popup_message += f"({error_source_message})"

    error_window = ctk.CTkToplevel(self.master)
    error_window.title(popup_title)
    error_window.geometry("500x500")
    error_window.minsize(500, 500)
    error_window.configure(fg_color="#E8F5E9")

    icon_path = "ico_logo.ico"
    if os.path.exists(icon_path):
        try:

```

```

        error_window.iconbitmap(icon_path)

    except Exception as e_icon:

        print(f"Failed to set icon for server down popup: {e_icon}")

else:

    print(f"Server down popup icon not found at: {icon_path}")


error_window.update_idletasks()

popup_width = error_window.winfo_width()
popup_height = error_window.winfo_height()
screen_width = error_window.winfo_screenwidth()
screen_height = error_window.winfo_screenheight()
x_pos = (screen_width // 2) - (popup_width // 2)
y_pos = (screen_height // 2) - (popup_height // 2)
error_window.geometry(f'{popup_width}x{popup_height}+{x_pos}+{y_pos - 30}')


error_window.lift()
error_window.attributes("-topmost", True)
error_window.after(100, lambda: error_window.attributes("-topmost", False))
error_window.grab_set()
error_window.focus_force()


image_path = "maintenance.png"
if os.path.exists(image_path):

    try:

        image = Image.open(image_path)
        image = image.resize((400, 400))
        photo = ImageTk.PhotoImage(image)

        img_label = Label(error_window, image=photo, bg="#E8F5E9")
        img_label.image = photo
        img_label.pack(pady=10)

    except Exception as e_img:

```

```

print(f"Failed to load image: {e_img}")

ctk.CTkLabel(error_window, text=popup_message,
             font=("Arial", 16), wraplength=420).pack(pady=20, padx=20)

else:

    ctk.CTkLabel(error_window, text=popup_message,
                 font=("Arial", 16), wraplength=420).pack(pady=20, padx=20)

MyButton(error_window, text="OK", command=self.master.destroy, width=100).pack(pady=(0, 20))

error_window.protocol("WM_DELETE_WINDOW", self.master.destroy)

def handle_server_critical_error(self, error_source_message=""):

    if self.server_error_active:

        return # Already handling or handled

    self.server_error_active = True

    print(f"CRITICAL SERVER ERROR: {error_source_message}")

    if self.session_timer:

        self.master.after_cancel(self.session_timer)

        self.session_timer = None

    # Attempt to close any modal popups first

    # This is tricky as we don't have a central list of popups

    # grab_release on master might help if a popup has grab_set

    try:

        current_grab = self.master.grab_current()

        if current_grab and current_grab != self.master:

            current_grab.grab_release()

            current_grab.destroy()

    except Exception as e:

        print(f"Error trying to release grab / destroy popup: {e}")

    self.clear_screen() # Clear current content of the main window

```

```

# Close network connection if it exists and seems open

if self.network and self.network.socket:

    try:

        self.network.close()

    except Exception as e:

        print(f"Error closing network connection during critical error: {e}")

self.network = None # Mark network as unusable


self._show_server_unavailable_popup(error_source_message)


def safe_network_request(self, request_data, source_description=""):

    if self.server_error_active: # If error already handled, don't make new requests

        return None


    if not self.network:

        self.master.after(0, lambda: self.handle_server_critical_error(

            f"Network client not initialized ({source_description})"))

        return None


    try:

        response = self.network.send_request(request_data)


        # Check for specific error messages that indicate total connection loss

        if isinstance(response, dict) and response.get("status") == "error":

            msg = response.get("message", "").lower()

            critical_phrases = [

                "network communication failed",

                "server response timed out",

                "server closed connection",

                "client not properly initialized" # If public key somehow got lost

            ]

```

```

        if any(phrase in msg for phrase in critical_phrases):

            self.master.after(0, lambda: self.handle_server_critical_error(

                f'{response.get("message")} ({source_description})'))

            return None

        return response

    except Exception as e: # Catch any other unexpected exception from send_request itself

        self.master.after(0, lambda: self.handle_server_critical_error(

            f'Unexpected error during network request: {e} ({source_description})'))

        return None

def start_session_timer(self):

    if self.session_timer:

        self.master.after_cancel(self.session_timer)

    self.session_timer = self.master.after(self.session_duration * 1000, self.handle_session_timeout)

def reset_session_timer(self):

    self.start_session_timer()

def handle_session_timeout(self):

    if self.server_error_active: return

    is_user_session_active = False

    if self.current_frame:

        if not hasattr(self, 'email_entry') or self.email_entry is None:

            is_user_session_active = True

        for child in self.master.winfo_children():

            if isinstance(child, AdminDashboard):

                is_user_session_active = True

                break

    if self.current_user_id_session:

        is_user_session_active = True

```

```

if is_user_session_active:

    messagebox.showwarning("Session Expired",

                           "Your session has expired due to inactivity. Please log in again.",

                           parent=self.master)

self.logout_and_show_login()


def logout_and_show_login(self):

    if self.server_error_active: return

    self.pending_user = None

    self.current_user_id_session = None

    self.current_username_session = None

    self.create_login_screen()


def on_closing(self):

    if self.network:

        self.network.close()

    if self.session_timer:

        self.master.after_cancel(self.session_timer)

    self.master.destroy()


def clear_screen(self):

    for widget in self.master.winfo_children():

        widget.destroy()

    self.current_frame = None

    for attr in ['email_entry', 'password_entry', 'message_label', 'show_password_btn']:

        if hasattr(self, attr):

            setattr(self, attr, None)


def create_login_screen(self):

    if self.server_error_active: return

    self.reset_session_timer()

    self.clear_screen() # Use the enhanced clear_screen

```

```

login_main_frame = ctk.CTkFrame(self.master)

login_main_frame.pack(fill="both", expand=True)

self.current_frame = login_main_frame

try:
    image = Image.open('logo1.png')

    self.logo_image_obj_login = ctk.CTkImage(light_image=image, dark_image=image, size=(400, 150))

    ctk.CTkLabel(login_main_frame, image=self.logo_image_obj_login, text="").pack(pady=(30, 20))
except Exception as e:
    print(f"Error loading logo1.png for login: {e}")

    ctk.CTkLabel(login_main_frame, text="[Logo]", font=("Arial", 12)).pack(pady=(30, 20))

self.email_entry = ctk.CTkEntry(login_main_frame, font=("Arial", 14), placeholder_text="Enter your
username",

                                height=40)

self.email_entry.pack(pady=10, padx=40, fill="x")

password_frame = ctk.CTkFrame(login_main_frame, fg_color="transparent")

password_frame.pack(pady=10, padx=40, fill="x")

self.password_entry = ctk.CTkEntry(password_frame, font=("Arial", 14), placeholder_text="Enter your
password",

                                show="*", height=40)

self.password_entry.pack(side="left", fill="both", expand=True)

try:
    self.eye_open_img_ref_login = ctk.CTkImage(Image.open("eye_open.png").resize((35, 35)), size=(35,
35))

    self.eye_closed_img_ref_login = ctk.CTkImage(Image.open("eye_closed.png").resize((35, 35)), size=(35,
35))

    current_eye_icon = self.eye_closed_img_ref_login if not self.show_password else
self.eye_open_img_ref_login

except Exception as e:
    print(f"Error loading eye icons for login: {e}")

```

```

self.eye_open_img_ref_login = self.eye_closed_img_ref_login = None

current_eye_icon = None

self.show_password_btn = MyButton(
    password_frame, image=current_eye_icon, text="", width=40,
    command=self.toggle_password_visibility, border_width=0,
    fg_color="transparent", hover_color="#dcdcdc"
)

self.show_password_btn.pack(side="right")

self.login_btn = MyButton(login_main_frame, text="Log In", font=("Consolas", 14, "bold"),
    height=40, width=250, command=self.validate_login)

self.login_btn.pack(pady=30, padx=40)

self.forgot_password_btn = MyButton(login_main_frame, text="Forgot Password?", font=("Arial", 12),
    fg_color="transparent", text_color="blue", hover_color="#e0e0e0",
    command=self.open_forgot_password_screen)

self.forgot_password_btn.pack(pady=5)

self.message_label = ctk.CTkLabel(login_main_frame, text="", font=("Arial", 12), text_color="red")

self.message_label.pack(pady=10)

self.email_entry.focus()

def toggle_password_visibility(self):
    if self.server_error_active: return

    self.reset_session_timer()

    self.show_password = not self.show_password

    if self.show_password:
        self.password_entry.configure(show="")

        if self.eye_open_img_ref_login:
            self.show_password_btn.configure(image=self.eye_open_img_ref_login)

    else:

```

```

self.password_entry.configure(show="*")

if self.eye_closed_img_ref_login:

    self.show_password_btn.configure(image=self.eye_closed_img_ref_login)

def validate_login(self):

    if self.server_error_active: return

    self.reset_session_timer()

    username = self.email_entry.get()

    password = self.password_entry.get()

    if not username or not password:

        self.message_label.configure(text="Please enter username and password.")

        return

    self.message_label.configure(text="Logging in...", text_color="blue")

    self.login_btn.configure(state="disabled")

    threading.Thread(target=self.process_login, args=(username, password), daemon=True).start()

def process_login(self, username, password):

    response = self.safe_network_request({

        "type": "login",

        "username": username,

        "password": password

    }, source_description="process_login")

    if response is None: # Critical server error handled

        self.master.after(0, lambda: self.login_btn.configure(state="normal")) # Re-enable button if UI still there

        return

    self.master.after(0, lambda: self.login_btn.configure(state="normal"))

    if response.get("status") == "ok":

        user_id = response.get("user_id")

```

```

is_admin = response.get("is_admin")

email = response.get("email")

first_login = response.get("first_login", False)


self.pending_user = (user_id, is_admin, username, email)

self.current_user_id_session = user_id

self.current_username_session = username


if first_login:

    self.master.after(0, self.create_first_login_screen)

    return


code_response = self.safe_network_request({

    "type": "send_code_to_user",

    "user_id": user_id

}, source_description="process_login.send_code")


if code_response is None: return


if code_response.get("status") == "ok":

    self.pending_code_context = "login"

    self.master.after(0, self.create_code_verification_screen)

else:

    self.master.after(0, lambda: self.message_label.configure(

        text=code_response.get("message", "Failed to send 2FA code."), text_color="red"))

else:

    self.master.after(0, lambda: self.message_label.configure(text=response.get("message", "Login failed."),

                                                                text_color="red"))


def create_first_login_screen(self):

    if self.server_error_active: return

    self.reset_session_timer()

```

```

self.clear_screen()

self.current_frame = ctk.CTkFrame(self.master)

self.current_frame.pack(fill="both", expand=True)

    ctk.CTkLabel(self.current_frame, text="Complete Your Profile & Set Password", font=("Arial", 18,
"bold")).pack(
        pady=10)

self.id_entry_first = ctk.CTkEntry(self.current_frame, placeholder_text="ID Number (9 digits)")
self.phone_entry_first = ctk.CTkEntry(self.current_frame, placeholder_text="Phone (9-10 digits)")
self.address_entry_first = ctk.CTkEntry(self.current_frame, placeholder_text="Address")
self.age_entry_first = ctk.CTkEntry(self.current_frame, placeholder_text="Age (e.g., 25)")
self.new_password_entry_first_login = ctk.CTkEntry(self.current_frame, placeholder_text="New Password",
                                                    show="*")

self.id_entry_first.pack(pady=5, padx=40, fill="x")
self.phone_entry_first.pack(pady=5, padx=40, fill="x")
self.address_entry_first.pack(pady=5, padx=40, fill="x")
self.age_entry_first.pack(pady=5, padx=40, fill="x")
self.new_password_entry_first_login.pack(pady=5, padx=40, fill="x")

self.first_login_message_label = ctk.CTkLabel(self.current_frame, text="", text_color="red")
self.first_login_message_label.pack(pady=5)

    MyButton(self.current_frame, text="Submit Information",
command=self.submit_first_login_info).pack(pady=10)

    MyButton(self.current_frame, text="Back to Login", command=self.create_login_screen).pack(pady=5)

self.id_entry_first.focus()

def submit_first_login_info(self):
    if self.server_error_active: return

    self.reset_session_timer()

    id_number = self.id_entry_first.get().strip()

```

```

phone = self.phone_entry_first.get().strip()

address = self.address_entry_first.get().strip()

age = self.age_entry_first.get().strip()

new_password = self.new_password_entry_first_login.get().strip()


from validators import is_valid_phone, is_valid_age

if not id_number.isdigit() or len(id_number) != 9:

    self.first_login_message_label.configure(text="Invalid ID number (must be 9 digits).")

    return

if not is_valid_phone(phone):

    self.first_login_message_label.configure(text="Invalid phone number (must be 9-10 digits).")

    return

if not address:

    self.first_login_message_label.configure(text="Address is required.")

    return

if not is_valid_age(age):

    self.first_login_message_label.configure(text="Invalid age (must be between 1 and 119).")

    return

if not is_strong_password(new_password):

    self.first_login_message_label.configure(text="Password: min 6 chars, letters & digits.")

    return


user_id, __, __, __ = self.pending_user


self.first_login_message_label.configure(text="Submitting...", text_color="blue")

MyButton_ref = self.current_frame.winfo_children()[-2]

MyButton_ref.configure(state="disabled")


threading.Thread(target=self._process_submit_first_login,

                  args=(user_id, id_number, phone, address, age, new_password), daemon=True).start()


def _process_submit_first_login(self, user_id, id_number, phone, address, age, new_password):

```

```

response = self.safe_network_request({
    "type": "complete_first_login",
    "user_id": user_id,
    "id_number": id_number,
    "phone": phone,
    "address": address,
    "age": age,
    "new_password": new_password
}, source_description="_process_submit_first_login")

if response is None: return

# Ensure button is re-enabled on main thread, only if not in server error state
if not self.server_error_active:
    self.master.after(0, lambda: self.current_frame.winfo_children()[-2].configure(state="normal"))

if response.get("status") == "ok":
    self.master.after(0, lambda: messagebox.showinfo("Success", "Profile updated. Please log in again.",
                                                    parent=self.master))
    self.master.after(0, self.create_login_screen)
else:
    self.master.after(0, lambda: self.first_login_message_label.configure(
        text=response.get("message", "Update failed.")))

def create_code_verification_screen(self):
    if self.server_error_active: return
    self.reset_session_timer()
    self.clear_screen()
    self.current_frame = ctk.CTkFrame(self.master)
    self.current_frame.pack(fill="both", expand=True)

    context_text = "Login" if self.pending_code_context == "login" else "Password Reset"

```

```

    ctk.CTkLabel(self.current_frame, text=f"Enter Verification Code for {context_text}",
                  font=("Arial", 18, "bold")).pack(pady=20)

    self.code_entry_verify = ctk.CTkEntry(self.current_frame, placeholder_text="6-digit code", font=("Arial", 14),
                                          height=40)
    self.code_entry_verify.pack(pady=10, padx=40, fill="x")
    self.code_entry_verify.focus()

    self.code_submit_btn_verify = MyButton(self.current_frame, text="Verify",
                                           command=self.verify_code_action,
                                           height=40)
    self.code_submit_btn_verify.pack(pady=20)

    self.code_message_info = ctk.CTkLabel(self.current_frame, text="A 6-digit code has been sent to your
    email.",
                                          font=("Arial", 12), text_color="gray")
    self.code_message_info.pack(pady=5)

    self.code_message_error = ctk.CTkLabel(self.current_frame, text="", font=("Arial", 12), text_color="red")
    self.code_message_error.pack(pady=5)

    MyButton(self.current_frame, text="Back to Login", command=self.create_login_screen).pack(pady=10)

def verify_code_action(self):
    if self.server_error_active: return
    self.reset_session_timer()
    code = self.code_entry_verify.get().strip()
    if not code or not self.pending_user:
        self.code_message_error.configure(text="Please enter the code.")
        return
    if not code.isdigit() or len(code) != 6:
        self.code_message_error.configure(text="Code must be 6 digits.")
        return

```

```

user_id, _, _ = self.pending_user

self.code_submit_btn_verify.configure(state="disabled", text="Verifying...")

self.code_message_error.configure(text="")

threading.Thread(target=self._process_verify_code, args=(user_id, code), daemon=True).start()

def _process_verify_code(self, user_id, code):
    response = self.safe_network_request({
        "type": "verify_code",
        "user_id": user_id,
        "code": code
    }, source_description="_process_verify_code")

    if response is None: return

    if not self.server_error_active:
        self.master.after(0, lambda: self.code_submit_btn_verify.configure(state="normal", text="Verify"))

    if response.get("status") == "ok":
        self.master.after(0, self._handle_successful_code_verification)
    else:
        self.master.after(0, lambda: self.code_message_error.configure(text="Invalid or expired code."))

def _handle_successful_code_verification(self):
    if self.server_error_active: return

    user_id, entity_specific_is_admin, username, email_val = self.pending_user

    if self.pending_code_context == "login":
        if entity_specific_is_admin:
            AdminDashboard(self.master, {"username": username, "is_admin": 1, "user_id": user_id},
                            app_instance=self, # Pass App instance
                            on_logout=self.create_login_screen)
        else:

```

```

        self.show_user_dashboard_options(user_id, username)

    elif self.pending_code_context == "reset_password":

        username_for_reset = entity_specific_is_admin

        email_for_reset = username

        self.create_new_password_screen(username_for_reset, email_for_reset)

def show_user_dashboard_options(self, user_id, username):

    if self.server_error_active: return

    self.reset_session_timer()

    self.clear_screen()

    self.current_frame = ctk.CTkFrame(self.master)

    self.current_frame.pack(fill="both", expand=True)

    display_username = self.current_username_session if self.current_username_session else username

    ctk.CTkLabel(self.current_frame, text=f"Welcome, {display_username}!", font=("Arial", 20,
"bold")).pack(pady=30)

    notifications_frame = ctk.CTkFrame(self.current_frame, fg_color="transparent")

    notifications_frame.pack(pady=(0, 10), fill="x", padx=20)

    self.notifications_label = ctk.CTkLabel(notifications_frame, text="Loading notifications...",

                                         font=("Arial", 12, "italic"), text_color="gray", justify="left",

                                         anchor="w")

    self.notifications_label.pack(anchor="w", fill="x")

    self._fetch_user_notifications(user_id)

    MyButton(self.current_frame, text="View Personal Data", width=200,

             command=lambda: self.create_user_data_dashboard(user_id, display_username),

             fg_color="#004d00", hover_color="#006400").pack(pady=10)

    MyButton(self.current_frame, text="View Meyunim", width=200,

             command=lambda: self.create_user_dashboard(user_id, display_username),

             fg_color="#004d00", hover_color="#006400").pack(pady=10)

    MyButton(self.current_frame, text="View Request History", width=200,

```

```

        command=lambda: self.show_request_history(user_id, display_username),
        fg_color="#004d00", hover_color="#006400").pack(pady=10)

    MyButton(self.current_frame, text="Messages to Admin", width=200,
        command=lambda: self.create_user_messages_screen(user_id, display_username),
        fg_color="#004d00", hover_color="#006400").pack(pady=10)

    logout_btn = MyButton(self.current_frame, text="Logout", command=self.logout_and_show_login)
    logout_btn.pack(side="bottom", pady=20)

def _fetch_user_notifications(self, user_id_to_fetch):
    if self.server_error_active: return
    if not user_id_to_fetch:
        if hasattr(self, 'notifications_label'): self.notifications_label.configure(
            text="Error: User context lost.")
        return
    threading.Thread(target=self.__send_fetch_notifications_req, args=(user_id_to_fetch,),
        daemon=True).start()

def __send_fetch_notifications_req(self, uid):
    response = self.safe_network_request({
        "type": "get_user_notifications",
        "user_id": uid
    }, source_description="__send_fetch_notifications_req")

    if response is None: return

    if hasattr(self, 'notifications_label') and not self.server_error_active:
        if response.get("status") == "ok":
            notifications = response.get("notifications", [])
            if notifications:
                notif_display_text = "\n".join([f"- {n}" for n in notifications[:3]])
                if len(notifications) > 3:

```

```

        notif_display_text += f"\n- ...and {len(notifications) - 3} more."

        self.master.after(0, lambda: self.notifications_label.configure(text=notif_display_text,
                                                                           text_color="#333333",
                                                                           font=("Arial", 12)))

    else:

        self.master.after(0, lambda: self.notifications_label.configure(text="No new notifications.",
                                                                           text_color="gray",
                                                                           font=("Arial", 12, "italic")))

    else:

        self.master.after(0, lambda: self.notifications_label.configure(text="Failed to load notifications.",
                                                                           text_color="red"))

def create_user_dashboard(self, user_id, username):

    if self.server_error_active: return

    self.reset_session_timer()

    self.clear_screen()

    self.current_frame = ctk.CTkFrame(self.master)

    self.current_frame.pack(fill="both", expand=True)

    ctk.CTkLabel(self.current_frame, text="Your Meyunim", font=("Arial", 20, "bold")).pack(pady=10)

    meyunim_scroll_frame = ctk.CTkScrollableFrame(self.current_frame)

    meyunim_scroll_frame.pack(fill="both", expand=True, padx=20, pady=10)

    response = self.safe_network_request({"type": "get_meyunim", "user_id": user_id},
                                          source_description="create_user_dashboard")

    if response is None: return

    if response.get("status") == "ok":

        meyunim_data = response.get("data", [])

    else:

        messagebox.showerror("Error", "Failed to load meyunim.", parent=self.master)

```

```

meyunim_data = []

if not meyunim_data:

    ctk.CTkLabel(meyunim_scroll_frame, text="No meyunim assigned yet.").pack(pady=10)

else:

    for assignment_id, title, desc, date, status, request_info in meyunim_data:

        card = ctk.CTkFrame(meyunim_scroll_frame, border_color="black", border_width=1,
fg_color="#f0f0f0")

        ctk.CTkLabel(card, text=f"{title} ({date})", font=("Arial", 16, "bold")).pack(anchor="w", padx=10,

                                pady=(5, 0))

        ctk.CTkLabel(card, text=f"Description: {desc}", font=("Arial", 12),

                                wraplength=self.master.winfo_width() - 80).pack(anchor="w", padx=10)

        status_text = f"Status: {status}"

        show_buttons = True

        if request_info:

            try:

                req_type, req_status = request_info.split(":")

                if req_status == "Pending":

                    status_text += f" (Pending {req_type.capitalize()})"; show_buttons = False

                elif req_status == "Approved":

                    status_text += f" (Previously {req_type} Approved)"; show_buttons = req_type != "cancel"

                elif req_status == "Rejected":

                    status_text += f" (Previous {req_type} Rejected)"

            except ValueError:

                pass

        ctk.CTkLabel(card, text=status_text, font=("Arial", 12)).pack(anchor="w", padx=10)

        if show_buttons and status.lower() != 'canceled':

            btn_frame = ctk.CTkFrame(card, fg_color="transparent")

            btn_frame.pack(anchor="w", padx=10, pady=5)

```

```

delay_btn = MyButton(btn_frame, text="Request Delay", width=120,
                      command=lambda aid=assignment_id: SubmitRequestPopup(self.master, aid, "delay",
                                                                              self.safe_network_request,
                                                                              user_id, username,
                                                                              self.create_user_dashboard,
                                                                              self.handle_server_critical_error),
                      fg_color="#004d00", hover_color="#006400")

cancel_btn = MyButton(btn_frame, text="Request Cancellation", width=150,
                      command=lambda aid=assignment_id: SubmitRequestPopup(self.master, aid,
                                                                              "cancel",
                                                                              self.safe_network_request,
                                                                              user_id, username,
                                                                              self.create_user_dashboard,
                                                                              self.handle_server_critical_error),
                      fg_color="#004d00", hover_color="#006400")

delay_btn.pack(side="left", padx=5)
cancel_btn.pack(side="left", padx=5)
card.pack(fill="x", padx=10, pady=10)

```

```

MyButton(self.current_frame, text="Back to Main Menu",
         command=lambda: self.show_user_dashboard_options(user_id, username),
         fg_color="#004d00", hover_color="#006400").pack(pady=20, side="bottom")

```

```

def create_user_data_dashboard(self, user_id, username):
    if self.server_error_active: return
    self.reset_session_timer()
    self.clear_screen()
    self.current_frame = ctk.CTkFrame(self.master)
    self.current_frame.pack(fill="both", expand=True)

    response = self.safe_network_request({"type": "get_user_data", "user_id": user_id},
                                         source_description="create_user_data_dashboard")

```

```

if response is None: return

if response.get("status") == "ok":
    d, p, h, i, hd = response.get("data", (None, None, None, "", None))
else:
    messagebox.showerror("Error", "Failed to load user data.", parent=self.master)
    d, p, h, i, hd = None, None, None, "", None

data_list = [
    ("DAPAR", d if d is not None else "N/A"), ("PROFILE", p if p is not None else "N/A"),
    ("HEBREW", h if h is not None else "N/A"), ("INTERVIEW", i if i else "N/A"),
    ("100 DAY", "Completed" if hd else ("Not Completed" if hd is not None else "N/A"))
]

cards_container_main = ctk.CTkFrame(self.current_frame, fg_color="transparent")
cards_container_main.pack(expand=True, pady=20)

for title, value in data_list:
    card = self.create_data_card_user(cards_container_main, title, value)
    card.pack(pady=10)

MyButton(self.current_frame, text="Back to Main Menu", font=("Arial", 14),
          command=lambda: self.show_user_dashboard_options(user_id, username),
          fg_color="#004d00", hover_color="#006400").pack(pady=20, side="bottom")

def create_data_card_user(self, master_frame, title, value):
    frame = ctk.CTkFrame(master_frame, fg_color="#4CAF50", width=250, height=100,
                        border_width=2, border_color="black", corner_radius=8)
    frame.pack_propagate(False)
    ctk.CTkLabel(frame, text=title, font=("Arial", 16, "bold"), text_color="white").pack(pady=(10, 2))
    ctk.CTkLabel(frame, text=str(value), font=("Arial", 14), text_color="white", wraplength=230).pack(pady=(2,
10),

```

```

        padx=10)

    return frame

def create_user_messages_screen(self, user_id, username):

    if self.server_error_active: return

    self.reset_session_timer()

    self.clear_screen()

    self.current_frame = ctk.CTkFrame(self.master)

    self.current_frame.pack(fill="both", expand=True, padx=10, pady=10)

    ctk.CTkLabel(self.current_frame, text="Send a Message to Admin", font=("Arial", 18, "bold")).pack(pady=10)

    self.subject_entry_usermsg = ctk.CTkEntry(self.current_frame, placeholder_text="Subject")

    self.subject_entry_usermsg.pack(pady=5, padx=20, fill="x")

    self.content_box_usermsg = ctk.CTkTextbox(self.current_frame, height=100)

    self.content_box_usermsg.pack(pady=5, padx=20, fill="x")

    self.user_msg_result_label = ctk.CTkLabel(self.current_frame, text="", text_color="green")

    self.user_msg_result_label.pack(pady=5)

    MyButton(self.current_frame, text="Send", command=lambda:
self.submit_user_message_action(user_id)).pack(

        pady=10)

    ctk.CTkLabel(self.current_frame, text="Your Message History", font=("Arial", 16, "bold")).pack(pady=(15, 5))

    self.messages_history_scroll_frame = ctk.CTkScrollableFrame(self.current_frame, height=200)

    self.messages_history_scroll_frame.pack(fill="both", expand=True, padx=20, pady=(0, 10))

    self._load_user_message_history_display(user_id)

    MyButton(self.current_frame, text="Back to Main Menu",

```

```

        command=lambda: self.show_user_dashboard_options(user_id, username),
        fg_color="#004d00", hover_color="#006400").pack(pady=15, side="bottom")

def submit_user_message_action(self, user_id_for_msg):
    if self.server_error_active: return

    self.reset_session_timer()

    subject = self.subject_entry_usermsg.get().strip()

    content = self.content_box_usermsg.get("1.0", "end-1c").strip()

    if not subject or not content:
        self.user_msg_result_label.configure(text="Please enter both subject and message.", text_color="red")
        return

    self.user_msg_result_label.configure(text="Sending...", text_color="blue")

    threading.Thread(target=self._process_submit_user_message, args=(user_id_for_msg, subject, content),
                     daemon=True).start()

def _process_submit_user_message(self, uid, subj, cont):
    response = self.safe_network_request({
        "type": "send_message", "user_id": uid, "subject": subj, "content": cont
    }, source_description="_process_submit_user_message")

    if response is None: return

    if response.get("status") == "ok":
        self.master.after(0, lambda: self.user_msg_result_label.configure(text="Message sent successfully.",
                                                                            text_color="green"))

        self.master.after(0, self.subject_entry_usermsg.delete, (0, "end"))

        self.master.after(0, self.content_box_usermsg.delete, ("1.0", "end"))

        self.master.after(0, lambda: self._load_user_message_history_display(uid))
    else:
        self.master.after(0, lambda: self.user_msg_result_label.configure(text="Failed to send message.",

```

```

text_color="red"))

def _load_user_message_history_display(self, user_id_to_load):
    if self.server_error_active: return

    for widget in self.messages_history_scroll_frame.winfo_children():
        widget.destroy()

    response = self.safe_network_request({"type": "get_messages", "user_id": user_id_to_load},
                                         source_description="_load_user_message_history_display")

    if response is None: return

    messages = response.get("data", []) if response.get("status") == "ok" else []

    if not messages:
        ctk.CTkLabel(self.messages_history_scroll_frame, text="No messages yet.").pack(pady=10)
        return

    for subject, content, admin_reply, timestamp in messages:
        box = ctk.CTkFrame(self.messages_history_scroll_frame, border_width=1, border_color="gray")
        box.pack(fill="x", pady=5)

        ctk.CTkLabel(box, text=f"Subject: {subject}", font=("Arial", 14, "bold")).pack(anchor="w", padx=10,
                                                                                      pady=(5, 0))

        ctk.CTkLabel(box, text=f"Your Message: {content}", font=("Arial", 12),
                      wraplength=self.master.winfo_width() - 100).pack(anchor="w", padx=10)

        ctk.CTkLabel(box, text=f"Sent: {timestamp}", font=("Arial", 10, "italic")).pack(anchor="w", padx=10,
                                                                                      pady=(0, 5))

        if admin_reply:
            ctk.CTkLabel(box, text=f"Admin's Reply: {admin_reply}", font=("Arial", 12, "bold"),
                          text_color="#004d00", wraplength=self.master.winfo_width() - 100).pack(anchor="w", padx=10,
                                                                                      pady=(0, 5))

        else:
            ctk.CTkLabel(box, text="Awaiting admin response...", font=("Arial", 12, "italic"),

```

```

text_color="gray").pack(anchor="w", padx=10, pady=(0, 5))

def show_request_history(self, user_id, username):
    if self.server_error_active: return

    self.reset_session_timer()

    self.clear_screen()

    self.current_frame = ctk.CTkFrame(self.master)

    self.current_frame.pack(fill="both", expand=True, padx=10, pady=10)

    ctk.CTkLabel(self.current_frame, text="Your Request History", font=("Arial", 20, "bold")).pack(pady=10)

    response = self.safe_network_request({"type": "get_my_requests", "user_id": user_id},
                                         source_description="show_request_history")

    if response is None: return

    requests_data = response.get("data", []) if response.get("status") == "ok" else []

    if not requests_data:

        ctk.CTkLabel(self.current_frame, text="No requests submitted yet.").pack(pady=10)

    else:

        scroll_frame = ctk.CTkScrollableFrame(self.current_frame)

        scroll_frame.pack(fill="both", expand=True, padx=10, pady=5)

        for title, req_type, status, message, timestamp in requests_data:

            box = ctk.CTkFrame(scroll_frame, border_width=1, border_color="gray", fg_color="#f9f9f9")

            box.pack(fill="x", pady=5, padx=5)

            ctk.CTkLabel(box, text=f"Meyun: {title if title else 'N/A'}", font=("Arial", 14, "bold")).pack(
                anchor="w", padx=10, pady=(5, 0))

            ctk.CTkLabel(box, text=f"Type: {req_type.capitalize()} | Status: {status}", font=("Arial", 12)).pack(
                anchor="w", padx=10)

            ctk.CTkLabel(box, text=f"Your Message: {message}", font=("Arial", 12),
                          wraplength=self.master.winfo_width() - 100).pack(anchor="w", padx=10, pady=(0, 5))

            ctk.CTkLabel(box, text=f"Sent on: {timestamp}", font=("Arial", 10, "italic")).pack(anchor="w", padx=10,

```

pady=(0, 5))

```

MyButton(self.current_frame, text="Back to Main Menu",
         command=lambda: self.show_user_dashboard_options(user_id, username),
         fg_color="#004d00", hover_color="#006400").pack(pady=15, side="bottom")

def open_forgot_password_screen(self):
    if self.server_error_active: return

    self.reset_session_timer()

    self.clear_screen()

    self.current_frame = ctk.CTkFrame(self.master)
    self.current_frame.pack(fill="both", expand=True)

    ctk.CTkLabel(self.current_frame, text="Reset Your Password", font=("Arial", 20, "bold")).pack(pady=20)

    self.username_entry_reset_pass = ctk.CTkEntry(self.current_frame, placeholder_text="Enter your
username",
                                                height=40)

    self.username_entry_reset_pass.pack(pady=10, padx=40, fill="x")
    self.username_entry_reset_pass.focus()

    self.reset_email_btn_pass = MyButton(self.current_frame, text="Send Code to Email", font=("Arial", 14),
                                         command=self.send_reset_code_thread, fg_color="#004d00",
                                         hover_color="#006400")

    self.reset_email_btn_pass.pack(pady=10)

    MyButton(self.current_frame, text="Back to Login", command=self.create_login_screen).pack(pady=10)

    self.reset_message_label_pass = ctk.CTkLabel(self.current_frame, text="", font=("Arial", 12),
text_color="red")

    self.reset_message_label_pass.pack(pady=10)

def send_reset_code_thread(self):

```

```

if self.server_error_active: return

self.reset_session_timer()

username = self.username_entry_reset_pass.get().strip()

if not username:

    self.reset_message_label_pass.configure(text="Please enter your username.")

    return

self.reset_email_btn_pass.configure(state="disabled", text="Sending...")

self.reset_message_label_pass.configure(text="")

threading.Thread(target=self._process_send_reset_code, args=(username,), daemon=True).start()


def _process_send_reset_code(self, username):

    response = self.safe_network_request({"type": "request_reset_code", "username": username},

                                         source_description="_process_send_reset_code")

    if response is None: return


    if not self.server_error_active:

        self.master.after(0, lambda: self.reset_email_btn_pass.configure(state="normal", text="Send Code to
Email"))


    if response.get("status") == "ok":

        self.pending_user = (response.get("user_id"), username, response.get("email"), None)

        self.pending_code_context = "reset_password"

        self.master.after(0, self.create_code_verification_screen)

    else:

        self.master.after(0,

                        lambda: self.reset_message_label_pass.configure(text=response.get("message", "Failed.")))


def create_new_password_screen(self, username, email):

    if self.server_error_active: return

    self.reset_session_timer()

    self.clear_screen()

    self.current_frame = ctk.CTkFrame(self.master)

```

```

self.current_frame.pack(fill="both", expand=True)

    ctk.CTkLabel(self.current_frame, text=f"Enter New Password for {username}", font=("Arial", 18,
"bold")).pack(

        pady=20)

self.new_password_entry_final_reset = ctk.CTkEntry(self.current_frame, placeholder_text="New Password",
                                                    height=40, show="*")

self.new_password_entry_final_reset.pack(pady=10, padx=40, fill="x")

self.new_password_entry_final_reset.focus()


self.reset_btn_final_action = MyButton(self.current_frame, text="Reset Password",
                                        command=lambda: self.reset_password_action_thread(username, email),
                                        height=40)

self.reset_btn_final_action.pack(pady=10)


self.final_reset_label_msg = ctk.CTkLabel(self.current_frame, text="", font=("Arial", 12), text_color="red")

self.final_reset_label_msg.pack(pady=10)

MyButton(self.current_frame, text="Back to Login", command=self.create_login_screen).pack(pady=10)


def reset_password_action_thread(self, username, email):

    if self.server_error_active: return

    self.reset_session_timer()

    new_password = self.new_password_entry_final_reset.get()

    if not new_password:

        self.final_reset_label_msg.configure(text="Please enter a new password.")

        return

    if not is_strong_password(new_password):

        self.final_reset_label_msg.configure(text="Password: min 6 chars, letters & digits.")

        return


self.reset_btn_final_action.configure(state="disabled", text="Resetting...")

```

```

self.final_reset_label_msg.configure(text="")

threading.Thread(target=self._process_reset_password, args=(username, email, new_password),
daemon=True).start()

def _process_reset_password(self, username, email, new_password):

    response = self.safe_network_request({

        "type": "reset_password", "username": username, "email": email, "new_password": new_password

    }, source_description="_process_reset_password")

    if response is None: return

    if not self.server_error_active:

        self.master.after(0, lambda: self.reset_btn_final_action.configure(state="normal", text="Reset Password"))

    if response.get("status") == "ok":

        self.master.after(0, lambda: messagebox.showinfo("Success", response.get("message", "Password
reset!"),

                                parent=self.master))

        self.master.after(0, self.create_login_screen)

    else:

        self.master.after(0, lambda: self.final_reset_label_msg.configure(

            text=response.get("message", "Reset failed.")))

if __name__ == "__main__":

    root = ctk.CTk()

    app = App(root)

    if app.server_available:

        # App.__init__ has already configured the root window and created the login screen.

        pass # UI is already running if server was available

    else:

        # Server was not available on startup. App.__init__ returned early.

        # self.network is None, self.server_error_active is False here

```

```
# Show the initial server down popup.

app._show_server_unavailable_popup("Server not available at startup.")
```

```
root.mainloop()
```

### **admin\_dashboard.py:**

```
import customtkinter as ctk

from tkinter import messagebox

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

import matplotlib.pyplot as plt

import hashlib

from PIL import Image

# from network.network_client import NetworkClient # No longer creating its own

from ui_elements import MyButton, BasePopup

from tkcalendar import DateEntry

import datetime

from validators import is_valid_email, is_valid_phone, is_valid_age, is_strong_password, is_valid_dapar,
is_valid_hebrew, is_valid_profile

import random

import string


class NewDatePopup(BasePopup):

    def __init__(self, master, request_id, on_success_callback, safe_request_method, critical_error_handler,
*args, **kwargs):

        super().__init__(master, title="Select New Date", geometry="300x200", *args, **kwargs)

        self.request_id = request_id

        self.on_success_callback = on_success_callback

        self.safe_request = safe_request_method

        self.critical_error_handler = critical_error_handler # To be called by App if needed


        ctk.CTkLabel(self, text="Choose a new date:", font=("Arial", 14)).pack(pady=10)
```

```

today = datetime.date.today()

self.date_picker = DateEntry(self, mindate=today, date_pattern="yyyy-mm-dd", width=18,
                             background='darkblue', foreground='white', borderwidth=2)

self.date_picker.pack(pady=5)

MyButton(self, text="Submit", command=self.save_date).pack(pady=10)

def save_date(self):
    new_date = self.date_picker.get_date().strftime("%Y-%m-%d")
    response = self.safe_request({ # Use the safe_request method passed from App
        "type": "update_date_for_request",
        "request_id": self.request_id,
        "new_date": new_date
    }, source_description="NewDatePopup.save_date")

    if response is None: # Critical server error was handled by App
        self.close_popup() # Close this popup
        return

    if response.get("status") == "ok":
        messagebox.showinfo("Success", "Date updated.", parent=self)
        if self.on_success_callback:
            self.on_success_callback()
        self.close_popup()
    else:
        messagebox.showerror("Error", "Failed to update date.", parent=self)

class AdminDashboard:
    def __init__(self, root, user, app_instance, on_logout): # app_instance holds network, safe_request, etc.
        self.root = root
        self.user = user

```

```

self.app = app_instance # Store the App instance

self.icons = self.load_icons()

self.on_logout_callback = on_logout


self.clear_screen()

self.create_sidebar()

self.create_content_frame()

self.show_default_content()


def load_icons(self):
    icons = {}

    try:
        icons["dashboard"] = ctk.CTkImage(Image.open("icons/dashboard.png"), size=(20, 20))
        icons["users"] = ctk.CTkImage(Image.open("icons/users.png"), size=(20, 20))
        icons["data"] = ctk.CTkImage(Image.open("icons/data.png"), size=(20, 20))
        icons["send"] = ctk.CTkImage(Image.open("icons/send.png"), size=(20, 20))
        icons["logs"] = ctk.CTkImage(Image.open("icons/logs.png"), size=(20, 20))
        icons["meyun"] = ctk.CTkImage(Image.open("icons/meyun.png"), size=(25, 25))
        icons["logout"] = ctk.CTkImage(Image.open("icons/logout.png"), size=(20, 20))
    except Exception as e:
        print("Icon loading failed:", e)

    return icons


def show_default_content(self):
    if self.app.server_error_active: return

    self.clear_content()

    default_frame = ctk.CTkFrame(self.content_frame)

    default_frame.pack(fill="both", expand=True)


    try:
        image = Image.open("logo_admin.png")

```

```

        ctk_image = ctk.CTkImage(light_image=image, dark_image=image, size=(300, 440))

        ctk.CTkLabel(default_frame, image=ctk_image, text="").pack(pady=(80, 20))

        self.default_image = ctk_image # Keep a reference

    except FileNotFoundError:

        ctk.CTkLabel(default_frame, text="Welcome to the Admin Panel\n(Logo not found: logo_admin.png)",
font=("Arial", 20, "bold")).pack(pady=30)

    except Exception as e:

        print(f"Error loading admin logo: {e}")

        ctk.CTkLabel(default_frame, text="Welcome to the Admin Panel", font=("Arial", 20,
"bold")).pack(pady=30)


def clear_screen(self):

    for widget in self.root.winfo_children():

        widget.destroy()


def create_sidebar(self):

    if self.app.server_error_active: return

    self.sidebar = ctk.CTkFrame(self.root, width=200)

    self.sidebar.pack(side="left", fill="y", padx=5, pady=5)

    buttons = [

        ("Dashboard", self.show_dashboard, self.icons.get("dashboard")),

        ("Manage Users", self.show_manage_users, self.icons.get("users")),

        ("Manage User Data", self.show_manage_user_data, self.icons.get("data")),

        ("User Messages", self.show_user_messages, self.icons.get("send")),

        ("Manage Meyunim", self.show_manage_meyunim, self.icons.get("meyun")),

        ("Logout", self.logout_action, self.icons.get("logout"))

    ]

    for text, command, icon in buttons:

        button = MyButton(self.sidebar, text=text, command=command, height=40, width=180,

            image=icon, anchor="w", compound="left", font=("Arial", 14))

```

```

button.pack(pady=7, padx=10)

def create_content_frame(self):
    if self.app.server_error_active: return

    self.content_frame = ctk.CTkFrame(self.root)
    self.content_frame.pack(side="left", fill="both", expand=True, padx=5, pady=5)

def clear_content(self):
    if hasattr(self, 'content_frame') and self.content_frame.winfo_exists():
        for widget in self.content_frame.winfo_children():
            widget.destroy()

    elif self.app.server_error_active:
        pass # Expected if server error occurred before content_frame creation

def show_dashboard(self):
    if self.app.server_error_active: return

    self.clear_content()

    dashboard_frame = ctk.CTkFrame(self.content_frame)
    dashboard_frame.pack(fill="both", expand=True, padx=20, pady=20)

    ctk.CTkLabel(dashboard_frame, text="Dashboard", font=("Arial", 20, "bold")).pack(pady=10)

    button_frame = ctk.CTkFrame(dashboard_frame) # No fg_color for default theme
    button_frame.pack(pady=10)

    self.graph_area = ctk.CTkFrame(dashboard_frame, height=500) # No fg_color
    self.graph_area.pack(fill="both", expand=True, padx=10, pady=10)

def clear_graph_area_local(): # Renamed to avoid conflict if self.clear_graph_area existed
    if self.app.server_error_active: return

```

```

for widget in self.graph_area.winfo_children():
    widget.destroy()

def show_age_distribution():
    if self.app.server_error_active: return

    clear_graph_area_local()

    response = self.app.safe_network_request({"type": "get_age_distribution"},
"AdminDashboard.show_age_distribution")

    if response is None: return

    fig, ax = plt.subplots(figsize=(10.5, 7.5)) # Default backend white
    fig.patch.set_facecolor('#ECECEC') # Match CTk light theme bg slightly
    ax.set_facecolor('FFFFFF')

    if response.get("status") == "ok":
        ages = response.get("ages", [])
        if ages:
            ax.hist(ages, bins=10, edgecolor='black', color='#007ACC') # Example color
            ax.set_title("Age Distribution", color='black')
            ax.set_xlabel("Age", color='black')
            ax.set_ylabel("Number of Users", color='black')
            ax.tick_params(axis='x', colors='black')
            ax.tick_params(axis='y', colors='black')

        else:
            ax.text(0.5, 0.5, "No data available", ha='center', va='center', color='black')
    else:
        ax.text(0.5, 0.5, "Failed to load data", ha='center', va='center', color='red')

    fig.tight_layout()

    canvas = FigureCanvasTkAgg(fig, master=self.graph_area)
    canvas.draw()

```

```

canvas.get_tk_widget().pack(fill="both", expand=True) # fill and expand

plt.close(fig)

# Similar styling for other graph functions:

def show_requests_by_type():

    if self.app.server_error_active: return

    clear_graph_area_local()

    response = self.app.safe_network_request({"type": "get_request_type_stats"},
"AdminDashboard.show_requests_by_type")

    if response is None: return

    if response.get("status") == "ok" and response.get("data"):

        types, counts = zip(*response["data"])

        fig, ax = plt.subplots(figsize=(10.5, 7.5)); fig.patch.set_facecolor('#ECECEC');
ax.set_facecolor('#FFFFFF')

        ax.bar(types, counts, color='skyblue', edgecolor='black')

        ax.set_title("Requests by Type", color='black'); ax.set_ylabel("Count", color='black')

        ax.tick_params(axis='x', colors='black'); ax.tick_params(axis='y', colors='black')

        fig.tight_layout(); canvas = FigureCanvasTkAgg(fig, master=self.graph_area); canvas.draw();
canvas.get_tk_widget().pack(fill="both", expand=True); plt.close(fig)

    else:

        fig, ax = plt.subplots(figsize=(10.5, 7.5)); fig.patch.set_facecolor('#ECECEC');
ax.set_facecolor('#FFFFFF')

        ax.text(0.5, 0.5, "No data or failed to load", ha='center', va='center', color='red')

        fig.tight_layout(); canvas = FigureCanvasTkAgg(fig, master=self.graph_area); canvas.draw();
canvas.get_tk_widget().pack(fill="both", expand=True); plt.close(fig)

def show_requests_by_status():

    if self.app.server_error_active: return

    clear_graph_area_local()

    response = self.app.safe_network_request({"type": "get_request_status_stats"},
"AdminDashboard.show_requests_by_status")

    if response is None: return

    if response.get("status") == "ok" and response.get("data"):

        statuses, counts = zip(*response["data"])

```

```

fig, ax = plt.subplots(figsize=(10.5, 7.5)); fig.patch.set_facecolor('#ECECECE');
ax.set_facecolor('#FFFFFF')

ax.bar(statuses, counts, color='lightgreen', edgecolor='black')

ax.set_title("Requests by Status", color='black'); ax.set_ylabel("Count", color='black')

ax.tick_params(axis='x', colors='black'); ax.tick_params(axis='y', colors='black')

fig.tight_layout(); canvas = FigureCanvasTkAgg(fig, master=self.graph_area); canvas.draw();
canvas.get_tk_widget().pack(fill="both", expand=True); plt.close(fig)

else:

fig, ax = plt.subplots(figsize=(10.5, 7.5)); fig.patch.set_facecolor('#ECECECE');
ax.set_facecolor('#FFFFFF')

ax.text(0.5, 0.5, "No data or failed to load", ha='center', va='center', color='red')

fig.tight_layout(); canvas = FigureCanvasTkAgg(fig, master=self.graph_area); canvas.draw();
canvas.get_tk_widget().pack(fill="both", expand=True); plt.close(fig)


def show_meyunim_by_month():

if self.app.server_error_active: return

clear_graph_area_local()

response = self.app.safe_network_request({"type": "get_meyunim_per_month"},
"AdminDashboard.show_meyunim_by_month")

if response is None: return

if response.get("status") == "ok" and response.get("data"):

months, counts = zip(*response["data"])

fig, ax = plt.subplots(figsize=(10.5, 7.5)); fig.patch.set_facecolor('#ECECECE');
ax.set_facecolor('#FFFFFF')

ax.plot(months, counts, marker='o', color='orange')

ax.set_title("Meyunim per Month", color='black'); ax.set_ylabel("Count", color='black');
ax.set_xlabel("Month", color='black')

plt.setp(ax.get_xticklabels(), rotation=45, ha="right", color='black')

ax.tick_params(axis='y', colors='black')

fig.tight_layout(); canvas = FigureCanvasTkAgg(fig, master=self.graph_area); canvas.draw();
canvas.get_tk_widget().pack(fill="both", expand=True); plt.close(fig)

else:

fig, ax = plt.subplots(figsize=(10.5, 7.5)); fig.patch.set_facecolor('#ECECECE');
ax.set_facecolor('#FFFFFF')

ax.text(0.5, 0.5, "No data or failed to load", ha='center', va='center', color='red')

fig.tight_layout(); canvas = FigureCanvasTkAgg(fig, master=self.graph_area); canvas.draw();
canvas.get_tk_widget().pack(fill="both", expand=True); plt.close(fig)

```

```

def show_messages_by_week():

    if self.app.server_error_active: return

    clear_graph_area_local()

    response = self.app.safe_network_request({"type": "get_messages_per_week"},
"AdminDashboard.show_messages_by_week")

    if response is None: return

    if response.get("status") == "ok" and response.get("data"):

        weeks, counts = zip(*response["data"])

        fig, ax = plt.subplots(figsize=(10.5, 7.5)); fig.patch.set_facecolor('#ECECECE');
ax.set_facecolor('#FFFFFF')

        ax.bar(weeks, counts, color='plum', edgecolor='black')

        ax.set_title("Messages per Week", color='black'); ax.set_ylabel("Count", color='black');
ax.set_xlabel("Week", color='black')

        plt.setp(ax.get_xticklabels(), rotation=45, ha="right", color='black')

        ax.tick_params(axis='y', colors='black')

        fig.tight_layout(); canvas = FigureCanvasTkAgg(fig, master=self.graph_area); canvas.draw();
canvas.get_tk_widget().pack(fill="both", expand=True); plt.close(fig)

    else:

        fig, ax = plt.subplots(figsize=(10.5, 7.5)); fig.patch.set_facecolor('#ECECECE');
ax.set_facecolor('#FFFFFF')

        ax.text(0.5, 0.5, "No data or failed to load", ha='center', va='center', color='red')

        fig.tight_layout(); canvas = FigureCanvasTkAgg(fig, master=self.graph_area); canvas.draw();
canvas.get_tk_widget().pack(fill="both", expand=True); plt.close(fig)

    MyButton(button_frame, text="Age Distribution", command=show_age_distribution).pack(side="left",
padx=5)

    MyButton(button_frame, text="Requests by Type", command=show_requests_by_type).pack(side="left",
padx=5)

    MyButton(button_frame, text="Requests by Status", command=show_requests_by_status).pack(side="left",
padx=5)

    MyButton(button_frame, text="Meyunim per Month",
command=show_meyunim_by_month).pack(side="left", padx=5)

    MyButton(button_frame, text="Messages per Week",
command=show_messages_by_week).pack(side="left", padx=5)

show_age_distribution() # Default graph

```

```

def show_manage_users(self):

    if self.app.server_error_active: return

    self.clear_content()

    frame = ctk.CTkFrame(self.content_frame)

    frame.pack(fill="both", expand=True, padx=20, pady=20)

    ctk.CTkLabel(frame, text="Manage Users", font=("Arial", 20, "bold")).pack(pady=10)

    form_frame = ctk.CTkFrame(frame) # Renamed to avoid conflict
    form_frame.pack(pady=10)

    self.username_entry = ctk.CTkEntry(form_frame, placeholder_text="Username")
    self.email_entry = ctk.CTkEntry(form_frame, placeholder_text="Email")
    self.id_entry = ctk.CTkEntry(form_frame, placeholder_text="ID Number (9 digits)")

    self.username_entry.pack(pady=4, padx=5, fill="x")
    self.email_entry.pack(pady=4, padx=5, fill="x")
    self.id_entry.pack(pady=4, padx=5, fill="x")

    self.is_admin_var = ctk.BooleanVar()
    self.is_admin_checkbox = ctk.CTkCheckBox(form_frame, text="Is Admin", variable=self.is_admin_var)
    self.is_admin_checkbox.pack(pady=4)

    add_btn = MyButton(form_frame, text="Add User", command=self.add_user, image=self.icons.get("users"),
compound="left")

    add_btn.pack(pady=8)

    self.users_table = ctk.CTkTextbox(frame, wrap="none", width=700, height=300) # Increased width
    self.users_table.pack(pady=10, fill="x", expand=True)

    self.populate_users_table()

```

```

def populate_users_table(self):

    if self.app.server_error_active: return

    self.users_table.configure(state="normal") # Ensure textbox is writable

    try:

        self.users_table.configure(state="normal")

        self.users_table.delete("1.0", "end")

    except Exception as e:

        print(f"Textbox delete error: {e}")

        return

    response = self.app.safe_network_request({"type": "get_all_users"},
"AdminDashboard.populate_users_table")

    if response is None: self.users_table.configure(state="disabled"); return

    if response.get("status") != "ok":

        self.users_table.insert("end", "Failed to load users.\n")

        self.users_table.configure(state="disabled")

        return

    users = response.get("data", [])

    if not users:

        self.users_table.insert("end", "No users found or all are admins.\n")

    else:

        header = f"{'ID':<5} | {'Username':<20} | {'Role':<10} | {'Phone':<15} | {'Address':<30} | {'Age':<5}\n"

        self.users_table.insert("end", header)

        self.users_table.insert("end", "-" * (len(header) - 1) + "\n") # Separator line

        for uid, username, is_admin, phone, address, age in users:

            if is_admin: # Skip admins as per original logic

                continue

            role = "User"

            # Ensure data is string and handle None for display

            s_uid = str(uid if uid is not None else 'N/A')

            s_username = str(username if username else 'N/A')

```

```

s_phone = str(phone if phone else 'N/A')

s_address = str(address if address else 'N/A')

s_age = str(age if age is not None else 'N/A')

line = f"{s_uid:<5} | {s_username:<20} | {role:<10} | {s_phone:<15} | {s_address:<30} | {s_age:<5}\n"

self.users_table.insert("end", line)

self.users_table.configure(state="disabled") # Make read-only

def add_user(self):

    if self.app.server_error_active: return

    username = self.username_entry.get().strip()

    email = self.email_entry.get().strip()

    password = generate_strong_password() # Generate password client-side

    id_number = self.id_entry.get().strip()

    is_admin_val = 1 if self.is_admin_var.get() else 0

    if not username:

        messagebox.showerror("Error", "Username is required.", parent=self.root)

        return

    if not is_valid_email(email):

        messagebox.showerror("Invalid Email", "Please enter a valid email address.", parent=self.root)

        return

    if not id_number.isdigit() or len(id_number) != 9:

        messagebox.showerror("Invalid ID", "ID number must be 9 digits.", parent=self.root)

        return

    response = self.app.safe_network_request({

        "type": "add_user",

        "username": username,

        "email": email,

        "password": password, # Server will hash this password for storage

        "raw_password": password, # For sending in email

```

```

        "id_number": id_number,

        "is_admin": is_admin_val # Send the admin status
    }, "AdminDashboard.add_user")

if response is None: return # Server error handled

if response.get("status") == "ok":

    messagebox.showinfo("Success", "User added successfully. Password sent to email.", parent=self.root)

    self.populate_users_table() # Refresh the list

    # Clear entry fields

    self.username_entry.delete(0, "end")

    self.email_entry.delete(0, "end")

    self.id_entry.delete(0, "end")

    self.is_admin_var.set(False)

else:

    messagebox.showerror("Error", response.get("message", "Failed to add user."), parent=self.root)

def show_manage_user_data(self):

    if self.app.server_error_active: return

    self.clear_content()

    manage_data_frame = ctk.CTkFrame(self.content_frame)

    manage_data_frame.pack(fill="both", expand=True, padx=20, pady=20)

    ctk.CTkLabel(manage_data_frame, text="Manage User Data", font=("Arial", 20, "bold")).pack(pady=10)

    search_frame = ctk.CTkFrame(manage_data_frame)

    search_frame.pack(pady=5, fill="x", padx=10)

    ctk.CTkLabel(search_frame, text="Search Username:").pack(side="left", padx=(0,5))

    self.search_var = ctk.StringVar()

    self.search_var.trace_add("write", self.update_user_selector)

    self.search_entry = ctk.CTkEntry(search_frame, textvariable=self.search_var)

    self.search_entry.pack(side="left", padx=5, fill="x", expand=True)

```

```

self.selected_user_var = ctk.StringVar()

self.user_selector = ctk.CTkComboBox(manage_data_frame, variable=self.selected_user_var,
command=lambda choice: self.load_user_data()) # Load on selection

self.user_selector.pack(pady=10, fill="x", padx=10)

# Frame for data entry fields

data_entry_fields_frame = ctk.CTkFrame(manage_data_frame)

data_entry_fields_frame.pack(pady=10, fill="x", padx=10)

# Configure columns for grid layout

data_entry_fields_frame.columnconfigure(0, weight=1)
data_entry_fields_frame.columnconfigure(1, weight=2)
data_entry_fields_frame.columnconfigure(2, weight=1)
data_entry_fields_frame.columnconfigure(3, weight=2)

ctk.CTkLabel(data_entry_fields_frame, text="Email:").grid(row=0, column=0, padx=5, pady=2, sticky="e")
self.email_data_entry = ctk.CTkEntry(data_entry_fields_frame, placeholder_text="Email")
self.email_data_entry.grid(row=0, column=1, padx=5, pady=2, sticky="ew")

ctk.CTkLabel(data_entry_fields_frame, text="Address:").grid(row=1, column=0, padx=5, pady=2, sticky="e")
self.address_data_entry = ctk.CTkEntry(data_entry_fields_frame, placeholder_text="Address")
self.address_data_entry.grid(row=1, column=1, padx=5, pady=2, sticky="ew")

ctk.CTkLabel(data_entry_fields_frame, text="Age:").grid(row=0, column=2, padx=5, pady=2, sticky="e")
self.age_data_entry = ctk.CTkEntry(data_entry_fields_frame, placeholder_text="Age")
self.age_data_entry.grid(row=0, column=3, padx=5, pady=2, sticky="ew")

ctk.CTkLabel(data_entry_fields_frame, text="Phone:").grid(row=1, column=2, padx=5, pady=2, sticky="e")
self.phone_entry = ctk.CTkEntry(data_entry_fields_frame, placeholder_text="Phone")
self.phone_entry.grid(row=1, column=3, padx=5, pady=2, sticky="ew")

```

```

    ctk.CTkLabel(data_entry_fields_frame, text="DAPAR:").grid(row=2, column=0, padx=5, pady=2, sticky="e")

    self.dapar_entry = ctk.CTkEntry(data_entry_fields_frame, placeholder_text="DAPAR (10-90)")

    self.dapar_entry.grid(row=2, column=1, padx=5, pady=2, sticky="ew")


    ctk.CTkLabel(data_entry_fields_frame, text="Profile:").grid(row=3, column=0, padx=5, pady=2, sticky="e")

    self.profile_entry = ctk.CTkEntry(data_entry_fields_frame, placeholder_text="Profile (e.g., 97, 82)")

    self.profile_entry.grid(row=3, column=1, padx=5, pady=2, sticky="ew")


    ctk.CTkLabel(data_entry_fields_frame, text="Hebrew:").grid(row=2, column=2, padx=5, pady=2, sticky="e")

    self.hebrew_entry = ctk.CTkEntry(data_entry_fields_frame, placeholder_text="Hebrew (5-8)")

    self.hebrew_entry.grid(row=2, column=3, padx=5, pady=2, sticky="ew")


    ctk.CTkLabel(data_entry_fields_frame, text="Interview:").grid(row=3, column=2, padx=5, pady=2, sticky="e")

    self.interview_entry = ctk.CTkEntry(data_entry_fields_frame, placeholder_text="Interview Notes")

    self.interview_entry.grid(row=3, column=3, padx=5, pady=2, sticky="ew")


    self.hundred_day_var = ctk.BooleanVar()

    self.hundred_day_checkbox = ctk.CTkCheckBox(data_entry_fields_frame, text="100 DAY Completed",
        variable=self.hundred_day_var)

    self.hundred_day_checkbox.grid(row=4, column=0, columnspan=2, padx=5, pady=10, sticky="w")


    buttons_frame = ctk.CTkFrame(manage_data_frame)

    buttons_frame.pack(pady=20)

    save_changes_btn = MyButton(buttons_frame, text="Save Changes", command=self.save_user_data)

    save_changes_btn.pack(side="left", padx=10)

    reset_changes_btn = MyButton(buttons_frame, text="Reset Changes",
        command=self.reset_user_data_fields) # Reñamed

    reset_changes_btn.pack(side="left", padx=10)


    self.load_users_to_selector() # Initial load

```

```

def load_users_to_selector(self): # Also called by update_user_selector

    if self.app.server_error_active: return

    current_search = self.search_var.get().lower() if hasattr(self, 'search_var') else ""

    response = self.app.safe_network_request({"type": "get_all_users"},
"AdminDashboard.load_users_to_selector")

    if response is None: self.user_selector.configure(values=[]); return

    if response.get("status") == "ok":

        users = response.get("data", [])

        # Filter out admins AND filter by search term

        regular_usernames = [u[1] for u in users if not u[2] and (not current_search or current_search in
u[1].lower())]

        self.user_selector.configure(values=regular_usernames)

        if not regular_usernames and current_search: # if search yielded no results

            self.user_selector.set("No matching users")

        elif not regular_usernames:

            self.user_selector.set("No users available")

        elif self.selected_user_var.get() not in regular_usernames: # If current selection invalid or empty

            if regular_usernames : self.selected_user_var.set(regular_usernames[0]) # Select first if list not empty

            else: self.selected_user_var.set("") # Clear if list became empty

        else:

            self.user_selector.configure(values=[])

            self.user_selector.set("Failed to load users")

        # After updating values, if a user is selected, trigger data load.

        # This is now handled by combobox command or explicit call after setting selection.

        if self.selected_user_var.get() and self.selected_user_var.get() not in ["No matching users", "Failed to load
users", "No users available"]:

            self.load_user_data() # Load data for the (newly) selected user

        else:

            self.clear_user_data_fields() # Clear fields if no valid user selected

def update_user_selector(self, *args): # Called on search_var change

```

```

self.load_users_to_selector() # Re-uses the logic to filter and update ComboBox

def clear_user_data_fields(self):
    if hasattr(self, 'email_data_entry'): self.email_data_entry.delete(0, "end")
    if hasattr(self, 'address_data_entry'): self.address_data_entry.delete(0, "end")
    if hasattr(self, 'age_data_entry'): self.age_data_entry.delete(0, "end")
    if hasattr(self, 'phone_entry'): self.phone_entry.delete(0, "end")
    if hasattr(self, 'dapar_entry'): self.dapar_entry.delete(0, "end")
    if hasattr(self, 'profile_entry'): self.profile_entry.delete(0, "end")
    if hasattr(self, 'hebrew_entry'): self.hebrew_entry.delete(0, "end")
    if hasattr(self, 'interview_entry'): self.interview_entry.delete(0, "end")
    if hasattr(self, 'hundred_day_var'): self.hundred_day_var.set(False)
    self.original_data_loaded = None # Clear loaded data cache

def load_user_data(self, event=None): # event=None for direct calls
    if self.app.server_error_active: return
    username = self.selected_user_var.get()
    if not username or username in ["No matching users", "Failed to load users", "No users available"]:
        self.clear_user_data_fields()
        return

    response = self.app.safe_network_request({
        "type": "load_user_data", "username": username
    }, f"AdminDashboard.load_user_data for {username}")
    if response is None:
        self.clear_user_data_fields()
        return

    if response.get("status") != "ok":
        messagebox.showerror("Error", f"Failed to load data for {username}.", parent=self.root)
        self.clear_user_data_fields()
        return

```

```

data = response.get("data", {})

self.original_data_loaded = data.copy() # Store a copy for reset


self.email_data_entry.delete(0, "end")

self.email_data_entry.insert(0, str(data.get("email", "") or ""))

self.address_data_entry.delete(0, "end")

self.address_data_entry.insert(0, str(data.get("address", "") or ""))

self.age_data_entry.delete(0, "end")

self.age_data_entry.insert(0, str(data.get("age", "") if data.get("age") is not None else ""))

self.phone_entry.delete(0, "end")

self.phone_entry.insert(0, str(data.get("phone", "") or ""))

self.dapar_entry.delete(0, "end")

self.dapar_entry.insert(0, str(data.get("dapar", "") if data.get("dapar") is not None else ""))

self.profile_entry.delete(0, "end")

self.profile_entry.insert(0, str(data.get("profile", "") if data.get("profile") is not None else ""))

self.hebrew_entry.delete(0, "end")

self.hebrew_entry.insert(0, str(data.get("hebrew", "") if data.get("hebrew") is not None else ""))

self.interview_entry.delete(0, "end")

self.interview_entry.insert(0, str(data.get("interview", "") or ""))

self.hundred_day_var.set(bool(data.get("hundred_day", 0)))


def reset_user_data_fields(self): # Renamed

    if self.app.server_error_active: return

    if hasattr(self, 'original_data_loaded') and self.original_data_loaded:

        data = self.original_data_loaded

        self.email_data_entry.delete(0, "end")

        self.email_data_entry.insert(0, str(data.get("email", "") or ""))

        self.address_data_entry.delete(0, "end")

        self.address_data_entry.insert(0, str(data.get("address", "") or ""))

        self.age_data_entry.delete(0, "end")

        self.age_data_entry.insert(0, str(data.get("age", "") if data.get("age") is not None else ""))

```

```

self.phone_entry.delete(0, "end")

self.phone_entry.insert(0, str(data.get("phone", "") or ""))

self.dapar_entry.delete(0, "end")

self.dapar_entry.insert(0, str(data.get("dapar", "") if data.get("dapar") is not None else ""))

self.profile_entry.delete(0, "end")

self.profile_entry.insert(0, str(data.get("profile", "") if data.get("profile") is not None else ""))

self.hebrew_entry.delete(0, "end")

self.hebrew_entry.insert(0, str(data.get("hebrew", "") if data.get("hebrew") is not None else ""))

self.interview_entry.delete(0, "end")

self.interview_entry.insert(0, str(data.get("interview", "") or ""))

self.hundred_day_var.set(bool(data.get("hundred_day", 0)))

messagebox.showinfo("Reset", "Fields have been reset to last loaded data.", parent=self.root)

else:

    messagebox.showwarning("Reset", "No data loaded to reset to. Please load user data first.",
                           parent=self.root)

def save_user_data(self):

    if self.app.server_error_active: return

    username = self.selected_user_var.get()

    if not username or username in ["No matching users", "Failed to load users", "No users available"]:

        messagebox.showerror("Error", "Please select a valid user.", parent=self.root)

        return

    email = self.email_data_entry.get().strip()

    address = self.address_data_entry.get().strip()

    age_str = self.age_data_entry.get().strip()

    phone = self.phone_entry.get().strip()

    dapar_raw = self.dapar_entry.get().strip()

    profile_raw = self.profile_entry.get().strip()

    hebrew_raw = self.hebrew_entry.get().strip()

    interview = self.interview_entry.get().strip() # Allow empty interview notes

    hundred_day = 1 if self.hundred_day_var.get() else 0

```

```

# Validations

if not is_valid_email(email): messagebox.showerror("Invalid Email", "Please enter a valid email address.",
parent=self.root); return

age = None

if age_str: # Age is optional

    if not is_valid_age(age_str): messagebox.showerror("Invalid Age", "Please enter a valid age (1-119).",
parent=self.root); return

    age = int(age_str)

if phone: # Phone is optional

    if not is_valid_phone(phone): messagebox.showerror("Invalid Phone", "Phone must be 9-10 digits.",
parent=self.root); return

dapar = None

if dapar_raw: # DAPAR is optional

    if not is_valid_dapar(dapar_raw): messagebox.showerror("Invalid DAPAR", "DAPAR must be 10-90 (steps
of 10).", parent=self.root); return

    dapar = int(dapar_raw)

profile = None

if profile_raw: # Profile is optional

    if not is_valid_profile(profile_raw): messagebox.showerror("Invalid Profile", "Profile value is not valid.",
parent=self.root); return

    profile = int(profile_raw)

hebrew = None

if hebrew_raw: # Hebrew is optional

    if not is_valid_hebrew(hebrew_raw): messagebox.showerror("Invalid Hebrew Score", "Hebrew score must
be 5-8.", parent=self.root); return

    hebrew = int(hebrew_raw)

response = self.app.safe_network_request({

    "type": "update_user_data", "username": username,

    "account_data": {"email": email, "address": address, "age": age, "phone": phone},

    "system_data": {"dapar": dapar, "profile": profile, "hebrew": hebrew, "interview": interview, "hundred_day":
hundred_day}

}, "AdminDashboard.save_user_data")

if response is None: return

```

```

if response.get("status") == "ok":

    messagebox.showinfo("Success", f"User data for {username} updated.", parent=self.root)

    self.load_user_data() # Reload to refresh original_data_loaded and fields

else:

    messagebox.showerror("Error", response.get("message", "Update failed."), parent=self.root)


def show_manage_meyunim(self):

    if self.app.server_error_active: return

    self.clear_content()

    tabs = ctk.CTkTabview(self.content_frame, width=800, height=600) # fg_color removed for default

    tabs.pack(padx=20, pady=20, fill="both", expand=True)

    self.build_create_meyun_tab(tabs.add("Create Meyun"))

    self.build_requests_tab(tabs.add("View Requests"))

    self.build_all_meyunim_tab(tabs.add("All Meyunim"))


def logout_action ככככ(self):

    if self.app.server_error_active:

        if hasattr(self.app, 'on_closing') and callable(self.app.on_closing):

            self.app.on_closing()

        return

    self.clear_screen()

    if callable(self.on_logout_callback):

        self.on_logout_callback()


def create_meyun_and_assign(self):

    if self.app.server_error_active: return

    title = self.meyun_title.get().strip()

    desc = self.meyun_description.get().strip()

    try:

        date = self.meyun_date.get_date().strftime("%Y-%m-%d")

    except AttributeError: # Handle case where DateEntry might not be fully initialized

        messagebox.showerror("Error", "Invalid date selected.", parent=self.root)

```

```

return

user_ids = [uid for uid, var in self.user_checkboxes.items() if var.get()]

if not title or not date or not user_ids: # Description can be optional
    messagebox.showerror("Error", "Title, date, and at least one user are required.", parent=self.root)
    return

response = self.app.safe_network_request({
    "type": "create_meyun", "title": title, "description": desc, "date": date, "user_ids": user_ids
}, "AdminDashboard.create_meyun_and_assign")

if response is None: return

if response.get("status") == "ok":
    messagebox.showinfo("Success", "Meyun created and assigned.", parent=self.root)
    self.show_manage_meyunim() # Refresh the whole meyunim management view
else:
    messagebox.showerror("Error", response.get("message", "Failed to create meyun."), parent=self.root)

def build_create_meyun_tab(self, parent_tab): # Renamed parameter
    if self.app.server_error_active: return

    ctk.CTkLabel(parent_tab, text="Create New Meyun", font=("Arial", 20, "bold")).pack(pady=10)

    self.meyun_title = ctk.CTkEntry(parent_tab, placeholder_text="Title"); self.meyun_title.pack(pady=4, fill="x",
    padx=20)

    self.meyun_description = ctk.CTkEntry(parent_tab, placeholder_text="Description (Optional)");
    self.meyun_description.pack(pady=4, fill="x", padx=20)

    date_frame = ctk.CTkFrame(parent_tab, fg_color="transparent"); date_frame.pack(pady=4)

    ctk.CTkLabel(date_frame, text="Date:").pack(side="left", padx=(0,5))

    self.meyun_date = DateEntry(date_frame, mindate=datetime.date.today(), date_pattern="yyyy-mm-dd",
    width=18, background='darkblue', foreground='white', borderwidth=2); self.meyun_date.pack(side="left")

    response = self.app.safe_network_request({"type": "get_all_users"},
    "AdminDashboard.build_create_meyun_tab.get_users")

    users_data = []

```

```

if response is None: # Error handled by safe_network_request

    users_data = []

elif response.get("status") == "ok":

    users_data = response.get("data", [])

else:

    ctk.CTkLabel(parent_tab, text="Failed to load users for assignment.", text_color="red").pack()

self.user_checkboxes = {}

user_frame = ctk.CTkScrollableFrame(parent_tab, height=200, label_text="Assign to Users");
user_frame.pack(pady=10, fill="x", padx=20)

if not any(not u[2] for u in users_data): # Check if there are any non-admin users

    ctk.CTkLabel(user_frame, text="No regular users available to assign.").pack(padx=5, pady=5)

else:

    for uid, username, is_admin, *_ in users_data:

        if is_admin: continue

        var = ctk.BooleanVar()

        cb = ctk.CTkCheckBox(user_frame, text=username, variable=var); cb.pack(anchor="w", padx=5)

        self.user_checkboxes[uid] = var

    MyButton(parent_tab, text="Create Meyun and Assign",
command=self.create_meyun_and_assign).pack(pady=20)

def build_requests_tab(self, parent_tab):

    if self.app.server_error_active: return

    filter_frame = ctk.CTkFrame(parent_tab); filter_frame.pack(pady=10, fill="x", padx=10)

    self.request_type_filter = ctk.StringVar(value="All")

    self.status_filter = ctk.StringVar(value="All")

    ctk.CTkLabel(filter_frame, text="Type:").pack(side="left", padx=(0,5))

    ctk.CTkComboBox(filter_frame, variable=self.request_type_filter, values=["All", "delay", "cancel"],
command=lambda _: self.refresh_requests_ui(), width=100).pack(side="left", padx=5)

    ctk.CTkLabel(filter_frame, text="Status:").pack(side="left", padx=(15,5))

    ctk.CTkComboBox(filter_frame, variable=self.status_filter, values=["All", "Pending", "Approved", "Rejected"],
command=lambda _: self.refresh_requests_ui(), width=120).pack(side="left", padx=5)

```

```

self.requests_frame = ctk.CTkScrollableFrame(parent_tab, label_text="User Requests");
self.requests_frame.pack(fill="both", expand=True, padx=10, pady=5)

self.refresh_requests_ui()

def refresh_requests_ui(self):
    if self.app.server_error_active: return

    if not hasattr(self, 'requests_frame') or not self.requests_frame.winfo_exists(): return

    for widget in self.requests_frame.winfo_children(): widget.destroy()

    response = self.app.safe_network_request({"type": "get_all_requests"},
"AdminDashboard.refresh_requests_ui")

    if response is None: ctk.CTkLabel(self.requests_frame, text="Error loading requests.",
text_color="red").pack(); return

    requests_data_filtered = []

    if response.get("status") == "ok":

        req_type_filter_val = self.request_type_filter.get()

        status_filter_val = self.status_filter.get()

        for req_tuple in response.get("data", []): # Renamed req to req_tuple

            if len(req_tuple) == 7:

                _, _, _, rtype, _, rstatus = req_tuple # Unpack directly

                if (req_type_filter_val == "All" or rtype == req_type_filter_val) and \

                    (status_filter_val == "All" or rstatus == status_filter_val):

                    requests_data_filtered.append(req_tuple)

            else:

                print(f"Warning: Malformed request data tuple: {req_tuple}")

    if not requests_data_filtered:

        ctk.CTkLabel(self.requests_frame, text="No requests match current filters.", font=("Arial", 14,
"italic")).pack(pady=20)

    return

    for req_id, username, title, date, req_type, msg, status in requests_data_filtered:

        safe_username = username if username else "N/A"

```

```

safe_title = title if title else "N/A"

safe_date = date if date else "N/A"

box = ctk.CTkFrame(self.requests_frame, border_width=1, border_color="gray"); box.pack(fill="x",
pady=5, padx=5) # Reduced padx

title_line = f"User: {safe_username} | Meyun: {safe_title} ({safe_date})"

if status == "Approved" and req_type == "cancel": title_line += " [MEYUN CANCELED]"

elif status == "Approved" and req_type == "delay": title_line += f" [DELAYED TO: {safe_date}]" # Date
should be updated one

ctk.CTkLabel(box, text=title_line, font=("Arial", 14, "bold")).pack(anchor="w", padx=10, pady=(5,0))

status_color = {"Pending": "#FFA500", "Approved": "green", "Rejected": "red"}.get(status, "black") #
Brighter pending

ctk.CTkLabel(box, text=f"Request: {req_type.capitalize()} | Status: {status}", font=("Arial", 12),
text_color=status_color).pack(anchor="w", padx=10)

ctk.CTkLabel(box, text=f"Message: {msg}", font=("Arial", 12),
wraplength=self.requests_frame.winfo_width() - 50).pack(anchor="w", padx=10, pady=(0,5)) # Dynamic
wraplength

if status == "Pending":

    btns_frame = ctk.CTkFrame(box, fg_color="transparent"); btns_frame.pack(pady=5, padx=10,
anchor="e")

    MyButton(btns_frame, text="Approve", command=lambda rid=req_id, rt=req_type:
self.handle_approval(rid, rt), width=80).pack(side="left", padx=5)

    MyButton(btns_frame, text="Reject", command=lambda rid=req_id: self.handle_rejection(rid),
width=80, fg_color="#B22222", hover_color="#8B0000").pack(side="left", padx=5)

def build_all_meyunim_tab(self, parent_tab):

    if self.app.server_error_active: return

    frame = ctk.CTkScrollableFrame(parent_tab, label_text="All Created Meyunim"); frame.pack(fill="both",
expand=True, padx=10, pady=5)

    response = self.app.safe_network_request({"type": "get_all_meyunim"},
"AdminDashboard.build_all_meyunim_tab")

    if response is None: ctk.CTkLabel(frame, text="Error loading meyunim.", text_color="red").pack(); return

    if response.get("status") != "ok":

        ctk.CTkLabel(frame, text="Failed to load meyunim.", font=("Arial", 14), text_color="red").pack(pady=10)

        return

    all_meyunim = response.get("data", [])

    if not all_meyunim:

```

```

    ctk.CTkLabel(frame, text="No meyunim created yet.", font=("Arial", 14, "italic")).pack(pady=10)

    return

for m_id, title, desc, date, usernames in all_meyunim:

    box = ctk.CTkFrame(frame, border_width=1, border_color="gray"); box.pack(fill="x", pady=5, padx=5)

    is_effectively_canceled = not usernames # Meyun exists but no one is assigned (could be after all
canceled)

    status_text = "[No Users Assigned]" if is_effectively_canceled else ""

    ctk.CTkLabel(box, text=f"{title} ({date}) {status_text}", font=("Arial", 14, "bold")).pack(anchor="w",
padx=10, pady=(5,0))

    ctk.CTkLabel(box, text=f"Description: {desc if desc else 'N/A'}", font=("Arial", 12)).pack(anchor="w",
padx=10)

    assigned_users_text = "Assigned to: " + (' , '.join(usernames) if usernames else "None")

    ctk.CTkLabel(box, text=assigned_users_text, font=("Arial", 11, "italic")).pack(anchor="w", padx=10,
pady=(0,5))

    MyButton(box, text="Delete Meyun", fg_color="red", hover_color="#cc0000", width=120,

        command=lambda mid=m_id: self.confirm_delete_meyun(mid)).pack(pady=5, padx=10, anchor="e")


def confirm_delete_meyun(self, meyun_id):

    if self.app.server_error_active: return

    if not messagebox.askyesno("Confirm Delete", f"Are you sure you want to permanently delete this meyun
(ID: {meyun_id}) and all its assignments/requests? This cannot be undone.", parent=self.root, icon='warning'):

        return

    response = self.app.safe_network_request({"type": "delete_meyun", "meyun_id": meyun_id},
"AdminDashboard.confirm_delete_meyun")

    if response is None: return

    if response.get("status") == "ok":

        messagebox.showinfo("Success", "Meyun deleted.", parent=self.root)

        self.show_manage_meyunim() # Refresh the view

    else:

        messagebox.showerror("Error", response.get("message", "Failed to delete meyun."), parent=self.root)


def handle_rejection(self, request_id):

    if self.app.server_error_active: return

```

```

response = self.app.safe_network_request({
    "type": "update_request_status", "request_id": request_id, "new_status": "Rejected"
}, "AdminDashboard.handle_rejection")

if response is None: return

if response.get("status") == "ok":
    messagebox.showinfo("Success", "Request rejected.", parent=self.root)
    self.refresh_requests_ui()
else:
    messagebox.showerror("Error", response.get("message", "Failed to reject request."), parent=self.root)

def handle_approval(self, request_id, request_type):
    if self.app.server_error_active: return

    if request_type == "cancel":
        response = self.app.safe_network_request({
            "type": "approve_cancel_request", "request_id": request_id
        }, "AdminDashboard.handle_approval.cancel")

        if response is None: return

        if response.get("status") == "ok":
            messagebox.showinfo("Success", "Cancel request approved. Meyun assignment removed for user.",
parent=self.root)

            self.refresh_requests_ui()
        else:
            messagebox.showerror("Error", response.get("message", "Failed to approve cancel request."),
parent=self.root)

    elif request_type == "delay":
        NewDatePopup(self.root, request_id,
            on_success_callback=self.refresh_requests_ui,
            safe_request_method=self.app.safe_network_request,
            critical_error_handler=self.app.handle_server_critical_error)

def show_user_messages(self):
    if self.app.server_error_active: return

```

```

self.clear_content()

main_messages_frame = ctk.CTkFrame(self.content_frame); main_messages_frame.pack(fill="both",
expand=True, padx=20, pady=20)

ctk.CTkLabel(main_messages_frame, text="Messages from Users", font=("Arial", 20,
"bold")).pack(pady=10)

scrollable_messages_frame = ctk.CTkScrollableFrame(main_messages_frame, label_text="Inbox");

scrollable_messages_frame.pack(fill="both", expand=True, padx=5, pady=5)


response = self.app.safe_network_request({"type": "get_all_messages"},
"AdminDashboard.show_user_messages")

if response is None: ctk.CTkLabel(scrollable_messages_frame, text="Error loading messages.",
text_color="red").pack(); return


all_messages = response.get("data", []) if response.get("status") == "ok" else []

if not all_messages:

    ctk.CTkLabel(scrollable_messages_frame, text="No messages found.", font=("Arial", 14,
"italic")).pack(pady=10)

    return


for msg_id, username, subject, content, msg_response, timestamp in all_messages:

    bg_color = "#E0F2F7" if msg_response else "FFFFFF" # Light blue for responded, white for not

    box = ctk.CTkFrame(scrollable_messages_frame, border_width=1, border_color="gray",
fg_color=bg_color); box.pack(fill="x", pady=5, padx=5)

    header_frame = ctk.CTkFrame(box, fg_color="transparent")

    header_frame.pack(fill="x", padx=10, pady=(5,0))

    ctk.CTkLabel(header_frame, text=f"From: {username if username else 'Unknown'}", font=("Arial", 12,
"bold")).pack(side="left")

    ctk.CTkLabel(header_frame, text=f"Received: {timestamp}", font=("Arial", 10, "italic"),
text_color="gray").pack(side="right")


    ctk.CTkLabel(box, text=f"Subject: {subject if subject else '(No Subject)'}", font=("Arial", 13,
"underline")).pack(anchor="w", padx=10, pady=(2,2))

    ctk.CTkLabel(box, text=f"Message: {content}", font=("Arial", 12),
wraplength=self.content_frame.winfo_width() - 70).pack(anchor="w", padx=10, pady=(0,5))


if msg_response:

    resp_frame = ctk.CTkFrame(box, fg_color="#D1E7DD") # Light green for response

```

```

resp_frame.pack(fill="x", padx=10, pady=5, ipady=5)

ctk.CTkLabel(resp_frame, text=f"Your Reply: {msg_response}", font=("Arial", 12, "italic"),
              text_color="#0F5132", wraplength=self.content_frame.winfo_width() - 90).pack(anchor="w",
padx=5) # Darker green text

else:

    # Each "Reply" button needs its own scope for the reply_entry and send_action

    # We use a closure (factory function) for this

    def create_reply_ui_and_handler(parent_box, message_id_for_reply):

        # This inner function will be the command for the "Reply" button

        def toggle_reply_area():

            if self.app.server_error_active: return

            # Check if reply area already exists for THIS box

            if hasattr(parent_box, "reply_area_active") and parent_box.reply_area_active:

                # Remove existing reply widgets if toggling off

                if hasattr(parent_box, "reply_widgets_tuple"):

                    for w in parent_box.reply_widgets_tuple: w.destroy()

                del parent_box.reply_widgets_tuple

                parent_box.reply_area_active = False

                reply_button.configure(text="Reply") # Reset button text

                return

            # Create reply widgets

            reply_button.configure(text="Cancel Reply") # Change button text

            parent_box.reply_area_active = True

            reply_entry_widget = ctk.CTkTextbox(parent_box, height=80)

            reply_entry_widget.pack(padx=10, pady=(5, 0), fill="x")

            reply_entry_widget.focus()

        def send_actual_reply():

            if self.app.server_error_active: return

```

```

reply_text = reply_entry_widget.get("1.0", "end-1c").strip()

if not reply_text:

    messagebox.showerror("Error", "Reply cannot be empty.", parent=self.root)

    return


send_response = self.app.safe_network_request({

    "type": "reply_to_message",

    "message_id": message_id_for_reply,

    "response": reply_text

}, f"AdminDashboard.send_reply_for_msg_{message_id_for_reply}")


if send_response is None: return # Critical error handled


if send_response.get("status") == "ok":

    messagebox.showinfo("Success", "Reply sent.", parent=self.root)

    self.show_user_messages() # Refresh the whole view

else:

    messagebox.showerror("Error", send_response.get("message", "Failed to send reply."),
parent=self.root)


height=30) send_button_widget = MyButton(parent_box, text="Send Reply", command=send_actual_reply,

send_button_widget.pack(pady=5, padx=10, anchor="e")

parent_box.reply_widgets_tuple = (reply_entry_widget, send_button_widget)

return toggle_reply_area


reply_button = MyButton(box, text="Reply", command=create_reply_ui_and_handler(box, msg_id),
width=80, height=30)

reply_button.pack(padx=10, pady=5, anchor="e")


def generate_strong_password(length=10):

    # Ensure there's at least one of each required type if stricter rules apply

```

```
# For now, simple random choice from a combined set

chars = string.ascii_letters + string.digits + "!@#$$%^&*()"

# Make sure length is reasonable

length = max(8, min(length, 16)) # e.g. enforce 8-16 char passwords

return "".join(random.choices(chars, k=length))
```

```
def safe_entry_delete(entry):

    try:

        if entry:

            entry.delete(0, "end")

    except Exception as e:

        print(f"[safe_entry_delete] Error: {e}")
```

### **network\_client.py:**

```
import socket

import threading

import json

import base64

import os

from cryptography.hazmat.primitives import serialization

from cryptography.hazmat.backends import default_backend

from cryptography.hazmat.primitives.asymmetric import padding, rsa

from cryptography.hazmat.primitives import hashes

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

SERVER_IP = '127.0.0.1'

SERVER_PORT = 1729

MAX_MESSAGE_LENGTH = 4096

# Helper functions for PKCS#7 padding

def pkcs7_pad(data, block_size=16):

    pad_len = block_size - len(data) % block_size
```

```

return data + bytes([pad_len] * pad_len)

def pkcs7_unpad(data):
    if not data:
        raise ValueError("Cannot unpad empty data")
    pad_len = data[-1]
    if pad_len > len(data) or pad_len == 0:
        raise ValueError("Invalid PKCS#7 padding or RCE attempt")
    for i in range(1, pad_len + 1):
        if data[-i] != pad_len:
            raise ValueError("Invalid PKCS#7 padding bytes or RCE attempt")
    return data[:-pad_len]

class ServerConnectionError(Exception):
    pass

class NetworkClient:
    def __init__(self):
        self.socket = None
        try:
            self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.socket.settimeout(5)
            self.socket.connect((SERVER_IP, SERVER_PORT))

            self.lock = threading.Lock()
            self.private_key = rsa.generate_private_key(
                public_exponent=65537,
                key_size=2048,
                backend=default_backend()
            )
            self.public_key_pem = self.private_key.public_key().public_bytes(
                encoding=serialization.Encoding.PEM,

```

```

        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ).decode()

    self.socket.send(json.dumps({
        "type": "send_client_key",
        "key": self.public_key_pem,
        "client_ip": self.socket.getsockname()[0]
    }).encode())

    ack = self.socket.recv(MAX_MESSAGE_LENGTH)

    if ack != b"ACK":
        raise ServerConnectionError("ACK not received from server.")

    self.socket.settimeout(None)

    self.public_key = None
    self.fetch_public_key_internal()

except (socket.timeout, ConnectionRefusedError, socket.gaierror, ServerConnectionError) as e:
    if self.socket:
        try: self.socket.close()
        except: pass

    if isinstance(e, ServerConnectionError): # Re-raise if it's already our specific error
        raise

    # Wrap other connection-related errors
    raise ServerConnectionError(f"Failed to connect or initialize with server: {e}")

except Exception as e_generic:
    if self.socket:
        try: self.socket.close()
        except: pass

    raise ServerConnectionError(f"Unexpected error during network client setup: {e_generic}")

```

```

def fetch_public_key_internal(self):

    # This method is only called during __init__ which is protected by its own try-except
    # ServerConnectionError

    data = self.send_raw_request({"type": "get_public_key"}) # send_raw_request can raise
    # ServerConnectionError

    if data.get("status") == "ok":

        key_pem = data.get("key").encode()

        self.public_key = serialization.load_pem_public_key(

            key_pem,

            backend=default_backend()

        )

    else:

        # This specific failure path within fetch_public_key_internal should also raise ServerConnectionError

        raise ServerConnectionError(f"Failed to fetch server public key: {data.get('message', 'Server error')}")


def send_raw_request(self, data):

    # This method can be called during __init__ (for public key) or later.

    # If called during __init__, its ServerConnectionError will be caught by __init__'s handler.

    # If called later (e.g. from send_request for get_public_key type),

    # send_request needs to handle its potential ServerConnectionError.

    try:

        with self.lock:

            self.socket.send(json.dumps(data).encode())

            # Set a timeout for this specific recv, in case it's a long-running app

            # and the socket's default timeout was None.

            current_timeout = self.socket.gettimeout()

            self.socket.settimeout(10)

            response_bytes = self.socket.recv(MAX_MESSAGE_LENGTH)

            self.socket.settimeout(current_timeout) # Restore previous timeout

            if not response_bytes: # Server closed connection

```

```

        raise ServerConnectionError("Server closed connection unexpectedly (empty response in
send_raw_request).")

        return json.loads(response_bytes.decode())

except socket.timeout:

    raise ServerConnectionError("Server response timed out for raw request.")

except (socket.error, json.JSONDecodeError, AttributeError) as e: # AttributeError if socket is None

    raise ServerConnectionError(f"Network or data error in send_raw_request: {e}")

```

```

def send_request(self, data_dict):

    # This is the main method called by the App. It should not raise ServerConnectionError itself,
    # but return a dict, so App.safe_network_request can decide to call handle_server_critical_error.

    try:

        if not self.public_key: # Should have been caught by __init__ or App's checks

            return {"status": "error", "message": "Client not properly initialized (no server public key)."}

        if data_dict.get("type") == "get_public_key":

            try:

                # If send_raw_request raises ServerConnectionError here, it means something went wrong
                # after successful initialization. Treat it like other runtime network errors.

                return self.send_raw_request(data_dict)

            except ServerConnectionError as sce:

                return {"status": "error", "message": f"Network error fetching public key (runtime): {sce}"}

```

```

aes_key = os.urandom(32)

request_iv = os.urandom(16)

json_data_bytes = json.dumps(data_dict).encode()

cipher_encrypt = Cipher(algorithms.AES(aes_key), modes.CBC(request_iv),
backend=default_backend())

encryptor = cipher_encrypt.encryptor()

padded_data = pkcs7_pad(json_data_bytes)

encrypted_data = encryptor.update(padded_data) + encryptor.finalize()

```

```

encrypted_key = self.public_key.encrypt(
    aes_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

payload = json.dumps({
    "key": base64.b64encode(encrypted_key).decode(),
    "iv": base64.b64encode(request_iv).decode(),
    "data": base64.b64encode(encrypted_data).decode()
}).encode()

with self.lock:
    # Similar timeout logic as in send_raw_request
    current_timeout = self.socket.gettimeout()
    self.socket.settimeout(10) # Timeout for send/recv
    self.socket.send(payload)
    response_bytes = self.socket.recv(MAX_MESSAGE_LENGTH)
    self.socket.settimeout(current_timeout) # Restore

    if not response_bytes:
        return {"status": "error", "message": "Server closed connection (empty response)."}

try:
    response_json = json.loads(response_bytes.decode())
    if "iv" in response_json and "data" in response_json:
        response_iv = base64.b64decode(response_json["iv"])
        encrypted_response_data = base64.b64decode(response_json["data"])

```

```

        cipher_decrypt = Cipher(algorithms.AES(aes_key), modes.CBC(response_iv),
                                backend=default_backend())

        decryptor = cipher_decrypt.decryptor()

        padded_decrypted_data = decryptor.update(encrypted_response_data) + decryptor.finalize()

        decrypted_data = pkcs7_unpad(padded_decrypted_data)

        return json.loads(decrypted_data.decode())

    else:

        return response_json

    except (json.JSONDecodeError, ValueError, base64.binascii.Error) as e_dec: # ValueError from
    unpad or b64decode

        return {"status": "error", "message": f"Error decoding/decrypting server response: {e_dec}"}

    except Exception as e_gen_dec:

        return {"status": "error", "message": f"General error processing server response: {e_gen_dec}"}

except socket.timeout:

    return {"status": "error", "message": "Server response timed out."}

except (socket.error, AttributeError) as se: # AttributeError if self.socket is None (e.g. init failed badly)

    return {"status": "error", "message": f"Network communication failed: {se}"}

except Exception as e: # Catch-all for other unexpected errors during the send process

    return {"status": "error", "message": f"Unexpected error in send_request: {str(e)}"}

def close(self):

    try:

        if self.socket:

            self.socket.close()

    except: # Ignore errors on close

        pass

```

### network\_server.py:

```

import os

import socket

import json

from concurrent.futures import ThreadPoolExecutor

```

```

import time

from collections import defaultdict

from send_email import send_verification_email, send_new_user_email

from cryptography.hazmat.primitives import serialization

from cryptography.hazmat.backends import default_backend

from cryptography.hazmat.primitives.asymmetric import padding

from cryptography.hazmat.primitives import hashes

import base64

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

from database import (
    check_user_credentials,
    generate_and_store_code,
    verify_code,
    get_user_data,
    get_user_meyunim,
    submit_request,
    send_message_to_admin,
    get_user_messages,
    reset_user_password,
    connect_db,
    hash_password,
    get_all_messages,
    get_all_users,
    update_request_status,
    get_all_requests,
    get_requests_by_user,
    delete_meyun,
    update_date_for_request,
    hash_id_number,
)

SERVER_IP = '0.0.0.0'

```

```
SERVER_PORT = 1729
```

```
MAX_MESSAGE_LENGTH = 4096
```

```
executor = ThreadPoolExecutor(max_workers=10)
```

```
connection_log = defaultdict(list)
```

```
blocked_ips = {}
```

```
RATE_LIMIT = 30
```

```
RATE_WINDOW = 10
```

```
BLOCK_TIME = 60
```

```
clients = {}
```

```
client_public_keys = {}
```

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
private_key_path = os.path.join(BASE_DIR, "private_key.pem")
```

```
public_key_path = os.path.join(BASE_DIR, "public_key.pem")
```

```
if not os.path.exists(private_key_path) or not os.path.exists(public_key_path):
```

```
    script_dir = os.path.dirname(os.path.abspath(__file__))
```

```
    private_key_path_alt = os.path.join(script_dir, "private_key.pem")
```

```
    public_key_path_alt = os.path.join(script_dir, "public_key.pem")
```

```
if os.path.exists(private_key_path_alt) and os.path.exists(public_key_path_alt):
```

```
    private_key_path = private_key_path_alt
```

```
    public_key_path = public_key_path_alt
```

```
    print(f"INFO: Using keys from script directory: {script_dir}")
```

```
else:
```

```
    raise FileNotFoundError(
```

```
        f"Private or public key not found. Checked BASE_DIR ('{BASE_DIR}') and script_dir ('{script_dir}').  
        Ensure keys exist."
```

```

    )

with open(private_key_path, "rb") as f:
    PRIVATE_KEY = serialization.load_pem_private_key(
        f.read(),
        password=None,
        backend=default_backend()
    )

with open(public_key_path, "rb") as f:
    PUBLIC_KEY = serialization.load_pem_public_key(
        f.read(),
        backend=default_backend()
    )

def pkcs7_pad(data, block_size=16):
    pad_len = block_size - len(data) % block_size
    return data + bytes([pad_len] * pad_len)

def pkcs7_unpad(data):
    if not data: raise ValueError("Cannot unpad empty data")
    pad_len = data[-1]
    if pad_len > len(data) or pad_len == 0: raise ValueError("Invalid PKCS#7 padding length")
    for i in range(1, pad_len + 1):
        if data[-i] != pad_len: raise ValueError("Invalid PKCS#7 padding bytes")
    return data[:-pad_len]

def handle_client(conn, addr):
    print(f"[NEW CONNECTION] {addr} connected.")
    ip = addr[0]
    now = time.time()

```

```

if ip in blocked_ips and now < blocked_ips[ip]:
    print(f"[BLOCKED] {ip} is currently blocked. Connection refused.")
    conn.close()
    return

elif ip in blocked_ips and now >= blocked_ips[ip]:
    del blocked_ips[ip]
    print(f"[UNBLOCKED] {ip} is no longer blocked.")

connection_log[ip] = [t for t in connection_log[ip] if now - t < RATE_WINDOW]
connection_log[ip].append(now)

if len(connection_log[ip]) > RATE_LIMIT:
    print(f"[DDOS PROTECTION] Blocking IP {ip} for {BLOCK_TIME} seconds due to high request rate.")
    blocked_ips[ip] = now + BLOCK_TIME
    if ip in connection_log: del connection_log[ip]
    conn.close()
    return

conn.settimeout(600)
aes_key_session = None

with conn:
    try:
        conn.settimeout(20)
        raw_init_data = conn.recv(MAX_MESSAGE_LENGTH)
        conn.settimeout(600)

        if not raw_init_data:
            print(f"[KEY INIT ERROR] {addr}: No data received for key init.")
            return

```

```

init_msg = json.loads(raw_init_data.decode())

if init_msg.get("type") == "send_client_key" and "key" in init_msg:

    client_ip_reported = init_msg.get("client_ip")

    if client_ip_reported != ip:

        print(f"[KEY INIT WARNING] {addr}: Reported IP {client_ip_reported} differs from actual source
IP {ip}.")

        key_pem = init_msg["key"].encode()

        client_key_obj = serialization.load_pem_public_key(key_pem, backend=default_backend())

        client_public_keys[ip] = client_key_obj

        conn.send(b"ACK")

        print(f"[KEY INIT SUCCESS] {addr}: Client public key received and ACK sent.")

    else:

        conn.send(json.dumps({"status": "error", "message": "Invalid key initialization
sequence."}).encode())

        print(f"[KEY INIT FAIL] {addr}: Invalid init message type or missing key.")

        return

except socket.timeout:

    print(f"[KEY INIT TIMEOUT] {addr}: Timeout during initial key exchange.")

    return

except (json.JSONDecodeError, UnicodeDecodeError) as e:

    print(f"[KEY INIT ERROR] {addr}: Invalid JSON/encoding for key init - {e}")

    return

except Exception as e:

    print(f"[KEY INIT UNEXPECTED ERROR] {addr}: {e}")

    return

while True:

    try:

        raw_data = conn.recv(MAX_MESSAGE_LENGTH)

        if not raw_data:

            print(f"[DISCONNECTED] {addr} closed connection (no data).")

            break

```

```

request_json_str = raw_data.decode()

request_dict = None

is_encrypted_payload = True

try:

    parsed_plain = json.loads(request_json_str)

    if isinstance(parsed_plain, dict) and parsed_plain.get("type") == "get_public_key":

        request_dict = parsed_plain

        is_encrypted_payload = False

        print(f"[UNENCRYPTED REQUEST] {addr}: get_public_key")

except json.JSONDecodeError:

    pass


if is_encrypted_payload:

    if not client_public_keys.get(ip):

        print(f"[ENCRYPTION ERROR] {addr}: No client public key found for decryption for an encrypted payload.")

        conn.send(json.dumps({"status": "error", "message": "Server error: client key missing."}).encode())

        break

    try:

        payload = json.loads(request_json_str)

        encrypted_aes_key_b64 = payload["key"]

        request_iv_b64 = payload["iv"]

        encrypted_data_b64 = payload["data"]


        encrypted_aes_key = base64.b64decode(encrypted_aes_key_b64)

        request_iv = base64.b64decode(request_iv_b64)

        encrypted_data = base64.b64decode(encrypted_data_b64)


        aes_key_session = PRIVATE_KEY.decrypt(

            encrypted_aes_key,

            padding.OAEP(

                mgf=padding.MGF1(algorithm=hashes.SHA256()),

```

```

        algorithm=hashes.SHA256(),

        label=None

    )

)

    cipher_decrypt = Cipher(algorithms.AES(aes_key_session), modes.CBC(request_iv),
backend=default_backend())

    decryptor = cipher_decrypt.decryptor()

    padded_decrypted_data = decryptor.update(encrypted_data) + decryptor.finalize()

    request_data_bytes = pkcs7_unpad(padded_decrypted_data)

    request_dict = json.loads(request_data_bytes.decode())

    except (json.JSONDecodeError, KeyError, base64.binascii.Error, ValueError) as e_dec:

        print(f"[DECRYPTION/PAYLOAD ERROR] {addr}: {e_dec}. Payload:
{request_json_str[:200]}")

        conn.send(json.dumps({"status": "error", "message": "Failed to parse/decrypt
payload."}).encode())

        continue

    except Exception as e_rsa_aes:

        print(f"[DECRYPTION RSA/AES ERROR] {addr}: {e_rsa_aes}")

        conn.send(json.dumps({"status": "error", "message": "Failed to decrypt request
components."}).encode())

        continue

    if not isinstance(request_dict, dict) or "type" not in request_dict:

        print(f"[INVALID REQUEST] {addr}: 'type' field missing or not a dict.")

        conn.send(json.dumps({"status": "error", "message": "Invalid request format: 'type' field
missing."}).encode())

        continue

    request_dict["request_ip"] = ip

    response_dict = handle_request(request_dict)

    if request_dict.get("type") == "get_public_key":

        conn.send(json.dumps(response_dict).encode())

    elif aes_key_session:

        try:

```

```

        encrypted_response_payload = encrypt_response_with_aes(response_dict,
aes_key_session)

        conn.send(json.dumps(encrypted_response_payload).encode())

    except Exception as e_enc_resp:

        print(f"[ENCRYPTION ERROR FOR RESPONSE] {addr}: {e_enc_resp}")

        conn.send(json.dumps({"status": "error", "message": "Server error during response
encryption."}).encode())

    else:

        print(f"[LOGIC ERROR] {addr}: AES key not available for encrypting response to an encrypted
request.")

        conn.send(json.dumps({"status": "error", "message": "Internal server error: encryption key
missing."}).encode())

except socket.timeout:

    print(f"[TIMEOUT] {addr} inactive for too long.")

    break

except ConnectionResetError:

    print(f"[CONNECTION RESET] {addr} closed connection abruptly.")

    break

except (UnicodeDecodeError, json.JSONDecodeError) as e_json:

    print(f"[DATA ERROR] {addr}: Could not decode JSON from client - {e_json}")

    try: conn.send(json.dumps({"status": "error", "message": "Invalid data format received."}).encode())

    except: pass

    break

except Exception as e_loop:

    print(f"Unexpected error handling client {addr} in loop: {e_loop}")

    try: conn.send(json.dumps({"status": "error", "message": "An unexpected server error
occurred."}).encode())

    except: pass

    break

print(f"[DISCONNECTED] {addr} processing finished.")

if ip in client_public_keys: del client_public_keys[ip]

if ip in connection_log: del connection_log[ip]

```

```

def handle_request(request):

    rtype = request.get("type")

    client_ip = request.get('request_ip', 'Unknown IP') # Get IP safely for logging

    # Basic logging for all request types
    # print(f"Handling request of type: {rtype} from {client_ip}")

    if rtype == "get_public_key":

        pem = PUBLIC_KEY.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ).decode()

        return {"status": "ok", "key": pem}

    # ... (All other specific rtype handlers remain unchanged from the previous full version) ...

    elif rtype == "login":

        username = request.get("username")

        password = request.get("password")

        success, message, is_admin, user_id, email = check_user_credentials(username, password)

        if success:

            conn_db = connect_db()

            cursor = conn_db.cursor()

            cursor.execute("SELECT phone, address, age FROM usersDB WHERE id=?", (user_id,))

            user_row = cursor.fetchone()

            conn_db.close()

            first_login = not user_row or any(val is None or str(val).strip() == "" for val in user_row)

            return {"status": "ok", "message": message, "is_admin": is_admin, "user_id": user_id, "email": email,
                    "first_login": first_login}

        else:

            return {"status": "error", "message": message}

```

```

elif rtype == "complete_first_login":

    user_id = request.get("user_id")

    entered_id_plain = request.get("id_number")

    phone = request.get("phone")

    address = request.get("address")

    age = request.get("age")

    new_password = request.get("new_password")

    conn_db = connect_db()

    cursor = conn_db.cursor()

    try:

        cursor.execute("SELECT id_number FROM usersDB WHERE id=?", (user_id,))

        row = cursor.fetchone()

        if not row: return {"status": "error", "message": "User not found."}

        hashed_id_in_db = row[0]

        if hashed_id_in_db != hash_id_number(entered_id_plain):

            return {"status": "error", "message": "ID number does not match records."}

        cursor.execute("UPDATE usersDB SET phone=?, address=?, age=?, password=? WHERE id=?",

                        (phone, address, age, hash_password(new_password), user_id))

        conn_db.commit()

        return {"status": "ok", "message": "Profile and password updated successfully."}

    except Exception as e:

        print(f"Error in complete_first_login for user {user_id}: {e}")

        conn_db.rollback()

        return {"status": "error", "message": "Server error during profile update."}

    finally:

        conn_db.close()

elif rtype == "send_code_to_user":

    user_id = request.get("user_id")

    conn_db = connect_db()

```

```

cursor = conn_db.cursor()

try:

    cursor.execute("SELECT email FROM usersDB WHERE id=?", (user_id,))

    row = cursor.fetchone()

    if not row: return {"status": "error", "message": "User not found."}

    email = row[0]

    code = generate_and_store_code(user_id)

    if not code: return {"status": "error", "message": "Failed to generate verification code."}

    email_sent = send_verification_email(email, code)

    if email_sent: return {"status": "ok", "message": "Verification code sent to email."}

    else: return {"status": "error", "message": "Failed to send verification email."}

except Exception as e:

    print(f"Error sending code to user {user_id}: {e}")

    return {"status": "error", "message": "Server error sending code."}

finally:

    conn_db.close()

elif rtype == "verify_code":

    user_id = request.get("user_id")

    code = request.get("code")

    valid = verify_code(user_id, code)

    if valid: return {"status": "ok", "message": "Code verified."}

    else: return {"status": "error", "message": "Invalid or expired code."}

elif rtype == "request_reset_code":

    username = request.get("username")

    conn_db = connect_db()

    cursor = conn_db.cursor()

    try:

        cursor.execute("SELECT id, email FROM usersDB WHERE username=?", (username,))

        row = cursor.fetchone()

        if not row: return {"status": "error", "message": "Username not found."}

```

```

user_id, email = row

code = generate_and_store_code(user_id)

if not code: return {"status": "error", "message": "Failed to generate reset code."}

email_sent = send_verification_email(email, code, subject="Password Reset Code")

if email_sent: return {"status": "ok", "user_id": user_id, "email": email, "message": "Password reset
code sent."}

else: return {"status": "error", "message": "Failed to send password reset email."}

except Exception as e:

    print(f"Error requesting reset code for {username}: {e}")

    return {"status": "error", "message": "Server error requesting reset code."}

finally:

    conn_db.close()

```

```

elif rtype == "reset_password":

```

```

    username = request.get("username")

    email = request.get("email") # This email is used for verification, not for uniqueness check during reset

    new_password = request.get("new_password")

    success, message = reset_user_password(username, email, new_password) # reset_user_password
    should find the user by username and verify the email

    if success: return {"status": "ok", "message": message}

    else: return {"status": "error", "message": message}

```

```

elif rtype == "get_user_data":

```

```

    user_id = request.get("user_id")

    data_tuple = get_user_data(user_id)

    if data_tuple: return {"status": "ok", "data": data_tuple}

    else: return {"status": "error", "message": "Could not retrieve user data."}

```

```

elif rtype == "get_meyunim":

```

```

    user_id = request.get("user_id")

    data = get_user_meyunim(user_id)

    return {"status": "ok", "data": data}

```

```

elif rtype == "submit_request":

    assignment_id = request.get("assignment_id")

    req_type = request.get("request_type")

    message_text = request.get("message")

    success = submit_request(assignment_id, req_type, message_text)

    if success: return {"status": "ok", "message": "Request submitted."}

    else: return {"status": "error", "message": "Failed to submit request."}


elif rtype == "send_message":

    user_id = request.get("user_id")

    subject = request.get("subject")

    content = request.get("content")

    success = send_message_to_admin(user_id, subject, content)

    if success: return {"status": "ok", "message": "Message sent to admin."}

    else: return {"status": "error", "message": "Failed to send message."}


elif rtype == "get_messages":

    user_id = request.get("user_id")

    messages_list = get_user_messages(user_id)

    return {"status": "ok", "data": messages_list}


elif rtype == "get_my_requests":

    user_id = request.get("user_id")

    requests_list = get_requests_by_user(user_id)

    return {"status": "ok", "data": requests_list}


elif rtype == "add_user":

    username = request.get("username")

    email = request.get("email")

    plain_password = request.get("raw_password")

    id_number_plain = request.get("id_number")

    is_admin_flag = request.get("is_admin", 0)

```

```

conn_db = connect_db()

cursor = conn_db.cursor()

try:

    cursor.execute("SELECT id FROM usersDB WHERE username=?", (username,))

    if cursor.fetchone():

        conn_db.close()

        return {"status": "error", "message": "Username already exists."}

    hashed_pw = hash_password(plain_password)

    hashed_id = hash_id_number(id_number_plain)

    cursor.execute("INSERT INTO usersDB (username, email, password, id_number, is_admin) VALUES
    (?, ?, ?, ?, ?)",

        (username, email, hashed_pw, hashed_id, is_admin_flag))

    conn_db.commit()

    email_sent = send_new_user_email(username, email, plain_password)

    if email_sent: msg = "User added and credentials emailed."

    else: msg = "User added, but failed to email credentials."

    return {"status": "ok", "message": msg}

except Exception as e:

    conn_db.rollback()

    print(f"Error adding user {username}: {e}")

    return {"status": "error", "message": "Server error adding user."}

finally:

    if conn_db:

        conn_db.close()

elif rtype == "get_all_users":

    users_list = get_all_users()

    return {"status": "ok", "data": users_list}

```

```

elif rtype == "load_user_data":

    username = request.get("username")

    conn_db = connect_db()

    cursor = conn_db.cursor()

    try:

        cursor.execute("SELECT id, email, address, age, phone FROM usersDB WHERE username=?",
            (username,))

        user_row = cursor.fetchone()

        if not user_row: return {"status": "error", "message": f"User '{username}' not found."}

        user_id, email, address, age, phone = user_row


        cursor.execute("SELECT dapar, profile, hebrew, interview, hundred_day FROM user_data WHERE
            user_id=?", (user_id,))

        system_row = cursor.fetchone()

        d, p, h, i, hd = system_row if system_row else (None, None, None, None, None)


        return {"status": "ok", "data": {

            "email": email, "address": address, "age": age, "phone": phone,

            "dapar": d, "profile": p, "hebrew": h, "interview": i, "hundred_day": hd

        }}

    except Exception as e:

        print(f"Error loading user data for {username}: {e}")

        return {"status": "error", "message": "Server error loading user data."}

    finally:

        conn_db.close()


elif rtype == "update_user_data":

    username = request.get("username")

    account_data = request.get("account_data", {})

    system_data = request.get("system_data", {})


    conn_db = connect_db()

```

```

cursor = conn_db.cursor()

try:

    cursor.execute("SELECT id FROM usersDB WHERE username=?", (username,))

    user_row = cursor.fetchone()

    if not user_row: return {"status": "error", "message": f"User '{username}' not found for update."}

    user_id = user_row[0]

    cursor.execute("UPDATE usersDB SET email=?, address=?, age=?, phone=? WHERE id=?",

                    (account_data.get("email"), account_data.get("address"),

                     account_data.get("age"), account_data.get("phone"), user_id))

    cursor.execute("SELECT id FROM user_data WHERE user_id=?", (user_id,))

    if cursor.fetchone():

        cursor.execute("UPDATE user_data SET dapar=?, profile=?, hebrew=?, interview=?,
hundred_day=? WHERE user_id=?",

                        (system_data.get("dapar"), system_data.get("profile"), system_data.get("hebrew"),

                         system_data.get("interview"), system_data.get("hundred_day"), user_id))

    else:

        cursor.execute("INSERT INTO user_data (user_id, dapar, profile, hebrew, interview, hundred_day)
VALUES (?, ?, ?, ?, ?, ?)",

                        (user_id, system_data.get("dapar"), system_data.get("profile"),
                         system_data.get("hebrew"),

                          system_data.get("interview"), system_data.get("hundred_day")))

    conn_db.commit()

    return {"status": "ok", "message": f"Data for {username} updated successfully."}

except Exception as e:

    conn_db.rollback()

    print(f"Error updating user data for {username}: {e}")

    return {"status": "error", "message": "Server error updating data."}

finally:

    conn_db.close()

elif rtype == "create_meyun":

```

```

title = request.get("title")

description = request.get("description", "")

date = request.get("date")

user_ids_to_assign = request.get("user_ids", [])

conn_db = connect_db()

cursor = conn_db.cursor()

try:

    cursor.execute("INSERT INTO meyunim (title, description, date) VALUES (?, ?, ?)", (title, description,
date))

    meyun_id = cursor.lastrowid

    if user_ids_to_assign:

        for u_id in user_ids_to_assign:

            cursor.execute("INSERT INTO meyunim_assignments (user_id, meyun_id) VALUES (?, ?)",
(int(u_id), meyun_id))

        conn_db.commit()

        return {"status": "ok", "message": "Meyun created and assigned."}

except Exception as e:

    conn_db.rollback()

    print(f"Error creating meyun '{title}': {e}")

    return {"status": "error", "message": "Server error creating meyun."}

finally:

    conn_db.close()

elif rtype == "get_all_meyunim":

    conn_db = connect_db()

    cursor = conn_db.cursor()

    meyunim_with_users = []

    try:

        cursor.execute("SELECT id, title, description, date FROM meyunim ORDER BY date DESC")

        all_meyunim_db = cursor.fetchall()

        for m_id, m_title, m_desc, m_date in all_meyunim_db:

            cursor.execute("SELECT u.username FROM meyunim_assignments ma JOIN usersDB u ON
ma.user_id = u.id WHERE ma.meyun_id=?", (m_id,))

```

```

        assigned_usernames = [row[0] for row in cursor.fetchall()]

        meyunim_with_users.append((m_id, m_title, m_desc, m_date, assigned_usernames))

    return {"status": "ok", "data": meyunim_with_users}

except Exception as e:

    print(f"Error getting all meyunim: {e}")

    return {"status": "error", "message": "Server error fetching meyunim."}

finally:

    conn_db.close()

elif rtype == "delete_meyun":

    meyun_id_to_delete = request.get("meyun_id")

    success = delete_meyun(meyun_id_to_delete)

    if success: return {"status": "ok", "message": "Meyun deleted successfully."}

    else: return {"status": "error", "message": "Failed to delete meyun."}

elif rtype == "get_all_requests":

    all_reqs = get_all_requests()

    return {"status": "ok", "data": all_reqs}

elif rtype == "update_request_status":

    request_id_update = request.get("request_id")

    new_status_update = request.get("new_status")

    success = update_request_status(request_id_update, new_status_update)

    if success: return {"status": "ok", "message": f"Request status updated to {new_status_update}."}

    else: return {"status": "error", "message": "Failed to update request status."}

elif rtype == "approve_cancel_request":

    request_id_cancel = request.get("request_id")

    conn_db = connect_db()

    cursor = conn_db.cursor()

    try:

```

```

        cursor.execute("SELECT assignment_id FROM meyunim_requests WHERE id=?",
(request_id_cancel,))

        assign_row = cursor.fetchone()

        if not assign_row:

            return {"status": "error", "message": "Request not found or no assignment linked."}

        assignment_id_to_delete = assign_row[0]

        if assignment_id_to_delete:

            cursor.execute("DELETE FROM meyunim_assignments WHERE id=?",
(assignment_id_to_delete,))

            cursor.execute("UPDATE meyunim_requests SET status='Approved' WHERE id=?",
(request_id_cancel,))

            conn_db.commit()

            return {"status": "ok", "message": "Cancel request approved, assignment removed."}

except Exception as e:

    conn_db.rollback()

    print(f"Error approving cancel request {request_id_cancel}: {e}")

    return {"status": "error", "message": "Server error approving cancel request."}

finally:

    conn_db.close()

elif rtype == "update_date_for_request":

    request_id_delay = request.get("request_id")

    new_date_delay = request.get("new_date")

    success = update_date_for_request(request_id_delay, new_date_delay)

    if success: return {"status": "ok", "message": "Delay request approved and date updated."}

    else: return {"status": "error", "message": "Failed to update date for delay request."}

elif rtype == "get_all_messages":

    messages_admin_inbox = get_all_messages()

    return {"status": "ok", "data": messages_admin_inbox}

```

```

elif rtype == "reply_to_message":

    message_id_reply = request.get("message_id")

    response_text = request.get("response")

    conn_db = connect_db()

    cursor = conn_db.cursor()

    try:

        cursor.execute("UPDATE messages SET response=?,
admin_responded_at=CURRENT_TIMESTAMP WHERE id=?", (response_text, message_id_reply))

        conn_db.commit()

        if cursor.rowcount > 0:

            return {"status": "ok", "message": "Reply sent."}

        else:

            error_msg = f"Message with ID {message_id_reply} not found, or no actual update occurred (e.g.
data identical)."
```

```

            print(f"[REPLY WARNING] {error_msg}")

            return {"status": "error", "message": error_msg}

    except Exception as e:

        conn_db.rollback()

        detailed_error_msg = f"Server failed to send reply for message ID {message_id_reply}. Database
error: {str(e)}"

        print(f"[REPLY EXCEPTION] Error replying to message {message_id_reply}: {e}")

        return {"status": "error", "message": detailed_error_msg}

    finally:

        conn_db.close()

elif rtype == "get_age_distribution":

    conn_db = connect_db(); cursor = conn_db.cursor()

    try:

        cursor.execute("SELECT age FROM usersDB WHERE age IS NOT NULL AND age != ''")

        ages = [row[0] for row in cursor.fetchall() if isinstance(row[0], (int, float))]

        return {"status": "ok", "ages": ages}

    except Exception as e: print(f"Age distribution error: {e}"); return {"status": "error", "data": []}

    finally: conn_db.close()

```

```

elif rtype == "get_request_type_stats":

    conn_db = connect_db(); cursor = conn_db.cursor()

    try:

        cursor.execute("SELECT type, COUNT(*) FROM meyunim_requests GROUP BY type")

        return {"status": "ok", "data": cursor.fetchall()}

    except Exception as e: print(f"Request type stats error: {e}"); return {"status": "error", "data": []}

    finally: conn_db.close()


elif rtype == "get_request_status_stats":

    conn_db = connect_db(); cursor = conn_db.cursor()

    try:

        cursor.execute("SELECT status, COUNT(*) FROM meyunim_requests GROUP BY status")

        return {"status": "ok", "data": cursor.fetchall()}

    except Exception as e: print(f"Request status stats error: {e}"); return {"status": "error", "data": []}

    finally: conn_db.close()


elif rtype == "get_meyunim_per_month":

    conn_db = connect_db(); cursor = conn_db.cursor()

    try:

        cursor.execute("SELECT strftime('%Y-%m', date) as month, COUNT(*) FROM meyunim GROUP BY month ORDER BY month")

        return {"status": "ok", "data": cursor.fetchall()}

    except Exception as e: print(f"Meyunim per month error: {e}"); return {"status": "error", "data": []}

    finally: conn_db.close()


elif rtype == "get_messages_per_week":

    conn_db = connect_db(); cursor = conn_db.cursor()

    try:

        cursor.execute("SELECT strftime('%Y-%W', timestamp) as week, COUNT(*) FROM messages GROUP BY week ORDER BY week")

        return {"status": "ok", "data": cursor.fetchall()}

    except Exception as e: print(f"Messages per week error: {e}"); return {"status": "error", "data": []}

```

```
finally: conn_db.close()
```

```
elif rtype == "get_user_notifications":
```

```
    user_id = request.get("user_id")
```

```
    conn_db = connect_db(); cursor = conn_db.cursor()
```

```
    notifications = []
```

```
    try:
```

```
        cursor.execute("SELECT m.title, m.date FROM meyunim_assignments a JOIN meyunim m ON  
a.meyun_id = m.id WHERE a.user_id = ? AND date(m.date) >= date('now') ORDER BY date(m.date)  
ASC", (user_id,))
```

```
        for title, date_str in cursor.fetchall(): notifications.append(f"Reminder: Meyun '{title}' on {date_str}.")
```

```
    cursor.execute("""
```

```
        SELECT m.title, r.type, r.status, r.id as request_id
```

```
        FROM meyunim_requests r
```

```
        JOIN meyunim_assignments a ON r.assignment_id = a.id
```

```
        JOIN meyunim m ON a.meyun_id = m.id
```

```
        WHERE r.status != 'Pending' AND a.user_id = ? AND (r.notified_user = 0 OR r.notified_user IS  
NULL)
```

```
    """, (user_id,))
```

```
    req_ids_to_mark = []
```

```
    for title, req_type, status, req_id in cursor.fetchall():
```

```
        notifications.append(f"Update: Your {req_type} request for '{title}' was {status.lower()}.")
```

```
        req_ids_to_mark.append(req_id)
```

```
    if req_ids_to_mark:
```

```
        cursor.execute(f"UPDATE meyunim_requests SET notified_user = 1 WHERE id IN  
({' '.join(['?']*len(req_ids_to_mark))})", req_ids_to_mark)
```

```
    cursor.execute("SELECT subject, id as message_id FROM messages WHERE sender_id = ? AND  
response IS NOT NULL AND (notified_user = 0 OR notified_user IS NULL) ORDER BY timestamp  
DESC", (user_id,))
```

```
    msg_ids_to_mark = []
```

```
    for subject, msg_id in cursor.fetchall():
```

```
        notifications.append(f"Admin replied to your message: '{subject}'.")
```

```
        msg_ids_to_mark.append(msg_id)
```

```

    if msg_ids_to_mark:

        cursor.execute(f"UPDATE messages SET notified_user = 1 WHERE id IN ({','.join(['?']*len(msg_ids_to_mark))})", msg_ids_to_mark)

        conn_db.commit()

        return {"status": "ok", "notifications": notifications}

    except Exception as e:

        conn_db.rollback()

        print(f"Error fetching notifications for user {user_id}: {e}")

        return {"status": "error", "message": "Failed to load notifications."}

    finally:

        conn_db.close()

else:

    client_ip_for_log = request.get('request_ip', 'Unknown IP')

    print(f"[UNKNOWN REQUEST TYPE] IP: {client_ip_for_log}, Type: {rtype}")

    return {"status": "error", "message": "Unknown request type received by server."}


def encrypt_response_with_aes(response_dict, aes_key):

    response_iv = os.urandom(16)

    cipher_encrypt = Cipher(algorithms.AES(aes_key), modes.CBC(response_iv),
                             backend=default_backend())

    encryptor = cipher_encrypt.encryptor()

    json_data_bytes = json.dumps(response_dict).encode('utf-8')

    padded_data = pkcs7_pad(json_data_bytes)

    encrypted_data = encryptor.update(padded_data) + encryptor.finalize()

    return {

        "iv": base64.b64encode(response_iv).decode('utf-8'),

        "data": base64.b64encode(encrypted_data).decode('utf-8')

    }

```

```

def start_server():

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    try:

        server.bind((SERVER_IP, SERVER_PORT))

    except OSError as e:

        print(f"!!! SERVER BIND FAILED: {e} !!!")

        print("Check if another process is using the port, or if IP is correct.")

        return

    server.listen()

    print(f"[LISTENING] Server running on {SERVER_IP}:{SERVER_PORT}")

    try:

        while True:

            try:

                conn, addr = server.accept()

                executor.submit(handle_client, conn, addr)

            except OSError as e:

                print(f"Server accept error: {e}. Shutting down executor.")

                break

    except KeyboardInterrupt:

        print("\n[SHUTTING DOWN] Keyboard interrupt received. Server is shutting down.")

    except Exception as e_main_loop:

        print(f"[CRITICAL SERVER ERROR] Main loop error: {e_main_loop}")

    finally:

        print("Closing server socket and shutting down thread pool...")

        executor.shutdown(wait=True)

        server.close()

        print("[CLOSED] Server has closed gracefully.")


if __name__ == "__main__":

```

```
start_server()
```

### **database.py:**

```
import sqlite3
```

```
import hashlib
```

```
import random
```

```
import string
```

```
from datetime import datetime, timedelta
```

```
def connect_db():
```

```
    try:
```

```
        conn = sqlite3.connect("users.db", timeout=10, check_same_thread=False)
```

```
        return conn
```

```
    except sqlite3.Error as e:
```

```
        print(f"Error while connecting to SQLite: {e}")
```

```
        return None
```

```
def setup_database():
```

```
    conn = connect_db()
```

```
    if conn:
```

```
        cursor = conn.cursor()
```

```
        try:
```

```
            cursor.execute("""
```

```
                CREATE TABLE IF NOT EXISTS usersDB (
```

```
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                    username TEXT NOT NULL UNIQUE,
```

```
                    email TEXT,
```

```
                    password TEXT NOT NULL,
```

```
                    id_number TEXT,
```

```
                    is_admin INTEGER DEFAULT 0,
```

```
                    phone TEXT,
```

```

        address TEXT,

        age INTEGER

    )

    """)

    print("Database setup completed successfully.")

except sqlite3.Error as e:

    print(f"Database setup error: {e}")

conn.commit()

conn.close()


def setup_user_data_table():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                CREATE TABLE IF NOT EXISTS user_data (

                    id INTEGER PRIMARY KEY AUTOINCREMENT,

                    user_id INTEGER NOT NULL,

                    dapar INTEGER DEFAULT 0,

                    profile INTEGER DEFAULT 0,

                    hebrew REAL DEFAULT 0,

                    interview TEXT DEFAULT "",

                    hundred_day INTEGER DEFAULT 0,

                    FOREIGN KEY (user_id) REFERENCES usersDB(id)

                )

            """)

            print("User data table setup completed successfully.")

        except sqlite3.Error as e:

            print(f"User data table setup error: {e}")

        conn.commit()

        conn.close()

```

```

def setup_meyunim_table():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                CREATE TABLE IF NOT EXISTS meyunim (

                    id INTEGER PRIMARY KEY AUTOINCREMENT,

                    title TEXT NOT NULL,

                    description TEXT,

                    date TEXT NOT NULL

                )

            """)

            print("Meyunim table setup completed successfully.")

        except sqlite3.Error as e:

            print(f"Meyunim table setup error: {e}")

        conn.commit()

        conn.close()


def setup_assignments_table():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                CREATE TABLE IF NOT EXISTS meyunim_assignments (

                    id INTEGER PRIMARY KEY AUTOINCREMENT,

                    user_id INTEGER NOT NULL,

                    meyun_id INTEGER NOT NULL,

                    status TEXT DEFAULT 'Scheduled',

                    FOREIGN KEY(user_id) REFERENCES usersDB(id),

                    FOREIGN KEY(meyun_id) REFERENCES meyunim(id)

```

```

    )

    """

    print("Assignments table setup completed successfully.")

except sqlite3.Error as e:

    print(f"Assignments table setup error: {e}")

conn.commit()

conn.close()


def update_date_for_request(request_id, new_date):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute('SELECT assignment_id FROM meyunim_requests WHERE id=?', (request_id,))

            row = cursor.fetchone()

            if not row:

                return False

            assignment_id = row[0]

            cursor.execute('SELECT meyun_id FROM meyunim_assignments WHERE id=?', (assignment_id,))

            row = cursor.fetchone()

            if not row:

                return False

            meyun_id = row[0]

            cursor.execute('UPDATE meyunim SET date=? WHERE id=?', (new_date, meyun_id))

            cursor.execute('UPDATE meyunim_requests SET status="Approved" WHERE id=?', (request_id,))

            conn.commit()

            return True

```

```

except Exception as e:

    print("update_date_for_request error:", e)

    return False

finally:

    conn.close()

return False

```

```

def setup_requests_table():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                CREATE TABLE IF NOT EXISTS meyunim_requests (

                    id INTEGER PRIMARY KEY AUTOINCREMENT,

                    assignment_id INTEGER NOT NULL,

                    type TEXT NOT NULL,

                    message TEXT,

                    status TEXT DEFAULT 'Pending',

                    timestamp TEXT DEFAULT (datetime('now')),

                    FOREIGN KEY(assignment_id) REFERENCES meyunim_assignments(id)

                )

            """)

            print("Requests table setup completed successfully.")

        except sqlite3.Error as e:

            print(f"Requests table setup error: {e}")

            conn.commit()

            conn.close()

```

```

def upgrade_requests_table():

    conn = connect_db()

```

```

if conn:

    cursor = conn.cursor()

    try:

        cursor.execute("ALTER TABLE meyunim_requests ADD COLUMN username TEXT")

    except sqlite3.Error:

        pass

    try:

        cursor.execute("ALTER TABLE meyunim_requests ADD COLUMN meyun_title TEXT")

    except sqlite3.Error:

        pass

    try:

        cursor.execute("ALTER TABLE meyunim_requests ADD COLUMN meyun_date TEXT")

    except sqlite3.Error:

        pass

    try:

        cursor.execute("ALTER TABLE meyunim_requests ADD COLUMN user_id INTEGER")

    except sqlite3.Error:

        pass

    try:

        cursor.execute("ALTER TABLE meyunim_requests ADD COLUMN notified_user INTEGER
DEFAULT 0")

    except sqlite3.Error:

        pass

    conn.commit()

    conn.close()

```

```

def setup_messages_table():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

```

```

try:
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS messages (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            sender_id INTEGER NOT NULL,
            subject TEXT,
            content TEXT,
            response TEXT DEFAULT "",
            timestamp TEXT,
            admin_responded_at DATETIME,
            notified_user INTEGER DEFAULT 0,
            FOREIGN KEY(sender_id) REFERENCES usersDB(id)
        )
    """)
    print("Messages table setup completed successfully.")
except sqlite3.Error as e:
    print(f"Messages table setup error: {e}")
conn.commit()
conn.close()

```

```

def setup_auth_codes_table():
    conn = connect_db()
    if conn:
        cursor = conn.cursor()
        try:
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS auth_codes (
                    user_id INTEGER,
                    code TEXT,
                    timestamp DATETIME,

```

```

        FOREIGN KEY(user_id) REFERENCES usersDB(id)

    )

    """)

    print("Auth codes table setup completed successfully.")

except sqlite3.Error as e:

    print(f"Auth codes table setup error: {e}")

conn.commit()

conn.close()


def hash_password(password):

    return hashlib.sha256(password.encode()).hexdigest()


def hash_id_number(id_number):

    return hashlib.sha256(id_number.encode()).hexdigest()


def check_user_credentials(username, password):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("SELECT id, password, is_admin, email FROM usersDB WHERE username=?",
                           (username,))

            row = cursor.fetchone()

            if row:

                user_id, db_password, is_admin, email = row

                if hash_password(password) == db_password:

                    return True, "Login successful.", is_admin, user_id, email

                else:

                    return False, "Incorrect password.", None, None, None

```

```

    else:

        return False, "Username not found.", None, None, None

except sqlite3.Error as e:

    print(f"Error during login check: {e}")

    return False, "Database error.", None, None, None

finally:

    conn.close()

return False, "Database connection error.", None, None, None


def get_user_data(user_id):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("SELECT dapar, profile, hebrew, interview, hundred_day FROM user_data WHERE
user_id=?", (user_id,))

            row = cursor.fetchone()

            if row:

                return row

            else:

                return (0, 0, 0, "", 0)

        except sqlite3.Error as e:

            print(f"Error fetching user data: {e}")

            return (0, 0, 0, "", 0)

        finally:

            conn.close()

    return (0, 0, 0, "", 0)


def get_all_users():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

```

```

try:

    cursor.execute("SELECT id, username, is_admin, phone, address, age FROM usersDB")

    users = cursor.fetchall()

    return users

except sqlite3.Error as e:

    print(f"Error fetching all users: {e}")

    return []

finally:

    conn.close()

return []

```

```

def update_user_data(user_id, dapar, profile, hebrew, interview, hundred_day):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("SELECT id FROM user_data WHERE user_id=?", (user_id,))

            exists = cursor.fetchone()

            if exists:

                cursor.execute("""

                    UPDATE user_data

                    SET dapar=?, profile=?, hebrew=?, interview=?, hundred_day=?

                    WHERE user_id=?

                    """, (dapar, profile, hebrew, interview, hundred_day, user_id))

            else:

                cursor.execute("""

                    INSERT INTO user_data (user_id, dapar, profile, hebrew, interview, hundred_day)

                    VALUES (?, ?, ?, ?, ?, ?)

                    """, (user_id, dapar, profile, hebrew, interview, hundred_day))

            conn.commit()

        except sqlite3.Error as e:

            print(f"Error updating user data: {e}")

```

```

finally:

    conn.close()

def generate_and_store_code(user_id):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cleanup_expired_codes()

            code = ''.join(random.choices(string.digits, k=6))

            now = datetime.now().isoformat()

            cursor.execute("INSERT INTO auth_codes (user_id, code, timestamp) VALUES (?, ?, ?)", (user_id,
code, now))

            conn.commit()

            return code

        except sqlite3.Error as e:

            print(f"Error generating auth code: {e}")

        finally:

            conn.close()

    return None

def verify_code(user_id, code):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            now = datetime.now()

            cursor.execute("SELECT code, timestamp FROM auth_codes WHERE user_id=? ORDER BY
timestamp DESC LIMIT 1", (user_id,))

            row = cursor.fetchone()

            if row:

                stored_code, timestamp = row

                timestamp_dt = datetime.fromisoformat(timestamp)

```

```

        if stored_code == code and (now - timestamp_dt).total_seconds() <= 600:

            return True

    except sqlite3.Error as e:

        print(f"Error verifying code: {e}")

    finally:

        conn.close()

return False


def cleanup_expired_codes():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            ten_minutes_ago = datetime.now() - timedelta(minutes=10)

            cursor.execute("DELETE FROM auth_codes WHERE timestamp < ?",
                           (ten_minutes_ago.isoformat(),))

            conn.commit()

        except sqlite3.Error as e:

            print(f"Error cleaning expired codes: {e}")

        finally:

            conn.close()


def reset_user_password(username, email, new_password):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("SELECT id FROM usersDB WHERE username=? AND email=?", (username,
                                                                                       email))

            row = cursor.fetchone()

            if row:

                user_id = row[0]

                hashed_password = hash_password(new_password)

```

```

        cursor.execute("UPDATE usersDB SET password=? WHERE id=?", (hashed_password, user_id))

        conn.commit()

        return True, "Password reset successfully."

    else:

        return False, "Username and email do not match."

except sqlite3.Error as e:

    print(f"Error resetting password: {e}")

    return False, "Database error."

finally:

    conn.close()

return False, "Database connection error."

def create_meyun(title, description, date):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("INSERT INTO meyunim (title, description, date) VALUES (?, ?, ?)",

                           (title, description, date))

            conn.commit()

            return cursor.lastrowid

        except sqlite3.Error as e:

            print(f"Error creating meyun: {e}")

        finally:

            conn.close()

    return None

def assign_meyun_to_users(meyun_id, user_ids):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            for user_id in user_ids:

                cursor.execute("

```

```

        INSERT INTO meyunim_assignments (user_id, meyun_id)

        VALUES (?, ?)

        "", (user_id, meyun_id))

    conn.commit()

except sqlite3.Error as e:

    print(f"Error assigning meyun: {e}")

finally:

    conn.close()

def get_user_meyunim(user_id):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                SELECT

                ma.id, m.title, m.description, m.date, ma.status,

                (

                    SELECT r.type || ':' || r.status

                    FROM meyunim_requests r

                    WHERE r.assignment_id = ma.id

                    ORDER BY r.id DESC

                    LIMIT 1

                ) as request_info

                FROM meyunim_assignments ma

                JOIN meyunim m ON ma.meyun_id = m.id

                WHERE ma.user_id=?

            """, (user_id,))

            return cursor.fetchall()

        except sqlite3.Error as e:

            print(f"Error fetching user meyunim: {e}")

        finally:

            conn.close()

```

```
return []
```

```
def delete_meyun(meyun_id):
```

```
    conn = connect_db()
```

```
    if conn:
```

```
        cursor = conn.cursor()
```

```
        try:
```

```
            cursor.execute("""
```

```
                DELETE FROM meyunim_requests
```

```
                WHERE assignment_id IN (
```

```
                    SELECT id FROM meyunim_assignments WHERE meyun_id=?
```

```
                )
```

```
            """, (meyun_id,))
```

```
        cursor.execute('DELETE FROM meyunim_assignments WHERE meyun_id=?', (meyun_id,))
```

```
        cursor.execute('DELETE FROM meyunim WHERE id=?', (meyun_id,))
```

```
        conn.commit()
```

```
        return True
```

```
    except Exception as e:
```

```
        print("Error deleting meyun:", e)
```

```
    finally:
```

```
        conn.close()
```

```
    return False
```

```
def get_requests_by_user(user_id):
```

```
    conn = connect_db()
```

```
    if conn:
```

```
        cursor = conn.cursor()
```

```
        try:
```

```

cursor.execute("""
    SELECT m.title, r.type, r.status, r.message, r.timestamp
    FROM meyunim_requests r
    JOIN meyunim_assignments ma ON r.assignment_id = ma.id
    JOIN meyunim m ON ma.meyun_id = m.id
    WHERE ma.user_id=?
    ORDER BY r.id DESC
""", (user_id,))

return cursor.fetchall()

except sqlite3.Error as e:

    print(f"Error fetching user requests: {e}")

finally:

    conn.close()

return []

```

```

def submit_request(assignment_id, request_type, message):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""
                SELECT u.id as user_id, u.username, m.title, m.date
                FROM meyunim_assignments a
                JOIN usersDB u ON u.id = a.user_id
                JOIN meyunim m ON m.id = a.meyun_id
                WHERE a.id=?
            """, (assignment_id,))

            row = cursor.fetchone()

            if not row:

                return False

```

```

actual_user_id, username, meyun_title, meyun_date = row

timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

cursor.execute("""

    INSERT INTO meyunim_requests

        (assignment_id, user_id, type, message, status, timestamp, username, meyun_title, meyun_date,
        notified_user)

        VALUES (?, ?, ?, ?, 'Pending', ?, ?, ?, ?, 0)

    """, (assignment_id, actual_user_id, request_type, message, timestamp, username, meyun_title,
    meyun_date))

    conn.commit()

    return True

except sqlite3.Error as e:

    print(f"Error submitting request: {e}")

finally:

    conn.close()

return False

```

```

def get_all_requests():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                SELECT r.id,

                    COALESCE(u.username, r.username) AS username,

                    COALESCE(m.title, r.meyun_title) AS title,

                    COALESCE(m.date, r.meyun_date) AS date,

                    r.type,

                    r.message,

                    r.status
            """)

```

```

        FROM meyunim_requests r

        LEFT JOIN meyunim_assignments ma ON r.assignment_id = ma.id

        LEFT JOIN usersDB u ON ma.user_id = u.id

        LEFT JOIN meyunim m ON ma.meyun_id = m.id

    "")

    return cursor.fetchall()

except sqlite3.Error as e:

    print("Error fetching requests:", e)

    return []

finally:

    conn.close()


def update_request_status(request_id, new_status):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                UPDATE meyunim_requests

                SET status=?

                WHERE id=?

            """, (new_status, request_id))

            if cursor.rowcount == 0:

                return False

            conn.commit()

            return True

        except Exception as e:

            print("Update request status error:", e)

            return False

        finally:

            conn.close()

    return False

```

```

def send_message_to_admin(sender_id, subject, content):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            timestamp = datetime.now().isoformat()

            cursor.execute("""

                INSERT INTO messages (sender_id, subject, content, timestamp)

                VALUES (?, ?, ?, ?)

            """, (sender_id, subject, content, timestamp))

            conn.commit()

            return True

        except sqlite3.Error as e:

            print(f"Error sending message: {e}")

        finally:

            conn.close()

    return False

```

```

def get_user_messages(user_id):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                SELECT subject, content, response, timestamp

                FROM messages

                WHERE sender_id=?

                ORDER BY timestamp DESC

            """, (user_id,))

```

```

        return cursor.fetchall()

    except sqlite3.Error as e:

        print(f"Error fetching user messages: {e}")

    finally:

        conn.close()

    return []

def get_all_messages():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                SELECT m.id, u.username, m.subject, m.content, m.response, m.timestamp

                FROM messages m

                JOIN usersDB u ON m.sender_id = u.id

                ORDER BY m.timestamp DESC

            """)

            return cursor.fetchall()

        except sqlite3.Error as e:

            print(f"Error fetching all messages: {e}")

        finally:

            conn.close()

    return []

def reply_to_message(message_id, response_text):

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("""

                UPDATE messages

```

```

        SET response=?

        WHERE id=?

        "", (response_text, message_id))

    conn.commit()

    return True

except sqlite3.Error as e:

    print(f"Error replying to message: {e}")

finally:

    conn.close()

return False


def upgrade_messages_table():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("ALTER TABLE messages ADD COLUMN notified_user INTEGER DEFAULT 0")

        except sqlite3.Error:

            pass

        conn.commit()

        conn.close()


def upgrade_users_table():

    conn = connect_db()

    if conn:

        cursor = conn.cursor()

        try:

            cursor.execute("ALTER TABLE usersDB ADD COLUMN id_number TEXT")

        except sqlite3.Error:

            pass

```

```
conn.commit()
```

```
conn.close()
```

```
if __name__ == "__main__":
```

```
    setup_database()
```

```
    setup_user_data_table()
```

```
    setup_auth_codes_table()
```

```
    setup_meyunim_table()
```

```
    setup_assignments_table()
```

```
    setup_requests_table()
```

```
    setup_messages_table()
```

```
    upgrade_requests_table()
```

```
    upgrade_messages_table()
```

```
    upgrade_users_table()
```

### **send\_email.py:**

```
import smtplib
```

```
from email.mime.text import MIMEText
```

```
from email.mime.multipart import MIMEMultipart
```

```
import os
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
SENDER_EMAIL = os.getenv("EMAIL_USER")
```

```
SENDER_PASSWORD = os.getenv("EMAIL_PASS")
```

```
def send_new_user_email(username, email, password):
```

```
    subject = "ברוך הבא למערכת" IlayDF
```

```
    body = f"שלום {username},
```

ברכות על כניסתך להליך הגיוס לשירות ביטחון

לניהול נתוני מלש"בים IlayDF לצורך הליך יעיל, מאובטח, וקל ככל האפשר, נוסף לך חשבון במערכת

{username}: שם המשתמש שלך

{password}: הסיסמה הזמנית שלך

אין לחשוף פרטים אלו לאיש

לאחר ההתחברות הראשונית תדרש להחליף את הסיסמא

בנוסף, תידרש לאמת את זהותך בעזרת מספר תעודת זהות

בברכת הצלחה והליך גיוס נעים,

IlayDF צוות

""""

```
message = MIMEMultipart()
message["From"] = SENDER_EMAIL
message["To"] = email
message["Subject"] = subject
message.attach(MIMEText(body, "plain"))

try:
    with smtplib.SMTP("smtp.gmail.com", 587) as server:
        server.starttls()
        server.login(SENDER_EMAIL, SENDER_PASSWORD)
        server.send_message(message)

    return True
except Exception as e:
    print(f"Failed to send new user email: {e}")

    return False
```

```
def send_verification_email(receiver_email, code):

    subject = "קוד אימות - IlayDF"

    body = f"שלום,"

    {code}: קוד האימות שלך הוא

    יש להזין את הקוד כדי להמשיך. הקוד תקף ל-10 דקות בלבד.

    אם לא ביקשת קוד זה, ניתן להתעלם מההודעה.

    תודה,

    IlayDF צוות האבטחה של

    ""

    message = MIMEMultipart()

    message["From"] = SENDER_EMAIL

    message["To"] = receiver_email

    message["Subject"] = subject

    message.attach(MIMEText(body, "plain"))

    try:

        with smtplib.SMTP("smtp.gmail.com", 587) as server:

            server.starttls()

            server.login(SENDER_EMAIL, SENDER_PASSWORD)

            server.send_message(message)

        return True

    except Exception as e:

        print(f"Failed to send email: {e}")

        return False
```

### ui\_elements.py:

```
import customtkinter as ctk
```

```

import os

class MyButton(ctk.CTkButton):

    def __init__(self, *args, **kwargs):

        kwargs.setdefault("fg_color", "#004d00")

        kwargs.setdefault("hover_color", "#006400")

        kwargs.setdefault("text_color", "white")

        kwargs.setdefault("height", 40)

        super().__init__(*args, **kwargs)

class BasePopup(ctk.CTkToplevel):

    def __init__(self, master, title="Popup", geometry="500x350", *args, **kwargs):

        super().__init__(master, *args, **kwargs)

        self.title(title)

        self.geometry(geometry)

        self.configure(fg_color="#E8F5E9")

        icon_path = "ico_logo.ico"

        if os.path.exists(icon_path):

            try:

                self.iconbitmap(icon_path)

            except Exception as e:

                print(f"Failed to set icon for popup: {e}")

        else:

            print(f"Popup icon not found at: {icon_path}")

        self.center_window()

        self.lift()

        self.attributes("-topmost", True)

        self.after(100, lambda: self.attributes("-topmost", False))

        self.grab_set()

```

```
self.focus_force()
```

```
def center_window(self):
    self.update_idletasks()
    width = self.winfo_width()
    height = self.winfo_height()
    x = (self.master.winfo_screenwidth() // 2) - (width // 2)
    y = (self.master.winfo_screenheight() // 2) - (height // 2)
    x = max(0, x)
    y = max(0, y)
    self.geometry(f'{width}x{height}+{x}+{y-30}')
```

```
def close_popup(self):
    self.grab_release()
    self.destroy()
```

### **validators.py:**

```
import re
```

```
def is_valid_email(email):
    return bool(re.match(r"^[^@]+@[^@]+\.[^@]+$", email))
```

```
def is_valid_phone(phone_str):
    return phone_str.isdigit() and 9 <= len(phone_str) <= 10
```

```
def is_valid_age(age_str):
    try:
        age = int(age_str)
        return 0 < age < 120
```

```
except ValueError:
```

```
    return False
```

```
def is_strong_password(password):
```

```
    if len(password) < 6:
```

```
        return False
```

```
    if not any(c.isalpha() for c in password):
```

```
        return False
```

```
    if not any(c.isdigit() for c in password):
```

```
        return False
```

```
    return True
```

```
def is_valid_dapar(value):
```

```
    try:
```

```
        val = int(value)
```

```
        return val in range(10, 91, 10)
```

```
    except ValueError:
```

```
        return False
```

```
def is_valid_hebrew(value):
```

```
    try:
```

```
        val = int(value)
```

```
        return 5 <= val <= 8
```

```
    except ValueError:
```

```
        return False
```

```
def is_valid_profile(value):
```

```
    try:
```

```

    val = int(value)

    return val in {97, 82, 72, 64, 45, 25, 24, 21, 0}

except ValueError:

    return False

```

### **add\_initial\_user.py:**

```

import sqlite3

from database import (
    hash_password,
    hash_id_number,
    connect_db,
    setup_database,
    setup_user_data_table,
    setup_auth_codes_table,
    setup_meyunim_table,
    setup_assignments_table,
    setup_requests_table,
    setup_messages_table,
    upgrade_requests_table,
    upgrade_users_table,
    upgrade_messages_table
)

def create_initial_admin():
    print("Initializing database schema...")

    setup_database()
    setup_user_data_table()
    setup_auth_codes_table()
    setup_meyunim_table()
    setup_assignments_table()
    setup_requests_table()
    setup_messages_table()

```

```

upgrade_requests_table()

upgrade_users_table()

upgrade_messages_table()

print("Database schema initialization/upgrade complete.")


conn = connect_db()

if not conn:

    print("Failed to connect to the database. Admin creation aborted.")

    return


cursor = conn.cursor()


try:

    cursor.execute("SELECT COUNT(*) FROM usersDB WHERE is_admin=1")

    count = cursor.fetchone()[0]


    if count > 0:

        print("An admin user already exists. No new admin will be created.")

        conn.close()

        return

except sqlite3.Error as e:

    print(f"Error checking for existing admin: {e}")

    conn.close()

    return


username = "admin"

email = "mailmail@gmail.com"

password = "admin123"

id_number = "123456789"


hashed_password = hash_password(password)

```

```
hashed_id = hash_id_number(id_number)
```

```
try:
```

```
    cursor.execute("""
        INSERT INTO usersDB (username, email, password, id_number, is_admin)
        VALUES (?, ?, ?, ?, 1)
        """, (username, email, hashed_password, hashed_id))
    conn.commit()
    print(f"Initial admin user '{username}' created successfully.")
    print(f"Username: {username}")
    print(f>Password: {password} (Change this password after first login!)")
```

```
except sqlite3.IntegrityError as e:
```

```
    print(f"Failed to create admin due to an integrity error (e.g., username/email already exists): {e}")
```

```
except sqlite3.Error as e:
```

```
    print(f"Failed to create admin due to a database error: {e}")
```

```
finally:
```

```
    conn.close()
```

```
if __name__ == "__main__":
```

```
    create_initial_admin()
```